

Universidad Nacional de Asunción
Facultad Politécnica

SIGA-Óptica

Sistema Integral de Gestión y Agendamiento para Ópticas

Informe de Trabajo Final de Grado

Autores: Matías Melgarejo

Matías Gaona

Carrera: Licenciatura en Ciencias Informáticas

Tutores: Prof. Lic. Diego Ruiz Diaz

Lic. Juan Fernando Dure

Índice general

1. Introducción	15
1.1. Antecedentes	15
1.2. Objetivo del documento	15
1.3. Alcance	15
2. Resumen del Proyecto	16
2.1. Problema	16
2.2. Solución propuesta	16
2.3. Alcance definitivo	16
I Arquitectura de la Solución	18
3. Arquitectura del Sistema	19
3.1. Visión general	19
3.2. Diagrama de componentes	19
3.3. Capas del backend	19
3.3.1. SIGA.Domain	19
3.3.2. SIGA.Application	19
3.3.3. SIGA.Infrastructure	20
3.3.4. SIGA.Api	21
3.4. Pipeline HTTP y seguridad (<code>Program.cs</code>)	21
3.5. Autorización: permisos, no roles	21
3.6. Multi-sucursal	21
3.7. Módulos de negocio y controllers	22
3.8. Trabajos en segundo plano	23
3.9. Generación de documentos y PDFs	23
3.10. Integraciones externas	23
3.11. Convenciones	24
3.12. Frontend como cliente	24
4. Arquitectura del Frontend	25
4.1. Visión general	25
4.2. Estructura de carpetas	25
4.3. Patrón vista, composable y servicio	26
4.4. Manejo de estado	26
4.5. Router	27
4.6. Sidebar y navegación (<code>config/menuConfig.ts</code>)	27
4.7. Responsive (mobile/tablet)	27
4.8. Sistema de diseño	28

II Construcción del Software	29
5. Módulo: Identidad y Acceso	30
5.1. Propósito	30
5.2. Entidades principales	30
5.3. Reglas de negocio clave	30
5.4. Endpoints	31
5.5. Flujo típico	31
5.5.1. Auto-registro de paciente y verificación de email	31
5.5.2. Cambio de contraseña obligatorio (cuenta nueva de staff)	31
5.6. Vistas de frontend	31
5.7. Interfaz	33
5.8. Estado	33
6. Módulo: Pacientes y Profesionales	34
6.1. Propósito	34
6.2. Entidades principales	34
6.3. Reglas de negocio clave	34
6.4. Endpoints	35
6.5. Flujo típico	35
6.5.1. Alta de profesional (transacción multi-entidad)	35
6.5.2. Alta de paciente sin cuenta (caso normal)	35
6.6. Vistas de frontend	36
6.7. Interfaz	36
6.8. Estado	36
7. Módulo: Personal	38
7.1. Propósito	38
7.2. Entidades principales	38
7.3. Reglas de negocio clave	38
7.4. Endpoints	39
7.5. Flujo típico	39
7.6. Vistas de frontend	39
7.7. Interfaz	39
7.8. Estado	39
8. Agenda y Turnos	41
8.1. Propósito	41
8.2. Entidades principales	41
8.3. Reglas de negocio clave	41
8.4. Endpoints	42
8.5. Flujo típico	42
8.6. Vistas de frontend	44
8.7. Interfaz	44
8.8. Estado	44

9. Módulo: Clínica	45
9.1. Propósito	45
9.2. Entidades principales	45
9.3. Reglas de negocio clave	45
9.4. Endpoints	46
9.5. Flujo típico	46
9.6. Vistas de frontend	47
9.7. Interfaz	47
9.8. Estado	48
10.Módulo: Clientes	49
10.1. Propósito	49
10.2. Entidades principales	49
10.3. Reglas de negocio clave	49
10.4. Endpoints	50
10.5. Flujo típico	50
10.6. Vistas de frontend	51
10.7. Interfaz	51
10.8. Estado	51
11.Inventario / Catálogo	52
11.1. Propósito	52
11.2. Entidades principales	52
11.3. Reglas de negocio clave	53
11.4. Endpoints	53
11.5. Flujo típico	54
11.6. Vistas de frontend	54
11.7. Interfaz	55
11.8. Estado	55
12.Ventas	56
12.1. Propósito	56
12.2. Entidades principales	56
12.3. Reglas de negocio clave	56
12.4. Endpoints	57
12.5. Flujo típico	58
12.6. Vistas de frontend	58
12.7. Interfaz	58
12.8. Estado	58
13.Laboratorio	61
13.1. Propósito	61
13.2. Entidades principales	61
13.3. Reglas de negocio clave	61
13.4. Endpoints	62
13.5. Flujo típico	62
13.6. Vistas de frontend	63
13.7. Interfaz	64
13.8. Estado	64

14.Compras	65
14.1. Propósito	65
14.2. Entidades principales	65
14.3. Reglas de negocio clave	65
14.4. Endpoints	66
14.5. Flujo típico	66
14.6. Vistas de frontend	68
14.7. Interfaz	68
14.8. Estado	69
15.Caja y Egresos	70
15.1. Propósito	70
15.2. Entidades principales	70
15.3. Reglas de negocio clave	70
15.4. Endpoints	71
15.5. Flujo típico	72
15.6. Vistas de frontend	72
15.7. Interfaz	73
15.8. Estado	73
16.Módulo: Roles y Permisos	74
16.1. Propósito	74
16.2. Entidades principales	74
16.3. Reglas de negocio clave	74
16.4. Endpoints	75
16.5. Flujo típico	75
16.6. Vistas de frontend	75
16.7. Interfaz	75
16.8. Estado	75
17.Notificaciones	77
17.1. Propósito	77
17.2. Entidades principales	77
17.3. Reglas de negocio clave	77
17.4. Endpoints	78
17.5. Flujo típico	78
17.6. Vistas de frontend	79
17.7. Interfaz	79
17.8. Estado	80
18.Reportes	81
18.1. Propósito	81
18.2. Entidades principales	81
18.3. Reglas de negocio clave	81
18.4. Endpoints	82
18.5. Flujo típico	82
18.6. Vistas de frontend	82
18.7. Interfaz	82
18.8. Estado	82

19. Sucursales	84
19.1. Propósito	84
19.2. Entidades principales	84
19.3. Reglas de negocio clave	84
19.4. Endpoints	85
19.5. Flujo típico	85
19.6. Vistas de frontend	86
19.7. Interfaz	86
19.8. Estado	86
20. Modelo de Datos	88
20.1. Grupo A — Identidad, Personas y Personal	88
20.1.1. Identidad y control de acceso (RBAC)	88
20.1.2. Roles funcionales de una persona	88
20.1.3. Personal: disponibilidad y sucursales	88
20.1.4. Diccionario de datos — Grupo A	88
20.2. Grupo B — Catálogo, Inventario, Clínica y Agenda	93
20.2.1. Catálogo de productos y catálogo óptico	93
20.2.2. Inventario, stock y transferencias	93
20.2.3. Clínica: consultas y recetas	93
20.2.4. Agenda: turnos	93
20.2.5. Diccionario de datos — Grupo B	93
20.3. Grupo C — Comercial: Ventas, Laboratorio, Compras, Caja y Egresos	98
20.3.1. Ventas	98
20.3.2. Comprobantes, facturación y devoluciones	99
20.3.3. Laboratorio óptico	99
20.3.4. Compras y proveedores	99
20.3.5. Egresos — herencia TPH	99
20.3.6. Caja	99
20.3.7. Diccionario de datos — Grupo C	99
20.4. Entidades transversales	99
20.5. Tabla de enums	105
20.6. Índices, constraints y convenciones de tipos	106
20.7. Patrones de diseño transversales	107
21. Implementación	108
21.1. Requisitos	108
21.2. Instalación	108
21.3. Configuración	109
22. Validación	110
22.1. Metodología	110
22.2. Casos de prueba y resultados	110
22.2.1. Aislamiento multi-sucursal	110
22.2.2. Reglas de complejidad y reuso de contraseña	110
22.2.3. Migración de estados de texto libre a enumerados	110
22.2.4. Unificación de la escala monetaria	111
22.2.5. Reportes operativos	111

22.2.6. Preferencias de notificación	111
22.2.7. Aislamiento por sucursal de la receta óptica	111
22.3. Capturas de evidencia	111
23. Cambios respecto al Proyecto I	114
23.1. Sucursal: de atributo simple a mecanismo de aislamiento transversal	114
23.2. Reposición de stock: de pedido automático a notificación	114
23.3. Descuento de stock: del momento de entrega al momento de emisión del comprobante	115
23.4. Cancelación de citas: de acción autónoma del paciente a solicitud resuelta por el personal	115
23.5. Interfaz de usuario: componentes propios en vez de una librería de componentes .	116
23.6. Notificaciones: un solo canal real en vez de multicanal desde el inicio	116
24. Conclusiones	117
24.1. Objetivos alcanzados	117
24.2. Limitaciones	117
24.3. Mejoras futuras	118
A. Referencia de API	120
A.1. 1. Identidad y Acceso	120
A.1.1. AuthController — /api/auth	120
A.1.2. UsersController — /api/users ([Authorize] a nivel de clase)	121
A.1.3. RolesController — /api/roles ([Authorize] a nivel de clase)	121
A.1.4. UserRolesController — /api/users/{userId}/roles (mismo archivo RolesController.cs, [Authorize] a nivel de clase)	121
A.2. 2. Personas	122
A.2.1. PatientsController — /api/patients ([Authorize])	122
A.2.2. ProfessionalsController — /api/professionals ([Authorize])	122
A.2.3. ClientesController — /api/clientes ([Authorize])	123
A.2.4. EspecialidadesController — /api/especialidades ([Authorize])	123
A.3. 3. Personal	124
A.3.1. EmpleadosController — /api/empleados (policy de clase ver_empleados; usa su propio ToResponse, no BaseController)	124
A.4. 4. Agenda	124
A.4.1. TurnosController — /api/turnos ([Authorize])	124
A.4.2. HorariosController — /api/professionals/{professionalId} ([Authorize], anidado bajo Professionals)	125
A.5. 5. Clínica	126
A.5.1. ConsultasController — /api/consultas ([Authorize])	126
A.5.2. RecetasController — /api/recetas ([Authorize])	127
A.6. 6. Catálogo / Inventario	127
A.6.1. ProductosController — /api/productos ([Authorize])	127
A.6.2. MarcasController — /api/marcas ([Authorize])	129
A.6.3. TipoLentesController — /api/tipos-lente ([Authorize])	129
A.6.4. TratamientosController — /api/tratamientos ([Authorize])	130
A.6.5. ServiciosController — /api/servicios ([Authorize])	130
A.6.6. MotivosMovimientoController — /api/motivos-movimiento ([Authorize])	131

A.6.7. StockLotesController — /api/stock/lotos (policy de clase ver_inventario)	131
A.6.8. UbicacionesController — /api/ubicaciones ([Authorize])	131
A.7. 7. Ventas	132
A.7.1. VentasController — /api/ventas ([Authorize])	132
A.7.2. TimbradosController — /api/timbrados ([Authorize])	133
A.8. 8. Laboratorio	134
A.8.1. LaboratorioController — /api/laboratorio ([Authorize])	134
A.9. 9. Compras	134
A.9.1. ComprasController — /api/compras/pedidos (clase: gestionar_pedidos)	134
A.9.2. ProveedoresController — /api/proveedores ([Authorize])	135
A.9.3. FacturasCompraController — /api/compras/facturas (policy de clase ver_inventario)	135
A.9.4. RecepcionesController — /api/compras/recepciones (policy de clase ver_inventario)	136
A.10.10. Caja y Egresos	136
A.10.1. CajaController — /api/caja (policy de clase ver_ventas)	136
A.10.2. EgresosController — /api/egresos (policy de clase gestionar_egresos)	137
A.11.11. Configuración	138
A.11.1. ConfiguracionController — /api/configuracion ([Authorize])	138
A.11.2. EstadosConfigController — /api/estados-config ([Authorize])	139
A.12.12. Sucursales	139
A.12.1. SucursalesController — /api/sucursales ([Authorize])	139
A.12.2. TransferenciasController — /api/transferencias (policy de clase transferir_stock)	140
A.13.13. Notificaciones	140
A.13.1. NotificacionesController — /api/notificaciones (policy de clase ver_notificaciones)	140
A.13.2. NotificacionPreferenciasController — /api/notificaciones/preferencias ([Authorize] a nivel de clase)	141
A.14.14. Reportes	141
A.14.1. ReportesController — /api/reportes ([Authorize])	141
B. Decisiones de Diseño	143
B.1. ADR 0001 — Person como raíz única de identidad; Cliente cuelga de Person	143
B.2. ADR 0002 — Stock derivado de movimientos aprobados, no un campo mutable	144
B.3. ADR 0003 — Autorización basada en permisos individuales, no en roles con nombre fijo	144
B.4. ADR 0004 — Precio de venta siempre derivado del margen de categoría	145
B.5. ADR 0005 — TrabajoPedido nace en Borrador junto al presupuesto	146
B.6. ADR 0006 — Cobro desacoplado del ciclo del laboratorio	146
B.7. ADR 0007 — El cristal graduado ya no se modela como Producto con stock	147
B.8. ADR 0008 — Receta puede ser clínica o externa/manual	147
B.9. ADR 0009 — Una sucursal fija por usuario, sin selector	148
B.10. ADR 0010 — Centro de notificaciones interno antes que integración externa	148
B.11. ADR 0011 — Egreso como jerarquía TPH unificando las salidas de dinero	149
B.12. ADR 0012 — Cambio de contraseña obligatorio solo para cuentas nuevas	150

C. Manual de Usuario	151
C.1. Qué es SIGA y para quién	151
C.1.1. Los roles que existen	151
C.1.2. Roles a medida	151
C.2. Primeros pasos	152
C.2.1. Cómo se ingresa	152
C.2.2. La primera vez: cambio de contraseña obligatorio	152
C.2.3. Cómo se recupera la contraseña	152
C.2.4. Registro de pacientes	152
C.2.5. Cómo se cierra sesión	153
C.2.6. Por qué cada uno ve un menú distinto	153
C.3. Zona General	153
C.3.1. Panel de Control	153
C.3.2. Notificaciones	153
C.3.3. Mis Turnos (paciente)	154
C.3.4. Mi Historial (paciente)	154
C.3.5. Mis Recetas (paciente)	154
C.4. Zona Atención	154
C.4.1. Pacientes	154
C.4.2. Agenda	155
C.4.3. Clínica: Recepción	155
C.4.4. Clínica: Consultas	156
C.4.5. Clínica: Historial	156
C.5. Zona Óptica	156
C.5.1. Catálogo: Productos	156
C.5.2. Catálogo: Categorías	157
C.5.3. Catálogo: Armazón (Marcas y Modelos)	157
C.5.4. Catálogo: Óptico (Tipos de Lente y Tratamientos)	157
C.5.5. Inventario: Niveles de Stock	157
C.5.6. Inventario: Movimientos	158
C.5.7. Inventario: Transferencias	158
C.5.8. Inventario: Registrar Conteo e Inventario Físico	158
C.5.9. Inventario: Aprobaciones (de movimientos y de conteos)	159
C.5.10. Configuración: Motivos	159
C.6. Zona Comercial	159
C.6.1. Compras: Proveedores	159
C.6.2. Compras: Órdenes de Compra	159
C.6.3. Compras: Aprobación de OC	160
C.6.4. Compras: Facturas de Compra	160
C.6.5. Compras: Recepciones	160
C.6.6. Compras: Reportes de Compras	161
C.6.7. Egresos	161
C.6.8. Ventas: Configuración (Timbrados, Clientes, Servicios)	162
C.6.9. Ventas: Presupuestos	162
C.6.10. Ventas: Nueva Venta	162
C.6.11. Ventas: Lista de Ventas y Detalle	163
C.6.12. Ventas: Cobros Pendientes y Cobrar	163
C.6.13. Ventas: Emitir documento (comprobante o factura)	163

C.6.14. Ventas: Facturas de Venta	164
C.6.15. Laboratorio	164
C.6.16. Caja	164
C.7. Zona Gestión	165
C.7.1. Reportes	165
C.7.2. Personal	165
C.7.3. Administración	166
C.8. Flujos completos de punta a punta	166
C.8.1. Flujo A: Atender a un paciente	166
C.8.2. Flujo B: Vender unos lentes	167
C.8.3. Flujo C: Abrir y cerrar la caja del día	167
C.8.4. Flujo D: Reponer stock	168
C.9. Preguntas frecuentes y errores comunes	168
C.10. Cobertura	169
D. Manual Técnico	171
D.1. Entorno de desarrollo	171
D.1.1. Requisitos	171
D.1.2. Levantar el sistema	171
D.2. Dónde está cada cosa	172
D.3. Trabajar con migraciones	172
D.4. Cómo agregar un endpoint nuevo	173
D.5. Cómo agregar una vista nueva	173
D.6. Convenciones a tener en cuenta	174
D.7. Construcción de las imágenes	175

Índice de figuras

3.1. Diagrama de componentes de SIGA: el frontend consume la API vía HTTPS con JWT Bearer; <code>SIGA.Api</code> depende de <code>SIGA.Application</code> y <code>SIGA.Domain</code> ; <code>SIGA.Infrastructure</code> implementa los contratos de <code>Application</code> (línea punteada) y depende de <code>Domain</code> y de los servicios externos (PostgreSQL, Resend, hCaptcha).	20
4.1. Flujo de datos en el frontend: de la vista al backend, con las dos rutas posibles (servicio directo o vía composable de dominio) y el acceso a estado global de Pinia.	26
5.1. Auto-registro público de paciente: alta con <code>IsEmailVerified=false</code> y envío del email de verificación.	32
5.2. Verificación de email: continuación del flujo de auto-registro tras el click en el enlace enviado por Resend.	32
5.3. Cambio de contraseña obligatorio: el guard del router intercepta cualquier navegación mientras <code>mustChangePassword</code> sea verdadero.	32
5.4. Pantalla de inicio de sesión de SIGA-Óptica, con acceso alternativo vía Google y enlace de recuperación de contraseña.	33
6.1. Alta de profesional: transacción única que crea <code>Person</code> , <code>User</code> , <code>Professional</code> , asigna el rol y las especialidades.	36
6.2. Listado de pacientes registrados en el sistema.	37
7.1. Gestión de empleados administrativos, con su cargo, fecha de ingreso y estado. .	40
8.1. Self-booking: búsqueda de disponibilidad y reserva (parte 1 de 2).	43
8.2. Self-booking: creación del turno y confirmación por email (parte 2 de 2).	43
8.3. Agenda de turnos del día, con los contadores de estado (pendientes, confirmados, en sala, completados, cancelados) y el filtro por profesional.	44
9.1. Alta de consulta y receta: el paciente ya puede ver la consulta y descargar el PDF desde el portal apenas se completa este flujo.	47
9.2. Listado de consultas clínicas, con paciente, profesional, diagnóstico y estado. . . .	47
10.1. Alta rápida de cliente embebida en el armado de una venta: si el cliente no existe, se busca primero una <code>Person</code> existente por CI antes de crear una nueva.	50
10.2. Gestión de clientes, con su tipo de facturación y estado.	51
11.1. Movimiento de stock: alta en <code>Pendiente</code> y aprobación posterior, que recién ahí impacta <code>vw_stock_actual</code>	54
11.2. Catálogo de productos, con categoría, SKU y estado de cada uno.	55
12.1. Venta a pedido (1/2): creación del presupuesto y confirmación, momento en el que el <code>TrabajoPedido</code> recién entra a la cola del laboratorio (ver Capítulo 13).	58

12.2. Venta a pedido (2/2): cobro y emisión de comprobante/factura. El laboratorio sigue su propio ciclo en paralelo desde que el <code>TrabajoPedido</code> pasa a <code>PendienteAprobacion</code> — no bloquea ni es bloqueado por el cobro.	59
12.3. Listado de ventas, con tipo (directa o a pedido), total, saldo pendiente y estado de cada una.	59
13.1. Origen del <code>TrabajoPedido</code> al confirmar una venta a pedido. Nace en <code>Borrador</code> (editable, fuera de la cola) y recién entra a la cola de laboratorio al confirmarse. .	62
13.2. Ciclo del laboratorio tras la aprobación del <code>TrabajoPedido</code> . La venta ya estaba <code>ListaParaCobrar</code> (o cobrada) desde su confirmación — este ciclo no la bloqueó en ningún punto (ver §13.3).	63
13.3. Cola de pedidos al laboratorio óptico externo, con el tipo de lente y su estado. . .	64
14.1. Desde la creación de la orden de compra hasta el registro de la factura del proveedor. La factura se registra antes de la recepción física.	67
14.2. Recepción física (una o más, parcial) y devolución opcional al proveedor.	67
14.3. Listado de órdenes de compra a proveedores, con la cantidad de ítems, el total y su estado.	68
15.1. Apertura de sesión, creación de un egreso y registro de su pago (interno o externo). Tras el pago interno, el estado del egreso pasa a <code>Pagado</code>	72
15.2. Cierre de sesión con arqueo: dentro de tolerancia cierra directo, fuera de tolerancia requiere aprobación de un supervisor.	72
15.3. Listado de egresos, agrupando facturas de compra, gastos generales y facturas de laboratorio.	73
16.1. Roles del sistema con la cantidad de permisos asignados a cada uno.	76
17.1. Trigger de bajo stock: polling cada 30 minutos, sin duplicar avisos ya generados y sin leer.	78
17.2. Consumo desde el frontend: badge de contador, listado desplegable y marcado de lectura.	79
17.3. Centro de notificaciones internas, con avisos de stock bajo y transferencias pendientes de aprobación.	79
18.1. Vista previa paginada vs. exportación completa: mismos filtros, dos caminos — el export trae todas las filas sin paginar y las entrega como PDF (QuestPDF) o CSV (UTF-8+BOM).	83
18.2. Panel de acceso a los reportes gerenciales (ventas, citas, inventario) y operativos (ventas, compras, movimientos de inventario), exportables a PDF y CSV.	83
19.1. Transferencia de stock: crea Salida en origen (<code>Pendiente</code>); solo el destino gestiona la aceptación/rechazo.	86
19.2. Listado de sucursales activas.	87
20.1. Identidad y control de acceso basado en permisos (RBAC). El JWT emite permisos resueltos, no el nombre del rol — ver Capítulo 3, §3.5.	89

20.2. Roles funcionales que puede tener una Person : Patient y Cliente cuelgan directo de Person ; Professional y Empleado cuelgan de User . La línea Professional-Empleado es solo de layout (no hay FK real entre ambos — son roles independientes que pueden coexistir en la misma Person).	90
20.3. Disponibilidad del profesional para agenda (horarios por sucursal, pausas, bloqueos) y catálogo geográfico de Paraguay reutilizado por Sucursal y Proveedor	91
20.4. Catálogo de productos (armazones, accesorios, etc.), catálogo óptico (TipoLente/Tratamiento , sin stock propio) y catálogo de servicios con tarifas por profesional/especialidad. TipoLente y Tratamiento se consumen desde TrabajoPedido (Grupo C), no desde Producto	94
20.5. El stock no es un campo de Producto : se deriva sumando MovimientoStock filtrados por Estado=.'probado' . StockActualView mapea la vista SQL vw_stock_actual (sin PK, HasNoKey()). TransferenciaStock mueve stock entre dos sucursales con flujo de aprobación.	95
20.6. ConsultaClinica y su receta óptica opcional. Una Receta puede originarse en una consulta (1:1, Cascade) o cargarse manual para una Person sin consulta previa (receta “externa”). EstadoConfigId es un estado cosmético/configurable, no reemplaza ningún enum.	96
20.7. Turno.Estado es el enum real que gobierna la lógica de negocio; EstadoCustomId es puramente cosmético/configurable (color, nombre visible), no lo reemplaza. . .	97
20.8. Venta es la cabecera única para venta directa y presupuesto/venta a pedido. Un Cobro puede tener varias CobroLinea (métodos de pago combinados).	98
20.9. FacturaVenta y Comprobante son dos documentos distintos, ambos 1:1 con Venta (Cascade). FacturaVenta está ligada a un Timbrado vigente de la sucursal.	100
20.10 TrabajoPedido es el núcleo del flujo “óptica a pedido”. El mismo catálogo de Proveedor sirve tanto para compras de mercadería como para laboratorios ópticos externos (EsLaboratorio).	101
20.11 Ciclo de compras: OC (PedidoProveedor , Borrador → Confirmada → Recibida-Parcial/Total → Facturada, o Cancelada) → una o más recepciones parciales → factura del proveedor opcional (ver Figura 20.12).	102
20.12 Egreso es la clase base abstracta (Table-Per-Hierarchy) de toda salida de dinero, mapeada a una única tabla física egresos con columna discriminadora Tipo . Los 5 subtipos solo agregan 1-3 columnas específicas y comparten todo el resto de la base. Ver ADR 0011 — Egreso como jerarquía TPH.	103
20.13 SesionCaja es “una caja abierta por sucursal a la vez”; cierre con conteo físico vs. esperado y flujo de aprobación si hay diferencia. Cada MovimientoCaja se origina en una Venta , un Egreso , o es manual.	104
22.1. Sesión autenticada como administrador — visibilidad completa de usuarios de ambas sucursales, sin la restricción que sí aplica a un usuario con sucursal fija. . .	112
22.2. Agenda de turnos verificada end-to-end: creación, confirmación y cancelación de citas.	112
22.3. Listado de ventas verificado tras ejercitar el ciclo completo de venta directa y venta a pedido, incluyendo presupuesto, confirmación, cobro y emisión de comprobante. . .	112
22.4. Panel de reportes desde el cual se ejercitaron las ocho combinaciones de exportación (cuatro reportes operativos por dos formatos) descriptas en la Sección 22.2.5. . .	113

Índice de cuadros

2.1. Módulos entregados en SIGA-Óptica.	16
3.1. Mapeo de módulos de negocio a controllers del backend.	22
5.1. Entidades principales del módulo de Identidad y Acceso.	30
5.2. Endpoints principales del módulo de Identidad y Acceso.	31
6.1. Entidades principales del módulo de Pacientes y Profesionales.	34
6.2. Endpoints principales del módulo de Pacientes y Profesionales.	35
7.1. Entidades principales del módulo de Personal.	38
7.2. Endpoints principales del módulo de Personal.	39
8.1. Entidades principales del módulo de Agenda.	41
8.2. Endpoints principales del módulo de Agenda.	42
9.1. Entidades principales del módulo de Clínica.	45
9.2. Endpoints principales del módulo de Clínica.	46
10.1. Entidades principales del módulo de Clientes.	49
10.2. Endpoints principales del módulo de Clientes.	50
11.1. Entidades principales del módulo de Inventario/Catálogo.	52
11.2. Controllers del módulo de Inventario/Catálogo.	53
12.1. Entidades principales del módulo de Ventas.	56
13.1. Entidades principales del módulo de Laboratorio.	61
13.2. Endpoints de transición de estado del TrabajoPedido.	62
13.3. Vistas de frontend del módulo Laboratorio.	63
14.1. Entidades principales del módulo de Compras.	65
14.2. Endpoints de transición de estado de la OC.	66
14.3. Vistas de frontend del módulo Compras.	68
15.1. Entidades principales del módulo de Caja y Egresos.	70
15.2. Endpoints principales de Caja y Egresos.	71
15.3. Vistas de frontend del módulo Caja y Egresos.	73
16.1. Entidades principales del módulo de Roles y Permisos.	74
16.2. Endpoints principales del módulo de Roles y Permisos.	75
17.1. Entidad principal del módulo de Notificaciones.	77
17.2. Endpoints del módulo de Notificaciones.	78
18.1. Entidades fuente de cada reporte.	81

18.2. Endpoints de Reportes.	82
19.1. Entidades principales del módulo de Sucursales.	84
19.2. Endpoints de Sucursales y Transferencias.	85
20.1. Los 28 enums de negocio del modelo, guardados como <code>int</code> vía <code>HasConversion<int>()</code>	105
21.1. Servicios del stack de producción.	108
21.2. Variables de entorno del stack de producción.	109
A.1. <code>AuthController</code>	120
A.2. <code>UsersController</code>	121
A.3. <code>RolesController</code>	121
A.4. <code>UserRolesController</code>	121
A.5. <code>PatientsController</code>	122
A.6. <code>ProfessionalsController</code>	122
A.7. <code>ClientesController</code>	123
A.8. <code>EspecialidadesController</code>	123
A.9. <code>EmpleadosController</code>	124
A.10. <code>TurnosController</code>	124
A.11. <code>HorariosController</code>	125
A.12. <code>ConsultasController</code>	126
A.13. <code>RecetasController</code>	127
A.14. <code>ProductosController</code>	127
A.15. <code>MarcasController</code>	129
A.16. <code>TipoLentesController</code>	129
A.17. <code>TratamientosController</code>	130
A.18. <code>ServiciosController</code>	130
A.19. <code>MotivosMovimientoController</code>	131
A.20. <code>StockLotesController</code>	131
A.21. <code>UbicacionesController</code>	132
A.22. <code>VentasController</code>	132
A.23. <code>TimbradosController</code>	133
A.24. <code>LaboratorioController</code>	134
A.25. <code>ComprasController</code>	134
A.26. <code>ProveedoresController</code>	135
A.27. <code>FacturasCompraController</code>	136
A.28. <code>RecepcionesController</code>	136
A.29. <code>CajaController</code>	137
A.30. <code>EgresosController</code>	137
A.31. <code>ConfiguracionController</code>	139
A.32. <code>EstadosConfigController</code>	139
A.33. <code>SucursalesController</code>	139
A.34. <code>TransferenciasController</code>	140
A.35. <code>NotificacionesController</code>	140
A.36. <code>NotificacionPreferenciasController</code>	141
A.37. <code>ReportesController</code>	142

Capítulo 1

Introducción

1.1. Antecedentes

El Centro Óptico Santa María gestionaba su operación — agendamiento de citas, historia clínica de pacientes, control de inventario y seguimiento de ventas — de forma manual y fragmentada, sin una plataforma que centralizara esa información. Sobre esa necesidad, en el marco de la Licenciatura en Ciencias Informáticas de la Facultad Politécnica de la Universidad Nacional de Asunción, el anteproyecto relevó el problema y propuso el desarrollo de SIGA-Óptica: un sistema integral de gestión y agendamiento que unifique las operaciones administrativas, clínicas y comerciales de la óptica y su consultorio oftalmológico asociado.

El anteproyecto definió el problema, los objetivos, el modelo de casos de uso de alto nivel y un diseño preliminar de entidades. Este documento no repite ese análisis; lo da por sentado y se concentra en reportar lo que efectivamente se construyó.

1.2. Objetivo del documento

Este documento constituye el informe del trabajo final de grado y reporta el resultado del desarrollo de SIGA-Óptica: la arquitectura de la solución finalmente adoptada, la construcción de cada módulo de negocio, el modelo de datos, el proceso de implementación y despliegue, la validación realizada sobre el sistema y los cambios introducidos respecto al diseño planteado en el anteproyecto, con su justificación.

1.3. Alcance

Quedan dentro del sistema implementado y reportado en este documento los quince módulos de negocio detallados a partir del Capítulo 5: identidad y acceso, pacientes y profesionales, personal, agenda de turnos, historia clínica, clientes, inventario y catálogo, ventas, laboratorio óptico, compras, caja y egresos, roles y permisos, notificaciones, reportes y soporte multi-sucursal.

Quedan fuera de este entregable — y se documentan como tales en el Capítulo 23 y en las Conclusiones (Capítulo 24) — las funcionalidades que el anteproyecto contemplaba y no se llegaron a implementar en esta etapa, así como las limitaciones de diseño conocidas del sistema actual.

Capítulo 2

Resumen del Proyecto

2.1. Problema

El Centro Óptico Santa María administraba el agendamiento de citas, la historia clínica de sus pacientes, el control de inventario de productos ópticos y el seguimiento de ventas mediante procesos manuales y herramientas desconectadas entre sí. Esa fragmentación dificultaba tener información confiable y actualizada para la atención al paciente y para la toma de decisiones administrativas, y exponía a la óptica a errores de registro, duplicidad de datos y demoras evitables.

2.2. Solución propuesta

SIGA-Óptica es un sistema web integral que centraliza la gestión administrativa, clínica y comercial de la óptica en una única plataforma. La solución se estructura como una API REST (ASP.NET Core Microsoft, 2026a / .NET 10, Entity Framework Core Microsoft, 2026b, PostgreSQL PostgreSQL Global Development Group, 2026) consumida por una aplicación web (Vue 3 Vue.js, 2026 / TypeScript Microsoft, 2026c), con control de acceso basado en permisos individuales y soporte para operar simultáneamente en más de una sucursal física. El Capítulo 3 describe la arquitectura adoptada en detalle.

Sobre esa base se construyeron los módulos de negocio necesarios para cubrir el ciclo completo de la óptica: desde el registro de un paciente y el agendamiento de su cita, pasando por la consulta clínica y la receta que de ella resulta, hasta la venta de productos ópticos — de mostrador o “a pedido”, con armazón y cristal a medida fabricados por un laboratorio externo —, su cobro, la emisión del comprobante fiscal correspondiente y el control del stock e inventario que esa venta consume.

2.3. Alcance definitivo

El sistema entregado comprende quince módulos de negocio:

Cuadro 2.1: Módulos entregados en SIGA-Óptica.

Módulo	Cubre
Identidad y Acceso (Cap. 5)	Autenticación JWT, registro, verificación de email, recuperación de contraseña, control de acceso por permisos
Pacientes y Profesionales (Cap. 6)	Alta y gestión de pacientes y de profesionales de la salud
Personal (Cap. 7)	Alta y gestión de empleados administrativos

Módulo	Cubre
Agenda de Turnos (Cap. 8)	Reserva de citas por el staff y autogestionada por el paciente, disponibilidad del profesional, confirmación y cancelación
Historia Clínica (Cap. 9)	Registro de consultas, diagnósticos y recetas ópticas
Clientes (Cap. 10)	Quién compra — puede o no ser también un paciente
Inventario y Catálogo (Cap. 11)	Productos, stock por lote y sucursal, movimientos y conteos
Ventas (Cap. 12)	Venta directa y venta “a pedido”, presupuestos, cobro y comprobante fiscal
Laboratorio (Cap. 13)	Ciclo del pedido óptico con el laboratorio externo
Compras (Cap. 14)	Pedidos a proveedores, recepción de mercadería, devoluciones
Caja y Egresos (Cap. 15)	Sesiones de caja, movimientos y egresos de dinero
Roles y Permisos (Cap. 16)	Administración de roles y permisos individuales
Notificaciones (Cap. 17)	Centro de notificaciones internas y preferencias por usuario
Reportes (Cap. 18)	Reportes gerenciales y operativos exportables
Sucursales (Cap. 19)	Aislamiento y operación multi-sucursal transversal a todo el sistema

Este alcance definitivo difiere en algunos puntos del planteado originalmente en el anteproyecto; esas diferencias, con su justificación, se documentan en el Capítulo 23.

Parte I

Arquitectura de la Solución

Capítulo 3

Arquitectura del Sistema

El sistema tiene 36 controllers (234 endpoints), 71 clases de entidad más 28 enums de negocio (99 tipos en 94 archivos de `SIGA.Domain/Entities`) y 90 vistas de frontend. Este capítulo describe su arquitectura: la organización en capas, el modelo de autorización basado en permisos y el aislamiento multi-sucursal.

3.1. Visión general

SIGA es un sistema de gestión integral para una óptica: identidad y acceso, pacientes/profesionales/clientes/empleados, agenda de turnos, historia clínica, catálogo e inventario (incl. catálogo óptico de armazones/tratamientos), ventas (venta directa y “a pedido” con receta más laboratorio externo), compras a proveedores, caja, egresos, notificaciones internas y reportes operativos — todo con soporte **multi-sucursal** y control de acceso basado en permisos.

Es un monorepo de dos repositorios separados:

- **Backend** (SIGA): ASP.NET Core Microsoft, 2026a Web API (.NET 10), C#, Entity Framework Core Microsoft, 2026b, PostgreSQL PostgreSQL Global Development Group, 2026 (hosteado en Neon).
- **Frontend** (SIGA-Web): Vue 3 Vue.js, 2026 SPA, TypeScript Microsoft, 2026c, Vite Vite, 2026 (ver el capítulo de Arquitectura del Frontend).

El modelo de datos gira en torno a la entidad **Person** (cualquier individuo físico del sistema — CI, nombre, contacto). A partir de ahí, roles funcionales (**User**, **Patient**, **Professional**, **Cliente**, **Empleado**) se cuelgan de **Person** según qué necesite hacer esa persona en el sistema. Ver el capítulo de Modelo de Datos para el detalle completo.

3.2. Diagrama de componentes

3.3. Capas del backend

3.3.1. `SIGA.Domain`

Entidades de dominio puras (**Entities**/, 71 clases más 28 enums — 99 tipos en 94 archivos; 3 archivos declaran más de un tipo: **Comprobante.cs** y **Devolucion.cs** con 2 enums más 1 clase cada uno, y **SesionCaja.cs** con 1 enum más 1 clase; todas en un único namespace plano `SIGA.Domain.Entities`) y abstracciones de seguridad (**Security/IPasswordHasher**). Sin dependencias hacia afuera — es la capa más interna.

3.3.2. `SIGA.Application`

Contratos que consume `SIGA.Api` e implementa `SIGA.Infrastructure`: interfaces de servicio (**Interfaces/IXxxService**), DTOs de request/response (DTOs/) y el wrapper **Common/Result<T>**

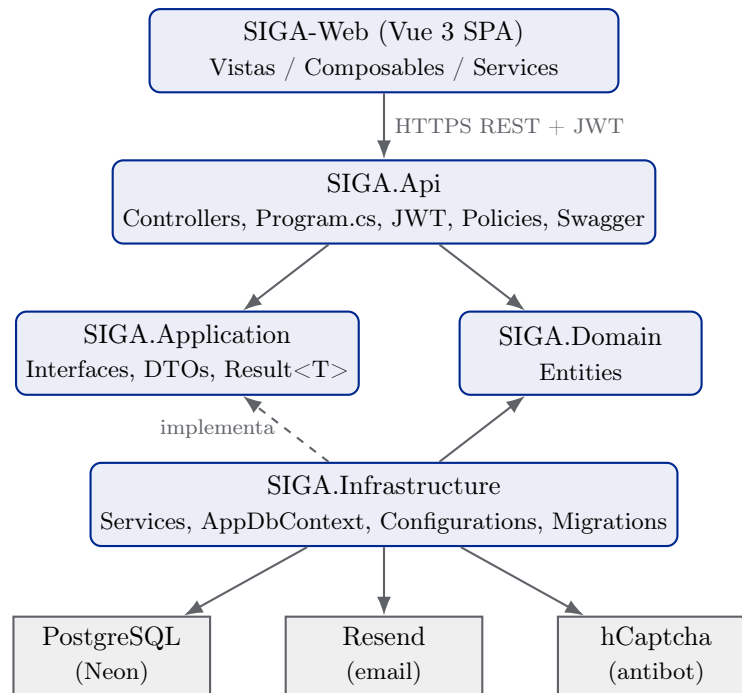


Figura 3.1: Diagrama de componentes de SIGA: el frontend consume la API vía HTTPS con JWT Bearer; SIGA.Api depende de SIGA.Application y SIGA.Domain; SIGA.Infrastructure implementa los contratos de Application (línea punteada) y depende de Domain y de los servicios externos (PostgreSQL, Resend, hCaptcha).

para modelar éxito/error sin excepciones de control de flujo. No contiene lógica de negocio ni acceso a datos.

3.3.3. SIGA.Infrastructure

La capa más grande — contiene toda la lógica de negocio y persistencia:

- **Services/** — implementación de cada `IXxxService` (48 servicios registrados, ver `DependencyInjection.AddInfrastructure`; 43 interfaces en `Application/Interfaces` — algunos servicios no exponen interface propia). Un servicio por módulo de negocio (`VentaService`, `LaboratorioService`, `EgresoService`, `TransferenciaStockService`, etc.), más 2 `BackgroundService` (`TurnoReminderService`, `StockBajoNotificadorService`, ver §3.8).
- **Persistence/** — `AppDbContext`, `Configurations/` (una clase `IEntityTypeConfiguration<T>` por entidad, Fluent API — fuente de verdad de constraints/índices/relaciones reales, más completa que las propiedades de la entidad sola) y `Migrations/` (EF Core, 88 migraciones ejecutables — cada una con su `*.Designer.cs`, más el snapshot del modelo).
- **Security/** — `Pbkdf2PasswordHasher`, `JwtTokenGenerator`, `CurrentUserContext` (implementa `ICurrentUserContext`: expone `UserId/SucursalId/EsGlobal/TienePermiso` vía `IHttpContextAccessor`, usado por los servicios transaccionales para filtrar/estampar por sucursal).
- **Options/** — POCOs de configuración (`ResendOptions`, `HCaptchaOptions`, `AppOptions`).

3.3.4. SIGA.Api

Controllers (36 archivos, uno por recurso — livianos, delegan a los servicios) y `Program.cs`: pipeline HTTP, autenticación JWT, autorización basada en permisos, Swagger y arranque de datos (ver abajo).

3.4. Pipeline HTTP y seguridad (`Program.cs`)

1. **Autenticación JWT Bearer** — `TokenValidationParameters` valida issuer / audience / lifetime / signing key (`Jwt:Issuer/Jwt:Audience/Jwt:Secret` en configuración).
2. **Autorización por permisos, no por rol** — el JWT lleva un claim "permission" por cada permiso concedido (no un claim de rol). `Program.cs` define 55 policies simples (una por permiso: `ver_pacientes`, `crear_venta`, `gestionar_laboratorio`, etc.) que exigen `RequireClaim("permission", perm)`, más 2 policies compuestas con `RequireAssertion` para OR de permisos: `cancelar_pedido` (creador `gestionar_pedidos` O aprobador `aprobar_pedidos`) y `ver_disponibles` (staff `ver_agenda` O paciente autogestionando `ver_mis_turnos`) — 57 policies en total. El seed (`DbSeeder.AllPermissions`) siembra 56 permisos (`ver_recepcion` existe en el seed pero no tiene policy declarada en `Program.cs` — asimetría real).
3. `app.UseAuthentication() → app.UseAuthorization() → app.MapControllers()`.
4. **Arranque** (dentro de un scope, antes de `app.Run()`):
 - `db.Database.MigrateAsync()` — aplica migraciones pendientes automáticamente al arrancar (no hay paso manual de `dotnet ef database update` en el flujo normal).
 - `DbSeeder.SeedAsync(db)` — siembra catálogos base (roles, permisos, sucursal "Casa Central", etc.), idempotente.
 - `DbSeeder.SeedAdminAsync(...)` — bootstrap de un usuario admin en **todos** los entornos (incl. producción), credenciales configurables por `Seed:AdminEmail/Seed:AdminPassword`.
 - `DevDataSeeder.SeedAsync(...)` — solo en `Development`, datos de prueba adicionales.

3.5. Autorización: permisos, no roles

El sistema es explícitamente **permission-based**: un `Role` es solo una colección con nombre de `Permissions` (tabla pivote `RolePermission`), y el JWT emite los permisos sueltos, no el nombre del rol. Los controllers protegen cada acción con `[Authorize(Policy = "permiso_especifico")]`. Esto permite crear roles ad-hoc desde `/roles` sin tocar código. `Role.Type` (`admin/professional/patient`, nullable) es la única noción de "rol de sistema" y se usa solo para lógica que necesita distinguir un rol inmutable (ej. bootstrap del admin), no para autorización de endpoints.

3.6. Multi-sucursal

Casi todo lo transaccional (stock, ventas, caja, timbrado, compras/recepciones, egresos, turnos/agenda, consultas clínicas) está scopeado por `SucursalId`. Quedan **globales**: catálogos (productos, marcas, categorías, servicios, etc.) y personas (`Patient`, `Cliente`, `Professional`,

User de rol admin). El scoping se resuelve centralmente en `ICurrentUserContext`, no repitiendo la lógica en cada servicio. Detalle completo en el capítulo de Modelo de Datos y en el capítulo de módulo de Sucursales ([Módulo 15 — Sucursales](#)).

El aislamiento entre sucursales se implementa de forma declarativa y centralizada: `AppDbContext` declara un *Global Query Filter* de Entity Framework Core sobre cada entidad con dimensión de sucursal (`HasQueryFilter(e => SucursalFiltro == null || e.SucursalId == SucursalFiltro)`), de modo que toda consulta a esas entidades queda filtrada por la sucursal del usuario autenticado sin que el servicio tenga que recordar hacerlo. `SucursalFiltro` es `null` para un usuario global (administrador), y en ese caso el filtro no restringe nada. `TransferenciaStock` usa un predicado dual (es visible desde la sucursal de origen y desde la de destino) y `Egreso` lo hereda en sus cinco subtipos TPH. La sucursal de escritura la resuelve `SucursalResolver`. `WriteBranchAsync`.

Dos entidades quedan deliberadamente fuera del filtro global. `User`, porque la administración de usuarios necesita verlos a todos. `Receta`, porque el flujo de venta a pedido exige poder leer las recetas de un cliente en cualquier sucursal: quien se hizo la receta en una sucursal debe poder comprar los lentes en otra. En `Receta` el guard por sucursal se aplica a mano solo donde corresponde, como en los conteos de reportes.

Conteo exacto: 18 objetos con dimensión sucursal. Quince entidades con columna `SucursalId` literal (`User`, `Venta`, `MovimientoCaja`, `MovimientoStock`, `SesionCaja`, `StockLote`, `Timbrado`, `Turno`, `ConsultaClinica`, `Receta`, `HorarioProfesional`, `Egreso`, `PedidoProveedor`, `RecepcionMercaderia`, `ConteoInventario`); una vista derivada que la arrastra (`StockActualView/vw_stock_actual`); una con la sucursal partida en dos columnas (`TransferenciaStock.SucursalOrigenId` y `SucursalDestinoId`); y una con nombre distinto (`NotificacionInterna.DestinatarioSucursalId`, para broadcast). De las quince primeras, el filtro global de EF cubre trece: quedan afuera `User` y `Receta` por las razones ya explicadas. `User.SucursalId` y `NotificacionInterna.DestinatarioSucursalId` son las únicas columnas nullable del grupo (`null` = global); en las demás la sucursal es obligatoria.

3.7. Módulos de negocio y controllers

Cuadro 3.1: Mapeo de módulos de negocio a controllers del backend.

Módulo	Controllers
Identidad y Acceso	<code>AuthController</code> , <code>UsersController</code> , <code>RolesController</code>
Personas	<code>PatientsController</code> , <code>ProfessionalsController</code> , <code>CientesController</code> , <code>EspecialidadesController</code>
Personal	<code>EmpleadosController</code>
Agenda	<code>TurnosController</code> , <code>HorariosController</code>
Clínica	<code>ConsultasController</code> , <code>RecetasController</code>
Catálogo / Inventario	<code>ProductosController</code> , <code>MarcasController</code> , <code>TipoLentes</code> <code>Controller</code> , <code>TratamientosController</code> , <code>Servicios</code> <code>Controller</code> , <code>MotivosMovimientoController</code> , <code>Stock</code> <code>LotesController</code> , <code>UbicacionesController</code>
Ventas	<code>VentasController</code> , <code>TimbradosController</code>

Módulo	Controllers
Laboratorio	LaboratorioController
Compras	ComprasController, ProveedoresController, Facturas CompraController, RecepcionesController
Caja y Egresos	CajaController, EgresosController
Configuración	ConfiguracionController, EstadosConfigController
Sucursales	SucursalesController, TransferenciasController
Notificaciones	NotificacionesController
Reportes	ReportesController

El detalle de cada endpoint (verbo, ruta, policy, request/response) está en el Apéndice de Referencia de API. Algunos catálogos menores del dominio (`CargoEmpleado`, `CategoriaGasto`, `CategoriaProducto`, `Modelo`, `Ciudad`, `Departamento`) se gestionan como sub-recursos de otro controller en vez de tener uno propio.

3.8. Trabajos en segundo plano

Dos `BackgroundService` registrados en `DependencyInjection`:

- `TurnoReminderService` — recordatorios de turnos por email.
- `StockBajoNotificadorService` — poller cada 30 minutos que compara `vw_stock_actual` contra `ProductoStockConfig` por producto más sucursal y genera `NotificacionInterna` de bajo stock (evita duplicar si ya hay una sin leer). Se eligió poller sobre hook inline porque los movimientos de stock “Aprobado” se originan desde 5 o más servicios distintos.

3.9. Generación de documentos y PDFs

QuestPDF (licencia Community, seteada en `Program.cs` antes del builder) para: receta óptica (`RecetaPdfGenerator`), movimientos de stock (`MovimientoStockPdfGenerator`) y reportes operativos (`ReporteExporter`, que también genera CSV con writer propio UTF-8 más BOM). Los exportadores reciben el DTO, no el HTTP request, para poder reutilizarse desde un futuro job de email.

3.10. Integraciones externas

- **PostgreSQL / Neon** — `Npgsql` con retry automático (`EnableRetryOnFailure`, 5 intentos). En desarrollo, `dotnet ef` corre en entorno `Production` por defecto y no toma la connection string de Neon — hay que forzar `ASPNETCORE_ENVIRONMENT=Development`.
- **Resend** — envío de email transaccional (verificación de cuenta, notificaciones externas de turnos). `HttpClient` nombrado `"resend"`.
- **hCaptcha** — protección del endpoint público de auto-registro de pacientes. `HttpClient` nombrado `"hcaptcha"`.

3.11. Convenciones

- Controllers livianos: reciben el request, llaman al servicio, devuelven `ToHttpResponse(result)`. Sin lógica de negocio ni acceso a `DbContext`.
- DTOs nunca exponen entidades de dominio directamente.
- Paginación: `page/pageSize/search/isActive` sí se usan en la práctica, con response shape `{ items, totalCount, page, pageSize, totalPages }`.
- Todos los servicios se registran en `SIGA.Infrastructure.DependencyInjection.AddInfrastructure()` — nunca directamente en `Program.cs`.

3.12. Frontend como cliente

SIGA-Web consume la API vía HTTP/JSON con el JWT en `Authorization: Bearer`. El token trae los claims `permission` (uno por permiso), `professional_id` (si aplica) y `sucursal_id` (si el usuario tiene sucursal fija) — el frontend los usa para mostrar/ocultar UI, pero la autorización real siempre se re-valida en el backend vía policies. Ver el capítulo de Arquitectura del Frontend (nota: solo aproximadamente 37 de las 56 policies del backend están mapeadas a rutas del router/menú del frontend — el resto se chequea a nivel de acción con `hasPermission` o todavía no se usa en la UI).

Capítulo 4

Arquitectura del Frontend

Este capítulo describe la arquitectura del cliente SIGA-Web: la organización de `src/`, el patrón vista-servicio y el manejo de estado.

4.1. Visión general

SIGA-Web es una SPA en **Vue 3** (Composition API con `<script setup>`), **TypeScript**, empaquetada con **Vite**, que consume la API REST de SIGA (backend) vía HTTP/JSON con JWT Bearer. El detalle del contrato de API (endpoints, DTOs, policies) está documentado del lado backend en el Apéndice de Referencia de API; este capítulo cubre solo la arquitectura del cliente.

Stack: Vue 3, TypeScript, Vite (proxy `/api/*` al backend en dev), Vue Router 5, Pinia, Axios, Tailwind CSS v4 con tema por variables CSS (claro/oscuro).

4.2. Estructura de carpetas

Todo cuelga de `src/`, un solo nivel de subcarpetas:

Carpeta	Contenido
<code>api/</code>	<code>http.ts</code> : instancia axios única + interceptores (auth, errores).
<code>assets/</code>	<code>main.css</code> : tokens del tema (colores, tipografía, radios, sombras).
<code>components/</code>	34 componentes reutilizables: <code>BaseButton</code> , <code>BaseTable</code> , <code>BaseModal</code> , <code>FilterChips</code> , <code>SearchableSelect</code> , <code>MultiSelect</code> , y componentes de dominio como <code>VentaEditor</code> , <code>TrabajoOpticoCard</code> , <code>ClienteSelector</code> .
<code>composables/</code>	11 archivos: <code>useHttp</code> (wrapper de axios), <code>useFieldStyles</code> (estilos de campo), <code>optica.ts</code> (estado derivado del editor de venta óptica) y 8 <code>useXxxPdf</code> (generación de PDFs client-side).
<code>config/</code>	<code>menuConfig.ts</code> : árbol de navegación del sidebar.
<code>router/</code>	<code>index.ts</code> : único archivo, todas las rutas + guard global.
<code>services/</code>	25 archivos, uno por dominio de negocio: <code>ventasService</code> , <code>inventarioService</code> , <code>clinicaService</code> , <code>laboratorioService</code> , etc.
<code>stores/</code>	Pinia: <code>auth.ts</code> , <code>sidebar.ts</code> , <code>theme.ts</code> .
<code>utils/</code>	Funciones utilitarias compartidas.
<code>views/</code>	90 vistas, directorio único flat (sin subcarpetas por módulo), convención <code><Modulo><Accion>View.vue</code> .

4.3. Patrón vista, composable y servicio

No es una cadena rígida de tres capas — depende de la complejidad de la vista:

- **Caso general (la mayoría de las vistas CRUD):** la vista llama **directo al servicio** correspondiente (ej. `ProductosView.vue` → `inventarioService.getProductos()`). El servicio es un módulo con tipos TS (interfaces que reflejan los DTOs del backend) más funciones delgadas que envuelven `useHttp()`. No hay lógica de negocio en el servicio, solo el contrato HTTP tipado.
- **Estado local de formulario:** vive en la vista misma (`ref/reactive` en `<script setup>`), no en un composable ni en un store.
- **Caso con lógica de estado derivado compleja:** un composable dedicado. El ejemplo real es `composables/optica.ts`, que centraliza el estado del editor de venta/presupuesto óptico (`VentaEditor.vue` más `TrabajoOpticoCard.vue`) — deriva líneas de venta y el bloque `TrabajoPedido` a partir de la selección de armazón/cristal/tratamientos.
- **Generación de PDF:** cada `useXxxPdf.ts` es un composable de un solo propósito (arma el documento a partir del DTO ya cargado), no hace fetch.
- `useHttp()` envuelve la instancia central de `api/http.ts` y devuelve `.data` directamente. Lo usan 5 de los 25 servicios (`egresosService`, `laboratorioService`, `empleadosService`, `reportesService`, `ventasService`); los otros 20 importan `httpdirecto` de `@api/http`. Ambos estilos comparten el mismo interceptor.

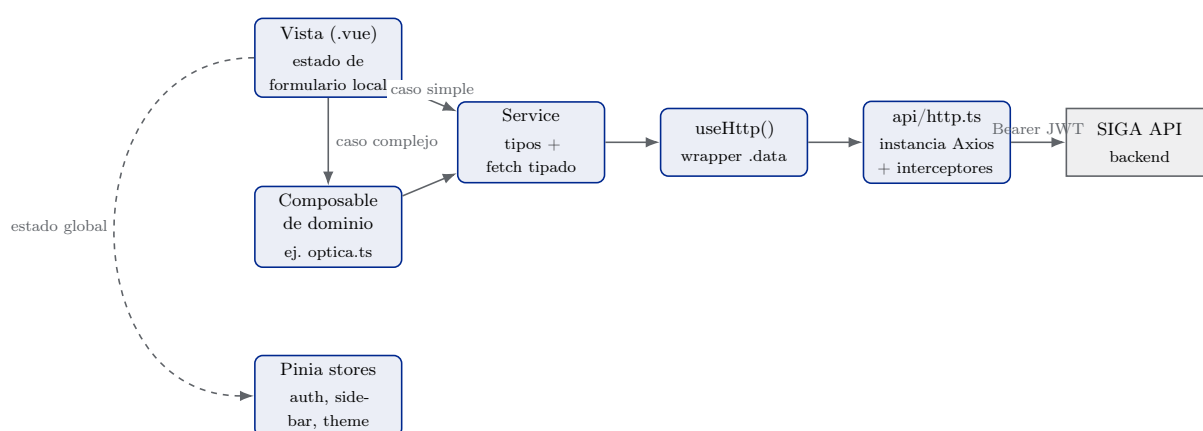


Figura 4.1: Flujo de datos en el frontend: de la vista al backend, con las dos rutas posibles (servicio directo o vía composable de dominio) y el acceso a estado global de Pinia.

4.4. Manejo de estado

Pinia, 3 stores con uso real:

- `auth.ts` — `token`, `user` (`AuthUser`: email, nombre, permisos, roles, `sucursalId/sucursalNombre`, `mustChangePassword`), persistidos en `localStorage` (`siga_token/siga_user`) para sobrevivir recargas/pestañas. Expone `hasPermission()`, usado tanto por el guard del router como por la UI para mostrar/ocultar acciones.
- `sidebar.ts` — estado de colapso/expansión del sidebar (`mobile/desktop`).

- `theme.ts` — modo claro/oscuro, aplica la clase `.dark` a `<html>`.

Todo lo demás (datos de listados, estado de modales, formularios) es estado **local de componente**, no global — consistente con que cada vista hace su propio fetch al entrar.

4.5. Router

Un único archivo `router/index.ts` con aproximadamente 90 rutas planas (no anidadas), cada una con `meta: { requiresAuth?, requiresGuest?, permission?, label? }`. La mayoría de los componentes se cargan con `import()` dinámico (code-splitting por ruta); solo `DashboardView`, `LoginView`, `RegisterView`, `PacientesView`, `PacienteDetailView` y `UsuariosView` se importan estáticos.

Guard global (`router.beforeEach`), en este orden:

1. `requiresAuth` sin sesión → redirige a `/login`.
2. Autenticado con `user.mustChangePassword = true` y no está ya en `/cambiar-contrasena` → fuerza redirección ahí (confirma en código lo que documenta el [ADR 0012](#): el guard cubre navegación directa y refresh, no solo el login inicial).
3. `requiresGuest` (login/registro) estando ya autenticado → redirige a la primera ruta accesible según permisos (`firstAccessibleRoute`, de `config/menuConfig.ts`).
4. Ruta con `meta.permission` que el usuario no tiene → redirige al primer fallback accesible; si no hay ninguno o el fallback es la misma ruta que falló, deja pasar (evita loop infinito; la vista debe manejar el estado vacío).

La autorización real siempre se revalida en el backend (policies) — este guard es solo UX, no seguridad.

Cobertura de permisos: el backend declara 57 policies (55 simples más 2 compuestas, ver §3.4), pero el router solo mapea **37 permisos distintos** a `meta.permission` (contado exacto sobre las aproximadamente 90 rutas). El resto se chequea a nivel de acción con `auth.hasPermission(...)` dentro de la vista (botones, ítems de `RowContextMenu`), o directamente no tiene UI todavía.

Paginación: dos estrategias conviven sin unificar: **server-side** en 8 servicios que devuelven `PagedResult<T>` (`clienteService`, `clinicaService`, `comprasService`, `egresosService`, `inventarioService`, `notificacionService`, `patientService`, `ventasService`) y **client-side** en aproximadamente 18 vistas que hacen slicing local con una constante `PAGE_SIZE` sobre la lista completa ya traída.

4.6. Sidebar y navegación (`config/menuConfig.ts`)

Árbol de navegación separado del router: cada nodo (`MenuItem/MenuChild`, hasta 3 niveles de anidamiento) tiene `permission` propio y opcionalmente `zone` (agrupa secciones bajo un título: *General, Atención, Óptica, Comercial, Gestión*). `AppSidebar.vue` renderiza este árbol filtrando por `auth.hasPermission()`.

4.7. Responsive (mobile/tablet)

Sidebar como drawer superpuesto en mobile/tablet (no reflow), controlado por la variable CSS `-sidebar-width` que el layout de `<main>` usa para su `margin-left` — en breakpoints angostos

se fuerza a 0 y el sidebar pasa a overlay. Tablas anchas usan scroll horizontal contenido, no reflow de columnas.

4.8. Sistema de diseño

Fuente de verdad visual: `design-system.md` (tokens de color, radius, sombras, motion, tipografía, layout, `useFieldStyles()`, badges, avatares, componentes base, anti-patrones).

Convenciones transversales: `rounded-xl` para modales; patrón de activar/desactivar entidades solo desde menú contextual (`RowContextMenu`), nunca desde el modal de edición; layout de página autenticada con padding `p-8` fijo, sin `max-w-*`.

Parte II

Construcción del Software

Capítulo 5

Módulo: Identidad y Acceso

5.1. Propósito

Gestiona quién puede entrar a SIGA y con qué credenciales: autenticación (login JWT), ciclo de vida de la cuenta (`User`) y los flujos de contraseña (self-service, reset por admin, cambio obligatorio en primer login, recuperación por email). Es la base sobre la que se apoyan todos los demás módulos — sin `User` autenticado no hay acceso a ningún endpoint protegido. El ABM del catálogo de `Role/Permission` en sí (qué permisos tiene cada rol) se documenta aparte en el [Módulo de Roles y Permisos](#); aquí el foco es la cuenta y su acceso.

5.2. Entidades principales

Ver Capítulo 20, Figura 20.1.

Cuadro 5.1: Entidades principales del módulo de Identidad y Acceso.

Entidad	Rol
<code>Person</code>	Datos personales de cualquier individuo del sistema — ver Capítulo 20.
<code>User</code>	Cuenta de acceso, 1:1 con <code>Person</code> . <code>SucursalId</code> nullable, <code>PasswordHash</code> , <code>IsActive</code> , <code>IsEmailVerified</code> , <code>MustChangePassword</code> , <code>PasswordResetToken</code> .
<code>Role / UserRole</code>	Roles asignados al usuario (many-to-many) — detalle en el Módulo de Roles y Permisos .

5.3. Reglas de negocio clave

- **Login es email más contraseña, resuelto vía `Person.Email`** (join `User`→`Person`), no hay username separado.
- **JWT lleva permisos, no roles** — el token emite un claim "permission" por cada permiso resuelto de los roles del usuario, más `professional_id` (si aplica) y `sucursal_id` (si el usuario tiene sucursal fija). Ver [ADR 0003](#) y Capítulo 3, §3.5.
- **Auto-registro de pacientes es el único flujo de alta pública** (POST `/api/auth/register/patient`): protegido con hCaptcha, crea `Person+User+Patient` con `IsEmailVerified=false` y exige verificar el email (GET `/api/auth/verify-email?token=`) antes de poder loguear. El alta de `Professional/Empleado` (por un admin) **no** pasa por este flujo.

- **IsEmailVerified=true bloquea el login si es false** — pero solo aplica de verdad al auto-registro público. `ProfessionalService.CreateAsync/EmpleadoService.CrearAsync` se-
tean `IsEmailVerified=true` directamente al crear la cuenta, porque son altas “vouched-for”
por un admin y no necesitan verificar email.
- **MustChangePassword solo aplica a cuentas nuevas, no retroactivo.** Se setea `true` al
crear un `Professional/Empleado`; no bloquea el login (a diferencia de `IsEmailVerified`) —
el frontend redirige *después* de autenticar, vía guard de router que cubre navegación directa
y refresh, no solo el submit del login. Ver [ADR 0012](#).
- **Tres formas de cambiar contraseña, cada una con su propio permiso/-
regla:** self-service (`POST /api/auth/change-password`, pide la actual), admin-
reset (`PUT /api/users/{id}/reset-password`, policy `editar_usuario`, no pide
la actual, fuerza `MustChangePassword=true`), y recuperación por email sin sesión
(`forgot-password/reset-password`, `AllowAnonymous`, con token).
- **Borrado lógico** vía `User.IsActive=false`, no hay borrado físico de cuentas.

5.4. Endpoints

Detalle completo (policies, DTOs) en el Apéndice de Referencia de API, §[A.1](#).

Cuadro 5.2: Endpoints principales del módulo de Identidad y Acceso.

Método	Ruta	Descripción
POST	<code>/api/auth/register/patient</code>	Auto-registro público de paciente (hCaptcha más verificación de email)
GET	<code>/api/auth/verify-email</code>	Confirma el email desde el link envia- do
POST	<code>/api/auth/login</code>	Login, devuelve JWT más claims
POST	<code>/api/auth/change-password</code>	Self-service, requiere contraseña ac- tual
POST	<code>/api/auth/forgot-password / reset-password</code>	Recuperación sin sesión, por token de email
GET / DELETE	<code>/api/users</code>	Listado y desactivación de cuentas
PUT	<code>/api/users/{id}/ reset-password</code>	Reset por admin, sin pedir la actual

5.5. Flujo típico

5.5.1. Auto-registro de paciente y verificación de email

5.5.2. Cambio de contraseña obligatorio (cuenta nueva de staff)

5.6. Vistas de frontend

- `LoginView.vue`, `RegisterView.vue`, `VerifyEmailView.vue`

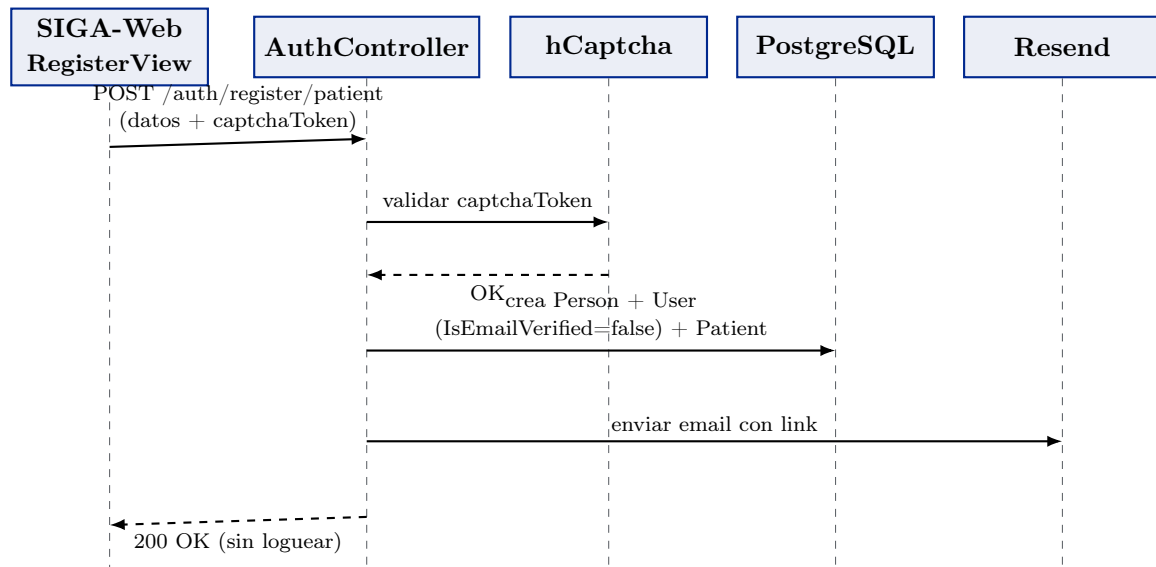


Figura 5.1: Auto-registro público de paciente: alta con `IsEmailVerified=false` y envío del email de verificación.

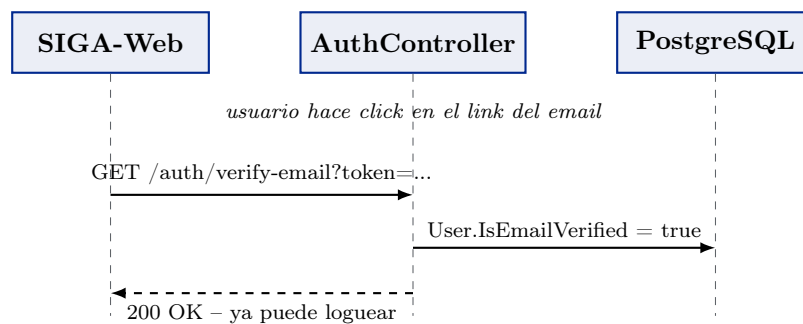


Figura 5.2: Verificación de email: continuación del flujo de auto-registro tras el click en el enlace enviado por Resend.

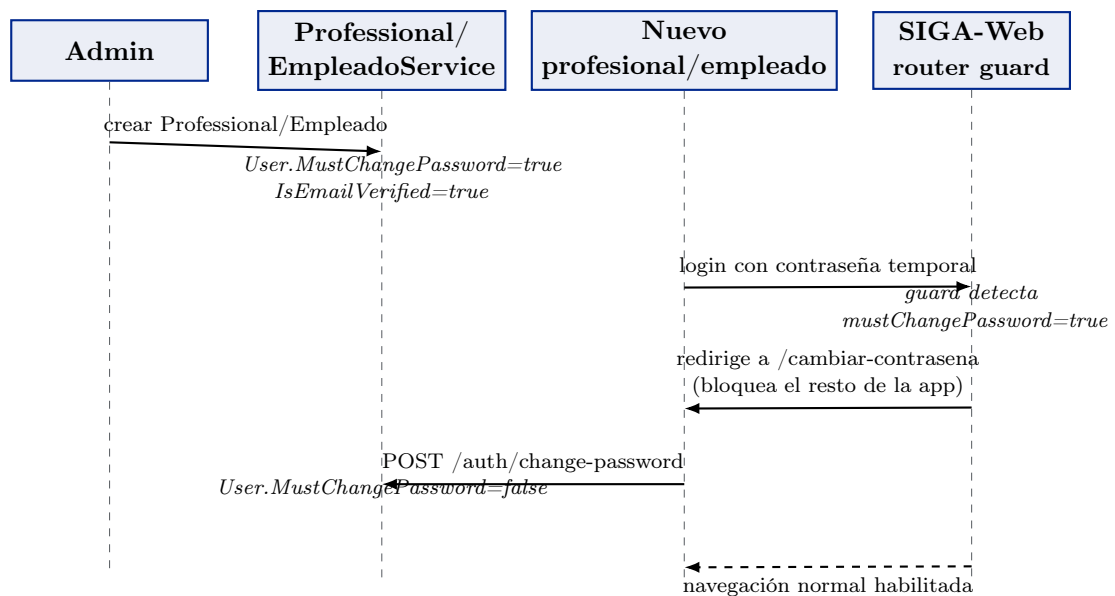


Figura 5.3: Cambio de contraseña obligatorio: el guard del router intercepta cualquier navegación mientras `mustChangePassword` sea verdadero.

- `OlvideContraseñaView.vue`, `RestablecerContraseñaView.vue` (recuperación por email)
- `CambiarContraseñaObligatoriaView.vue` (pantalla aislada, forzada por router guard)
- `UsuariosView.vue` (listado, roles por usuario, activar/desactivar, “Restablecer contraseña” en el modal de gestión)
- Modal “Cambiar contraseña” en `AppHeader.vue` (self-service, no es una vista propia)

5.7. Interfaz

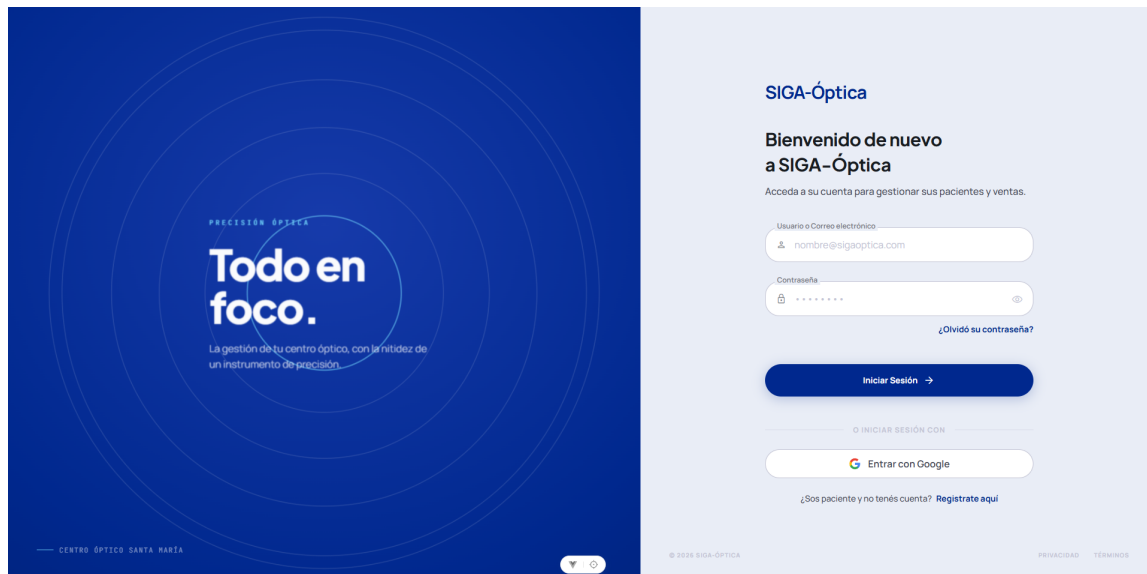


Figura 5.4: Pantalla de inicio de sesión de SIGA-Óptica, con acceso alternativo vía Google y enlace de recuperación de contraseña.

5.8. Estado

[Implementado], incluida la recuperación de contraseña por email (`forgot-password/reset-password`).

Capítulo 6

Módulo: Pacientes y Profesionales

6.1. Propósito

Administra las dos bases de personas que participan directamente de la atención clínica: los pacientes de la óptica y los profesionales de salud (optómetras/oftalmólogos) que los atienden. Ambos parten de **Person**, pero con reglas de acceso opuestas: el paciente puede existir sin cuenta, el profesional siempre la necesita.

6.2. Entidades principales

Ver Capítulo 20 para el diagrama completo.

Cuadro 6.1: Entidades principales del módulo de Pacientes y Profesionales.

Entidad	Rol
Patient	Marca que una Person es paciente. UserId nullable — puede existir sin acceso al sistema.
Professional	Marca que una Person es profesional de salud. UserId obligatorio (1:1) .
Especialidad / Profesional Especialidad	Catálogo de especialidades, many-to-many con Professional .
Horario Profesional / PausaHorario / BloqueoFecha	Disponibilidad del profesional — consumida por el módulo Agenda, se gestiona desde aquí (formulario de Profesional).

6.3. Reglas de negocio clave

- **Paciente sin cuenta es el caso normal, no la excepción:** recepción da de alta un paciente con solo CI/nombre/contacto (**Patient.UserId = null**). Solo si el paciente necesita el portal de autogestión (ver sus turnos/recetas/historial) se crea un **User** vinculado — vía el flujo de auto-registro público, no desde el ABM interno de pacientes. Ver Capítulo 5.
- **Profesional siempre requiere User** — no existe un “profesional sin login”. El alta de un profesional crea **Person+User+Professional** en una sola operación transaccional, con **IsEmailVerified=true** e **IsUserMustChangePassword=true** (cuenta vouched-for por el admin, no auto-registro).

- **La sucursal del profesional se asigna en este formulario, no en UsuariosView** — es una decisión explícita del rediseño multi-sucursal (2026-06-28): `User.SucursalId` se sigue guardando en `User`, pero la UX de asignación vive en `ProfesionalesView/EmpleadosView`, no en la pantalla de usuarios (que solo gestiona roles y activar/desactivar).
- **Un profesional puede tener más de una especialidad** (many-to-many vía `ProfesionalEspecialidad`), a diferencia de un diseño más simple de “una especialidad por profesional”.
- **Disponibilidad del profesional es multi-capa:** `HorarioProfesional` (horario semanal recurrente, por sucursal — índice único `ProfessionalId+SucursalId+DiaSemana`), `PausaHorario` (recesos dentro de un horario, ej. almuerzo) y `BloqueoFecha` (excepciones puntuales, ej. vacaciones). Los tres alimentan el cálculo de slots disponibles del módulo Agenda, pero se gestionan desde el form de Profesional.
- **Borrado lógico** (`Patient.IsActive` / desactivación de `User` para `Professional`) — sin borrado físico.

6.4. Endpoints

Detalle completo en el Apéndice de Referencia de API (`PatientsController`, `ProfessionalsController`, `EspecialidadesController`, `HorariosController` anidado bajo `/api/professionals/{id}`).

Cuadro 6.2: Endpoints principales del módulo de Pacientes y Profesionales.

Método / Ruta	Descripción
GET/POST/PUT/DELETE/ <code>api/patients</code>	CRUD más desactivación de pacientes.
GET/POST/PUT/DELETE/ <code>api/professionals</code>	CRUD más desactivación de profesionales.
GET/POST/PUT/DELETE/ <code>api/especialidades</code>	Catálogo de especialidades.
GET/PUT <code>/api/professionals/{id}/horarios</code>	Horario semanal (reemplaza el set completo por sucursal).
GET/POST/DELETE <code>/api/professionals/{id}/bloqueos</code>	Fechas bloqueadas.

6.5. Flujo típico

6.5.1. Alta de profesional (transacción multi-entidad)

6.5.2. Alta de paciente sin cuenta (caso normal)

Recepción completa un formulario simple (CI/nombre/contacto) → `POST /api/patients` crea `Person+Patient` sin `User`. El paciente queda disponible de inmediato para agendar turnos, tener historia clínica y figurar en ventas — no hace falta que tenga acceso al portal.

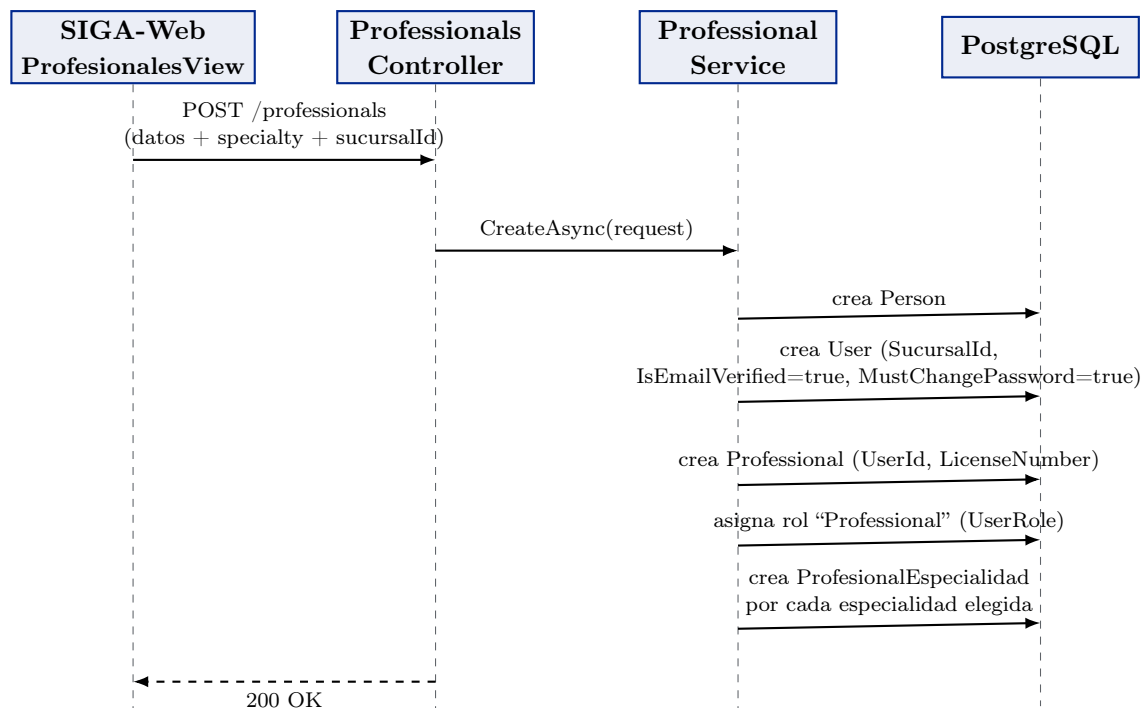


Figura 6.1: Alta de profesional: transacción única que crea `Person`, `User`, `Professional`, asigna el rol y las especialidades.

6.6. Vistas de frontend

- `PacientesView.vue`, `PacienteNuevoView.vue`, `PacienteDetailView.vue`
- `ProfesionalesView.vue` (incluye asignación de sucursal, horarios y especialidades)
- `EspecialidadesView.vue`
- Portal del paciente (autogestión, requiere `Patient.UserId` no nulo): `MiHistorialView.vue`, `MisRecetasView.vue` — documentadas en detalle en el [Módulo de Clínica](#), ya que son vistas de historial clínico, no de datos personales.

6.7. Interfaz

6.8. Estado

[Implementado]. Sin limitaciones conocidas relevantes para la tesis más allá de las ya anotadas en el Apéndice de Referencia de API (ej. `DELETE /api/professionals/{id}` usa la policy `editar_profesional` en vez de una policy de desactivación dedicada, a diferencia de `Patient/Cliente`).

SIGA Óptica PRECISIÓN CLÍNICA

Todas las sucursales

Gestión de Pacientes

Administre la base de datos de pacientes y su historial clínico con precisión.

Estado: ▼

Buscar por nombre, C.I., c:

[Añadir Paciente](#)

NOMBRE DEL PACIENTE	C.I.	FECHA DE REGISTRO	ESTADO
PP PruebaSideBar Paciente prueba.sidebar.paciente@test.local	90000051	03 de jul de 2026	INACTIVO
JG Juana Garcia 097175616	1234684516	26 de jun de 2026	ACTIVO
MG Matias Gaona matias.gaona23@hotmail.com	4553980	26 de jun de 2026	ACTIVO
MM Matias Melgarejo matmelga02@gmail.com	5414885	10 de may de 2026	ACTIVO
GR Gabriela Rios gabriela.rios031@gmail.com	1000032	10 de may de 2026	ACTIVO
SP Sebastián Pinto sebastian.pinto026@gmail.com	1000027	10 de may de 2026	ACTIVO
RG Ricardo Gómez ricardogomez023@gmail.com	1000024	10 de may de 2026	ACTIVO
AS Amanda Salinas amanda.salinas027@gmail.com	1000028	10 de may de 2026	ACTIVO
PF Paola Fernández			

Figura 6.2: Listado de pacientes registrados en el sistema.

Capítulo 7

Módulo: Personal

7.1. Propósito

Gestiona al personal administrativo/operativo de la óptica (recepción, cajeros, vendedores, etc.) que necesita acceso al sistema pero no es un profesional de salud — es la contraparte de RR.HH. de `Professional`. Alimenta al módulo de Egresos (`SalarioEmpleado`) para el registro de nóminas.

7.2. Entidades principales

Ver Capítulo 20, Grupo A.

Cuadro 7.1: Entidades principales del módulo de Personal.

Entidad	Rol
Empleado	Marca que un <code>User</code> es empleado. <code>CargoId</code> FK, <code>FechaIngreso/FechaEgreso</code> , <code>SalarioBase</code> nullable.
CargoEmpleado	Catálogo de cargos/puestos (ej. Recepcionista, Cajero).

7.3. Reglas de negocio clave

- **Estructura idéntica a `Professional`:** `Empleado` siempre requiere `User` (1:1), se crea con `IsEmailVerified=true` y `MustChangePassword=true` en la misma transacción (`Person+User+Empleado+asignación de rol`) — mismo patrón que el alta de profesional (Capítulo 6).
- `CargoEmpleado` es un catálogo simple sin controller propio — se gestiona como sub-recurso de `EmpleadosController` (`/api/empleados/cargos`), no tiene su ruta base independiente.
- La sucursal del empleado se asigna en este mismo formulario (`EmpleadosView`), igual que para profesionales — no desde `UsuariosView`.
- `SalarioBase` es nullable y opcional — no todo empleado tiene un salario fijo cargado aquí; cuando existe, es la base que consume el módulo de Egresos para generar `SalarioEmpleado` (un `Egreso` de tipo salario). Ver Capítulo 15.
- `FechaEgreso` nullable — permite registrar la baja de un empleado sin necesariamente desactivar el `User` en el mismo momento (son dos campos independientes: la fecha de egreso es dato de RR.HH., `IsActive` del `User` es control de acceso).

- **Borrado lógico** vía desactivación del **User** asociado, igual que el resto de las cuentas de staff.

7.4. Endpoints

Detalle completo en el Apéndice de Referencia de API, §3 (**EmpleadosController**, que usa su propio **ToResponse** en vez de **BaseController** — única excepción en los 36 controllers).

Cuadro 7.2: Endpoints principales del módulo de Personal.

Método	Ruta	Descripción
GET	/api/ empleados?soloActivos	Listado
POST/PUT/DELETE	/api/empleados	Alta (transacción multi-entidad), edición, desactivación
GET/POST/PUT	/api/empleados/cargos	Catálogo de cargos, anidado bajo este mismo controller

7.5. Flujo típico

Idéntico en estructura al alta de profesional (ver el diagrama de secuencia del Capítulo 6), reemplazando **Professional/LicenseNumber/ProfesionalEspecialidad** por **Empleado/CargoId/FechaIngreso**. No se repite el diagrama aquí para no duplicar contenido — la única diferencia estructural es que no hay equivalente a “especialidades” (many-to-many), la relación con **CargoEmpleado** es N:1 simple.

7.6. Vistas de frontend

- **EmpleadosView.vue**
- **CargosEmpleadoView.vue**

7.7. Interfaz

7.8. Estado

[Implementado]. Sin limitaciones conocidas específicas de este módulo más allá de las ya generales de gestión de cuentas (Capítulo 5, §5.8).

SIGA Óptica PRECISIÓN CLÍNICA

SIGA Óptica Todas las sucursales

Admin SIGA ADMINISTRADOR

Gestión de Empleados

Administre el personal no médico: vendedores, recepcionistas y cajeros.

Estado

Añadir Empleado

Buscar: Juan

EMPLEADO	C.I.	CARGO	INGRESO	ESTADO
Juan Pérez empleado@optica.com	4345788	Vendedor	30 de mayo de 2026	ACTIVO

Mostrando 1 de 7 empleados

3
EMPLEADOS ACTIVOS

7
TOTAL REGISTROS

Figura 7.1: Gestión de empleados administrativos, con su cargo, fecha de ingreso y estado.

Capítulo 8

Agenda y Turnos

8.1. Propósito

Gestiona el calendario de turnos entre pacientes y profesionales: reserva por parte de recepción/staff, auto-reserva del propio paciente desde el portal (self-booking), disponibilidad del profesional (horarios semanales, pausas, bloqueos puntuales) y el ciclo de confirmación/cancelación de un turno. Es la puerta de entrada habitual a una **ConsultaClinica**.

8.2. Entidades principales

Cuadro 8.1: Entidades principales del módulo de Agenda.

Entidad	Rol
Turno	El turno en sí: paciente, profesional, sucursal, fecha/hora, estado. Ver Capítulo 20 (Grupo B).
Horario Profesional	Disponibilidad semanal recurrente de un profesional, por sucursal. Ver Capítulo 20 (Grupo A).
PausaHorario	Ventana de pausa dentro de un HorarioProfesional (ej. almuerzo). Ver Capítulo 20 (Grupo A).
BloqueoFecha	Excepción puntual a la disponibilidad (ej. vacaciones, feriado). Ver Capítulo 20 (Grupo A).
EstadoConfig	Estado cosmético configurable (color/nombre) para Turno . EstadoCustomId , no gobierna la lógica. Ver Capítulo 20 (Grupo B).

8.3. Reglas de negocio clave

- **Turno.Estado** (enum **TurnoEstado**) es la fuente de verdad de la lógica; **EstadoCustomId** (FK a **EstadoConfig**) es puramente cosmético/configurable por el negocio (color, nombre visible) y no la reemplaza.
- **Un paciente solo puede tener un turno activo a la vez.** “Activo” = **Pendiente**, **Confirmado** o **Presente**. Un turno **Completado** no cuenta como activo (corrección explícita: antes bloqueaba con cualquier estado no cancelado, incluido **Completado**, dejando al paciente sin poder reservar nunca más tras su primera consulta).
- **Self-booking exige SucursalId obligatorio** en **SelfBookTurnoRequest** — impacto directo del proyecto multi-sucursal, ver [ADR 0009 — Sucursal fija por usuario](#). El paciente,

al ser global (sin sucursal propia), elige la sucursal en el momento de reservar.

- **HorarioProfesional está scopeado por sucursal**, con índice único `ProfessionalId + SucursalId + DiaSemana` — un mismo profesional puede tener horarios distintos en cada sucursal donde atiende.
- **Policy ver_disponibles es un OR** (`ver_agenda` del staff o `ver_mis_turnos` del paciente) — permite que ambos perfiles consulten slots libres sin exponerle al paciente el resto de la agenda.
- **Confirmación por token de email:** `POST /api/turnos/confirmar/{token}` es público (`AllowAnonymous`) — el paciente confirma el turno desde el link del email sin necesidad de estar logueado.
- **Cancelación en dos pasos:** el paciente solo puede *solicitar* la cancelación (`solicitar-cancelacion`, marca `Turno.SolicitudCancelacion = true`); el staff es quien la resuelve (`gestionar-cancelacion`, aprueba o rechaza). El paciente nunca cancela directamente su propio turno.
- **Recordatorios automáticos:** `TurnoReminderService` (`BackgroundService`, ver Capítulo 3, §3.8) envía recordatorios por email antes del turno.

8.4. Endpoints

Cuadro 8.2: Endpoints principales del módulo de Agenda.

Método	Ruta	Descripción
GET	<code>/api/turnos</code>	Listado filtrado (staff)
GET	<code>/api/turnos/disponibles</code>	Slots libres (staff u paciente, policy OR)
GET	<code>/api/turnos/profesionales-disponibles</code>	Profesionales con horario activo ese día (paso 1 del self-booking)
POST	<code>/api/turnos</code>	Alta manual (staff)
PUT	<code>/api/turnos/{id}/estado</code>	Cambio de estado
GET/POST	<code>/api/turnos/mis-turnos, /api/turnos/self-book</code>	Portal del paciente
POST	<code>/api/turnos/{id}/solicitar-cancelacion, /gestionar-cancelacion</code>	Cancelación en dos pasos
POST	<code>/api/turnos/confirmar/{token}</code>	Confirmación pública por email
GET/PUT	<code>/api/professionals/{id}/horarios, /bloqueos</code>	Disponibilidad del profesional (<code>HorariosController</code>)

Detalle completo (policies, DTOs) en el Apéndice de Referencia de API, § 4 Agenda.

8.5. Flujo típico

Self-booking desde el portal del paciente, dividido en dos figuras por cantidad de pasos.

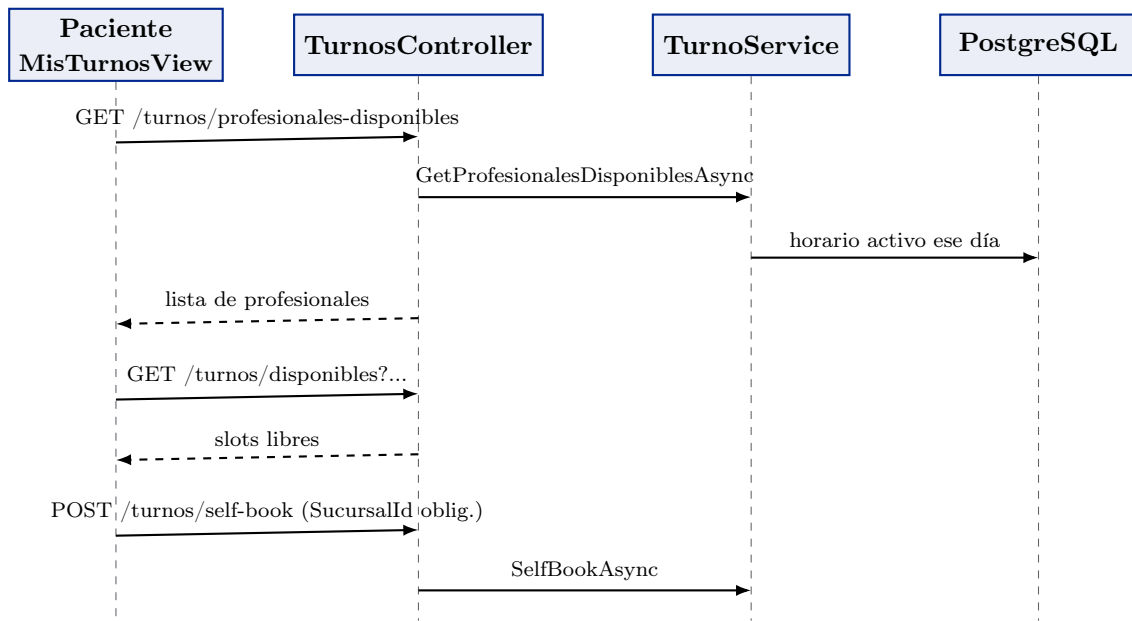


Figura 8.1: Self-booking: búsqueda de disponibilidad y reserva (parte 1 de 2).

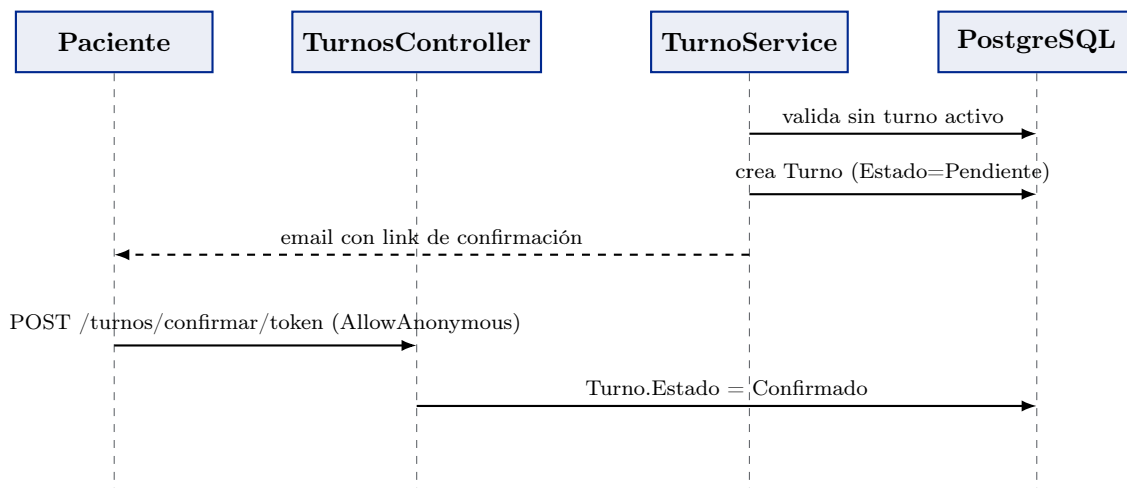


Figura 8.2: Self-booking: creación del turno y confirmación por email (parte 2 de 2).

Cancelación: el paciente solicita (**solicitar-cancelacion**) → el staff ve la bandera **Solicitud Cancelacion** en **AgendaView** → aprueba o rechaza (**gestionar-cancelacion**).

8.6. Vistas de frontend

- **AgendaView.vue** — agenda del staff.
- **MisTurnosView.vue** — portal del paciente: calendario mensual propio (grilla 7×6, sin librería externa), modal de reserva en dos pasos (profesional → horario), respeta la regla de “un solo turno activo”.
- **ConfirmarTurnoView.vue** — landing pública del link de confirmación por email.
- La gestión de horarios/pausas/bloqueos del profesional no tiene vista propia — vive embebida en **ProfesionalesView.vue**.

8.7. Interfaz

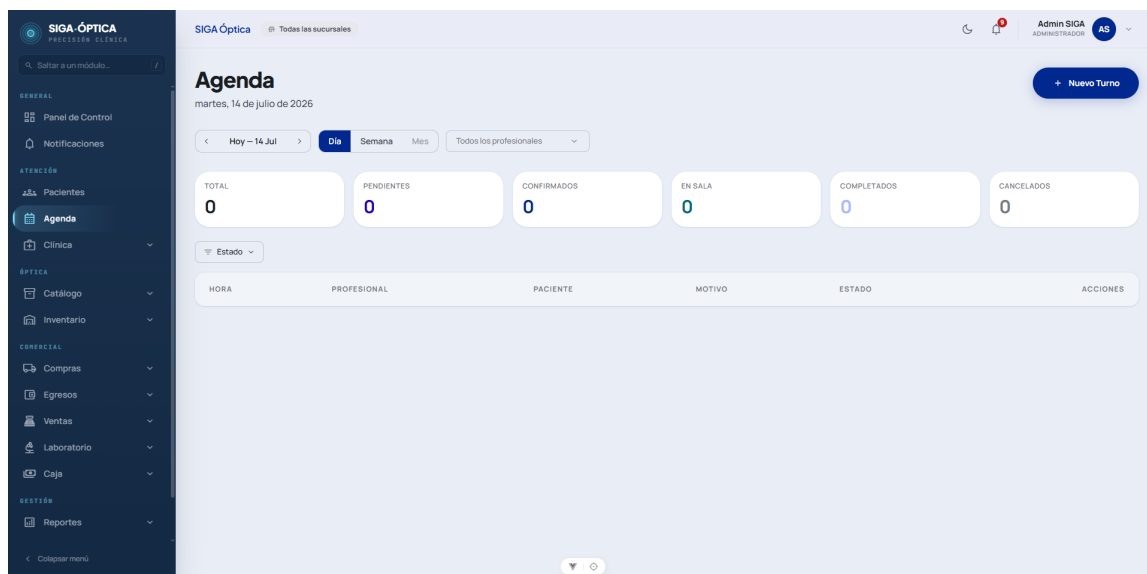


Figura 8.3: Agenda de turnos del día, con los contadores de estado (pendientes, confirmados, en sala, completados, cancelados) y el filtro por profesional.

8.8. Estado

[Implementado], incluyendo self-booking, confirmación por email y scoping multi-sucursal (Fase 4 del proyecto de sucursales).

Capítulo 9

Módulo: Clínica

9.1. Propósito

Registra la historia clínica de cada paciente (consultas) y la receta óptica resultante, con generación de PDF y acceso propio del paciente a su historial y recetas desde el portal. Una receta puede nacer de una consulta o cargarse manualmente sin consulta previa (“externa”), para cubrir el caso de un cliente que trae una receta de otra óptica.

9.2. Entidades principales

Cuadro 9.1: Entidades principales del módulo de Clínica.

Entidad	Rol
ConsultaClinica	Cabecera de la consulta: paciente, profesional, sucursal, motivo, diagnóstico. Ver Capítulo 20, Grupo B.
Receta	Graduación óptica (OD/OI: esférico, cilindro, eje, adición), 1:1 opcional con una consulta o vinculada directo a una Person (externa). Ver Capítulo 20, Grupo B.

9.3. Reglas de negocio clave

- **El profesional se autoasigna desde el JWT** (claim `professional_id`), nunca se selecciona en el formulario: al crear/editar una consulta, si quien llama es un profesional, el backend sobrescribe `ProfessionalId` con el propio, ignorando lo que venga en el request. El listado (GET `/api/consultas`) también se auto-filtra a las consultas del profesional autenticado.
- **Receta clínica o externa** — ver [ADR 0008](#): una `Receta` puede colgar de una `ConsultaClinica` (1:1, Cascade) o cargarse manual para una `Person` sin consulta (`PersonId`, sin `ConsultaClinicaId`). El flujo de venta a pedido consume ambas por igual vía GET `/api/recetas?clienteId=`.
- **El estado de una consulta NO es un enum C#**, a diferencia de `Turno`. `ConsultaClinica` no tiene columna `Estado`: su único campo de estado es `EstadoConfigId` (FK a `EstadoConfig`, la misma tabla de configuración que en `Turno` es puramente cosmética — ver Capítulo 8). Para `Consulta`, en cambio, `EstadoConfig` **sí es funcional**: `ConsultaClinicaService.CreateAsync` busca la fila con `Entidad == "Consulta"` && `CodigoInterno == "Abierta"` para asignar el estado inicial. Los 4 estados reales

(verificados en `DbSeeder.EstadosIniciales`) son **Pendiente**, **Abierta**, **Cerrada**, **Cancelada**.

- **PDF de receta** (QuestPDF, A5, 1.5cm de margen): `GET /api/consultas/{id}/receta/pdf` devuelve 404 si la consulta no tiene receta cargada. El paciente tiene su propio endpoint equivalente (`/mi-receta/pdf`) que valida que la consulta le pertenezca antes de generar el archivo.
- **Portal del paciente de solo lectura**: `mis-consultas` y `mi-receta/pdf` resuelven el `patientId` a partir del `userId` del JWT (el JWT no trae `patient_id` directo), replicando el mismo patrón de resolución que usa `TurnoService`.

9.4. Endpoints

Cuadro 9.2: Endpoints principales del módulo de Clínica.

Método	Ruta	Descripción
GET	<code>/api/consultas</code>	Listado paginado (auto-filtrado si el caller es profesional)
GET	<code>/api/consultas/{id} · /patient/{patientId}</code>	Detalle / por paciente
GET	<code>/api/consultas/profesional/stats</code>	Estadísticas del profesional autenticado
POST/PUT/DELETE	<code>/api/consultas</code>	Alta, edición, baja
PATCH	<code>/api/consultas/{id}/estado</code>	Cambio de estado
POST	<code>/api/consultas/{id}/receta</code>	Upsert de la receta 1:1
GET	<code>/api/consultas/{id}/receta/pdf</code>	PDF de receta (staff)
GET	<code>/api/consultas/mis-consultas · /{id}/mi-receta/pdf</code>	Portal del paciente
GET	<code>/api/recetas?clienteId=</code>	Recetas del cliente (clínicas + manuales), consumido por Ventas
POST	<code>/api/recetas</code>	Alta de receta manual/externa

Detalle completo (policies, DTOs) en el Apéndice de Referencia de API, §5. Clínica.

9.5. Flujo típico

Consumo posterior desde Ventas: al armar una venta “a pedido” para un cliente, `RecetaSelector` llama a `GET /api/recetas?clienteId=` y ofrece elegir una receta existente (clínica o externa) o cargar una nueva manual — ver Capítulo 12.

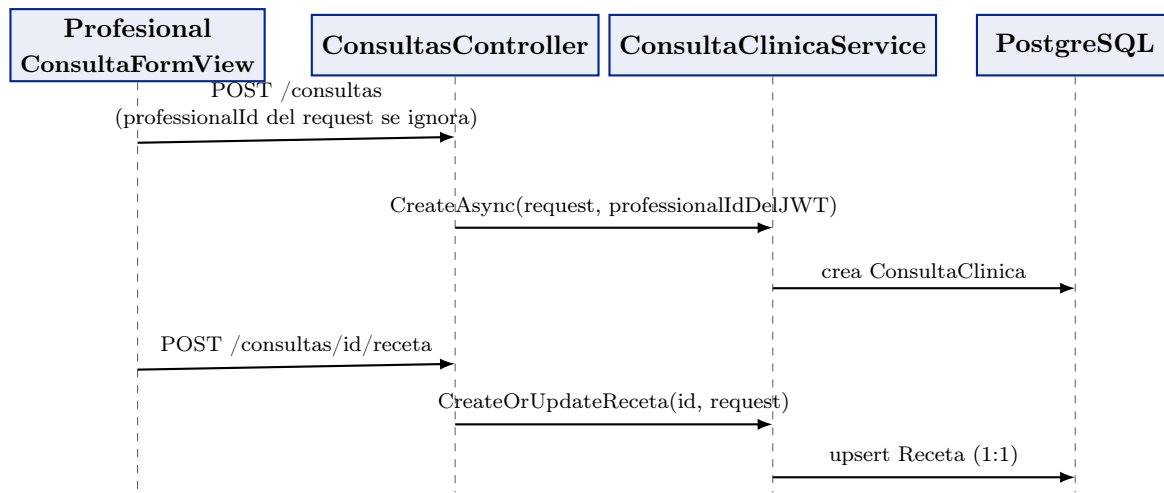


Figura 9.1: Alta de consulta y receta: el paciente ya puede ver la consulta y descargar el PDF desde el portal apenas se completa este flujo.

9.6. Vistas de frontend

- `ConsultaListView.vue`, `ConsultaFormView.vue`, `ConsultaEditView.vue` — ABM de consultas (staff/profesional).
- `HistorialClinicoView.vue` — historial completo con filtros (búsqueda más profesional), carga diferida.
- `PacienteDetailView.vue` — tab clínico con las consultas del paciente.
- `MiHistorialView.vue`, `MisRecetasView.vue` — portal del paciente, solo lectura más descarga de PDF.

9.7. Interfaz

FECHA	PACIENTE	PROFESIONAL	DIAGNÓSTICO	ESTADO
25 de jun de 2026	SP Sebastián Pinto CI1000027	Ana Pérez	miopia	Abierta Con receta
24 de jun de 2026	MM Matias Meigarejo CI5414885	Ana Pérez	TEST	Abierta Con receta
07 de jun de 2026	OP Oscar Pinto CI1000022	Ana Pérez	Miopia	Abierta Con receta
07 de jun de 2026	FM Federico Miranda CI1000061	Jorge Martínez		Pendiente
07 de jun de 2026	SV Sofia Vega CI1000024	Laura González		Abierta
04 de jun de 2026	MM Matias Meigarejo CI5414885	Ana Pérez	Miopia	Cerrada Con receta
27 de may de 2026	DO Daniela Gutiérrez CI1000014	Carlos Rodríguez	Miopia	Cerrada Con receta
16 de may de 2026	MS Mateo Bravo CI1000046	na Pérez	Cansancio	Cerrada

Figura 9.2: Listado de consultas clínicas, con paciente, profesional, diagnóstico y estado.

9.8. Estado

[Implementado] y verificado end-to-end (self-booking del paciente, portal, PDF, receta externa).

Capítulo 10

Módulo: Clientes

10.1. Propósito

Gestiona el ABM de clientes de venta — distintos de los pacientes clínicos, aunque pueden ser la misma persona física — con sus datos de facturación (persona física o jurídica). Es el módulo que le da a **Venta** a quién facturarle.

10.2. Entidades principales

Cuadro 10.1: Entidades principales del módulo de Clientes.

Entidad	Rol
Cliente	Identidad de facturación de una Person : tipo (física/jurídica), razón social, RUC/CI fiscal, contacto. Ver Capítulo 20, Grupo A.

10.3. Reglas de negocio clave

- **Cliente siempre cuelga de Person** (FK único **PersonId**) — no es una entidad de facturación separada. **TipoFacturacion** más **RazonSocial** más **RucCiFiscal** más **Direccion/Email/Telefono** son simplemente el dato de facturación de esa persona *en su rol de cliente*. Ver [ADR 0001 — Person como raíz de identidad](#).
- **Cliente y Patient pueden compartir la misma Person** sin relación directa entre ellos — se relacionan únicamente a través de **Person** (no hay FK **Cliente**↔**Patient**). Un paciente que además compra productos no genera una persona duplicada.
- **Alta por reutilización de Person**: GET `/api/clientes/buscar-persona?ci=` busca si ya existe una **Person** con esa CI; si existe, se reutiliza (evita duplicar datos si la persona ya es paciente); si no, se crea una nueva junto con el **Cliente**.
- **No hay borrado físico simple**: DELETE `/api/clientes/{id}` desactiva (soft delete); reactivar es POST `/api/clientes/{id}/activar`. En el frontend, ambas acciones viven exclusivamente en el menú contextual de la tabla (modales de confirmación verde/rojo), nunca en el formulario de edición.
- **Venta.ClienteId es nullable** — null significa “Consumidor Final”, una venta no exige tener un cliente registrado.

- **Origen histórico:** hasta la migración 035_Clientes, los datos de facturación colgaban 1:1 de *Patient* (*DatosFacturacion*, hoy eliminada); esa migración copió los datos existentes hacia *Cliente* (mapeando *PatientId*→*PersonId*) y dropeó la tabla vieja.

10.4. Endpoints

Detalle completo (policies, DTOs) en el Apéndice de Referencia de API, §2. Personas (*ClientesController*).

Cuadro 10.2: Endpoints principales del módulo de Clientes.

Método	Ruta	Descripción
GET	/api/clientes	Listado paginado (filtros: búsqueda, estado, tipo)
GET	/api/clientes/{id}	Detalle
GET	/api/clientes/ buscar-persona?ci=	Busca <i>Person</i> existente por CI, para reutilizar al dar de alta
POST	/api/clientes	Alta
PUT	/api/clientes/{id}	Edición
DELETE · POST	/api/clientes/{id} · /{id}/activar	Desactivar / reactivar

10.5. Flujo típico

Alta de cliente durante el armado de una venta (el caso de uso más frecuente en la práctica, más que el ABM directo):

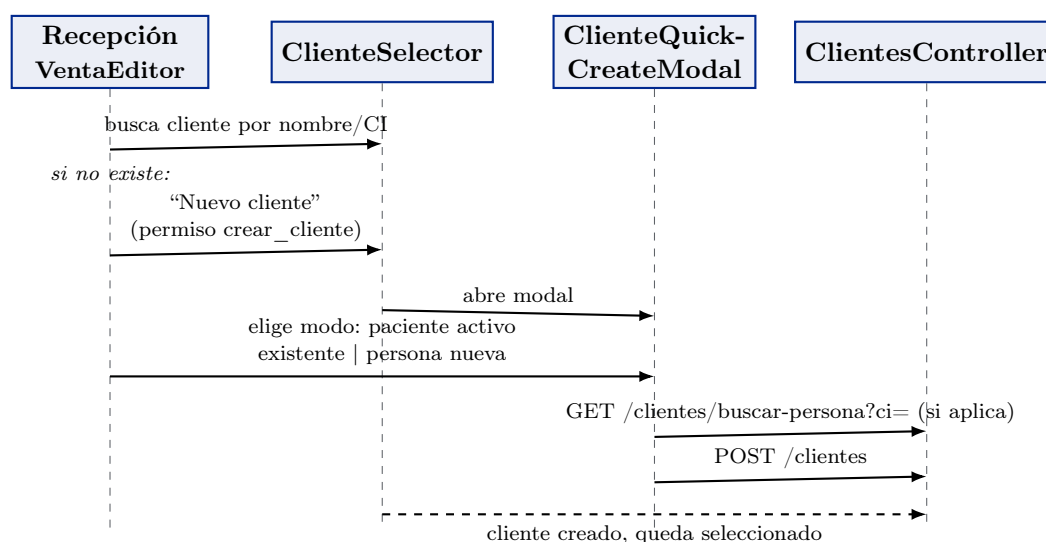


Figura 10.1: Alta rápida de cliente embebida en el armado de una venta: si el cliente no existe, se busca primero una *Person* existente por CI antes de crear una nueva.

Si el cliente ya existe, *ClienteSelector* simplemente lo deja seleccionado sin pasar por el modal.

10.6. Vistas de frontend

- `ClientesView.vue` — ABM principal (listado, alta/edición, activar/desactivar por menú contextual).
- `ClienteSelector.vue` (componente, no vista) — buscador de clientes activos más opción “Consumidor Final” más botón “Nuevo cliente”, usado embebido en los editores de venta/presupuesto.
- `ClienteQuickCreateModal.vue` (componente) — alta rápida en dos modos (paciente activo existente / persona nueva) con datos de facturación, invocado desde `ClienteSelector`.

10.7. Interfaz

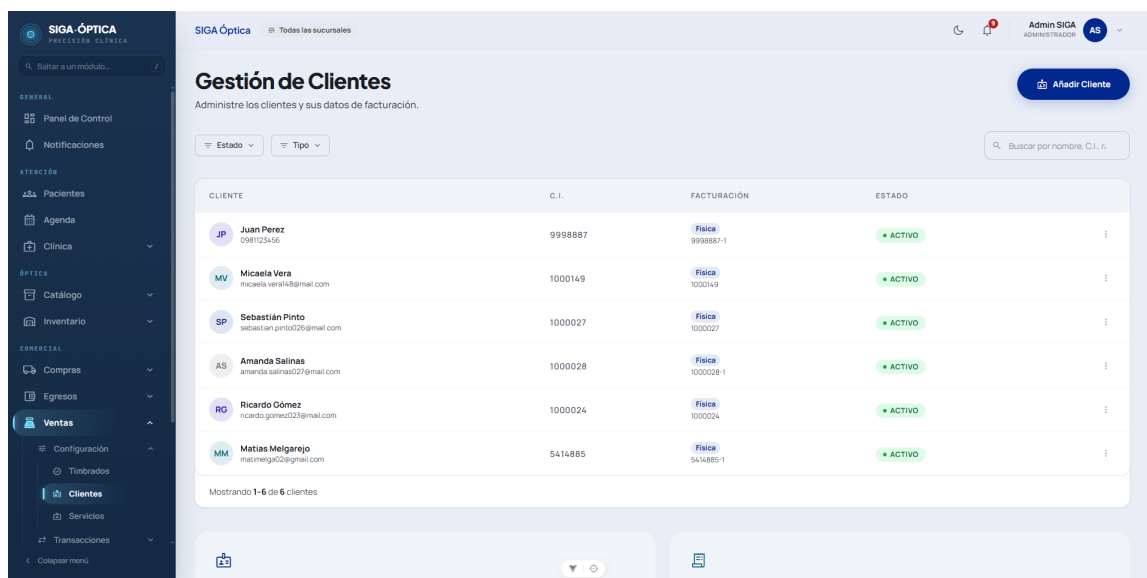


Figura 10.2: Gestión de clientes, con su tipo de facturación y estado.

10.8. Estado

[Implementado], integrado con Ventas (`Venta.ClienteId` reemplazó a la referencia previa por `PatientId`). `ventasService` y todas las vistas de venta/presupuesto/trabajo muestran `clienteNombre`, con fallback a “Consumidor Final”.

Capítulo 11

Inventario / Catálogo

11.1. Propósito

Administra el catálogo de productos de la óptica (armazones, accesorios y el catálogo óptico legado de cristales) y controla el stock por sucursal mediante un registro auditable de movimientos, en vez de un campo mutable. Incluye trazabilidad de lotes con vencimiento, conteos físicos periódicos con ajuste, y el catálogo de clasificaciones ópticas (**TipoLente**, **Tratamiento**) que consume el módulo de Ventas (Capítulo 12) para el flujo “a pedido”.

11.2. Entidades principales

Cuadro 11.1: Entidades principales del módulo de Inventario/Catálogo.

Entidad	Rol
Producto	Ítem de catálogo con stock (armazones, accesorios); PrecioVenta siempre derivado
CategoriaProducto	Categoría con Margen (define el precio de venta) y Tipo (Genérico/Armazón/ Cristal obsoleto)
Marca / Modelo	Catálogo jerárquico de marca → modelo
ProductoStock Config	Mín/máx de stock, 1:1 con Producto , global (no por sucursal)
MovimientoStock	Entrada/Salida con Estado Pendiente/Aprobado/Rechazado — única fuente del stock real
MotivoMovimiento	Catálogo de motivos, filtra por tipo de movimiento
StockLote	Trazabilidad de lotes con FechaVencimiento , originados en una recepción
ConteoInventario / ConteoInventario Linea	Conteo físico periódico con diferencia sistema-vs-físico
StockActualView	Entidad de solo lectura sobre la vista SQL <code>vw_stock_actual</code>
TipoLente	Diseño de lente (monofocal/bifocal/progresivo/ocupacional) con PrecioBase — usado por Ventas/Laboratorio
Tratamiento	Tratamiento óptico con precio (antirreflejo, fotocromático, etc.) — N:N con TrabajoPedido
Transferencia Stock / Item	Transferencia entre sucursales — detalle completo en el Capítulo 19

Ver diagrama ER completo en el Capítulo 20, Figura 20.5. Los servicios (exámenes con tarifa por profesional) se documentan en el Capítulo 12.

11.3. Reglas de negocio clave

- **El stock no es un campo, se deriva de movimientos aprobados.** Ver [ADR 0002](#). `vw_stock_actual` agrupa por producto más sucursal sumando Entrada (+) / Salida (−) sobre los movimientos en estado Aprobado (la vista compara contra el ordinal del enum, no contra una cadena).
- **PrecioVenta siempre derivado, nunca manual.** $\text{PrecioCosto} \times (1 + \text{CategoriaProducto}.\text{Margen}/100)$, centralizado en `Producto.AplicarCosto`. Ver [ADR 0004](#). Cambiar el margen de una categoría recalcula todos sus productos.
- **El catálogo óptico de cristales está obsoleto.** `CategoriaProducto.Tipo = Cristal` ya no se ofrece al dar de alta categorías (se conserva solo por compatibilidad de datos existentes). El cristal a pedido se especifica en el `TrabajoPedido` del módulo Ventas, no como `Producto`. Ver [ADR 0007](#) — **no confundir con TipoLente/Tratamiento** de esta lista, que siguen vigentes (son la especificación del cristal, no un producto con stock).
- **ProductoStockConfig es global, no por sucursal** — decisión explícita para no invalidar la relación 1:1 durante la migración multi-sucursal.
- **Crear un conteo de inventario exige solo `ver_inventario`**; únicamente *gestionar* (aprobar/rechazar, lo que ajusta el stock) exige `gestionar_inventario` — asimetría real del código, no error de este documento.
- **Los estados de inventario son enums, igual que en el resto del sistema:** `MovimientoStock.Estado/.Tipo`, `ConteoInventario.Estado`, `TransferenciaStock.Estado` y `MotivoMovimiento.Tipo` son enums C# persistidos como int, con la misma convención que `Venta/TrabajoPedido/Egreso`. Ver el Capítulo 20, Cuadro 20.1.

11.4. Endpoints

Detalle completo en el Apéndice de Referencia de API, § Catálogo/Inventario. Resumen:

Cuadro 11.2: Controllers del módulo de Inventario/Catálogo.

Controller	Ruta base	Cubre
<code>ProductosController</code>	<code>/api/productos</code>	CRUD de productos, movimientos de stock, stock-config, imagen, y sub-recursos <i>categorias</i> (<code>CategoriaProducto</code>)
<code>MarcasController</code>	<code>/api/marcas</code>	CRUD de marcas y sub-recurso <i>modelos</i> (<code>Modelo</code>)
<code>TipoLentesController</code>	<code>/api/tipos-lente</code>	CRUD de diseños de lente
<code>TratamientosController</code>	<code>/api/tratamientos</code>	CRUD de tratamientos

Controller	Ruta base	Cubre
MotivosMovimiento Controller	/api/motivos- movimiento	CRUD de motivos, filtrable por tipo
StockLotesController	/api/stock/ lotes	Lotes con vencimiento más conteos físicos (crear/listar/gestionar)
Ubicaciones Controller	/api/ ubicaciones	Departamento/Ciudad — catálo- go geográfico compartido con Su- cursal/Proveedor, no exclusivo de este módulo

11.5. Flujo típico

Movimiento de stock con aprobación:

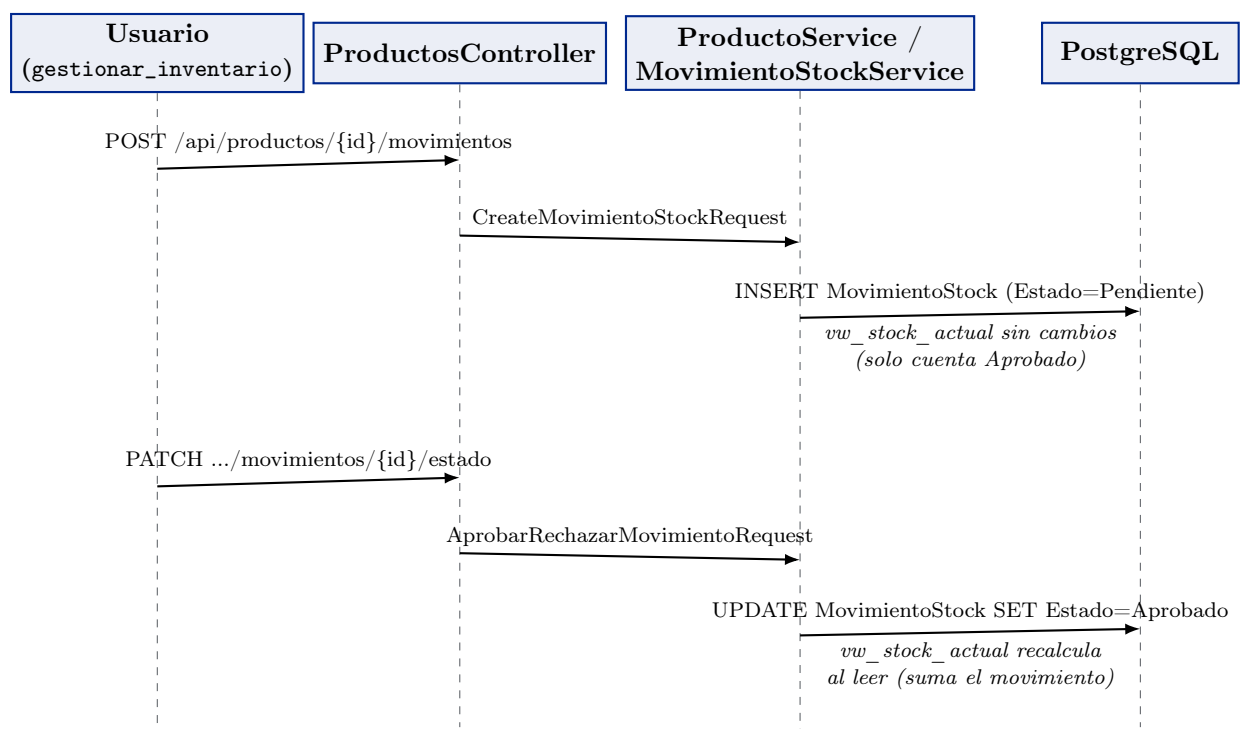


Figura 11.1: Movimiento de stock: alta en *Pendiente* y aprobación posterior, que recién ahí impacta *vw_stock_actual*.

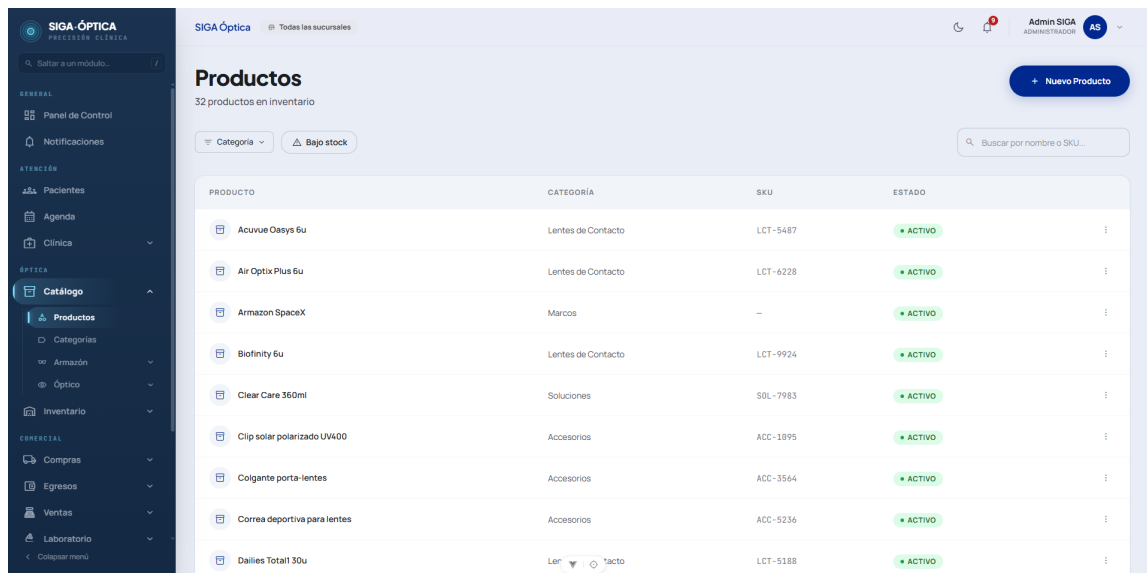
Conteo físico: cualquier usuario con *ver_inventario* registra un *ConteoInventario* con sus *ConteoInventarioLinea* (cantidad física por producto); el sistema calcula *Diferencia* contra *vw_stock_actual*. Solo un usuario con *gestionar_inventario* puede aprobar el conteo — la aprobación genera los *MovimientoStock* de ajuste necesarios para que el stock del sistema coincida con lo contado.

11.6. Vistas de frontend

ProductosView.vue, *CategoriasProductoView.vue*, *MarcasView.vue*, *ModelosView.vue*, *TipoLentesView.vue*, *TratamientosView.vue*, *StockView.vue*, *MovimientosView.vue*,

MovimientoFormView.vue, MotivosMovimientoView.vue, ConteoFormView.vue, ConteoRevisarView.vue, ConteoAprobacionView.vue, ConteoAprobacionPendientesView.vue (todas en SIGA-Web/src/views/).

11.7. Interfaz



PRODUCTO	CATEGORÍA	SKU	ESTADO
Acuvue Oasys 6u	Lentes de Contacto	LCT-5487	ACTIVO
Air Optix Plus 6u	Lentes de Contacto	LCT-6228	ACTIVO
Armazon SpaceX	Marcos	-	ACTIVO
Biofinity 6u	Lentes de Contacto	LCT-9924	ACTIVO
Clear Care 360ml	Soluciones	SOL-7983	ACTIVO
Clip solar polarizado UV400	Accesorios	ACC-1895	ACTIVO
Coligante porta-lentes	Accesorios	ACC-3564	ACTIVO
Correa deportiva para lentes	Accesorios	ACC-5236	ACTIVO
Dailies Total1 30u	Lentes de Contacto	LCT-5188	ACTIVO

Figura 11.2: Catálogo de productos, con categoría, SKU y estado de cada uno.

11.8. Estado

[Implementado] y en uso. Dos decisiones de diseño explícitas que conviene tener presentes:

- **ProductoStockConfig** define los niveles mínimo y máximo de forma global, no por sucursal.
- Las categorías con **Tipo = Cristal** y los productos asociados a ellas quedan como datos legado: siguen existiendo en la base por trazabilidad histórica, pero no se ofrecen en altas nuevas (ver [ADR 0007](#)).

Capítulo 12

Ventas

12.1. Propósito

Registra las ventas de la óptica en dos modalidades sobre una misma cabecera (**Venta.Tipo**): **venta directa** de mostrador (productos/servicios de stock inmediato) y **venta “a pedido”** óptica (armazón + cristal a medida + tratamientos + laboratorio externo, con receta obligatoria al confirmar). Cubre también presupuestos (la misma entidad **Venta** en estado **Borrador**), cobro flexible (contado/crédito, métodos combinados), emisión de documento fiscal (recibo simple o factura timbrada con numeración automática) y devoluciones/cambios.

12.2. Entidades principales

Cuadro 12.1: Entidades principales del módulo de Ventas.

Entidad	Rol
Venta	Cabecera única para venta directa, presupuesto y venta a pedido (Tipo/Estado)
VentaLinea	Línea de producto, servicio o lente a pedido (Tipo)
Cliente	A quién se le vende (nullable = Consumidor Final) — ver Capítulo 10
Cobro / CobroLinea	Pago (seña o cuota) con métodos combinados
FacturaVenta / Comprobante	Los dos documentos fiscales posibles, mutuamente excluyentes
Timbrado	Serie fiscal con numeración correlativa automática
Devolucion / DevolucionLinea	Devolución o cambio de producto post-venta
TrabajoPedido	El “pedido óptico” de una venta a pedido — detalle completo en el Capítulo 13
Receta	Graduación que respalda una venta a pedido — ver Capítulo 9
Servicio / ServicioTarifa	Exámenes/servicios con tarifa por profesional o especialidad, vendibles como línea

Ver diagrama ER completo en el Capítulo 20, Figura 20.8.

12.3. Reglas de negocio clave

- **Precio de línea autocompleta desde Producto.PrecioVenta** (derivado por margen, [ADR 0004](#)) pero queda editable por línea para un ajuste puntual.

- **El cristal a pedido ya no es un Producto con stock.** Se especifica en el TrabajoPedido como TipoLente (diseño, con PrecioBase que autocompleta el precio) más Tratamientos más precio editable. En VentaLinea esto se representa como Tipo = Lente, sin ProductoId — no descuenta stock. Ver [ADR 0007](#).
- **El TrabajoPedido de una venta a pedido nace en Borrador** junto al presupuesto, editable libremente y **sin aparecer en la cola del laboratorio**. Al confirmar la venta, si sigue en Borrador, se exige asignar laboratorio y pasa a PendienteAprobacion. Ver [ADR 0005](#).
- **Confirmar una venta TrabajoAPedido exige RecetaId.** La receta puede ser de una consulta clínica o cargada manualmente para el cliente (receta “externa”) — ver Capítulo 9.
- **El cobro está desacoplado del ciclo del laboratorio.** Toda venta confirmada (directa o a pedido) pasa directo a ListaParaCobrar; el envío/recepción del trabajo en el laboratorio corre en paralelo sin bloquear el cobro. EstadoVenta.EnProceso quedó sin uso para ventas nuevas. Ver [ADR 0006](#).
- **Documento fiscal: recibo o factura, nunca ambos.** Comprobante (recibo simple) y FacturaVenta (factura timbrada) son mutuamente excluyentes sobre la misma Venta; ambos producen los mismos efectos (egreso de stock por línea de producto más ingreso de caja, **sin duplicar** si ya hubo cobros previos) y mueven la venta a ComprobanteEmitido. La emisión es un paso dedicado (VentaComprobanteView), separado de la pantalla de cobro.
- **Numeración de factura automática por timbrado.** Al emitir una factura, el cajero elige un Timbrado activo y vigente; el backend genera {Establecimiento}-{PuntoExpedicion}-{correlativo:D7} y avanza Timbrado.UltimoNumero — no hay carga manual del número (VentaService, confirmado en código: EmitirFacturaRequest solo lleva TimbradoId, no número/establecimiento manuales).
- **Servicio/ServicioTarifa** permiten un precio distinto por profesional o por especialidad para el mismo servicio (ej. examen); GET /api/servicios/{id}/precio?professionalId= resuelve el precio aplicable en el momento de armar la línea.
- **El plan de cuotas de una venta a crédito no se materializa como filas.** La venta guarda solo CantidadCuotas y FrecuenciaCuotasDias; el monto de cada cuota, cuántas van pagadas, el vencimiento de la próxima y si está vencida se derivan de esos dos campos más los Cobro registrados. No existe una tabla de cuotas que pueda quedar desincronizada de los cobros reales.

12.4. Endpoints

Detalle completo en el Apéndice de Referencia de API, § Ventas (VentasController, TimbradosController) y § Catálogo/Inventario para ServiciosController (agrupado ahí en el documento por archivo de origen, pero funcionalmente de este módulo). Flujo de estados de Venta cubierto por: POST /api/ventas (crea presupuesto/borrador, con bloque opcional TrabajoPedido) → PUT /api/ventas/{id}/confirmar → POST /api/ventas/cobros (uno o más) → POST /api/ventas/{id}/comprobante o POST /api/ventas/facturas.

12.5. Flujo típico

Venta a pedido, de presupuesto a comprobante (dividido en 2 figuras por la cantidad de pasos):

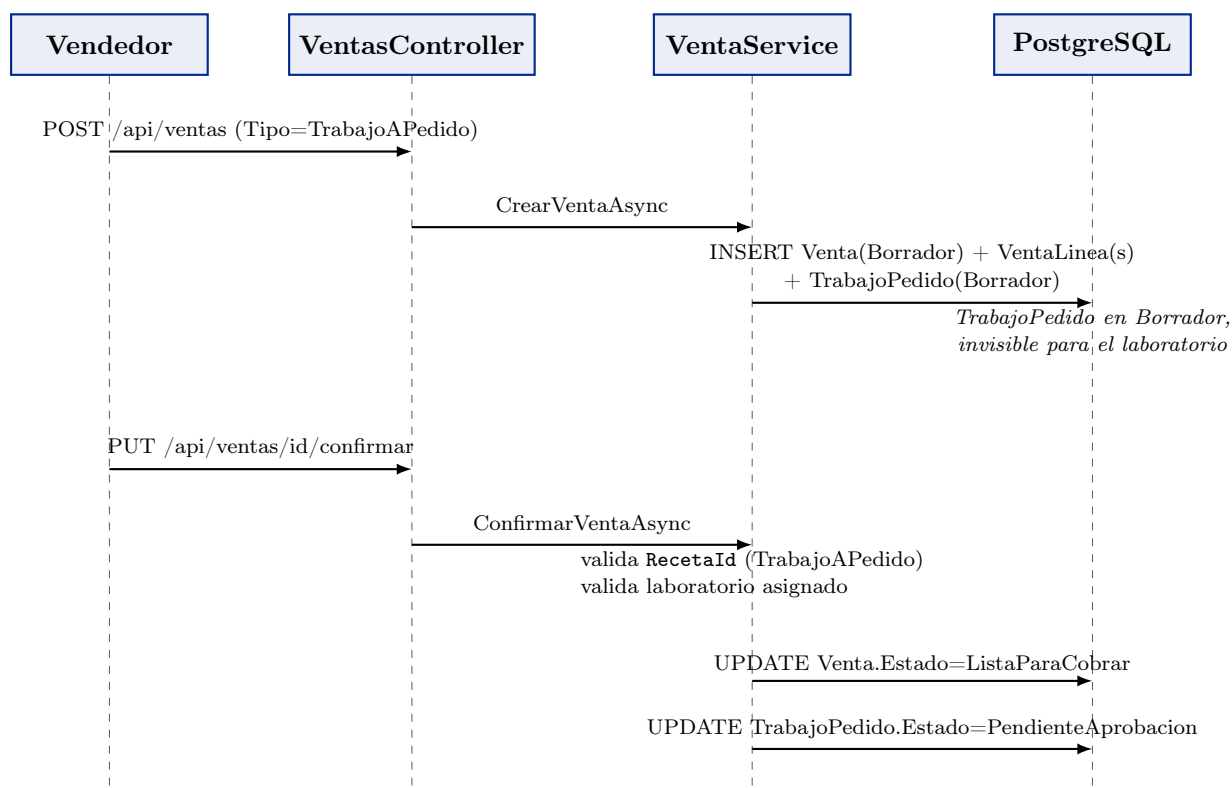


Figura 12.1: Venta a pedido (1/2): creación del presupuesto y confirmación, momento en el que el TrabajoPedido recién entra a la cola del laboratorio (ver Capítulo 13).

12.6. Vistas de frontend

VentasView.vue, VentasNuevaView.vue, PresupuestosView.vue, PresupuestoNuevoView.vue (wrappers del editor unificado), components/VentaEditor.vue (prop esPresupuesto, toggle A pedido/Directa), VentaDetalleView.vue, VentaCobrarView.vue, VentaComprobanteView.vue, CobrosPendientesView.vue, FacturasVentaView.vue, FacturaDetailView.vue, TimbradosView.vue, VentasCierreView.vue; componentes components/RecetaSelector.vue, components/TrabajoOpticoCard.vue, components/ClienteSelector.vue, components/ClienteQuickCreateModal.vue, y el composable composables/optica.ts (arma OpticaState → líneas más bloque TrabajoPedido). Las devoluciones y cambios no tienen vista propia: viven dentro de VentaDetalleView.vue, como un modal “Devolución / Cambio” desde el que se solicitan y un bloque que lista las ya registradas sobre esa venta.

12.7. Interfaz

12.8. Estado

[Implementado] y en uso, con dos particularidades a tener en cuenta:

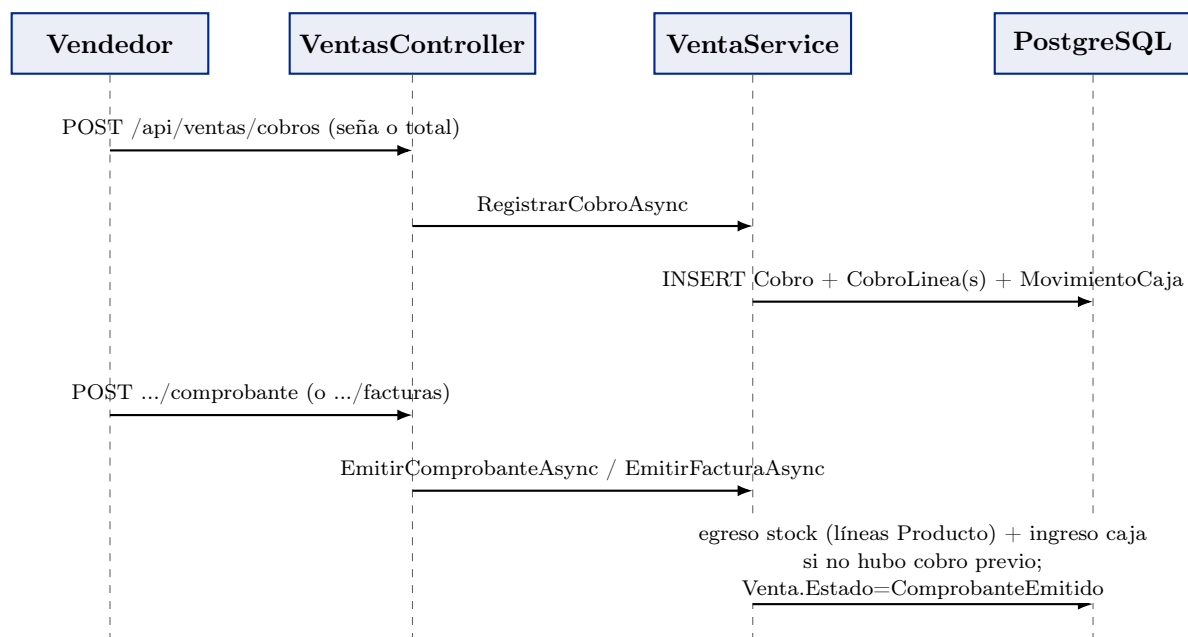


Figura 12.2: Venta a pedido (2/2): cobro y emisión de comprobante/factura. El laboratorio sigue su propio ciclo en paralelo desde que el `TrabajoPedido` pasa a `PendienteAprobacion` — no bloquea ni es bloqueado por el cobro.

COMPROBANTE	CLIENTE	TIPO	FECHA	TOTAL	SALDO	ESTADO
REC-0000088	Juan Perez	A pedido	10 jul. 2026	Gs. 26.500	—	Emitido
REC-0000087	Amenda Salinas	A pedido	10 jul. 2026	Gs. 27.700	Gs. 17.700	Lista cobrar
REC-0000086	Sebastián Pinto	A pedido	10 jul. 2026	Gs. 22.500	—	Lista cobrar
REC-0000085	Ricardo Gómez	A pedido	10 jul. 2026	Gs. 26.500	—	Lista cobrar
REC-0000084	Ricardo Gómez	A pedido	10 jul. 2026	Gs. 26.500	Gs. 26.500	Borrador
REC-0000083	Micaela Vera	A pedido	10 jul. 2026	Gs. 22.500	—	Lista cobrar
REC-0000082	Juan Perez	A pedido	10 jul. 2026	Gs. 302.500	—	Emitido
REC-0000081	Micaela Vera	A pedido	10 jul. 2026	Gs. 705.000	Gs. 705.000	Borrador
REC-0000080	Ricardo Gómez	A pedido	05 jul. 2026	Gs. 1.260.000	Gs. 1.134.000	Lista cobrar
REC-0000079	Sebastián Pinto	Directa	01 jul. 2026	Gs. 590.000	—	Lista cobrar

Figura 12.3: Listado de ventas, con tipo (directa o a pedido), total, saldo pendiente y estado de cada una.

- El flujo de “generar venta desde presupuesto” carga líneas más cliente más receta y confirma, pero **no rehidrata** la `TrabajoOpticoCard` (la config óptica ya quedó fija en el `TrabajoPedido` del presupuesto) — no hay endpoint para editar receta/laboratorio de un presupuesto existente al confirmarlo.
- `Venta` tiene varios totales (`Total`, `MontoSeñala`, `TotalCobrado`, `SaldoPendiente`, montos por categoría fiscal) que son propiedades **calculadas en memoria** (`Ignore()` por EF), no columnas — no confundir con datos persistidos al leer el esquema.

Capítulo 13

Laboratorio

13.1. Propósito

Gestiona el ciclo de vida de un `TrabajoPedido` (la parte “a pedido” de una venta óptica: cristal graduado más armazón más tratamientos) una vez que la venta que lo originó fue confirmada — desde que se aprueba internamente hasta que el laboratorio externo lo fabrica, lo envía, se recibe y se factura. Se separó de Ventas el 2026-06-08 porque el ciclo de vida de “fabricar y recibir un pedido de un proveedor externo” es operativamente distinto del ciclo de venta/cobro.

13.2. Entidades principales

Cuadro 13.1: Entidades principales del módulo de Laboratorio.

Entidad	Rol
<code>TrabajoPedido</code>	Núcleo del módulo — 1:1 con la <code>Venta</code> que lo originó. Ver Capítulo 20 (Grupo C).
<code>FacturaLaboratorio</code>	Factura del laboratorio externo, 1:1 con <code>TrabajoPedido</code> (<code>Cascade</code>).
<code>Proveedor</code>	El mismo catálogo de proveedores de Compras sirve como catálogo de laboratorios ópticos externos.
<code>TipoLente</code> / Tratamiento	Especificación del cristal a pedido — el cristal no es un <code>Producto</code> (ver ADR 0007).
<code>EgresoFacturaLaboratorio</code>	Subtipo de <code>Egreso</code> (TPH) creado automáticamente al emitir la factura del laboratorio — ver Capítulo 15 (Caja y Egresos).

13.3. Reglas de negocio clave

- El `TrabajoPedido` **nace en estado Borrador** junto con el presupuesto/venta, y no aparece en la cola de laboratorio hasta que la venta se confirma. Ver [ADR 0005](#). `GetPedidosAsync` excluye explícitamente `Borrador`.
- El **cobro de la venta está desacoplado** del ciclo de laboratorio: la venta pasa a `ListaParaCobrar` al confirmarse, sin esperar a que el laboratorio reciba o entregue nada. `RegistrarRecepcionAsync` **no** toca el estado de la `Venta`. Ver [ADR 0006](#).
- El ciclo de estados es **lineal y estricto** — cada transición valida el estado actual exacto y rechaza si no corresponde (`PendienteAprobacion`→`PendienteEnvio`/`Rechazado` →

Enviado → Recibido, factura solo desde Recibido).

- Al emitir la factura del laboratorio (EmitirFacturaAsync), el servicio crea automáticamente un EgresoFacturaLaboratorio (Estado = Pendiente) además de la Factura Laboratorio — el pago real de esa factura se gestiona luego desde el módulo de Egresos, no desde Laboratorio.
- Solo puede haber una factura por TrabajoPedido (tp.Factura != null bloquea una segunda).
- Al recibir un pedido, se genera una NotificacionInterna de tipo pedido_lab_recibido dirigida a toda la sucursal de la venta (broadcast por sucursal, no a un usuario puntual).

13.4. Endpoints

Ver el Apéndice de Referencia de API § Laboratorio para la tabla completa (Laboratorio Controller, /api/laboratorio/pedidos*, policies ver_laboratorio/gestionar_laboratorio).

Cuadro 13.2: Endpoints de transición de estado del TrabajoPedido.

Método	Ruta	Transición
GET	/api/laboratorio/pedidos	Listado (excluye Borrador)
POST	/api/laboratorio/pedidos/{id}/gestionar	PendienteAprobacion → PendienteEnvio Rechazado
PUT	/api/laboratorio/pedidos/{id}/enviar	PendienteEnvio → Enviado
PUT	/api/laboratorio/pedidos/{id}/recibir	Enviado → Recibido
POST	/api/laboratorio/pedidos/{id}/factura	Recibido → (sin cambio de estado del TP; crea FacturaLaboratorio + EgresoFacturaLaboratorio)

13.5. Flujo típico

El ciclo completo se muestra en dos figuras: el origen del TrabajoPedido desde la venta (Figura 13.1) y el ciclo propio del laboratorio una vez que el pedido entra a la cola (Figura 13.2).

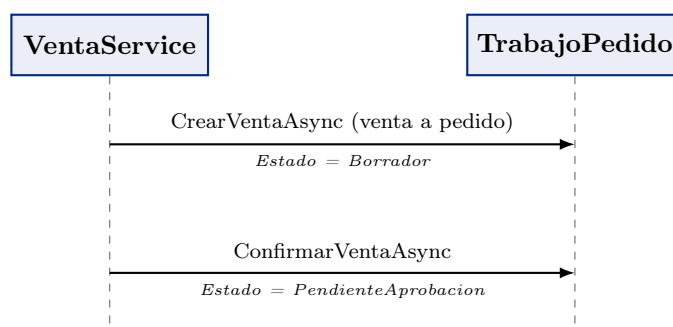


Figura 13.1: Origen del TrabajoPedido al confirmar una venta a pedido. Nace en Borrador (editable, fuera de la cola) y recién entra a la cola de laboratorio al confirmarse.

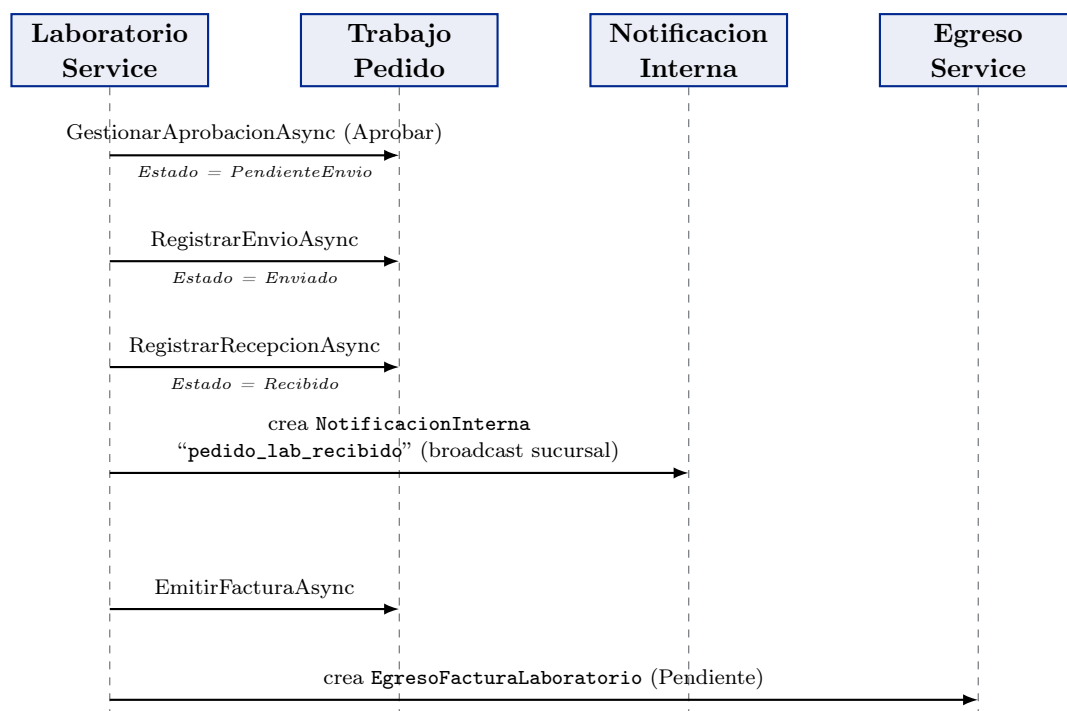


Figura 13.2: Ciclo del laboratorio tras la aprobación del TrabajoPedido. La venta ya estaba ListaParaCobrar (o cobrada) desde su confirmación — este ciclo no la bloqueó en ningún punto (ver §13.3).

13.6. Vistas de frontend

Rutas bajo `/laboratorio/*` (SIGA-Web/src/router/index.ts), grupo de menú “Laboratorio”:

Cuadro 13.3: Vistas de frontend del módulo Laboratorio.

Vista	Ruta	Permiso
TrabajosPedidoView.vue	/laboratorio/pedidos	ver_laboratorio
TrabajosPedido AprobacionView.vue	/laboratorio/aprobaciones	gestionar_laboratorio
TrabajosPedido RecepcionesView.vue	/laboratorio/recepciones	gestionar_laboratorio
TrabajosPedidoFacturas View.vue	/laboratorio/facturas	gestionar_laboratorio

`VentaDetalleView.vue` también expone acciones de crear/enviar/recibir el TP asociado a una venta puntual (repunteadas a `laboratorioService`, no a `ventasService`).

13.7. Interfaz

SIGA Óptica PRECISIÓN AL SECTOR

SIGA Óptica Todas las sucursales

Admin SIGA ADMINISTRADOR AS

Pedidos a Laboratorio

14 pedidos

Estado

Buscar por cliente, comprobante

COMPROBANTE	CLIENTE	TIPO DE LENTE	LABORATORIO	FECHA	ESTADO	FACTURA
REC-0000088	Juan Perez	Lente de Contacto Blando	Accesorios Ópticos JM	10 jul. 2026	Pend. aprobación	—
REC-0000087	Amanda Salinas	Lente de Contacto Blando	Accesorios Ópticos JM	10 jul. 2026	Pend. aprobación	—
REC-0000086	Sebastián Pinto	Lente de Contacto Blando	Accesorios Ópticos JM	10 jul. 2026	Pend. aprobación	—
REC-0000085	Ricardo Gómez	Lente de Contacto Blando	Accesorios Ópticos JM	10 jul. 2026	Pend. aprobación	—
REC-0000083	Micaela Vera	Lente de Contacto Blando	Accesorios Ópticos JM	10 jul. 2026	Pend. aprobación	—
REC-0000082	Juan Perez	Monofocal	Accesorios Ópticos JM	09 jul. 2026	Pend. aprobación	—
REC-0000080	Ricardo Gómez	Monofocal	Accesorios Ópticos JM	05 jul. 2026	Pend. aprobación	—
REC-0000076	Amanda Salinas	Lente de Contacto Blando	Accesorios Ópticos JM	26 jun. 2026	Enviado	—
REC-0000074	Ricardo Gómez	Lente de Contacto Rígido	Accesorios Ópticos JM	26 jun. 2026	Recibido	—

Figura 13.3: Cola de pedidos al laboratorio óptico externo, con el tipo de lente y su estado.

13.8. Estado

[Implementado] end-to-end (backend y frontend). Sin limitaciones conocidas propias del módulo — las limitaciones del flujo óptico en general (no poder editar receta/laboratorio de un presupuesto ya confirmado) están documentadas en el Capítulo 12 (Ventas).

Capítulo 14

Compras

14.1. Propósito

Gestiona el ciclo de abastecimiento de mercadería desde proveedores externos: alta de proveedores, órdenes de compra (OC), registro de la factura del proveedor, recepción física de la mercadería (con generación de lotes y stock) y devoluciones al proveedor.

14.2. Entidades principales

Cuadro 14.1: Entidades principales del módulo de Compras.

Entidad	Rol
Proveedor / ProveedorContacto	Catálogo de proveedores (compartido con Laboratorio vía el flag <code>EsLaboratorio</code>).
PedidoProveedor / PedidoProveedorItem	La orden de compra (OC) y sus líneas.
FacturaCompra (sub-tipo de Egreso, TPH)	Factura del proveedor — ver Capítulo 15 (Caja y Egresos) para la jerarquía completa.
RecepcionMercaderia / RecepcionMercaderiaItem	Recepción física (puede ser parcial, una OC admite varias).
StockLote	Generado por cada línea de recepción con lote/vencimiento (ver Capítulo 11).
DevolucionProveedor	Devolución de mercadería ya recibida hacia el proveedor. No confundir con Devolucion/DevolucionLinea , que es la devolución/cambio del lado del cliente en una Venta (Capítulo 12) — son dos entidades distintas sin relación entre sí, solo comparten la palabra “devolución”.

Ver el capítulo de Modelo de Datos (§20, Grupo C) para el diccionario de datos completo.

14.3. Reglas de negocio clave

- Una OC solo se puede **editar** (reemplazo completo de ítems) mientras está en **Borrador**.

- Una OC **no se puede cancelar** si ya está `RecibidaTotal`.
- Las **devoluciones a proveedor** (`DevolucionProveedor`) solo se permiten sobre OCs con mercadería recibida (`RecibidaTotal/RecibidaParcial`), y la cantidad a devolver no puede superar la cantidad recibida del ítem. Cada devolución genera un `MovimientoStock` de **Salida** en estado **Aprobado** (descuenta stock inmediatamente, sin flujo de aprobación adicional).
- Alta de proveedor exige **al menos un contacto con nombre** (validado en backend y frontend).
- El pago en efectivo de una factura de compra exige una **SesionCaja abierta** en la sucursal — si no hay, la operación falla explícitamente en vez de crear un movimiento huérfano.
- **El ciclo de una OC** es Borrador → Confirmada → Facturada → RecibidaParcial/RecibidaTotal. La factura del proveedor se registra apenas la OC está **Confirmada** (`RegistrarFacturaAsync`), antes de la recepción física de la mercadería; recién con la factura registrada (**Facturada** o **RecibidaParcial**) se pueden aceptar recepciones (`RecepcionesService`).

14.4. Endpoints

Ver el Apéndice de Referencia de API § Compras para la tabla completa (`ComprasController`, `ProveedoresController`, `FacturasCompraController`, `RecepcionesController`).

Cuadro 14.2: Endpoints de transición de estado de la OC.

Método	Ruta	Transición / acción
POST	<code>/api/compras/pedidos</code>	Crea OC en Borrador
PUT	<code>/api/compras/pedidos/{id}/confirmar</code>	Borrador → Confirmada (policy <code>aprobar_pedidos</code> , distinta de crear/editar)
POST	<code>/api/compras/pedidos/{id}/factura</code>	Confirmada → Facturada
POST	<code>/api/compras/recepciones</code>	Facturada/RecibidaParcial → RecibidaParcial/RecibidaTotal (según ítems pendientes)
POST	<code>/api/compras/pedidos/{id}/devolucion</code>	Devolución sobre OC recibida (no cambia el estado de la OC)
PUT	<code>/api/compras/pedidos/{id}/cancelar</code>	→ Cancelada (policy <code>OR: creador gestionar_pedidos</code> o <code>aprobador aprobar_pedidos</code>)

14.5. Flujo típico

El ciclo se muestra en dos figuras: desde la creación de la OC hasta la factura del proveedor (Figura 14.1), y desde la recepción física hasta una eventual devolución (Figura 14.2).

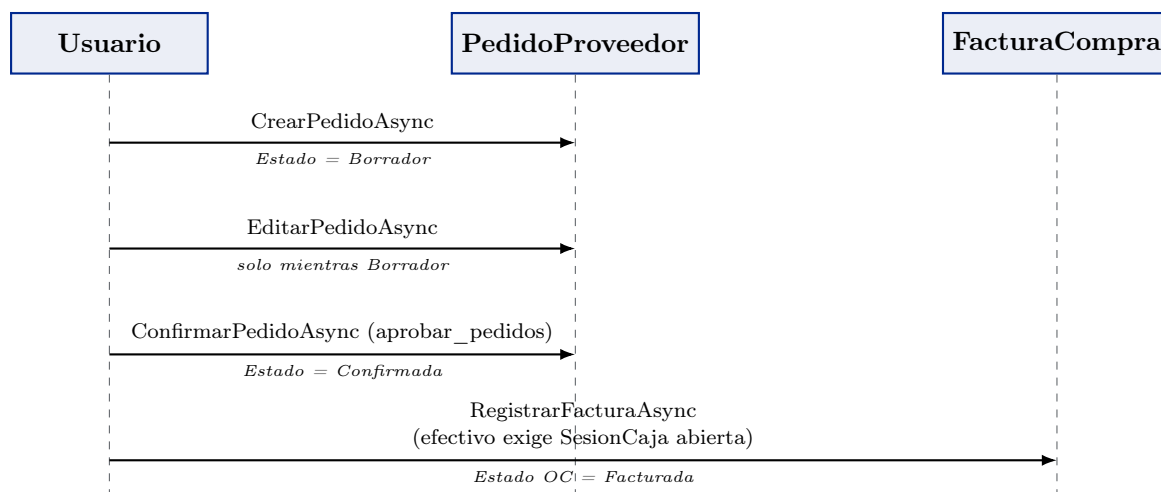


Figura 14.1: Desde la creación de la orden de compra hasta el registro de la factura del proveedor. La factura se registra **antes** de la recepción física.

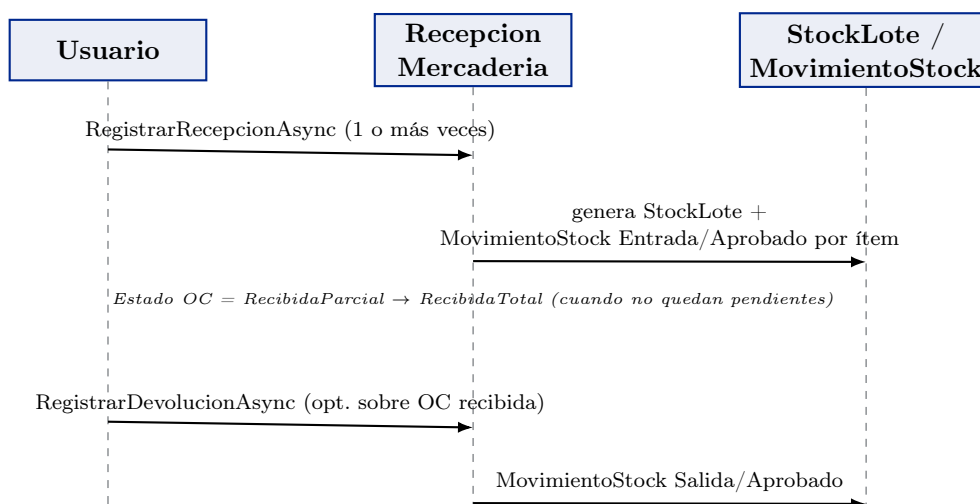


Figura 14.2: Recepción física (una o más, parcial) y devolución opcional al proveedor.

14.6. Vistas de frontend

Rutas bajo /compras/* (SIGA-Web/src/router/index.ts):

Cuadro 14.3: Vistas de frontend del módulo Compras.

Vista	Ruta
PedidosView.vue	/compras/oc
OcFormView.vue	/compras/oc/nueva, /compras/oc/:id/editar
OcDetailView.vue	/compras/oc/:id
OcAprobacionView.vue	/compras/aprobaciones
FacturasCompraView.vue / FacturaFormView.vue / FacturaDetailView.vue	/compras/facturas*
RecepcionesView.vue / RecepcionFormView.vue / RecepcionDetailView.vue	/compras/recepciones*
ProveedoresView.vue	ruta propia, fuera de /compras
ReportesComprasView.vue	/compras/reportes

No hay una vista dedicada a DevolucionProveedor — la acción vive embebida en OcDetailView.vue.

14.7. Interfaz

# OC	PROVEEDOR	FECHA	ÍTEMS / TOTAL	ESTADO
#23	Armazones del Paraguay	11 de jul de 2026	1 ítem — Gs. 3.200	Cancelada
#22	Armazones del Paraguay	11 de jul de 2026	1 ítem — Gs. 3.200	Facturada
#21	Accesorios Ópticos JM	10 de jul de 2026	1 ítem — Gs. 160.000	Recibida
#20	Cristales y Monturas PY	26 de jun de 2026	2 ítems — Gs. 4.400	Recibida
#19	Armazones del Paraguay	26 de jun de 2026	3 ítems — Gs. 7.300	Facturada
#18	Importadora Óptica Central	17 de jun de 2026	1 ítem — Gs. 450.000	Recibida
#17	Importadora Óptica Central	17 de jun de 2026	1 ítem — Gs. 450.000	Confirmada

Mostrando 1-7 de 7 órdenes

Figura 14.3: Listado de órdenes de compra a proveedores, con la cantidad de ítems, el total y su estado.

14.8. Estado

[Implementado] end-to-end.

Capítulo 15

Caja y Egresos

15.1. Propósito

Dos dominios estrechamente acoplados: **Caja** controla el efectivo físico por sucursal (apertura/cierre de sesión, arqueo) y registra todo movimiento de dinero (ingreso o egreso); **Egresos** modela las distintas formas en que el negocio gasta dinero (factura de proveedor, honorario profesional, gasto general, salario, factura de laboratorio) bajo una única jerarquía de datos.

15.2. Entidades principales

Cuadro 15.1: Entidades principales del módulo de Caja y Egresos.

Entidad	Rol
SesionCaja	Una caja abierta por sucursal a la vez. Apertura con monto inicial, cierre con arqueo físico.
MovimientoCaja	Cada entrada/salida de dinero, siempre asociada a una SesionCaja ; opcionalmente a una Venta (cobro) o un Egreso (pago).
Egreso (clase base TPH)	Ver ADR 0011 — una única tabla física egresos , discriminador Tipo (TipoEgreso).
FacturaCompra / Honorario / GastoGeneral / SalarioEmpleado / EgresoFacturaLaboratorio	Los 5 subtipos concretos de Egreso .
CategoriaGasto	Catálogo para clasificar GastoGeneral (sin controller propio — vive bajo EgresosController).

Ver el capítulo de Modelo de Datos (§20, Grupo C) para el diccionario de datos completo, y el [ADR 0011](#) para el razonamiento detrás de la herencia TPH.

15.3. Reglas de negocio clave

- **Una sola SesionCaja abierta por sucursal a la vez** (**AbrirSesionAsync** rechaza si ya hay una **Abierta** o **PendienteAprobacion**). El monto de apertura, si no se especifica, se autocompleta con el **EfectivoContado** del último cierre de esa sucursal (el efectivo del cajón se traslada entre sesiones).
- **Cierre con tolerancia de arqueo:** al cerrar, se calcula **EfectivoEsperado** = **MontoInicial** + **ingresosEfectivo** - **egresosEfectivo** y se compara contra

EfectivoContado (conteo físico). Si la diferencia está dentro de la tolerancia (± 50.000 , fijada como constante **Tolerancia** en **CajaService**), la sesión pasa directo a **Cerrada**; si la excede, pasa a **PendienteAprobacion** y requiere que alguien con **aprobar_cierres_caja** la apruebe o la rechace. Rechazar **reabre** la sesión (vuelve a **Abierta**, limpiando los campos de cierre) para que el cajero pueda re-arquear.

- **Egreso centraliza sus propias transiciones de estado como métodos de dominio** (`egreso.RegistrarPago(...)`, `.Aprobar()`, `.Rechazar(motivo)`), no como lógica dispersa en el servicio — lanzan `InvalidOperationException` si la transición no es válida en el estado actual, que el servicio traduce a `ErrorType.Conflict`.
- **Pago externo:** un egreso puede pagarse “fuera de caja” (`PagoExterno=true` + `MotivoPagoExterno` obligatorio) sin generar `MovimientoCaja` ni exigir sesión abierta — para pagos que no pasaron por la caja física del negocio.
- Un egreso **pagado no se puede anular**; solo egresos en otros estados.
- **Dos caminos generan una FacturaCompra, no uno solo:** (a) `POST /api/egresos/facturas` (alta directa, sin OC) y (b) `ComprasService.RegistrarFacturaAsync` cuando la factura está ligada a una `PedidoProveedor` (ver Capítulo 14). Ambos crean el mismo tipo de entidad `Egreso/FacturaCompra`, por caminos de código distintos.
- **El pago de un egreso registra el MovimientoCaja asociado** (`EgresoId = egreso.Id`) cuando el pago se hace vía `EgresoService.RegistrarPagoAsync`; el permiso que exige esta acción es `pagar_egresos`, definido a nivel de método en `EgresosController.RegistrarPago` (sobrescribe `gestionar_egresos` de la clase). El chequeo adicional de `aprobar_egresos` aplica solo para pago externo.

15.4. Endpoints

Ver el Apéndice de Referencia de API § Caja y Egresos para la tabla completa (`CajaController`, `EgresosController`).

Cuadro 15.2: Endpoints principales de Caja y Egresos.

Método	Ruta	Acción
POST	<code>/api/caja/sesiones</code>	Abrir sesión de caja
POST	<code>/api/caja/sesiones/{id}/cerrar</code>	Cerrar con arqueo → Cerrada o PendienteAprobacion
POST	<code>/api/caja/sesiones/{id}/aprobar-cierre rechazar-cierre</code>	Resolver una diferencia de arqueo
POST	<code>/api/egresos/{facturas honorarios salarios gastos}</code>	Crear el subtipo correspondiente de Egreso
PUT	<code>/api/egresos/{id}/pago</code>	Registrar pago (interno → MovimientoCaja , o externo)
PUT	<code>/api/egresos/{id}/{aprobar rechazar anular}</code>	Transiciones de estado

15.5. Flujo típico

El ciclo se muestra en dos figuras: creación y pago de un egreso (Figura 15.1), y cierre de sesión con tolerancia de arqueo (Figura 15.2).

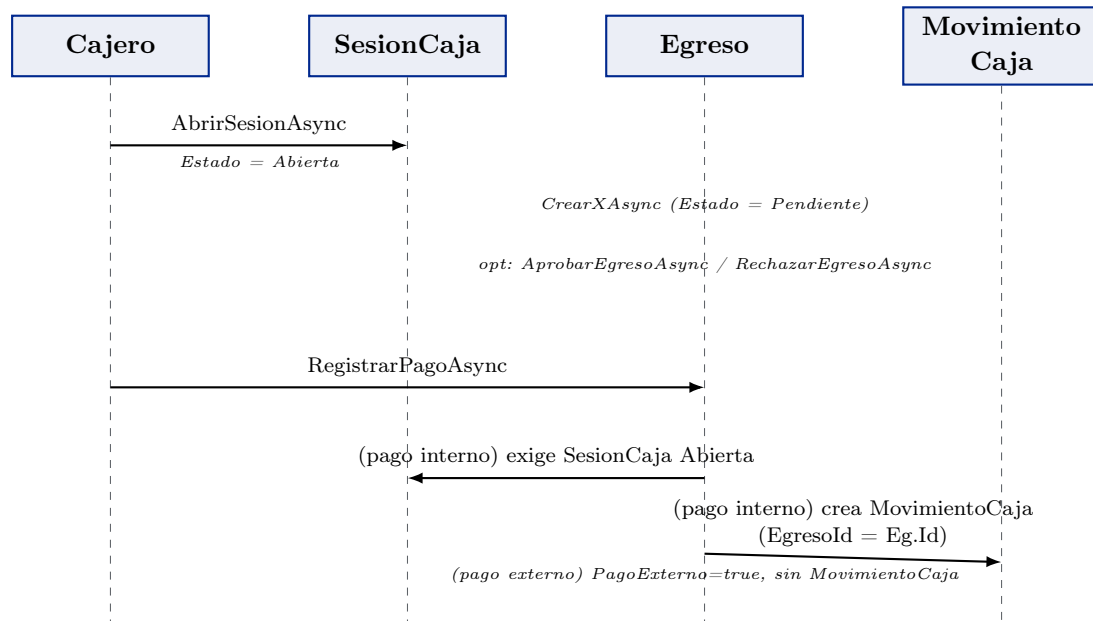


Figura 15.1: Apertura de sesión, creación de un egreso y registro de su pago (interno o externo). Tras el pago interno, el estado del egreso pasa a **Pagado**.

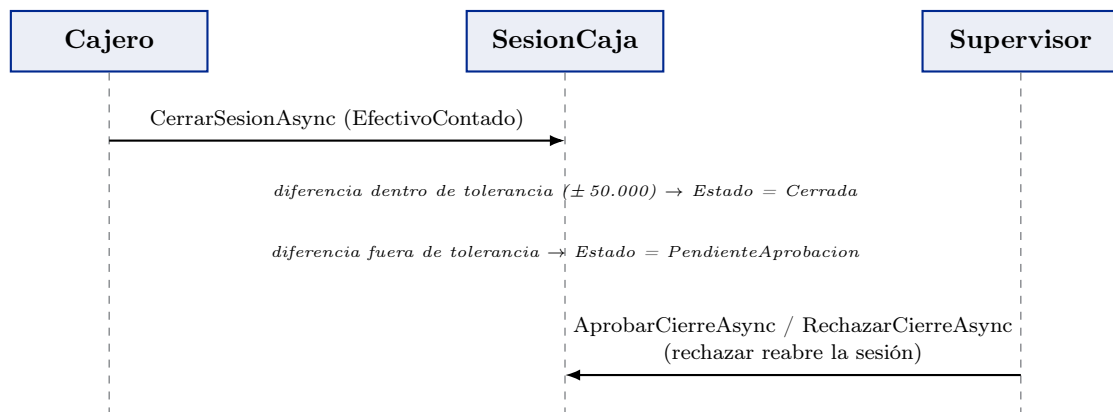


Figura 15.2: Cierre de sesión con arqueo: dentro de tolerancia cierra directo, fuera de tolerancia requiere aprobación de un supervisor.

15.6. Vistas de frontend

Rutas bajo `/caja/*` y `/egresos/*` (SIGA-Web/src/router/index.ts):

Cuadro 15.3: Vistas de frontend del módulo Caja y Egresos.

Vista	Ruta	Permiso
CajaHistorialView.vue	/caja/historial	ver_ventas
CajaAprobacionesView.vue	/caja/aprobaciones	aprobar_ cierres_caja
EgresosView.vue	/egresos	ver_egresos
NuevoEgresoView.vue	/egresos/nuevo	gestionar_ egresos
AprobacionEgresosView.vue	/egresos/aprobacion	aprobar_egresos
PagosEgresosView.vue	/ /egresos/pagos,	/ pagar_egresos
PagoEgresoView.vue	egresos/pagos/:id	
CategoriasGastoView.vue	/egresos/categorias	ver_egresos

15.7. Interfaz

TIPO	CONCEPTO	MONTO	EMISIÓN	ESTADO
Factura compra	Factura de compra – OC #22 – Armazones del Paraguay	Gs. 3.200	11 jul. 2026	Pagado
Gasto	alquiler	Gs. 130.000	10 jul. 2026	Pagado
Gasto	alquiler	Gs. 130.000	10 jul. 2026	Aprobado
Factura compra	Factura de compra – OC #21 – Accesorios Ópticos JM	Gs. 160.000	10 jul. 2026	Pagado
Factura compra	Factura de compra directa – 001-001-0000566 – Armazones del Paraguay	Gs. 5.450	26 jun. 2026	Pagado
Factura compra	Factura de compra – OC #20 – Cristales y Monturas PY	Gs. 4.400	26 jun. 2026	Pendiente
Factura compra	Factura de compra – OC #19 – Armazones del Paraguay	Gs. 7.300	26 jun. 2026	Pagado
Factura lab.	Factura lab. Accesorios Ópticos JM – REC-0000073 (Ricardo Gómez)	Gs. 230.000	26 jun. 2026	Pendiente
Factura compra	Factura de compra – OC #18 – Importadora Óptica Central	Gs. 450.000	17 jun. 2026	Aprobado

Figura 15.3: Listado de egresos, agrupando facturas de compra, gastos generales y facturas de laboratorio.

15.8. Estado

[Implementado] end-to-end.

Capítulo 16

Módulo: Roles y Permisos

16.1. Propósito

ABM del catálogo de `Role/Permission` y su asignación: define qué combinaciones de permisos existen como “rol” seleccionable y qué permisos tiene cada uno. Es el panel de configuración detrás de la autorización descrita en el Capítulo 5 — aquí se define *qué puede hacer cada rol*, allí se gestiona *qué rol tiene cada usuario*.

16.2. Entidades principales

Ver Capítulo 20, Grupo A.

Cuadro 16.1: Entidades principales del módulo de Roles y Permisos.

Entidad	Rol
Role	Rol configurable con nombre. Type (admin professional patient, nullable e immutable) marca los 3 roles de sistema.
Permission	Catálogo fijo de 56 permisos granulares sembrados por <code>DbSeeder.AllPermissions</code> (<code>ver_pacientes</code> , <code>crear_venta</code> , <code>gestionar_laboratorio</code> , etc.). 55 tienen policy homónima en <code>Program.cs</code> ; la excepción es <code>ver_recepcion</code> , que está en el seed pero no tiene policy declarada — no se puede usar hoy en un <code>[Authorize(Policy=...)]</code> aunque exista en la base.
RolePermission	Pivote many-to-many — la asignación real de permisos a un rol.

16.3. Reglas de negocio clave

- **Un Role es solo un nombre más una colección de Permissions** — no hay lógica de negocio codificada por nombre de rol en ningún endpoint. Ver [ADR 0003](#).
- **Se pueden crear roles ad-hoc desde /roles sin modificar código ni desplegar** — ej. “Recepcionista Senior” con un permiso extra sobre “Recepcionista” — porque la autorización nunca consulta `Role.Name`, solo los permisos resueltos en el JWT.
- **Role.Type es immutable y no es un permiso** — identifica los 3 roles de sistema (`admin`, `professional`, `patient`) para lógica interna puntual (ej. bootstrap del usuario admin, o que el seeder sepa a qué rol darle todos los permisos). No participa en ninguna policy de autorización de endpoint.

- **El admin tiene todos los permisos excepto `ver_mis_turnos`** (ese es específico del portal de autogestión del paciente) — sembrado por el `DbSeeder`, no codificado de forma fija en cada policy.
- **Vista propia separada de la gestión de usuarios:** `RolesView/RolFormView` (cards por módulo de negocio, para elegir permisos agrupados visualmente) es una pantalla distinta de `UsuariosView` — esta última solo asigna *qué rol* tiene cada usuario, no edita el contenido de un rol. Decisión de diseño explícita para no mezclar ambas responsabilidades en una sola pantalla.

16.4. Endpoints

Detalle completo en el Apéndice de Referencia de API, §1 (`RolesController`, `UserRolesController` — ambos viven en el mismo archivo `RolesController.cs`).

Cuadro 16.2: Endpoints principales del módulo de Roles y Permisos.

Método	Ruta	Descripción
GET	<code>/api/roles</code>	Listado de roles
GET	<code>/api/roles/{id}</code>	Detalle (incluye sus permisos)
POST/PUT/DELETE	<code>/api/roles</code>	ABM del rol y su combinación de permisos
GET	<code>/api/roles/{id}/users</code>	Usuarios que tienen ese rol
GET/POST/DELETE	<code>/api/users/{userId}/roles</code>	Asignación de roles a un usuario (documentado también en el Capítulo 5, vive en <code>UsuariosView</code>)

16.5. Flujo típico

CRUD estándar sin pasos multi-capas no triviales — no amerita diagrama de secuencia. La única mecánica particular es que el formulario de rol (`RolFormView`) presenta los 55 `Permission` agrupados por módulo de negocio (cards), y guarda la selección completa como el nuevo set de `RolePermission` del rol (reemplazo total, no diffing incremental).

16.6. Vistas de frontend

- `RolesView.vue` (listado)
- `RolFormView.vue` (alta/edición, cards de permisos por módulo)

16.7. Interfaz

16.8. Estado

[Implementado].

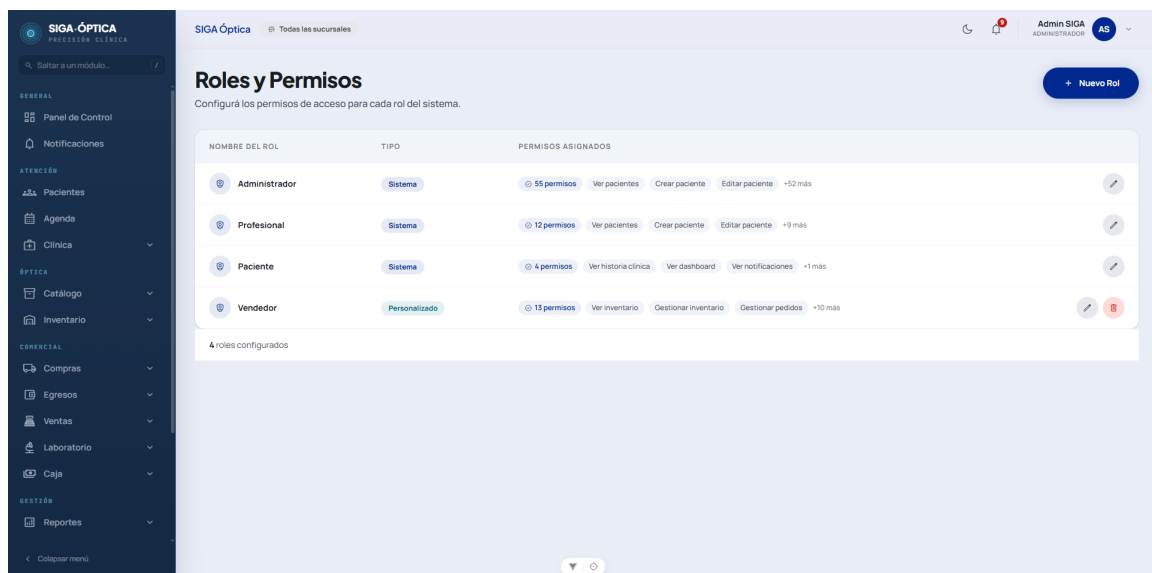


Figura 16.1: Roles del sistema con la cantidad de permisos asignados a cada uno.

Capítulo 17

Notificaciones

17.1. Propósito

Centro de alertas internas para el staff de la óptica: avisa de eventos que requieren atención (stock bajo, una transferencia de stock pendiente de aceptar, un pedido de laboratorio recibido) sin que nadie tenga que revisar manualmente cada pantalla. El diseño de destinatario (usuario, sucursal o global) es la base sobre la que se apoyan las preferencias de notificación configurables por cada persona.

17.2. Entidades principales

Cuadro 17.1: Entidad principal del módulo de Notificaciones.

Entidad	Rol
Notificacion Interna	Una alerta: destinatario (usuario / sucursal / global), tipo, mensaje, leída o no

Ver Capítulo 20 (entidad transversal, no pertenece a ninguno de los 3 grupos del DER — se referencia desde ahí).

17.3. Reglas de negocio clave

- **Scoping de destinatario en 3 niveles**, resuelto vía `ICurrentUserContext`: `Destinatario UsuarioId` (individual), `DestinatarioSucursalId` (broadcast a todo el staff de esa sucursal), o ambos `null` (broadcast global). No existe scoping por rol — los roles son dinámicos y `ICurrentUserContext` no expone el nombre de rol de forma robusta.
- **Leído es un único flag compartido** en las notificaciones de broadcast, no por-usuario — decisión explícita: si dos personas del staff comparten una notificación de sucursal, cuando una la marca como leída desaparece para ambas. Ver [ADR 0010 — Notificaciones internas antes que externas](#).
- **La visibilidad de broadcasts exige el permiso concreto `ver_todas_sucursales`**, no se infiere de `ICurrentUserContext.EsGlobal` (que también es `true` para pacientes, ver [ADR 0009](#)) — así un paciente con `ver_notificaciones` nunca ve los broadcasts operativos de otras sucursales.
- **Tres triggers reales conectados** (no hooks genéricos, cada uno cableado a mano):

1. **Bajo stock** — `StockBajoNotificadorService`, un `BackgroundService` que hace polling cada 30 minutos (mismo patrón que `TurnoReminderService`) comparando `vw_stock_actual` contra `ProductoStockConfig` por Producto más Sucursal. Se eligió poller en vez de hook inline porque los movimientos de stock “Aprobado” se originan desde al menos 5 servicios distintos (Compras, Ventas, Recepciones, Conteos, Transferencias) — instrumentar los 5 hubiera sido más invasivo que un poller. No duplica si ya hay una notificación sin leer para el mismo producto más sucursal.
2. **Transferencia pendiente** — hook directo en `TransferenciaStockService.CreateAsync`.
3. **Pedido de laboratorio recibido** — hook directo en `LaboratorioService.RegistrarRecepcionAsync`.

17.4. Endpoints

Cuadro 17.2: Endpoints del módulo de Notificaciones.

Método	Ruta	Descripción
GET	<code>/api/notificaciones?soloNoLeidas&page&pageSize</code>	Notificaciones del usuario autenticado
GET	<code>/api/notificaciones/contador</code>	Cantidad de no leídas (badge del header)
PUT	<code>/api/notificaciones/{id}/leer</code>	Marca una como leída
PUT	<code>/api/notificaciones/leer-todas</code>	Marca todas como leídas

Todos bajo policy `ver_notificaciones`. Ver Apéndice A, §A.13.

17.5. Flujo típico

El trigger de bajo stock (poller en background) y el consumo desde el frontend son dos flujos independientes; se muestran por separado.

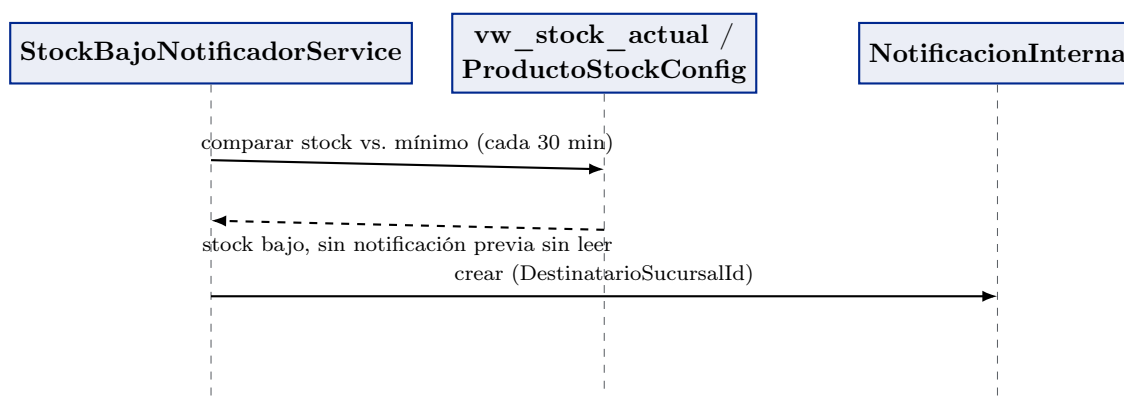


Figura 17.1: Trigger de bajo stock: polling cada 30 minutos, sin duplicar avisos ya generados y sin leer.

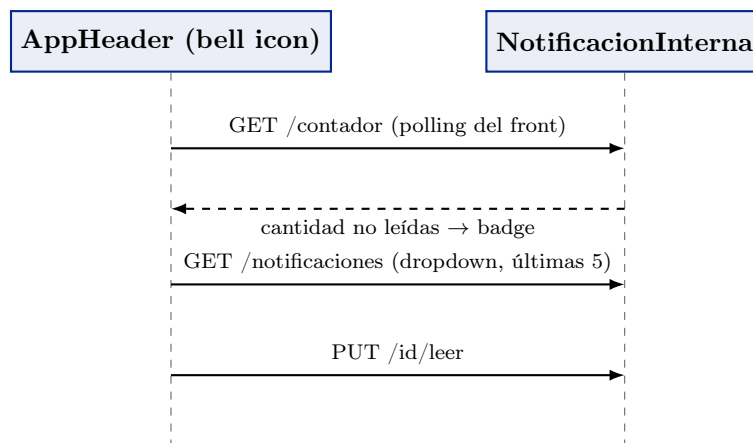


Figura 17.2: Consumo desde el frontend: badge de contador, listado desplegable y marcado de lectura.

Los otros dos triggers (transferencia pendiente, laboratorio recibido) son hooks síncronos dentro del flujo de negocio correspondiente, sin polling — se disparan en el mismo request que crea la transferencia o registra la recepción.

17.6. Vistas de frontend

- `NotificacionesView.vue` — listado completo (reemplazó un placeholder `ComingSoonView` en `/notificaciones`)
- Bell icon más dropdown en `AppHeader.vue` (componente global, no vista): badge de contador, últimas 5, marcar leída, link “Ver todas”

17.7. Interfaz

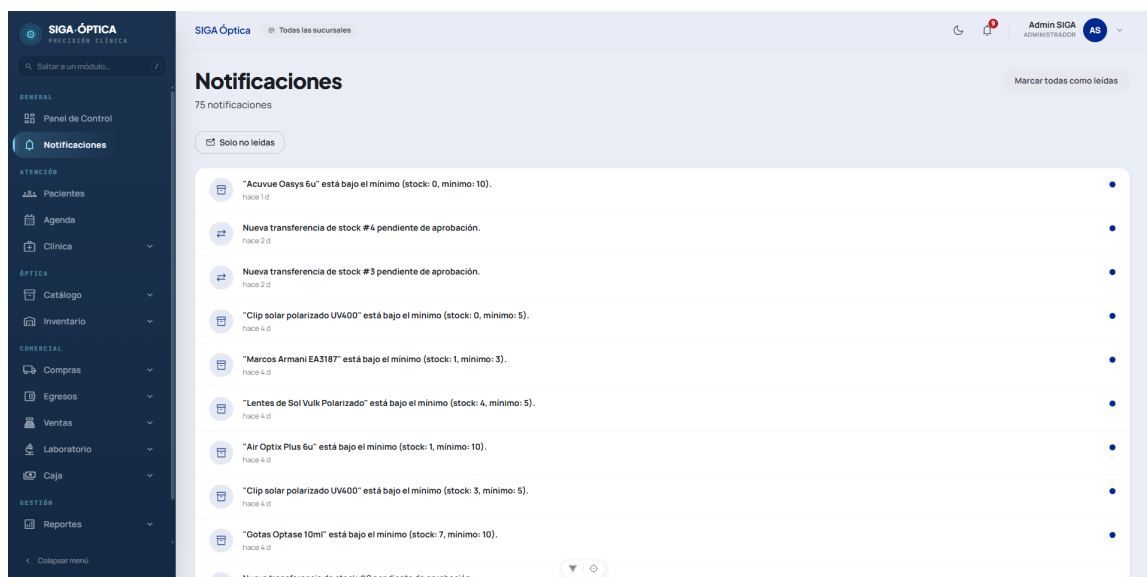


Figura 17.3: Centro de notificaciones internas, con avisos de stock bajo y transferencias pendientes de aprobación.

17.8. Estado

[Implementado] Fase 1 (centro interno): migración `055_NotificacionesInternas`, poller de bajo stock generando notificaciones reales, y los 4 endpoints del centro de notificaciones.

Capítulo 18

Reportes

18.1. Propósito

Reportes **operativos de control** — listados filtrables con totales, exportables a PDF/CSV, para auditar el día a día (qué se vendió, qué se compró, qué movió stock, qué movió caja). Es distinto de los reportes **analíticos** (KPIs agregados del dashboard), que son client-side con `jsPDF` y no forman parte de este módulo.

18.2. Entidades principales

Este módulo no tiene entidades propias — es una capa de agregación/exportación sobre entidades ya documentadas en el capítulo de Modelo de Datos (§20):

Cuadro 18.1: Entidades fuente de cada reporte.

Reporte	Entidades fuente
Ventas	Venta, Cobro, VentaLinea (Grupo C)
Compras	PedidoProveedor, FacturaCompra, Recepcion Mercaderia (Grupo C)
Movimientos de inventario	MovimientoStock, vw_stock_actual (Grupo B)
Movimientos de caja	MovimientoCaja, SesionCaja (Grupo C)

Dos campos se agregaron específicamente para este módulo (filtro de “operador” uniforme entre reportes): `Venta.VendedorId` ($\rightarrow$ `User`, quién creó la venta) y `Egreso.RegistradoPorId` ($\rightarrow$ `User`).

18.3. Reglas de negocio clave

- **Generación en backend, no client-side** — decisión arquitectónica explícita por volumen (listados de miles de filas), consistencia de formato entre PDF y CSV, y para poder reutilizar el mismo exportador desde un futuro job de envío por email (los exportadores reciben el DTO, no el `HttpRequest`).
- **PDF con QuestPDF** (mismo patrón que `MovimientoStockPdfGenerator` y la receta óptica), **CSV con un writer propio** (UTF-8 con BOM, separador `;`) — sin librería nueva.
- **La matriz de filtros no es uniforme entre los 4 reportes:** fecha y sucursal son nativos en todos; en Inventario y Caja los demás filtros (categoría, método de pago) son nativos también, pero en Ventas y Compras “método de pago” y “categoría” son filtros de tipo “contiene” porque hay que atravesar `Cobro/VentaLinea` en vez de un campo directo.

- **Filtros combinables:** fechas, sucursal (scoping automático vía `ICurrentUserContext`, igual que el resto del sistema — ver [ADR 0009](#)), método de pago, categoría/producto, operador.

18.4. Endpoints

Cuadro 18.2: Endpoints de Reportes.

Método	Ruta	Descripción
GET	<code>/api/reportes/ventas?desde&hasta&agrupacion</code>	Serie temporal agregada (día/semana/mes)
GET	<code>/api/reportes/citas?desde&hasta&agrupacion</code>	Turnos + consultas + recetas agregado
GET	<code>/api/reportes/inventario?desde&hasta&agrupacion</code>	Snapshot de stock (valorización, crítico, por categoría) + movimientos del rango
GET	<code>/api/reportes/compras?desde&hasta&agrupacion</code>	OC, facturas, recepciones, compras por proveedor
GET	<code>/api/reportes/operativo/{tipo}?desde&hasta&sucursalId&metodoPago&categoría&operadorId&tipoMov&page&pageSize</code>	Listado operativo paginado y filtrable (tipo: ventas compras inventario caja)
GET	<code>/api/reportes/operativo/{tipo}/export?formato&...</code>	Archivo <code>application/pdf</code> o <code>text/csv</code> , mismos filtros sin paginar

18.5. Flujo típico

18.6. Vistas de frontend

- `ReportesHubView.vue` — landing con cards a cada reporte.
- `ReportesVentasView.vue`, `ReportesComprasView.vue`, `ReportesInventarioView.vue`, `ReportesCitasView.vue` — reportes analíticos (dashboard, no exportables por este módulo).
- `ReporteOperativoView.vue` — vista **genérica** reutilizada para los 4 reportes operativos de este módulo (ventas/compras/inventario/caja), parametrizada por **tipo**.

18.7. Interfaz

18.8. Estado

[Implementado] (backend `ReporteOperativoService` + `ReporteExporter` + `ReportesController`; frontend `reportesOperativosService.ts` + `ReporteOperativoView.vue` + cards en `ReportesHubView`).

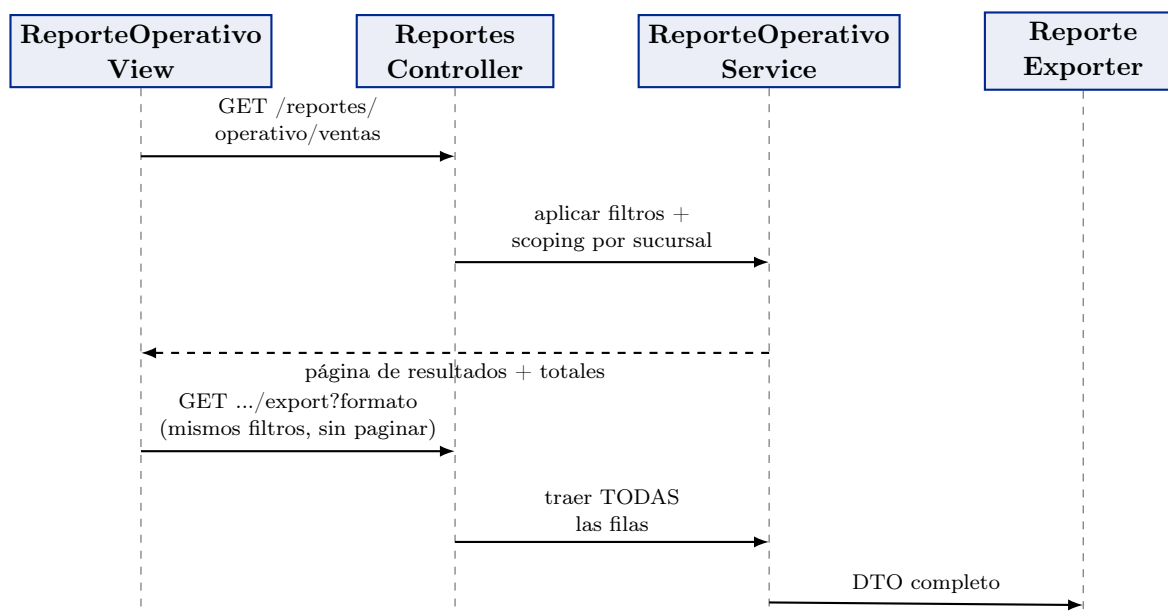


Figura 18.1: Vista previa paginada vs. exportación completa: mismos filtros, dos caminos — el export trae todas las filas sin paginar y las entrega como PDF (QuestPDF) o CSV (UTF-8+BOM).

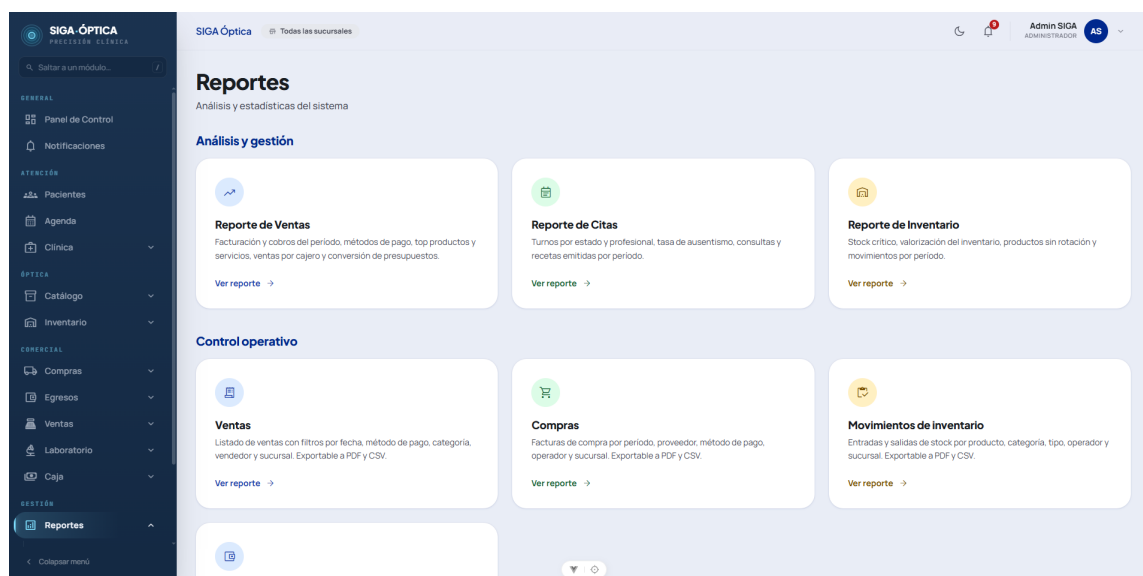


Figura 18.2: Panel de acceso a los reportes gerenciales (ventas, citas, inventario) y operativos (ventas, compras, movimientos de inventario), exportables a PDF y CSV.

Capítulo 19

Sucursales

19.1. Propósito

Vuelve a SIGA un sistema **multi-sucursal**: casi todo lo transaccional (stock, ventas, caja, timbrado, compras/recepciones, egresos, turnos/agenda, consultas clínicas) queda scopeado por sucursal, con transferencias de stock entre sucursales con flujo de aprobación. Es un proyecto transversal que tocó prácticamente todos los demás módulos del sistema, no solo las entidades propias listadas abajo.

19.2. Entidades principales

Cuadro 19.1: Entidades principales del módulo de Sucursales.

Entidad	Rol
Sucursal	Unidad de scoping — nombre, código único, ubicación (Ciudad).
TransferenciaStock	Cabecera de una transferencia entre dos sucursales (origen, destino, estado).
TransferenciaStock Item	Líneas de la transferencia (producto, cantidad).
Ciudad Departamento	/ Catálogo de ubicación geográfica de Paraguay, reutilizado por Sucursal y Proveedor.

Además, `SucursalId` (FK) fue agregado a ~15 entidades transaccionales ya documentadas en el capítulo de Modelo de Datos (Grupos A, B y C): `User`, `HorarioProfesional`, `ConsultaClinica`, `Turno`, `MovimientoStock`, `StockLote`, `ConteoInventario`, `Venta`, `Timbrado`, `PedidoProveedor`, `RecepcionMercaderia`, `Egreso` (y sus subtipos vía TPH), `SesionCaja`, `MovimientoCaja`. Quedan explícitamente **globales** (sin `SucursalId`): catálogos (`Producto`, `Marca`, `Servicio`, etc.) y personas (`Patient`, `Cliente`, `Professional`, y el `User` del admin).

19.3. Reglas de negocio clave

- **Una sucursal fija por usuario, sin selector** — ver [ADR 0009](#). `User.SucursalId = null` significa usuario global (admin, y todos los pacientes por diseño). El scoping se resuelve centralmente en `ICurrentUserContext` (`UserId/SucursalId/EsGlobal/TienePermiso`), no repetido en cada servicio.

- **La sucursal del staff se asigna desde el form de Profesional o de Empleado**, no desde la pantalla de Usuarios (esa solo gestiona roles + activar/desactivar).
- **Pacientes son globales**: no pertenecen a una sucursal fija, la eligen al reservar cada turno (paso propio en el modal de reserva).
- **ProductoStockConfig (mín/máx de stock) quedó global por producto**, no por sucursal — desvío de diseño explícito para no invalidar la relación 1:1 durante la migración.
- **TrabajoPedido no tiene SucursalId propio** — deriva de `Venta.SucursalId` (el `LaboratorioService` scopea a través de esa relación).
- **Transferencias de stock con flujo de aprobación**: crear = `Salida` en origen (estado `Pendiente`); aceptar = `Entrada` en destino; rechazar = `Entrada` de vuelta al origen. **Solo la sucursal destino gestiona** la aceptación o el rechazo — la sucursal origen no puede cancelar unilateralmente una vez creada.
- **Usuario global (admin) ve y filtra todas las sucursales**; el resto del staff solo ve/opera la propia.
- **El scoping por sucursal de las ~15 entidades listadas arriba sigue el mismo patrón centralizado en `ICurrentUserContext`** descrito en el Capítulo 3, §3.6.

19.4. Endpoints

Cuadro 19.2: Endpoints de Sucursales y Transferencias.

Método	Ruta	Descripción
GET	<code>/api/sucursales?soloActivas</code>	Listado — abierto a cualquier autenticado (incl. pacientes, lo necesitan para reservar turno)
GET	<code>/api/sucursales/{id}</code>	Detalle
POST / PUT / DELETE	<code>/api/sucursales(/{id})</code>	ABM, policy <code>gestionar_sucursales</code>
GET	<code>/api/transferencias?estado</code>	Listado, policy <code>transferir_stock</code>
POST	<code>/api/transferencias</code>	Crear (<code>Salida</code> en origen, <code>Pendiente</code>), policy <code>transferir_stock</code>
POST	<code>/api/transferencias/{id}/gestionar</code>	Aceptar/rechazar, policy <code>transferir_stock</code>

19.5. Flujo típico

Transferencia de stock entre dos sucursales (Figura 19.1).

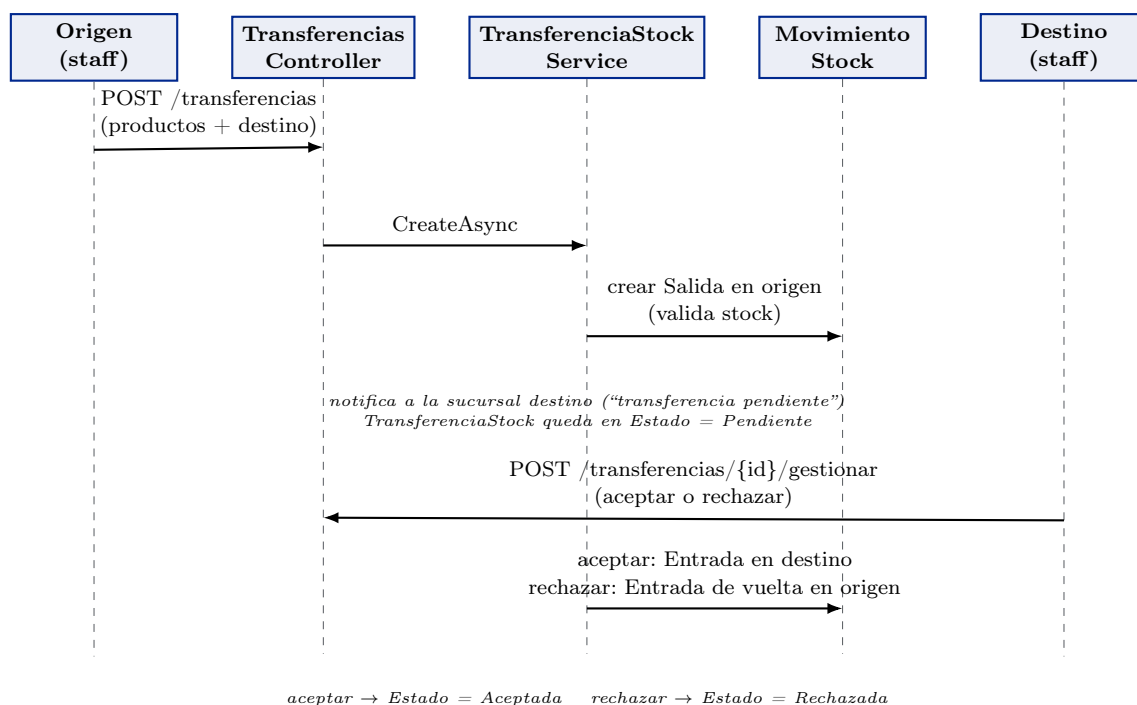


Figura 19.1: Transferencia de stock: crea Salida en origen (Pendiente); solo el destino gestiona la aceptación/rechazo.

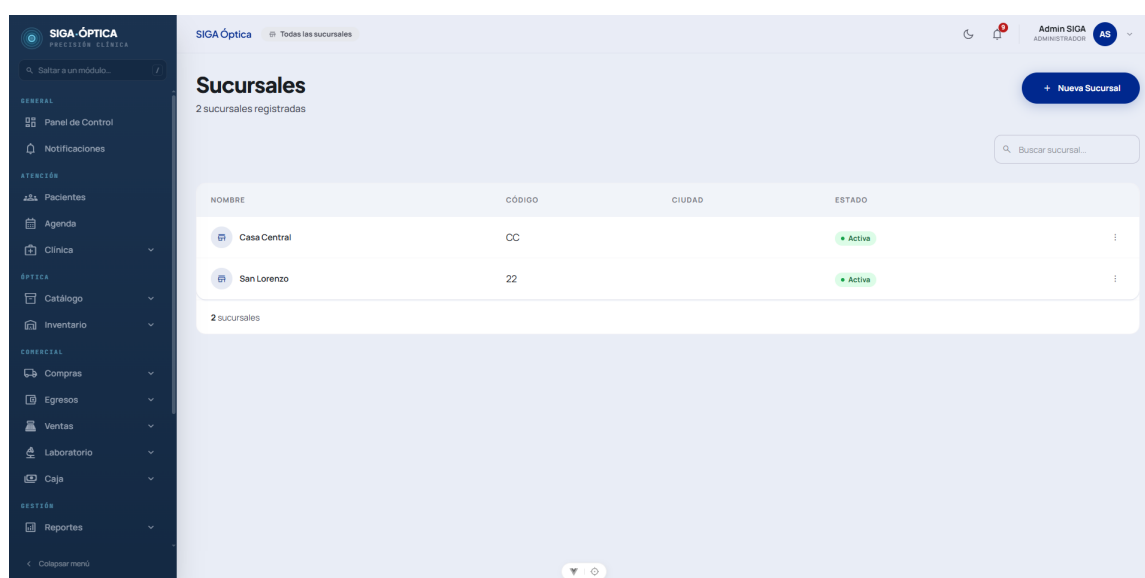
19.6. Vistas de frontend

- `SucursalesView.vue` — ABM de sucursales (`/admin/sucursales`).
- `TransferenciasView.vue` — crear transferencia (búsqueda de productos) + gestionar entrantes (aceptar/rechazar), ruta `/stock/transferencias`.
- Badge de sucursal en `AppHeader.vue` (componente, no vista): visible solo si el usuario tiene sucursal asignada (staff) o el permiso `ver_todas_sucursales` (admin → muestra “Todas”).
- Paso de selección de sucursal en el modal de reserva de turno de `MisTurnosView.vue` (portal del paciente).

19.7. Interfaz

19.8. Estado

[Implementado] Las 7 fases del proyecto (0 a 6) están completas en código: Fase 0 (entidad Sucursal + CRUD + JWT + `ICurrentUserContext` + permisos), Fase 1 (stock por sucursal), Fase 2 (ventas+caja+timbrado por sucursal), Fase 3 (compras+egresos por sucursal), Fase 4 (turnos/agenda+clínica por sucursal), Fase 5 (transferencias de stock), Fase 6 (reportes por sucursal). Migraciones 047 a 052.



SIGA Óptica PRECISIÓN CLÍNICA

Panel de Control, Notificaciones, Pacientes, Agenda, Clínica, Catálogo, Inventario, Compras, Egresos, Ventas, Laboratorio, Caja, Reportes

Sucursales
2 sucursales registradas

+ Nueva Sucursal

Buscar sucursal...

NOMBRE	CÓDIGO	CIUDAD	ESTADO
Casa Central	CC		Activa
San Lorenzo	22		Activa

2 sucursales

Figura 19.2: Listado de sucursales activas.

Capítulo 20

Modelo de Datos

Este capítulo describe el esquema de la base de datos de SIGA tal como lo define el código (SIGA.Domain/Entities + SIGA.Infrastructure/Persistence/Configurations).

Los diagramas usan el **nombre de la clase C#** (PascalCase) como identificador de entidad, no el nombre físico de la tabla. Postgres usa snake_case (confirmado vía `builder.ToTable("...")`) para varias entidades — ej. Egreso→egresos, Cliente→clientes, Venta→ventas, TrabajoPedido→trabajos_pedido); para el resto, asumir la misma convención pero confirmar contra el `*Configuration.cs` puntual antes de escribir SQL a mano.

Cada diagrama muestra PK, FK y los campos de negocio más relevantes de cada entidad — no todos los campos; el detalle completo está en el diccionario de datos que sigue a cada grupo. El esquema físico es **uno solo**; se divide en diagramas más chicos únicamente por legibilidad (69 entidades en un solo ER es ilegible en una página impresa), no porque sean bases de datos separadas. Las entidades marcadas con un asterisco (*) en un diagrama ya aparecieron completas en un diagrama anterior — se repiten como referencia de conexión, sin repetir su lista de campos.

Convención transversal no repetida en cada entidad: la gran mayoría tiene `CreatedAt` (DateTime) y muchas `UpdatedAt`; varias tienen `IsActive/Activo` (bool) para borrado lógico. Solo se listan explícitamente cuando su ausencia o comportamiento es relevante.

20.1. Grupo A — Identidad, Personas y Personal

Todo lo relacionado a “quién es quién” en el sistema: autenticación, roles/permisos, y los distintos roles funcionales que puede tener una `Person` (paciente, profesional, cliente, empleado), más ubicaciones y sucursales.

20.1.1. Identidad y control de acceso (RBAC)

20.1.2. Roles funcionales de una persona

`Professional` no tiene `PersonId` propio: llega a `Person` atravesando `User` (`Professional.User.PersonId`).

20.1.3. Personal: disponibilidad y sucursales

20.1.4. Diccionario de datos — Grupo A

`Person` — núcleo de identidad. CI único, `Email` único y nullable (usado para login vía `User`, con `AreNullsDistinct(true)`: coexisten muchas filas con `Email = NULL`). Toda persona del sistema (paciente, profesional, cliente, empleado, admin) tiene exactamente un `Person`. Ver [ADR 0001 — Person como raíz de identidad](#).

`User` — cuenta de acceso, 1:1 con `Person`. `SucursalId` nullable: `null` = usuario global (admin y pacientes); el resto del staff tiene una sucursal fija asignada desde el formulario de Profesional o de Empleado (no desde la pantalla de Usuarios, que solo gestiona roles + activar/desactivar).

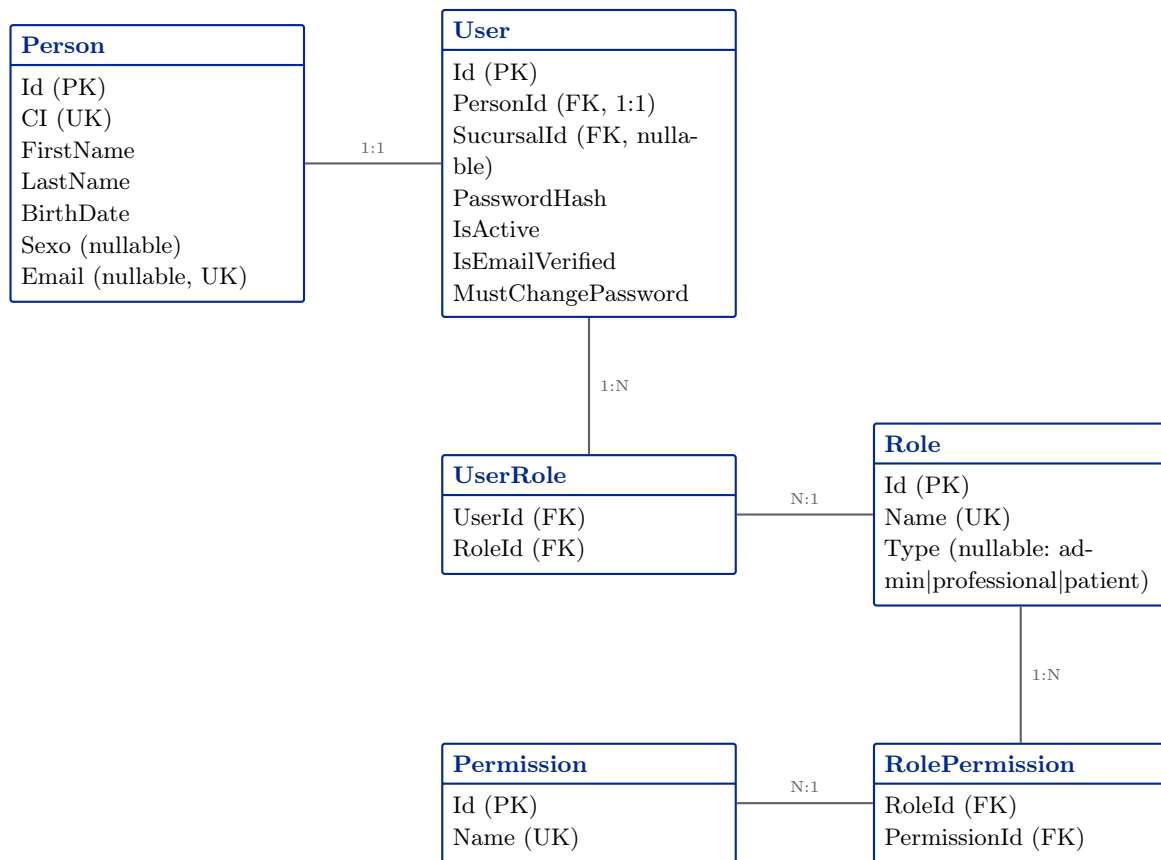


Figura 20.1: Identidad y control de acceso basado en permisos (RBAC). El JWT emite permisos resueltos, no el nombre del rol — ver Capítulo 3, §3.5.

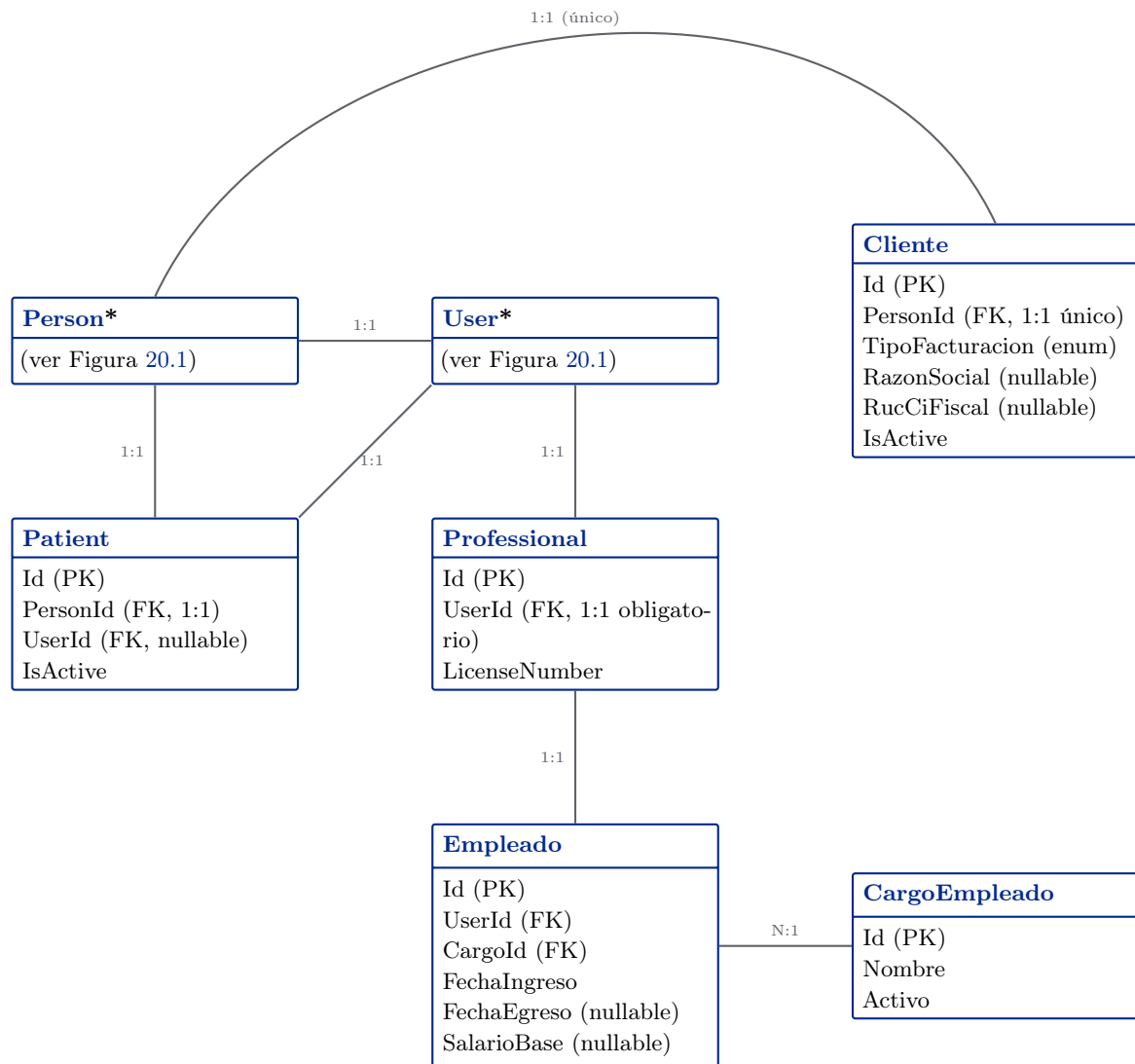


Figura 20.2: Roles funcionales que puede tener una **Person**: **Patient** y **Cliente** cuelgan directo de **Person**; **Professional** y **Empleado** cuelgan de **User**. La línea **Professional-Empleado** es solo de layout (no hay FK real entre ambos — son roles independientes que pueden coexistir en la misma **Person**).

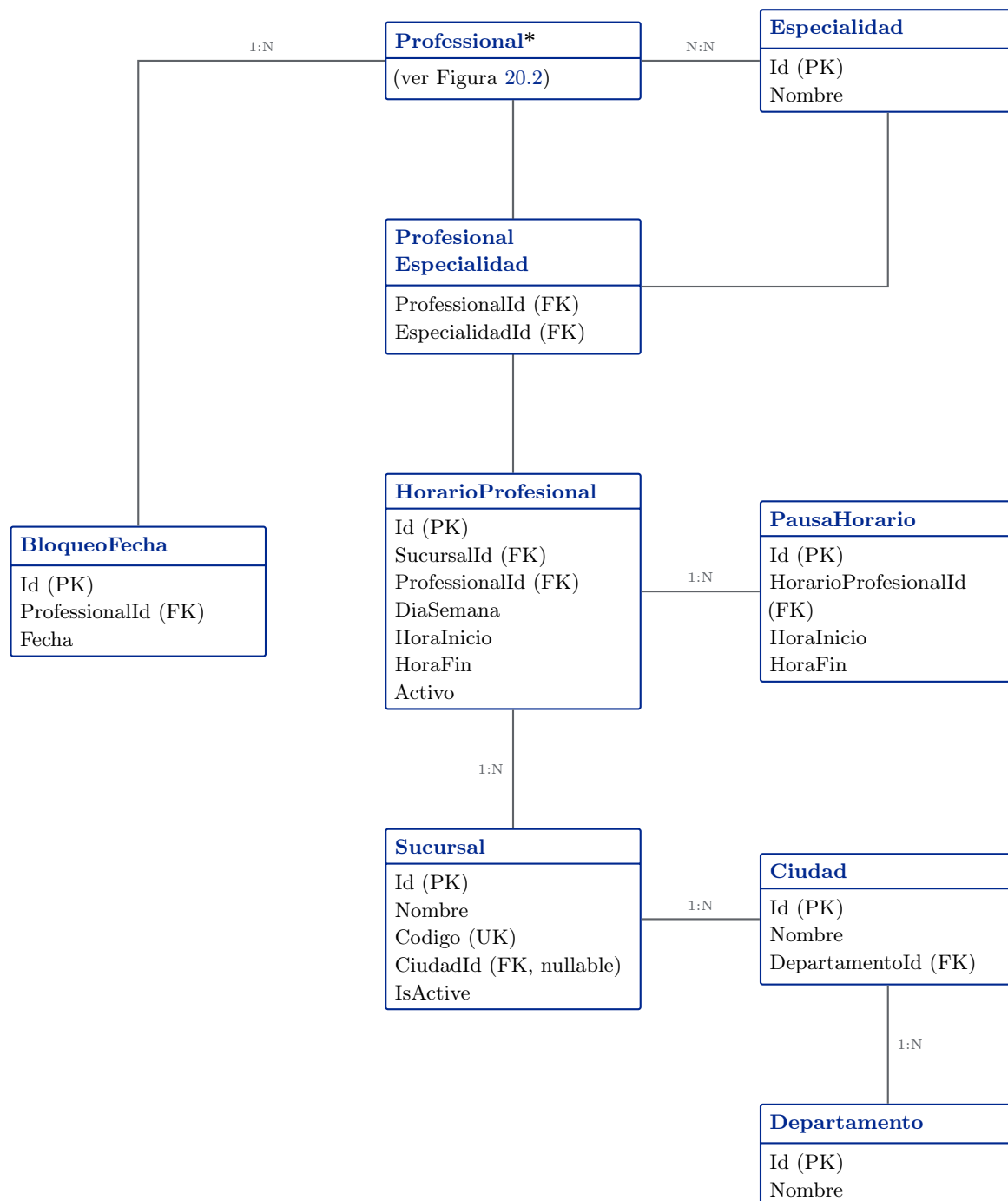


Figura 20.3: Disponibilidad del profesional para agenda (horarios por sucursal, pausas, bloqueos) y catálogo geográfico de Paraguay reutilizado por **Sucursal** y **Proveedor**.

Role / Permission / RolePermission / UserRole — autorización many-to-many pura. **Role.Type** (admin|professional|patient, nullable) es la única noción de rol “de sistema”, usada solo para bootstrap/lookups internos — la autorización de endpoints siempre pasa por permisos individuales, no por **Type** ni por **Role.Name**. Ver [ADR 0003 — Autorización por permisos](#).

Patient — marca que una **Person** es paciente. **UserId** nullable: puede existir sin cuenta de acceso (alta por recepción con solo CI/contacto); es **global** (sin **SucursalId** propio) — elige sucursal al reservar cada turno.

Professional — siempre requiere **User** (1:1, obligatorio). **LicenseNumber** es la matrícula. La especialidad es many-to-many vía **ProfesionalEspecialidad**.

HorarioProfesional / PausaHorario / BloqueoFecha — disponibilidad del profesional para agenda. **HorarioProfesional** es por sucursal (índice único **ProfessionalId+SucursalId+DiaSemana**). **BloqueoFecha** son excepciones puntuales (ej. vacaciones).

Cliente — siempre cuelga de **Person** (FK único **PersonId**). No es una entidad de facturación separada: **TipoFacturacion+RazonSocial+RucCiFiscal+Direccion/Email/Telefono** son solo el dato de facturación de esa persona como cliente. Un **Cliente** y un **Patient** pueden compartir el mismo **Person** sin relación directa entre sí.

Empleado / CargoEmpleado — capa de RR.HH. sobre **User** (1:1). **SalarioBase** nullable, usado por el módulo de Egresos (**SalarioEmpleado**, Grupo C).

Sucursal / Ciudad / Departamento — **Sucursal** es la unidad de scoping multi-sucursal (ver el capítulo de Arquitectura del Backend, § Multi-sucursal). **Ciudad/Departamento** son catálogo de ubicación geográfica de Paraguay.

20.2. Grupo B — Catálogo, Inventario, Clínica y Agenda

Todo lo que no es directamente “dinero entrando o saliendo”: el catálogo de productos (incl. el catálogo óptico), el stock, la historia clínica y la agenda de turnos.

Decisión. El cristal graduado a pedido no se modela como **Producto** con stock: se resuelve como **TipoLente** (clasificación) más una colección de **Tratamientos** dentro del propio **TrabajoPedido**, y en **VentaLinea** la línea correspondiente usa **TipoLineaVenta.Lente = 2** (no descuenta stock). **CategoriaProducto.Tipo = Cristal** queda como valor legado del enum, sin ofrecerse en altas nuevas. Ver [ADR 0007](#).

20.2.1. Catálogo de productos y catálogo óptico

20.2.2. Inventario, stock y transferencias

20.2.3. Clínica: consultas y recetas

20.2.4. Agenda: turnos

20.2.5. Diccionario de datos — Grupo B

Producto — **PrecioVenta** siempre se deriva de $\text{PrecioCosto} \times (1 + \text{CategoriaProducto.Margen}/100)$, nunca se carga a mano (centralizado en **Producto.AplicarCosto**). Ver [ADR 0004](#). **Categoria** (string) es un campo legado que matchea **CategoriaProducto.Nombre** por texto — el FK real **CategoriaProductoId** existe pero conviven ambos.

ProductoStockConfig — mín/máx de stock, 1:1 con **Producto**, **global** (no por sucursal) — decisión de diseño explícita para no invalidar la relación 1:1 durante la migración multi-sucursal.

MovimientoStock / **StockActualView** — el stock no es un campo de **Producto**; se deriva sumando movimientos (**Entrada+**, **Salida-**) filtrados por **Estado = "Aprobado"**. Ver [ADR 0002](#) — [Stock derivado de movimientos](#).

StockLote / **ConteoInventario** — trazabilidad de lotes con vencimiento (originados en una recepción de mercadería) y conteos físicos periódicos con diferencia sistema-vs-físico por sucursal.

TransferenciaStock / **TransferenciaStockItem** — flujo de aprobación: crear = **Salida** en origen (**Pendiente**); aceptar = **Entrada** en destino; rechazar = **Entrada** de vuelta al origen. Solo la sucursal destino gestiona la aceptación/rechazo.

Receta — puede originarse en una **ConsultaClinica** (1:1, Cascade) o cargarse manualmente para una **Person** sin consulta previa (**PersonId**, receta “externa”). Ambos FKs son nullable. Ver [ADR 0008](#) — [Receta clínica o externa](#).

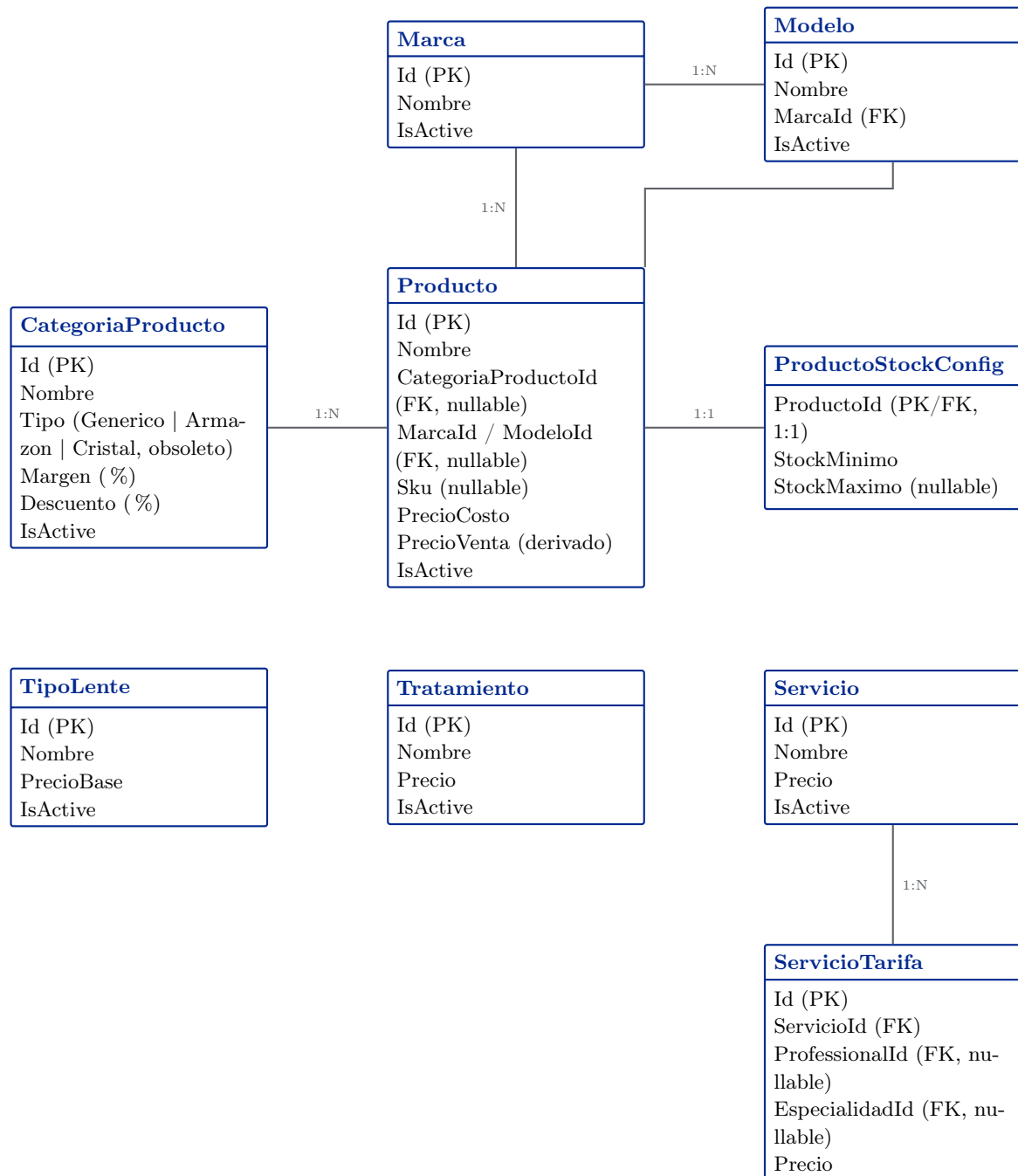


Figura 20.4: Catálogo de productos (armazones, accesorios, etc.), catálogo óptico (TipoLente/Tratamiento, sin stock propio) y catálogo de servicios con tarifas por profesional/especialidad. TipoLente y Tratamiento se consumen desde TrabajoPedido (Grupo C), no desde Producto.

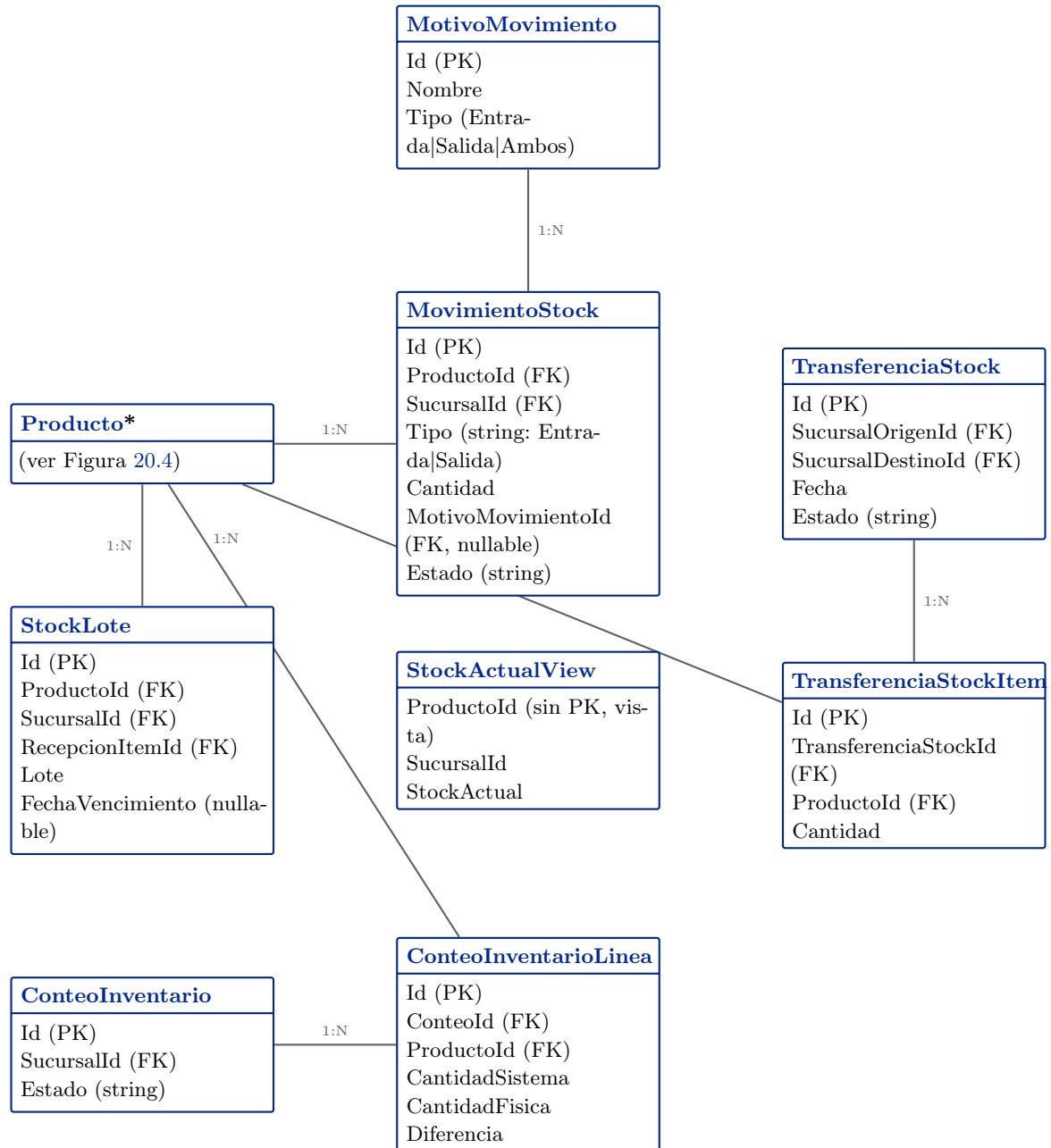


Figura 20.5: El stock no es un campo de Producto: se deriva sumando MovimientoStock filtrados por Estado=.*probado*. StockActualView mapea la vista SQL vw_stock_actual (sin PK, HasNoKey()). TransfendenciaStock mueve stock entre dos sucursales con flujo de aprobación.

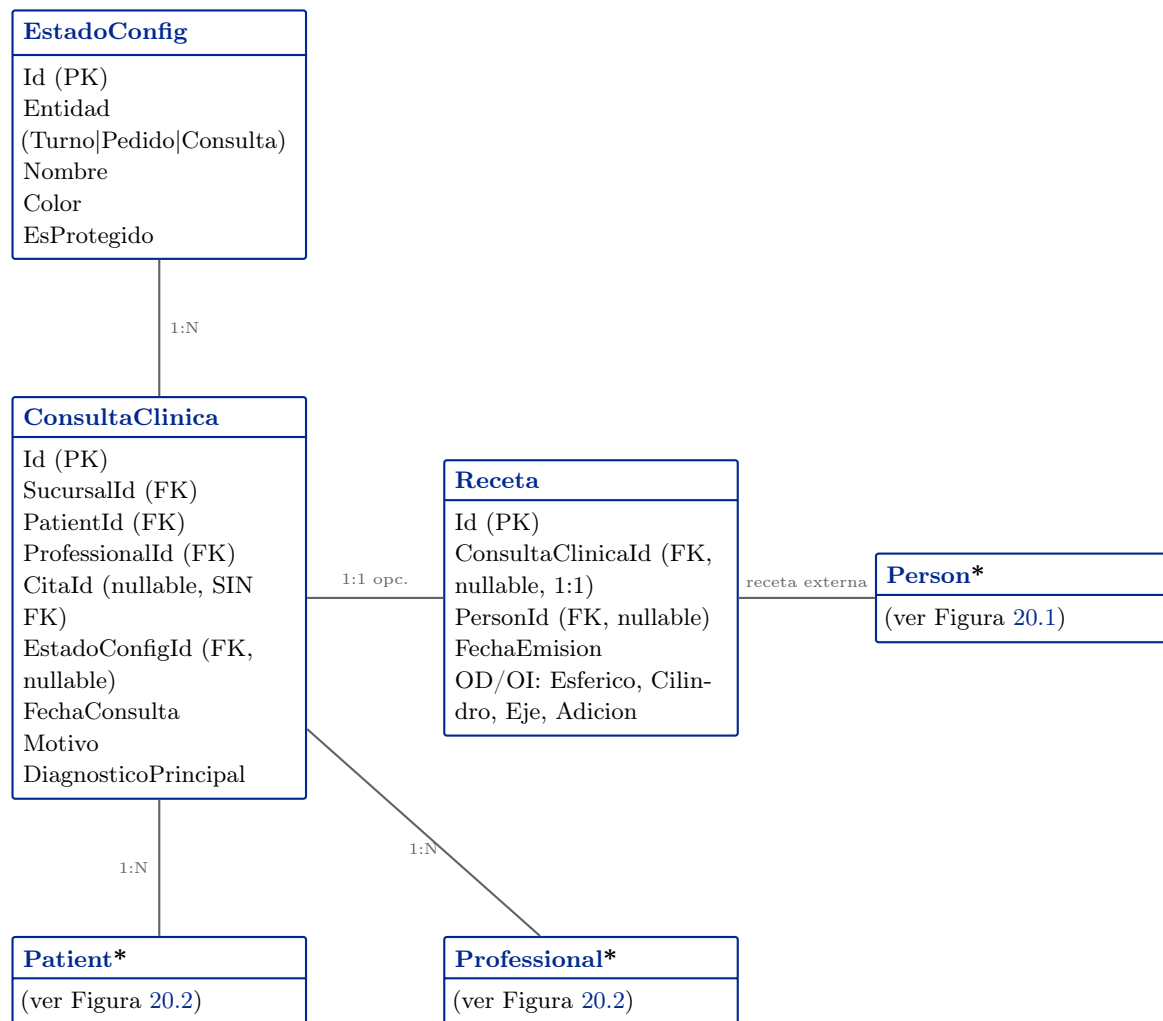


Figura 20.6: ConsultaClinica y su receta óptica opcional. Una Receta puede originarse en una consulta (1:1, Cascade) o cargarse manual para una Person sin consulta previa (receta “externa”). EstadoConfigId es un estado cosmético/configurable, no reemplaza ningún enum.

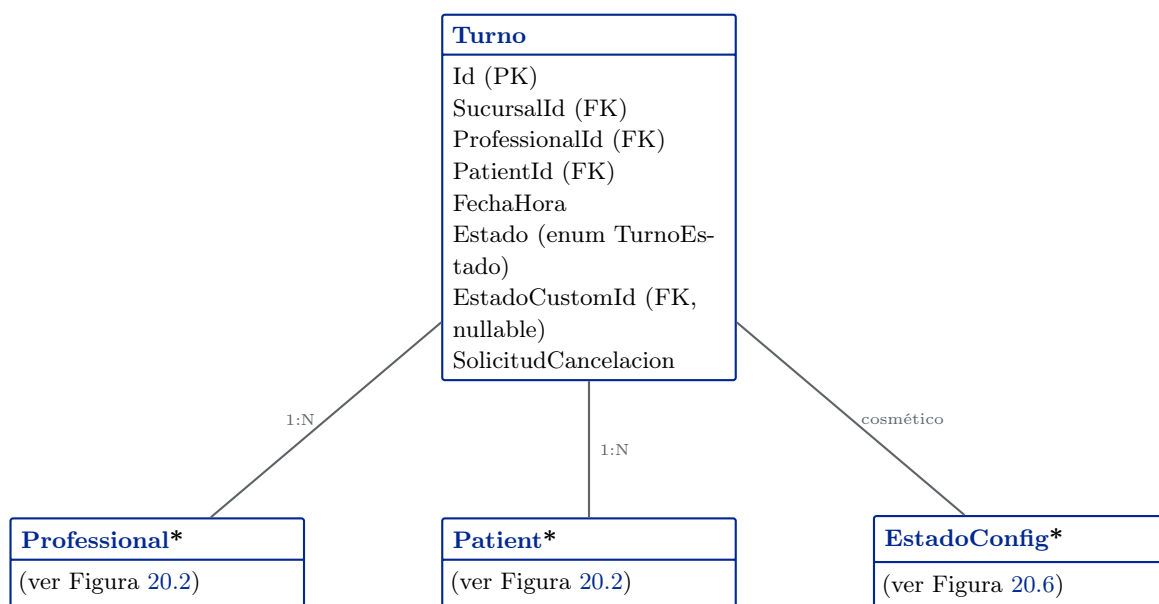


Figura 20.7: `Turno.Estado` es el enum real que gobierna la lógica de negocio; `EstadoCustomId` es puramente cosmético/configurable (color, nombre visible), no lo reemplaza.

20.3. Grupo C — Comercial: Ventas, Laboratorio, Compras, Caja y Egresos

Todo lo que mueve dinero o compromisos comerciales — el grupo más grande del modelo (27 entidades).

20.3.1. Ventas

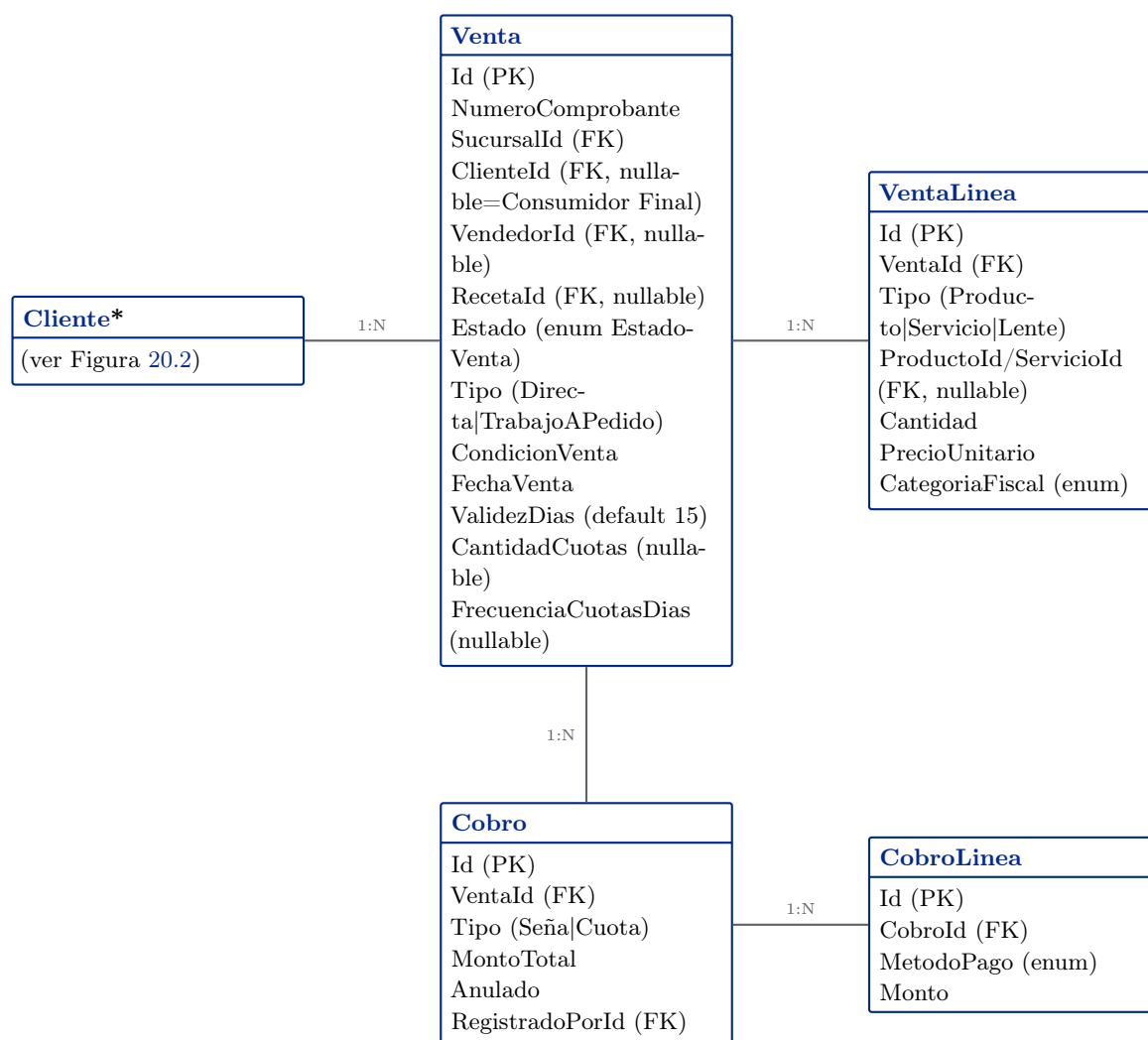


Figura 20.8: *Venta* es la cabecera única para venta directa y presupuesto/venta a pedido. Un *Cobro* puede tener varias *CobroLinea* (métodos de pago combinados).

Decisión. *EstadoVenta* recorre Borrador→Confirmada→ListaParaCobrar→ComprobanteEmitido (o Cancelada en cualquier punto): toda venta confirmada, directa o a pedido, pasa directo a *ListaParaCobrar*, mientras el ciclo del laboratorio corre en paralelo sin bloquear el cobro — ver [ADR 0006](#). Los totales (*Total*, *MontoSeñáa*, *TotalCobrado*, *SaldoPendiente*) son propiedades calculadas en memoria, no columnas.

Decisión. Una venta a crédito guarda su plan de cuotas como dos números en la propia cabecera — *CantidadCuotas* y *FrecuenciaCuotasDias* — en vez de materializar una tabla de cuotas con su vencimiento y su estado. Todo lo que el plan necesita se deriva de esos dos campos más los *Cobro* ya registrados: *MontoCuota*, *CuotasPagadas*, *ProximaCuotaVencimiento*

y `CuotaVencida` son propiedades calculadas, del mismo modo que los totales. Es el mismo criterio que gobierna el stock ([ADR 0002](#)): no guardar como dato lo que puede derivarse de los movimientos, para que no exista la posibilidad de que ambos queden en desacuerdo.

20.3.2. Comprobantes, facturación y devoluciones

20.3.3. Laboratorio óptico

Decisión. `TrabajoPedido.Estado` incluye `Borrador` para la configuración óptica de un presupuesto aún no confirmado, que no entra a la cola del laboratorio hasta que la venta se confirma. `AmazonDelCliente=true` permite un trabajo sin `AmazonProductoId` cuando el cliente trae su propio armazón. Ver [ADR 0005](#) y [ADR 0007](#).

20.3.4. Compras y proveedores

20.3.5. Egresos — herencia TPH

20.3.6. Caja

20.3.7. Diccionario de datos — Grupo C

`VentaLinea` — `Tipo = Lente` es la línea de un cristal graduado a pedido (no descuenta stock). `CategoriaFiscal` por línea permite mezclar ítems con distinto tratamiento de IVA paraguayo en la misma venta.

`Cobro / CobroLinea` — un `Cobro` puede tener múltiples `CobroLinea` (métodos de pago combinados, ej. parte efectivo + parte tarjeta). `Tipo` distingue seña (venta a crédito, primer pago) de cuota (pagos siguientes).

`Proveedor.EsLaboratorio` — el mismo catálogo de proveedores sirve tanto para compras de mercadería como para laboratorios ópticos externos; `TrabajoPedido.LaboratorioProveedorId` filtra por este flag.

`PedidoProveedor` → `RecepcionMercaderia` → `FacturaCompra` — ciclo de compras completo: se crea la OC, se registran una o más recepciones parciales (cada una con sus ítems, que pueden generar `StockLote`), y opcionalmente se registra la factura del proveedor (ligada o no a la OC vía `PedidoProveedorId` nullable).

`Egreso` — ver [Figura 20.12: FacturaCompra : Egreso](#) no es un error de modelado — una factura de compra a proveedor *es* (no “tiene”) un egreso.

`MovimientoCaja / SesionCaja` — cada `MovimientoCaja` puede originarse en una `Venta` (cobro), un `Egreso` (pago) o ser manual, y siempre pertenece a una `SesionCaja`.

20.4. Entidades transversales

`ConfiguracionNegocio` — fila única (singleton lógico) con los datos fiscales/de contacto del negocio (`NombreFantasia`, `RazonSocial`, `RUC`, etc.), usada por generación de PDFs y comprobantes. Sin FKs.

`NotificacionInterna` — centro de notificaciones internas al staff. Destinatario por `DestinatarioUsuarioId` (individual, nullable), `DestinatarioSucursalId` (broadcast a staff de esa sucursal, nullable) o ambos null (broadcast global). `EntidadOrigenTipo/EntidadOrigenId` referencian polimórficamente la entidad que disparó la notificación (sin FK real — referencia débil por diseño). `Leido` es un único flag compartido para notificaciones de broadcast, no por-usuario. Ver [ADR 0010](#).

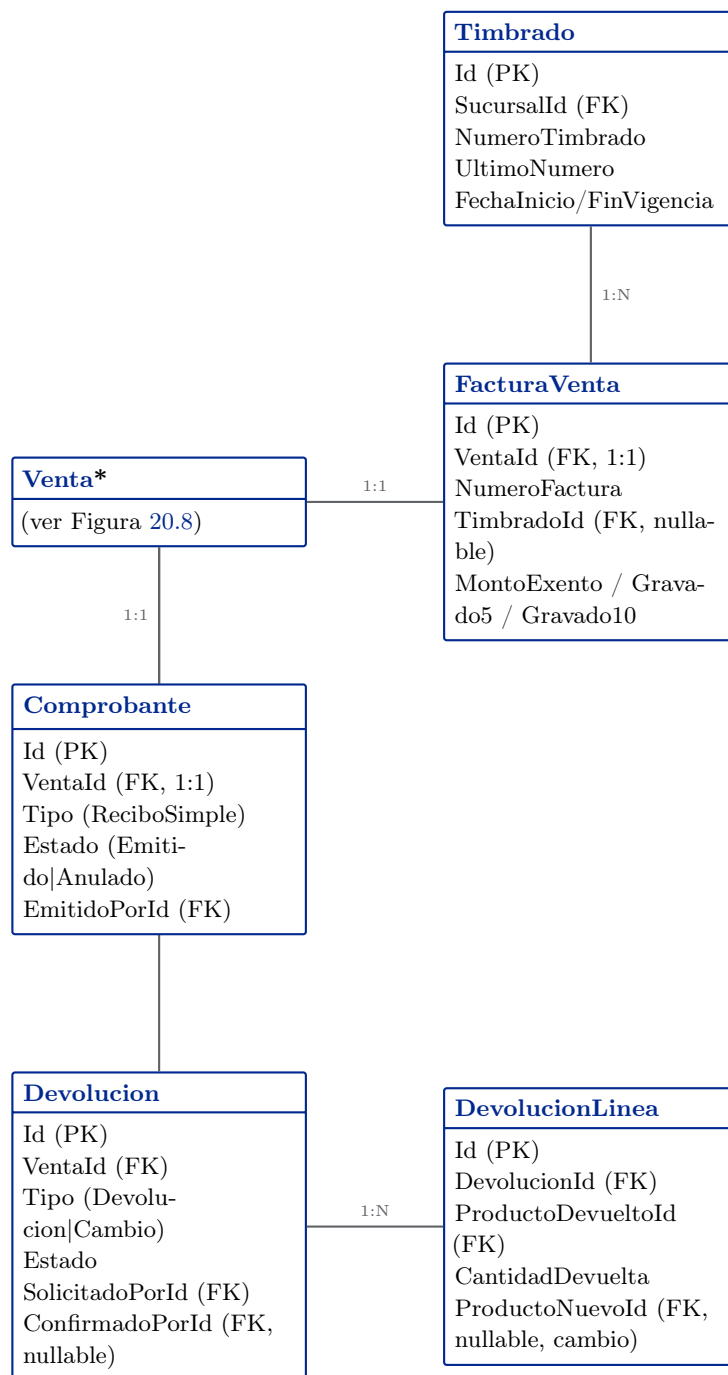


Figura 20.9: FacturaVenta y Comprobante son dos documentos distintos, ambos 1:1 con Venta (Cascade). FacturaVenta está ligada a un Timbrado vigente de la sucursal.

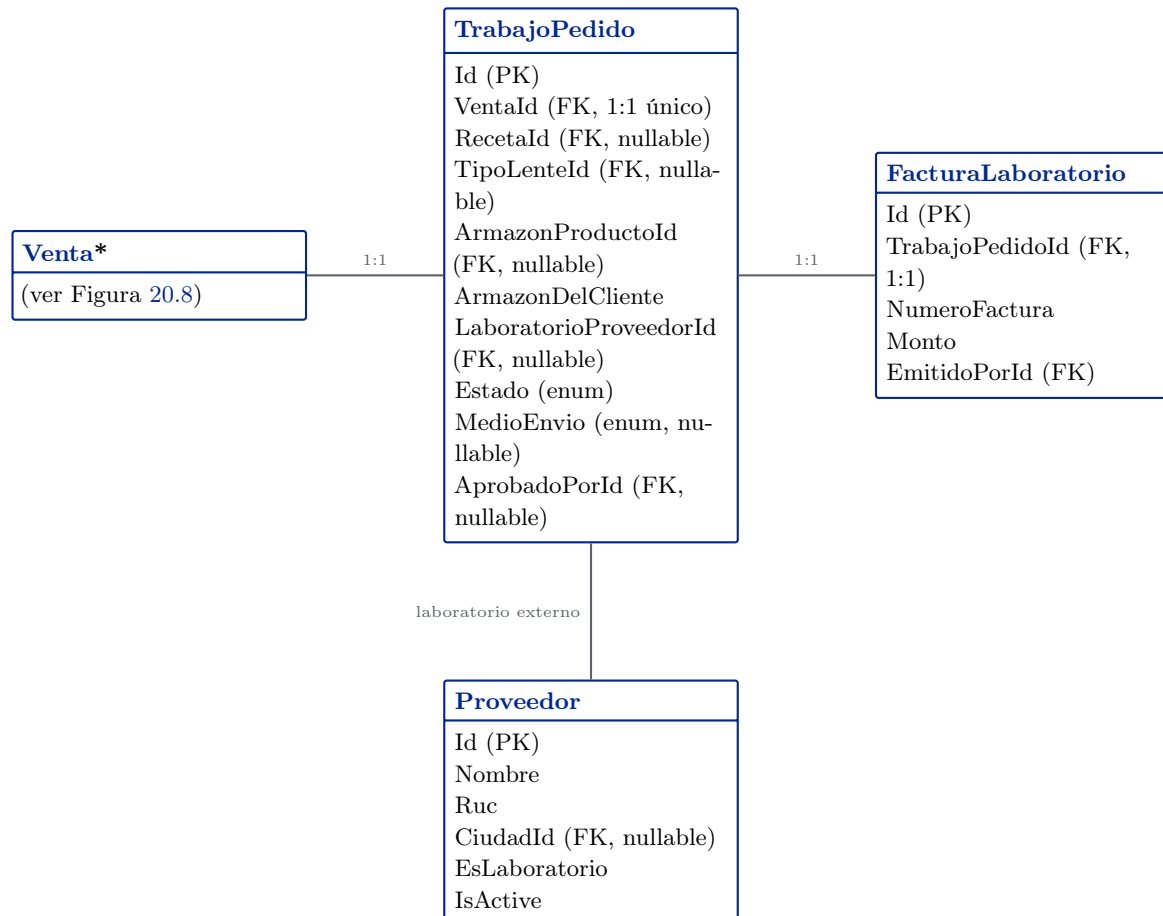


Figura 20.10: TrabajoPedido es el núcleo del flujo “óptica a pedido”. El mismo catálogo de Proveedor sirve tanto para compras de mercadería como para laboratorios ópticos externos (EsLaboratorio).

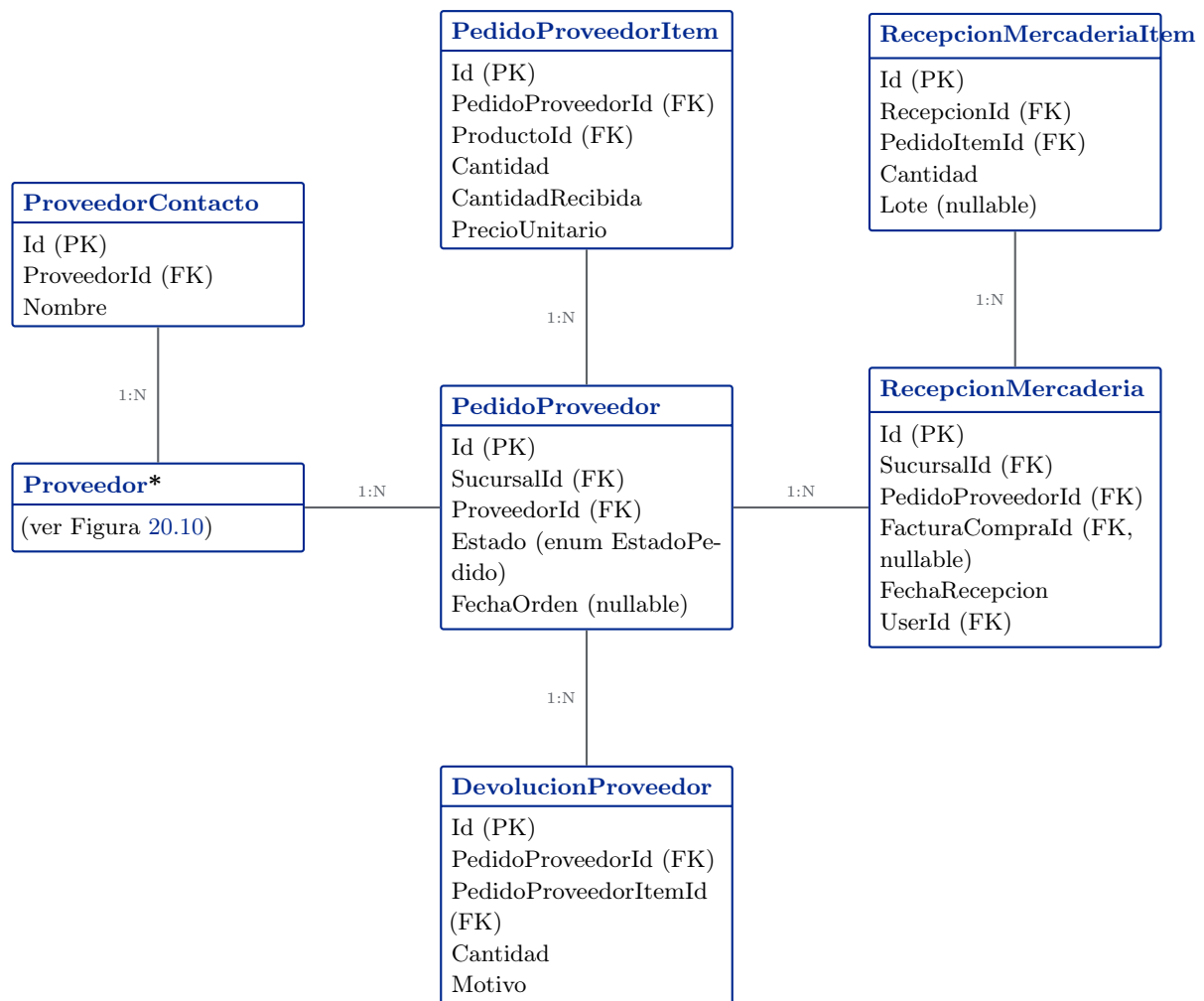


Figura 20.11: Ciclo de compras: OC (**PedidoProveedor**, Borrador → Confirmada → RecibidaParcial/Total → Facturada, o Cancelada) → una o más recepciones parciales → factura del proveedor opcional (ver Figura 20.12).

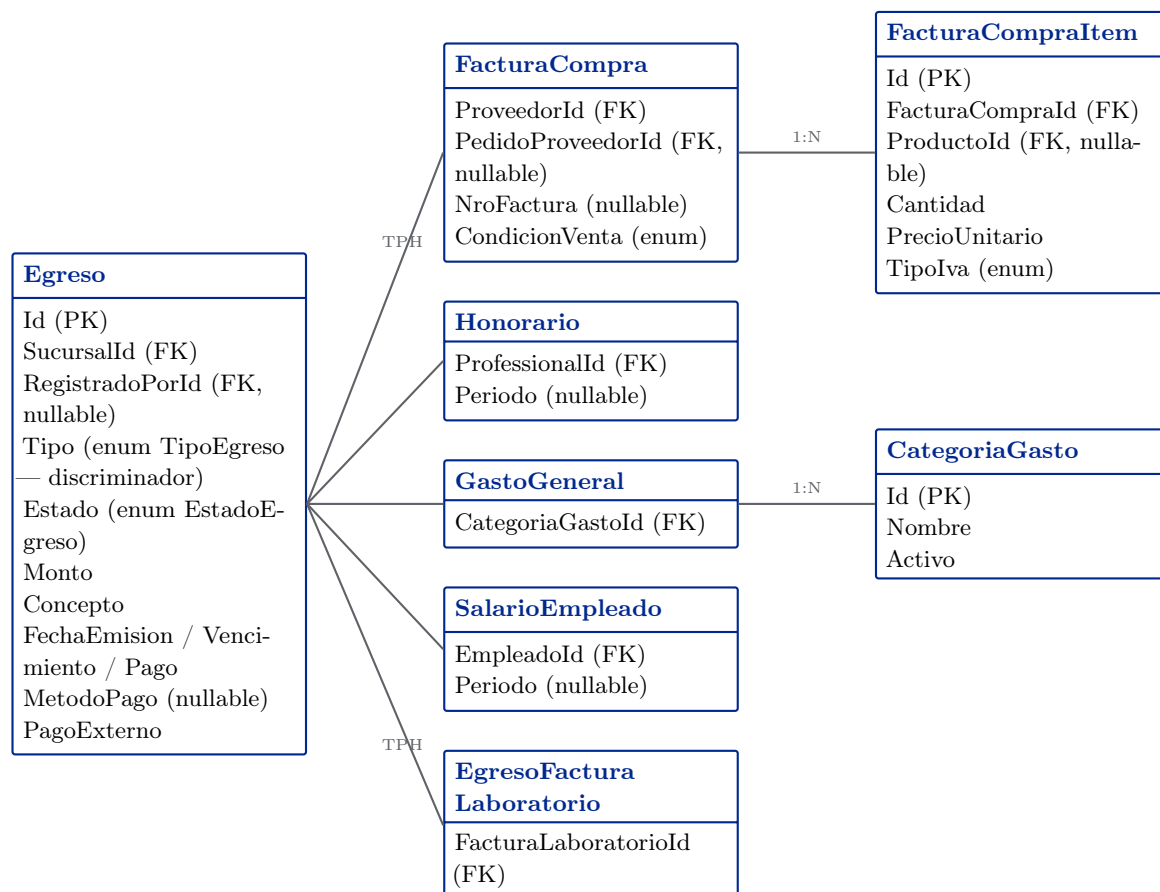


Figura 20.12: Egreso es la clase base abstracta (Table-Per-Hierarchy) de toda salida de dinero, mapeada a una única tabla física **egresos** con columna discriminadora **Tipo**. Los 5 subtipos solo agregan 1-3 columnas específicas y comparten todo el resto de la base. Ver [ADR 0011 — Egreso como jerarquía TPH](#).

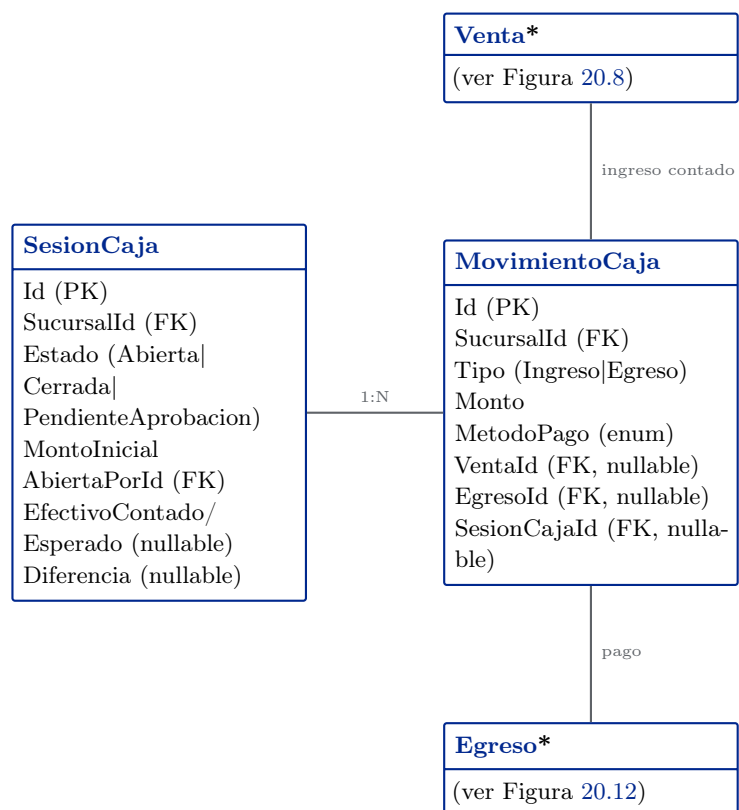


Figura 20.13: *SesionCaja* es “una caja abierta por sucursal a la vez”; cierre con conteo físico vs. esperado y flujo de aprobación si hay diferencia. Cada *MovimientoCaja* se origina en una *Venta*, un *Egreso*, o es manual.

20.5. Tabla de enums

Cuadro 20.1: Los 28 enums de negocio del modelo, guardados como `int` vía `HasConversion<int>()`.

Enum	Valores	Usado en
EstadoVenta	Borrador=1, Confirmada=2, EnProceso=3 (sin uso en ventas nuevas), ListaParaCobrar=4, ComprobanteEmitido=5, Cancelada=6	Venta.Estado
TipoVenta	Directa=1, TrabajoAPedido=2	Venta.Tipo
CondicionVenta	Contado=0, Credito=1	Venta.CondicionVenta, FacturaCompra, CondicionVenta
TipoLineaVenta	Producto=0, Servicio=1, Lente=2	VentaLinea.Tipo
CategoriaFiscal	Exento=0, Gravado5=1, Gravado10=2	VentaLinea.CategoriaFiscal
TipoCobro	Seña=1, Cuota=2	Cobro.Tipo
MetodoPago	Efectivo=0, Tarjeta=1, Transferencia=2, Cheque=3	CobroLinea, MovimientoCaja, Egreso
TipoComprobante / EstadoComprobante	ReciboSimple=1 / Emitido=1, Anulado=2	Comprobante
TipoDevolucion / EstadoDevolucion	Devolucion=1, Cambio=2 / Pendiente=1, Confirmada=2, Rechazada=3	Devolucion
EstadoTrabajo Pedido	PendienteAprobacion=0, PendienteEnvio=1, Enviado=2, Recibido=3, Rechazado=4, Borrador=5	TrabajoPedido.Estado
MedioEnvio Laboratorio	WhatsApp=0, Email=1, Portal=2, Telefono=3, EnPersona=4, Otro=5	TrabajoPedido.MedioEnvio
EstadoPedido	Borrador=0, Confirmada=1, RecibidaParcial=2, RecibidaTotal=3, Cancelada=4, Facturada=5	PedidoProveedor.Estado
TipoIvaFactura	ver <code>TipoIvaFactura.cs</code> (incluye <code>Iva10</code> como default)	FacturaCompraItem.TipoIva
TipoEgreso	FacturaCompra=0, Honorario=1, GastoGeneral=2, Salario=3, FacturaLaboratorio=4	Egreso.Tipo (discriminador TPH)
EstadoEgreso	Borrador=0, Pendiente=1, Pagado=2, Anulado=3, Aprobado=4, Rechazado=5	Egreso.Estado
TipoMovimiento Caja	Ingreso=0, Egreso=1	MovimientoCaja.Tipo
EstadoSesionCaja	Abierta, Cerrada, PendienteAprobacion	SesionCaja.Estado
TurnoEstado	Pendiente=0, Completado=1, Cancelado=2, Confirmado=3, Presente=4	Turno.Estado

Enum	Valores	Usado en
TipoCategoria Producto	Generico=0, Amazon=1, Cristal=2 (obsoleto)	CategoriaProducto.Tipo
TipoFacturacion	Fisica, Juridica	Cliente. TipoFacturacion
TipoMovimiento Stock	Entrada=0, Salida=1, Ajuste=2	MovimientoStock.Tipo
EstadoMovimiento Stock	Pendiente=0, Aprobado=1, Rechazado=2	MovimientoStock.Estado
EstadoConteo Inventario	Pendiente=0, Aprobado=1, Rechazado=2	ConteoInventario. Estado
Estado Transferencia Stock	Pendiente=0, Aceptada=1, Rechazada=2	TransferenciaStock. Estado
TipoMotivo Movimiento	Entrada=0, Salida=1, Ambos=2	MotivoMovimiento.Tipo
TipoNotificacion	BajoStock=0, TransferenciaPendiente=1, PedidoLabRecibido=2	NotificacionInterna. Tipo

20.6. Índices, constraints y convenciones de tipos

Las convenciones de mapeo las fija `Persistence/Configurations/*.cs`:

- **26 índices únicos** (`HasIndex(...).IsUnique()`) en todo el modelo. Los más relevantes: `persons.CI`, `persons.Email` (con `AreNullsDistinct(true)` — permite múltiples NULL), `professionals.LicenseNumber` y `professionals.UserId`, `users.PersonId` y `patients.PersonId`, `timbrados` compuesto (`NumeroTimbrado`, `Establecimiento`, `PuntoExpedicion`), `turnos` (`ProfessionalId`, `FechaHora`), `horarios_profesional` (`ProfessionalId`, `SucursalId`, `DiaSemana`), `bloqueos_fecha` (`ProfessionalId`, `Fecha`), `sucursales.Codigo`, `permissions` (`Entidad`, `Nombre`).
- **FKs con OnDelete explícito**: 56 `Restrict` (default defensivo, no deja borrar un padre con hijos), 19 `Cascade` (reservado para hijos de composición: líneas de venta, de pedido, etc.) y 12 `SetNull` (el hijo sobrevive sin el padre). EF autoindexa cada FK.

Dinero se mapea siempre a `numeric`, nunca a `float/double`, y con una escala uniforme: `numeric(18,0)` en las 21 columnas monetarias del modelo. El Guaraní no tiene centavos, así que ninguna de ellas admite decimales, y el frontend trata el dinero como entero en consecuencia. Queda una sola excepción: `Tratamiento.Precio`, que sigue en `numeric(14,2)`. Los porcentajes, que sí necesitan decimales, van en `numeric(5,2)`.

- **Fechas/horas**: `DateTime` mapea a `timestamp with time zone` (UTC) en todo el modelo; fechas puras (`BirthDate`, campos `Fecha`) mapean a `date`; horas de disponibilidad a `time`.
- **Auditoría** (`CreatedAt/UpdatedAt`) se setea en la aplicación (`DateTime.UtcNow` en la entidad), no hay `DEFAULT now()` ni trigger en la base — un `INSERT/UPDATE` por SQL crudo deja esas columnas sin valor.

- **Soft delete** (`IsActive`) es manual, sin query filter global — cada servicio decide si filtra por `IsActive`; no asumir que una consulta ya excluye inactivos.
- **Enums de negocio** se guardan como `int` (31 llamadas a `HasConversion<int>()` repartidas en 21 archivos de configuración). No hay `CHECK` en la base que valide el rango — un valor fuera de rango entra sin error por SQL crudo. Tampoco hay `CHECK` para montos ≥ 0 ni stock no negativo.

20.7. Patrones de diseño transversales

- **Herencia TPH** (`Egreso` y sus 5 subtipos) — única jerarquía de herencia real del modelo; todo lo demás es composición/FK plana.
- **Vistas SQL como entidad de solo lectura** (`StockActualView` $\rightarrow$ `vw_stock_actual`, `HasNoKey()`) — patrón a reutilizar si aparecen más cálculos agregados que no deben vivir como columna física.
- **Estados siempre como enum** — todo campo de estado o de tipo del modelo es un enum `C#` persistido como `int` vía `HasConversion<int>()`, tanto en los módulos comerciales (`Venta`, `TrabajoPedido`, `PedidoProveedor`, `Egreso`) como en inventario (`MovimientoStock`, `ConteoInventario`, `TransferenciaStock`, `MotivoMovimiento`) y notificaciones (`NotificacionInterna`). El conjunto de valores válidos queda definido en un solo lugar — el tipo — en vez de depender de que cada servicio escriba la cadena correcta.
- **Precio siempre derivado, nunca manual** — `Producto.PrecioVenta` (por margen de categoría). Ver [ADR 0004](#).
- **Scoping por sucursal** — la mayoría de entidades transaccionales de los Grupos B y C tienen `SucursalId` (nullable solo donde la entidad es intencionalmente global). Ver el capítulo de Arquitectura del Backend, § Multi-sucursal.
- **Person como raíz de identidad** — todo rol funcional (`User`, `Patient`, `Cliente`) cuelga de `Person`, nunca al revés.

Capítulo 21

Implementación

21.1. Requisitos

El sistema se despliega como un conjunto de contenedores Docker sobre un servidor Linux. El anteproyecto fijó como restricción de hardware un mínimo de 4 núcleos, 16 GB de RAM y 1 TB de almacenamiento; sobre esa base, el ambiente de producción real es un VPS (Contabo) con Ubuntu 24.04 LTS.

Software requerido en el servidor:

- Docker Engine y Docker Compose Docker Inc., 2026 (plugin `compose`).
- Ningún otro runtime — .NET, Node y PostgreSQL corren únicamente dentro de sus propios contenedores, no instalados en el host.

21.2. Instalación

El despliegue se define en `docker-compose.prod.yml`, con cuatro servicios sobre una red interna de Docker (`siganet`):

Cuadro 21.1: Servicios del stack de producción.

Servicio	Rol
db	PostgreSQL PostgreSQL Global Development Group, 2026 16. Sin puerto publicado — solo accesible desde la red interna de Docker, nunca directamente desde internet
api	Backend ASP.NET Core, compilado en una imagen multi-stage (SDK de .NET 10 para build, runtime ASP.NET para ejecución). Tampoco publica puerto — solo se alcanza a través de <code>caddy</code>
web	Frontend Vue 3, compilado a estático y servido por su propio contenedor
caddy	Proxy inverso de borde. Único servicio con puertos publicados al exterior (80 y 443)

El repositorio del frontend (`SIGA-Web`) se espera como directorio hermano del backend en el servidor, ya que la imagen de `web` se construye con contexto relativo `../SIGA-Web`. La instalación consiste en clonar ambos repositorios, completar el archivo de variables de entorno y levantar el stack:

```

1 cd ~/siga/SIGA
2 cp .env.example .env # completar los valores
3 docker compose -f docker-compose.prod.yml up -d --build

```

21.3. Configuración

Toda la configuración de producción se resuelve por variables de entorno, sin archivos de configuración editados a mano dentro de las imágenes:

Cuadro 21.2: Variables de entorno del stack de producción.

Variable	Uso
POSTGRES_DB/_USER/_ PASSWORD	Credenciales de la base de datos, compartidas entre <code>db</code> y <code>api</code>
JWT_SECRET	Clave de firma de los tokens de autenticación (JWT Jones et al., 2015)
DOMAIN / ACME_EMAIL	Dominio público del sistema y correo de contacto para el certificado TLS
SEED_ADMIN_EMAIL/_ PASSWORD	Credenciales de la cuenta administradora inicial, creadas por el <i>seeder</i> en el primer arranque
RESEND_API_KEY/_FROM_ EMAIL/_FROM_NAME	Envío de emails transaccionales (verificación de cuenta, recuperación de contraseña, recordatorios)
HCAPTCHA_SECRET / VITE_HCAPTCHA_SITE_ KEY	Protección anti-bot en formularios públicos (backend y frontend, respectivamente)

Caddy Caddy, 2026 resuelve dos responsabilidades de borde sin configuración manual adicional: emite y renueva automáticamente el certificado TLS del dominio configurado (protocolo ACME, Let's Encrypt), y enruta cada request según su ruta — `/api/*` y `/uploads/*` al backend, cualquier otra ruta a la aplicación Vue servida como *single-page application* con *fallback* a `index.html` para que el enrutamiento del lado del cliente funcione en una recarga de página directa sobre una ruta interna.

Capítulo 22

Validación

Este capítulo describe cómo se validó el sistema: la metodología de prueba empleada y los resultados obtenidos módulo por módulo.

22.1. Metodología

La validación de cada módulo se realizó de forma manual y end-to-end, contra la base de datos real de desarrollo (PostgreSQL, Neon) y con el sistema corriendo localmente: el backend expuesto vía `dotnet run` y ejercitado con peticiones HTTP autenticadas (token JWT real de un usuario), y el frontend ejercitado con Playwright conduciendo un navegador real sobre los flujos de pantalla correspondientes.

Sobre esa base, cada módulo se validó con dos criterios: que la operación produjera el efecto esperado en la base de datos (no solo una respuesta HTTP exitosa), y que ese efecto fuera visible correctamente en la interfaz.

22.2. Casos de prueba y resultados

22.2.1. Aislamiento multi-sucursal

Se creó una segunda sucursal de prueba y un usuario asociado a ella, con datos operativos reales cargados en ambas sucursales. Resultado: el usuario de la nueva sucursal no ve en los listados los registros de la otra sucursal, y una consulta directa por identificador de un recurso de la otra sucursal devuelve *no encontrado* en vez de exponer o permitir modificar el dato. El administrador (sin sucursal fija) sigue viendo y operando sobre ambas sin restricción. Una transferencia de stock entre las dos sucursales resulta visible desde ambas puntas, como corresponde a una operación que involucra a las dos. Los procesos de fondo del sistema (recordatorio de turnos, aviso de stock bajo) siguen evaluando todas las sucursales sin regresión.

22.2.2. Reglas de complejidad y reuso de contraseña

Sobre el endpoint de cambio de contraseña se ejecutaron los cinco casos que cubren la regla de negocio completa: contraseña nueva demasiado corta (rechazada), sin ningún número (rechazada), sin ninguna letra (rechazada), igual a la contraseña actual (rechazada), y una contraseña válida y distinta de la actual (aceptada). Los cinco casos devolvieron el resultado esperado.

22.2.3. Migración de estados de texto libre a enumerados

Antes de aplicar la migración que convierte los campos de estado de texto libre a tipos enumerados, se verificaron contra la base real los valores efectivamente almacenados en cada columna afectada, para asegurar que el mapeo de cada valor de texto a su nuevo valor numérico fuera exhaustivo y correcto. Tras aplicar la migración, los flujos de negocio que dependen de esos

estados (movimientos de stock, conteos de inventario, transferencias) se ejercitaron de punta a punta y devolvieron exactamente el mismo formato de respuesta que antes del cambio.

22.2.4. Unificación de la escala monetaria

Antes de reducir la escala de las columnas monetarias de dos decimales a cero decimales, se ejecutó una consulta de solo lectura sobre la base real para confirmar que ninguna de esas columnas tuviera efectivamente valores no enteros almacenados. El resultado confirmó que la migración no perdía información en ningún registro existente.

22.2.5. Reportes operativos

Se ejercitó la exportación de los cuatro tipos de reporte operativo (ventas, compras, inventario, caja) en sus dos formatos disponibles (PDF y CSV), ocho combinaciones en total. Las ocho devolvieron contenido real: los PDF con estructura de archivo válida y una o más páginas según el volumen de datos, los CSV con filas de datos reales.

22.2.6. Preferencias de notificación

Se verificó el flujo completo de preferencias de notificación por usuario: lectura y actualización directa contra el backend con un token real, y desde la interfaz — tanto el modal de autoservicio accesible desde cualquier cuenta como la pestaña dedicada en la ficha de un paciente, visible solo para quien tiene el permiso correspondiente —, confirmando en ambos casos que el cambio persiste tras recargar la página.

22.2.7. Aislamiento por sucursal de la receta óptica

Al incorporar la receta óptica al esquema de aislamiento por sucursal, se verificó con datos reales que cada sucursal ve únicamente las recetas que le corresponden, que una receta nueva cargada manualmente en una sucursal aparece correctamente contabilizada en esa sucursal, y que — a la vez — una receta sigue siendo accesible desde cualquier sucursal cuando el flujo de negocio lo requiere: un cliente que sacó su receta en una sucursal puede comprar los lentes en otra.

22.3. Capturas de evidencia

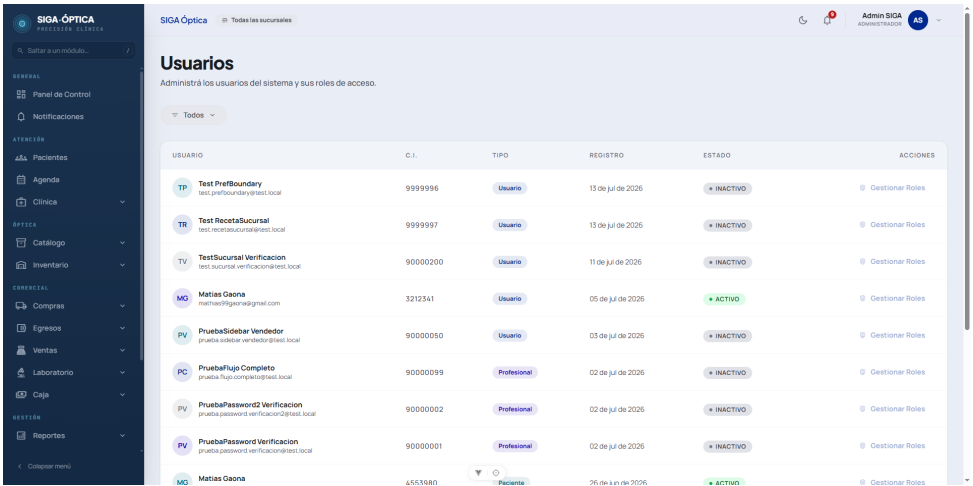


Figura 22.1: Sesión autenticada como administrador — visibilidad completa de usuarios de ambas sucursales, sin la restricción que sí aplica a un usuario con sucursal fija.

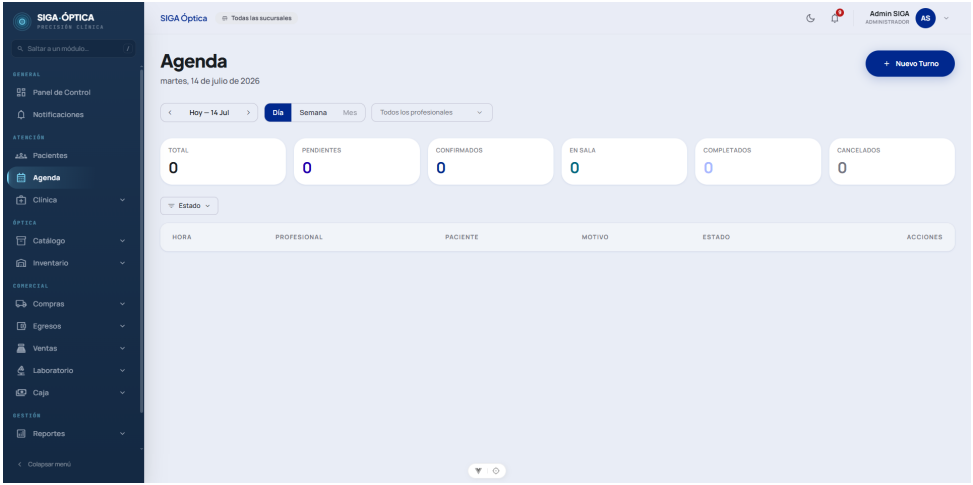


Figura 22.2: Agenda de turnos verificada end-to-end: creación, confirmación y cancelación de citas.

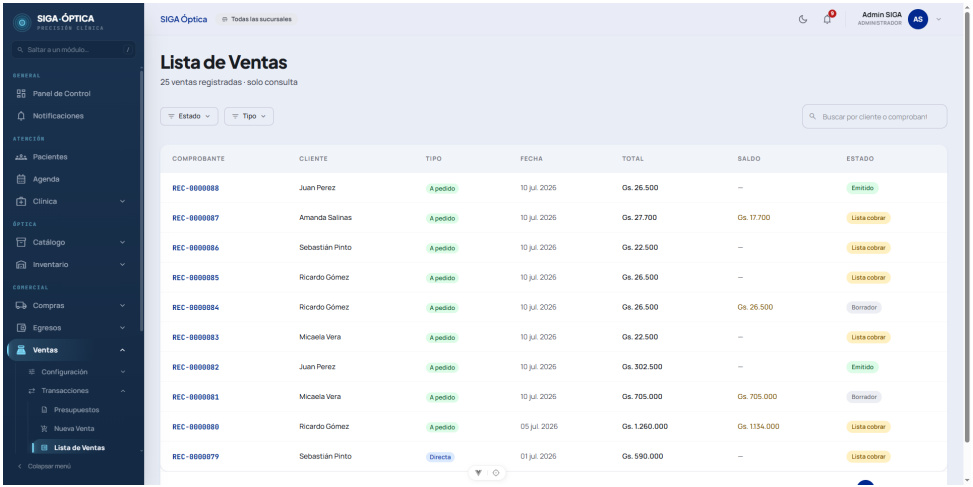


Figura 22.3: Listado de ventas verificado tras ejercitar el ciclo completo de venta directa y venta a pedido, incluyendo presupuesto, confirmación, cobro y emisión de comprobante.

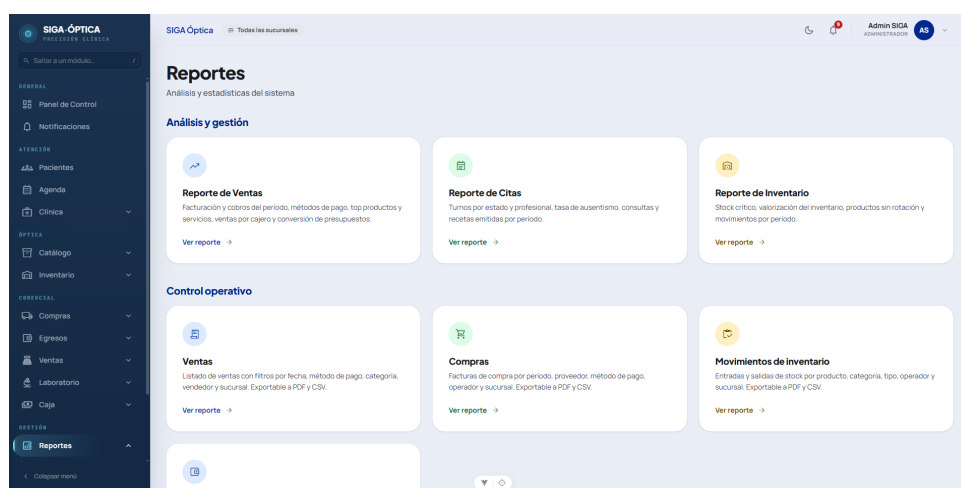


Figura 22.4: Panel de reportes desde el cual se ejercitaron las ocho combinaciones de exportación (cuatro reportes operativos por dos formatos) descritas en la Sección 22.2.5.

Capítulo 23

Cambios respecto al Proyecto I

Proyecto I definió el problema, los objetivos y un diseño preliminar de entidades y casos de uso para SIGA-Óptica. Durante la construcción del software (Proyecto II) algunas de esas decisiones iniciales se revisaron a la luz de reglas de negocio reales o de restricciones que no eran evidentes en la etapa de anteproyecto. Este capítulo documenta esos cambios: qué se planteó originalmente, qué problema apareció al construirlo, qué solución se adoptó y por qué.

23.1. Sucursal: de atributo simple a mecanismo de aislamiento transversal

Diseño inicial. Proyecto I modelaba `Sucursal` como un atributo más de `User`: “unidad/ubicación operativa a la que pertenece el usuario”. El diagrama de clases no le asignaba responsabilidad más allá de esa referencia.

Problema encontrado. Al construir los módulos operativos (ventas, compras, caja, inventario, agenda) quedó claro que una sucursal no es solo un dato descriptivo del usuario: cada venta, cada movimiento de caja, cada movimiento de stock y cada sesión de caja pertenecen a una sucursal concreta, y un usuario de una sucursal no debe poder ver ni menos aún modificar los datos operativos de otra.

Solución adoptada. `Sucursal` se convirtió en una dimensión transversal del modelo de datos. Quince entidades llevan su propio `SucursalId` (`Venta`, `MovimientoCaja`, `MovimientoStock`, `SesionCaja`, `PedidoProveedor`, `ConsultaClinica`, entre otras), y el acceso a esas entidades se filtra automáticamente por la sucursal del usuario autenticado mediante un *Global Query Filter* de Entity Framework Core, descrito en el Capítulo 19.

Justificación del cambio. Un filtro centralizado a nivel de `DbContext` garantiza el aislamiento en todas las rutas de lectura y escritura de una sola vez, en lugar de depender de que cada servicio recuerde aplicar el filtro a mano en cada consulta — el patrón original, más liviano, exigía justamente esa disciplina manual y no la garantizaba.

23.2. Reposición de stock: de pedido automático a notificación

Diseño inicial. Proyecto I planteaba que el módulo de inventario, al alcanzar el stock un nivel mínimo, “genere alertas” y “facilite la reposición con proveedores” mediante “pedidos automáticos”.

Problema encontrado. Generar automáticamente un pedido de compra implica decidir por el usuario — cantidad a pedir, proveedor elegido, condición de pago — decisiones de negocio que en una óptica real las toma una persona, no un proceso desatendido. Automatizar esa decisión sin supervisión humana arriesgaba generar pedidos incorrectos o innecesarios.

Solución adoptada. El sistema detecta el stock bajo mínimo de forma automática y periódica, y genera una notificación interna dirigida a la sucursal correspondiente. La creación del pedido de

compra en sí sigue siendo una acción manual del usuario, hecha desde el módulo de Compras (Capítulo 14) a partir de esa alerta.

Justificación del cambio. La automatización se aplicó a la parte mecánica y repetible del problema — detectar el cruce bajo el mínimo, algo que ningún usuario debería tener que revisar manualmente — y se dejó en manos de una persona la parte que requiere criterio de negocio: qué comprar, a quién y en qué condiciones.

23.3. Descuento de stock: del momento de entrega al momento de emisión del comprobante

Diseño inicial. La intuición de diseño más directa — y la primera implementada — descontaba el stock vendido en el momento en que la mercadería se entregaba físicamente al cliente.

Problema encontrado. Entre la confirmación de una venta y la entrega efectiva del producto puede pasar tiempo (especialmente en una venta “a pedido”, donde el cristal se fabrica en un laboratorio externo), y en ese intervalo el comprobante fiscal ya fue emitido y el cobro ya pudo haberse realizado. Descontar el stock recién en la entrega dejaba una ventana en la que el sistema mostraba disponible un producto que ya estaba fiscal y comercialmente comprometido.

Solución adoptada. El descuento de stock de las líneas de producto de una venta ocurre en el mismo paso que la emisión del comprobante o factura (`Venta.Estado` pasa a `Comprobante Emitido`), no en un paso de entrega separado. El flujo completo está descrito en el Capítulo 12, §12.5.

Justificación del cambio. Emitir el comprobante es el punto del proceso en el que la venta queda formalizada — fiscal y contablemente — y es, en consecuencia, el punto correcto para que el inventario refleje ese compromiso. Atarlo a la entrega física introducía una inconsistencia entre lo que el sistema fiscal ya daba por vendido y lo que el módulo de inventario todavía mostraba disponible.

23.4. Cancelación de citas: de acción autónoma del paciente a solicitud resuelta por el personal

Diseño inicial. Proyecto I planteaba que el paciente pudiera “programar, confirmar, reprogramar y cancelar citas de manera autónoma”, y su especificación de caso de uso ponía al Paciente como actor principal de la cancelación, junto al Vendedor y al Administrador.

Problema encontrado. Una cancelación no es un evento aislado: libera un horario del profesional, puede requerir avisar a alguien de una lista de espera y, si ocurre sobre la hora, tiene un costo real para la óptica. Dejar que el paciente cancele directamente desde el portal significaba que un turno podía desaparecer de la agenda sin que nadie del negocio se enterara ni pudiera intervenir.

Solución adoptada. La cancelación quedó en dos pasos: el paciente *solicita* cancelar desde el portal (queda marcada la solicitud sobre el turno) y el personal de la óptica la resuelve, aprobándola o rechazándola desde la agenda. El paciente nunca cancela directamente su propio turno. El flujo está descrito en el Capítulo 8.

Justificación del cambio. El paciente conserva la capacidad de iniciar la cancelación sin llamar por teléfono — que era el problema real que el objetivo buscaba resolver — pero la óptica mantiene visibilidad y control sobre su propia agenda. El costo es un paso adicional en un flujo poco frecuente; el beneficio es que ningún turno se cancela sin que el negocio lo sepa.

23.5. Interfaz de usuario: componentes propios en vez de una librería de componentes

Diseño inicial. El cronograma de Proyecto I fijaba como entorno de frontend “Vue 3, Vite, Vuetify, Axios”, es decir una librería de componentes ya armada (Vuetify) sobre la que montar las pantallas.

Problema encontrado. Vuetify impone su propio lenguaje visual (Material Design) y su propio sistema de temas. Uno de los objetivos específicos era una interfaz amigable y consistente para el Centro Óptico Santa María, y adoptar la identidad visual de la librería significaba, en la práctica, pelearse con sus estilos por defecto cada vez que el diseño requerido no coincidía con ellos.

Solución adoptada. El frontend no usa ninguna librería de componentes: las pantallas se arman con Tailwind CSS y un conjunto de componentes propios (`BaseButton`, `BaseModal` y demás), sobre un sistema de tokens de diseño propio que soporta tema claro y oscuro. El detalle está en el Capítulo 4.

Justificación del cambio. Con un catálogo de componentes acotado y repetitivo — formularios, tablas, modales — el trabajo de construirlos una vez resultó menor que el de sobrecribir permanentemente los de una librería, y dejó control total sobre la identidad visual del sistema. El resto del stack planteado (Vue 3, Vite, Axios) se mantuvo tal cual.

23.6. Notificaciones: un solo canal real en vez de multicanal desde el inicio

Diseño inicial. Proyecto I definía la notificación automática como un mensaje enviado “a través de canales digitales (correo, WhatsApp o SMS)”.

Problema encontrado. Integrar WhatsApp y SMS implica contratar y configurar proveedores externos de mensajería que la óptica no tenía contratados, y construir esa integración sin un canal real para probarla en producción hubiera dejado código sin ejercitar — estructura pensada para una funcionalidad que en los hechos nadie iba a poder usar todavía.

Solución adoptada. El sistema implementa un centro de notificaciones interno (Capítulo 17) con un único canal externo real, email, y preferencias de notificación configurables por usuario. El diseño del módulo no cierra la puerta a agregar otros canales más adelante, pero no incluye código de WhatsApp/SMS sin un proveedor real detrás.

Justificación del cambio. Se priorizó tener un canal completo y verificado — desde el disparo del evento hasta la entrega real al destinatario — por sobre tener varios canales parcialmente implementados. Agregar un canal nuevo hoy es extender el sistema existente, no diseñarlo desde cero.

Capítulo 24

Conclusiones

Este capítulo recoge las conclusiones del trabajo: los objetivos alcanzados, las limitaciones conocidas del sistema construido y las líneas de trabajo futuro.

24.1. Objetivos alcanzados

De los cinco objetivos específicos planteados en el anteproyecto, tres se alcanzaron de forma directa: el módulo de pacientes (registro, historial clínico e historial de compras), el módulo clínico digital (consultas, diagnósticos y recetas) y el módulo de gestión de óptica con control de stock en tiempo real y registro de ventas. El objetivo de una interfaz amigable y accesible se alcanzó de forma cualitativa — el sistema completo sigue un diseño visual consistente y responsive — aunque sin una medición formal de usabilidad que lo confirme con datos.

El objetivo de agendamiento en línea se alcanzó parcialmente. El paciente reserva su cita de forma autónoma desde el portal — elige sucursal, profesional y horario disponible — y la confirma desde el enlace que recibe por correo, todo desde el navegador de un dispositivo móvil o de escritorio. De las cuatro acciones que el objetivo enumeraba quedaron fuera dos: el sistema no permite reprogramar una cita, es decir cambiarle fecha y hora conservando el registro (para moverla hay que cancelarla y reservar una nueva), y la cancelación no es autónoma sino en dos pasos — el paciente la solicita y el personal de la óptica la resuelve, decisión de diseño documentada en el Capítulo 23.

Otros dos aspectos de los objetivos originales se resolvieron de forma distinta a la planteada, y se documentan con su justificación en ese mismo capítulo: la generación de pedidos a proveedores no quedó automática sino asistida por una notificación, y el objetivo cuantitativo de mejorar la eficiencia administrativa en al menos un 20 % no llegó a instrumentarse — el sistema no mide ese indicador, por lo que no puede darse por alcanzado ni descartado con la evidencia disponible.

24.2. Limitaciones

Además de lo anterior, hay casos de uso contemplados en el anteproyecto que no llegaron a implementarse en esta etapa. Se listan aquí de forma explícita, en vez de omitirlos, porque forman parte del alcance propuesto originalmente:

- **Reprogramación de citas.** No existe forma de cambiar la fecha y hora de un turno conservando el registro; la operación equivalente es cancelar y reservar de nuevo.
- **Auditoría de accesos y operaciones.** El sistema registra fecha de creación y modificación en casi todas las tablas, pero no hay una bitácora consultable de accesos y operaciones ni la pantalla para explorarla: la sección de Auditoría existe en el menú como espacio reservado, sin funcionalidad detrás.
- **Adjuntar documentos.** No se pueden anexas archivos — ni documentos del paciente ni resultados de estudios a una consulta. El historial clínico es enteramente estructurado.

- **Unificación de pacientes duplicados.** No hay detección ni fusión de registros duplicados. El sistema previene el duplicado en el alta mediante la unicidad de la cédula, pero no ofrece forma de resolverlo si llega a producirse.
- **Productividad por profesional.** Cada profesional dispone de un panel con sus propios indicadores (consultas del día, de la semana y del mes, pacientes únicos y recetas emitidas), pero no existe el reporte comparativo entre profesionales que el anteproyecto planteaba, con facturación generada, tiempo promedio de atención y ausencias.
- **Segundo factor de autenticación.** La autenticación es por usuario y contraseña; no se implementó MFA.

Las limitaciones de diseño del sistema construido son:

- El proyecto no cuenta con una batería de pruebas automatizadas; toda la validación descrita en el Capítulo 22 se hizo de forma manual.
- Cambiar o resetear una contraseña no invalida los tokens de sesión ya emitidos — quedan válidos hasta su expiración natural.
- El centro de notificaciones cubre hoy un solo canal externo (email); no incluye WhatsApp ni SMS.
- El módulo de notificaciones tiene funcionalidad planificada y no implementada: registro histórico de los envíos por email que hoy se mandan sin dejar rastro en el sistema, aviso de retiro (*pickup*) para ventas a pedido ya recibidas, y plantillas de mensaje administrables.
- El soporte multi-sucursal tiene refinamientos pendientes: los niveles mínimo y máximo de stock son globales, no configurables por sucursal; algunos listados no ofrecen todavía un selector para que un usuario con visibilidad global filtre por sucursal.

24.3. Mejoras futuras

- Incorporar una suite de pruebas automatizadas, al menos sobre las reglas de negocio más sensibles (aislamiento por sucursal, cálculo de stock, reglas de cobro y facturación).
- Evaluar canales de notificación adicionales (WhatsApp, SMS) si la óptica llega a contratar un proveedor real para alguno de ellos.
- Completar las fases pendientes del módulo de notificaciones: bitácora de envíos, aviso de retiro y plantillas administrables.
- Extender el soporte multi-sucursal con los refinamientos identificados: configuración de stock mínimo/máximo por sucursal y selectores cross-sucursal en los listados que lo requieran.
- Instrumentar los indicadores de gestión necesarios para medir, con datos reales de uso, si el sistema efectivamente mejora la eficiencia administrativa del Centro Óptico Santa María en la magnitud que el anteproyecto se propuso.

Bibliografía

- Caddy. (2026). *Caddy Documentation*. Consultado el 14 de julio de 2026, desde <https://caddyserver.com/docs/>
- Docker Inc. (2026). *Docker Documentation*. Consultado el 14 de julio de 2026, desde <https://docs.docker.com/>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. Internet Engineering Task Force (IETF). Consultado el 14 de julio de 2026, desde <https://www.rfc-editor.org/rfc/rfc7519>
- Microsoft. (2026a). *Documentación de ASP.NET Core*. Consultado el 14 de julio de 2026, desde <https://learn.microsoft.com/en-us/aspnet/core/>
- Microsoft. (2026b). *Documentación de Entity Framework Core*. Consultado el 14 de julio de 2026, desde <https://learn.microsoft.com/en-us/ef/core/>
- Microsoft. (2026c). *The TypeScript Handbook*. Consultado el 14 de julio de 2026, desde <https://www.typescriptlang.org/docs/>
- PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. Consultado el 14 de julio de 2026, desde <https://www.postgresql.org/docs/>
- Vite. (2026). *Vite Guide*. Consultado el 14 de julio de 2026, desde <https://vitejs.dev/guide/>
- Vue.js. (2026). *Vue.js Guide*. Consultado el 14 de julio de 2026, desde <https://vuejs.org/guide/>

Apéndice A

Referencia de API

Inventario completo de los 36 controllers de `SIGA.Api/Controllers/*.cs` (234 endpoints). Agrupado en los mismos 14 módulos de negocio usados en el resto del documento.

Cómo leer estas tablas:

- **Ruta:** todas empiezan con `/api`, incluido en la propia celda.
- **Policy:** el nombre exacto de `[Authorize(Policy = "...")]`. “(heredado)” = el controller entero está protegido por esa policy a nivel de clase, el método no agrega una propia. `[Authorize]` sin policy = requiere estar autenticado, cualquier permiso. `AllowAnonymous` = público.
- **Request/Response:** nombre del tipo C# tal cual en el código. Toda respuesta exitosa devuelve `result.Value` envuelto por `Result<T>` (ver Capítulo 3). Los controllers heredan de `BaseController` (`ToHttpResponse`) salvo `EmpleadosController`, que define su propio `ToResponse` equivalente.

A.1. 1. Identidad y Acceso

A.1.1. AuthController — `/api/auth`

Cuadro A.1: AuthController

Método	Ruta	Policy	Request	Response
POST	<code>/api/auth/register</code>	<code>AllowAnonymous</code>	<code>RegisterRequest</code>	—
POST	<code>/api/auth/register/patient</code>	<code>AllowAnonymous</code>	<code>RegisterPatientRequest</code>	—
GET	<code>/api/auth/verify-email?token=</code>	<code>AllowAnonymous</code>	— (query token)	—
POST	<code>/api/auth/login</code>	<code>AllowAnonymous</code>	<code>LoginRequest</code>	<code>LoginResponse</code> (incl. <code>jwtToken</code> , <code>roleClaims</code> , <code>sucursalId/sucursal</code> Nombre)
POST	<code>/api/auth/change-password</code>	<code>[Authorize]</code> (cualquier autenticado)	<code>ChangePasswordRequest</code>	—
POST	<code>/api/auth/forgot-password</code>	<code>AllowAnonymous</code>	<code>ForgotPasswordRequest</code>	—

Método	Ruta	Policy	Request	Response
POST	/api/auth/reset-password	AllowAnonymous	ResetPassword WithToken Request	—

A.1.2. UsersController — /api/users ([Authorize] a nivel de clase)

Cuadro A.2: UsersController

Método	Ruta	Policy	Request	Response
GET	/api/users	ver_usuarios	—	listado de usuarios
DELETE	/api/users/{id}	editar_usuario	—	— (desactiva, soft delete)
PUT	/api/users/{id}/reset-password	editar_usuario	ResetPassword Request	— (reset admin, fuerza cambio en próximo login)

A.1.3. RolesController — /api/roles ([Authorize] a nivel de clase)

Cuadro A.3: RolesController

Método	Ruta	Policy	Request	Response
GET	/api/roles	ver_rols	—	listado de roles
GET	/api/roles/{id}	ver_rols	—	detalle de rol
POST	/api/roles	crear_rol	RoleRequest	—
PUT	/api/roles/{id}	editar_rol	RoleRequest	—
DELETE	/api/roles/{id}	eliminar_rol	—	—
GET	/api/roles/{id}/users	ver_rols	—	usuarios que tienen ese rol

A.1.4. UserRolesController — /api/users/{userId}/roles (mismo archivo RolesController.cs, [Authorize] a nivel de clase)

Cuadro A.4: UserRolesController

Método	Ruta	Policy	Request	Response
GET	/api/users/{userId}/roles	ver_usuarios	—	roles del usuario

Método	Ruta	Policy	Request	Response
POST	/api/users/{userId}/roles	editar_usuario	AssignRole Request	—
DELETE	/api/users/{userId}/roles/{roleId}	editar_usuario	—	—

A.2. 2. Personas

A.2.1. PatientsController — /api/patients ([Authorize])

Cuadro A.5: PatientsController

Método	Ruta	Policy	Request	Response
GET	/api/patients?page&pageSize&search&status	ver_pacientes	—	listado paginado (pageSize máx. 500)
GET	/api/patients/{id}	ver_pacientes	—	detalle
POST	/api/patients	crear_paciente	CreatePatient Request	—
PUT	/api/patients/{id}	editar_paciente	UpdatePatient Request	—
DELETE	/api/patients/{id}	desactivar_paciente	—	—

A.2.2. ProfessionalsController — /api/professionals ([Authorize])

Cuadro A.6: ProfessionalsController

Método	Ruta	Policy	Request	Response
GET	/api/professionals	ver_profesionales	—	listado (sin paginar)
GET	/api/professionals/{id}	ver_profesionales	—	detalle
POST	/api/professionals	crear_profesional	Create Professional Request	—
PUT	/api/professionals/{id}	editar_profesional	Update Professional Request	—
DELETE	/api/professionals/{id}	editar_profesional	—	— (desactiva)

Horarios de profesional viven en un controller aparte, ver `HorariosController` (§A.4).

A.2.3. `CientesController` — `/api/clientes` ([Authorize])

Cuadro A.7: `CientesController`

Método	Ruta	Policy	Request	Response
GET	<code>/api/clientes?page&pageSize&search&status&tipo</code>	<code>ver_clientes</code>	—	listado paginado
GET	<code>/api/clientes/{id}</code>	<code>ver_clientes</code>	—	detalle
GET	<code>/api/clientes/buscar-persona?ci=</code>	<code>crear_cliente</code>	— (query ci)	<code>Person</code> existente o null — para re-utilizar persona al dar de alta
POST	<code>/api/clientes</code>	<code>crear_cliente</code>	<code>CreateClienteRequest</code>	—
PUT	<code>/api/clientes/{id}</code>	<code>editar_cliente</code>	<code>UpdateClienteRequest</code>	—
DELETE	<code>/api/clientes/{id}</code>	<code>desactivar_cliente</code>	—	— (desactiva)
POST	<code>/api/clientes/{id}/activar</code>	<code>desactivar_cliente</code>	—	—

A.2.4. `EspecialidadesController` — `/api/especialidades` ([Authorize])

Cuadro A.8: `EspecialidadesController`

Método	Ruta	Policy	Request	Response
GET	<code>/api/especialidades</code>	[Authorize] (here-dado, sin policy propia)	—	listado
GET	<code>/api/especialidades/{id}</code>	[Authorize] (here-dado)	—	detalle
POST	<code>/api/especialidades</code>	<code>gestionar_especialidades</code>	<code>CreateEspecialidadRequest</code>	—
PUT	<code>/api/especialidades/{id}</code>	<code>gestionar_especialidades</code>	<code>UpdateEspecialidadRequest</code>	—
DELETE	<code>/api/especialidades/{id}</code>	<code>gestionar_especialidades</code>	—	—

A.3. 3. Personal

A.3.1. EmpleadosController — /api/empleados (**policy de clase ver_empleados; usa su propio ToResponse, no BaseController**)

Cuadro A.9: EmpleadosController

Método	Ruta	Policy	Request	Response
GET	/api/empleados?soloActivos	ver_empleados (heredado)	—	listado
GET	/api/empleados/{id}	ver_empleados (heredado)	—	detalle
POST	/api/empleados	gestionar_empleados	CrearEmpleado Request	—
PUT	/api/empleados/{id}	gestionar_empleados	ActualizarEmpleado Request	—
DELETE	/api/empleados/{id}	gestionar_empleados	—	— (desactiva)
GET	/api/empleados/cargos	ver_empleados (heredado)	—	listado de CargoEmpleado — sin controller propio
POST	/api/empleados/cargos	gestionar_empleados	CrearCargoEmpleado Request	—
PUT	/api/empleados/cargos/{id}	gestionar_empleados	ActualizarCargoEmpleado Request	—

A.4. 4. Agenda

A.4.1. TurnosController — /api/turnos ([Authorize])

Cuadro A.10: TurnosController

Método	Ruta	Policy	Request	Response
GET	/api/turnos?fecha&professionalId&estado&patientId	ver_agenda	—	listado filtrado

Método	Ruta	Policy	Request	Response
GET	/api/turnos/disponibles?professionalId&fecha&sucursalId	ver_disponibles (OR: ver_agenda staff u ver_mis_turnos paciente)	—	slots libres
GET	/api/turnos/profesionales-disponibles?fecha&sucursalId	ver_mis_turnos	—	profesionales con horario activo — flujo de reserva del paciente
POST	/api/turnos	gestionar_agenda	CreateTurno Request	—
PUT	/api/turnos/{id}/estado	gestionar_agenda	UpdateTurno EstadoRequest	—
DELETE	/api/turnos/{id}	gestionar_agenda	—	—
GET	/api/turnos/mis-turnos	ver_mis_turnos	—	(usa turnos del paciente autenticado)
POST	/api/turnos/self-book	ver_mis_turnos	SelfBookTurno Request (incl. SucursalId, obligatorio)	—
POST	/api/turnos/{id}/solicitar-cancelacion	ver_mis_turnos	—	—
POST	/api/turnos/{id}/gestionar-cancelacion	gestionar_agenda	Gestionar Cancelacion Request	—
POST	/api/turnos/confirmar/{token}	AllowAnonymous	— (token en URL, del email de confirmación)	—

A.4.2. HorariosController — /api/professionals/{professionalId} ([Authorize], anidado bajo Professionals)

Cuadro A.11: HorariosController

Método	Ruta	Policy	Request	Response
GET	/api/professionals/{professionalId}/horarios	ver_profesionales	—	horarios semanales
PUT	/api/professionals/{professionalId}/horarios	editar_profesional	SetHorarios Request	— (reemplaza el set completo por sucursal del usuario)

Método	Ruta	Policy	Request	Response
GET	/api/professionals/{professionalId}/bloqueos	ver_profesionales	—	fechas bloqueadas
POST	/api/professionals/{professionalId}/bloqueos	editar_profesional	BloqueoFechaRequest	—
DELETE	/api/professionals/{professionalId}/bloqueos/{bloqueoId}	editar_profesional	—	—

A.5. 5. Clínica

A.5.1. ConsultasController — /api/consultas ([Authorize])

Cuadro A.12: ConsultasController

Método	Ruta	Policy	Request	Response
GET	/api/consultas?page&pageSize&search&patientId&professionalId	ver_consultas	—	listado paginado; si el caller es profesional se autofiltra a sus propias consultas
GET	/api/consultas/patient/{patientId}	ver_consultas	—	consultas de un paciente
GET	/api/consultas/{id}	ver_consultas	—	detalle
GET	/api/consultas/profesional/stats	ver_consultas	— (requiere claim estadísticas profesional_id, si no 403)	del profesional autenticado
POST	/api/consultas	registrar_consulta	CreateConsultaClinicaRequest	— (si el caller es profesional, fuerza ProfessionalId al propio)
PUT	/api/consultas/{id}	editar_consulta	UpdateConsultaClinicaRequest	—
DELETE	/api/consultas/{id}	eliminar_consulta	—	—
PATCH	/api/consultas/{id}/estado	editar_consulta	CambiarEstadoConsultaRequest	—
POST	/api/consultas/{id}/receta	editar_consulta	CreateRecetaRequest	— (crea o actualiza la receta 1:1 de la consulta)

Método	Ruta	Policy	Request	Response
GET	/api/consultas/{id}/receta/pdf	ver_consultas	—	PDF (QuestPDF) — 404 si no tiene receta
GET	/api/consultas/mis-consultas	ver_mis_turnos	—	portal del paciente — sus propias consultas
GET	/api/consultas/{id}/mi-receta/pdf	ver_mis_turnos	—	portal del paciente — PDF de su propia receta

A.5.2. RecetasController — /api/recetas ([Authorize])

Cuadro A.13: RecetasController

Método	Ruta	Policy	Request	Response
GET	/api/recetas?clienteId=	ver_ventas	—	recetas del cliente (clínicas + manuales) — flujo de venta a pedido
POST	/api/recetas	registrar_venta	CreateReceta ManualRequest	— (receta externa, sin Consulta ClinicaId, vinculada directo a un cliente)

A.6. 6. Catálogo / Inventario

A.6.1. ProductosController — /api/productos ([Authorize])

Cuadro A.14: ProductosController

Método	Ruta	Policy	Request	Response
GET	/api/productos?page&pageSize&search&categoria&bajoStock&tipoCategoria	ver_inventario	—	listado paginado
GET	/api/productos/{id}	ver_inventario	—	detalle
POST	/api/productos	gestionar_inventario	CreateProducto Request	—

Método	Ruta	Policy	Request	Response
PUT	/api/productos/{id}	gestionar_inventario	UpdateProducto Request	—
DELETE	/api/productos/{id}	gestionar_inventario	—	— (desactiva)
DELETE	/api/productos/{id}/permanente	gestionar_inventario	—	— (borrado físico)
POST	/api/productos/{id}/movimientos	gestionar_inventario	Create Movimiento StockRequest	—
GET	/api/productos/{id}/movimientos	ver_inventario	—	movimientos de un producto
GET	/api/productos/movimientos?page&pageSize&tipo&estado	ver_inventario	—	movimientos globales
PATCH	/api/productos/movimientos/{id}/estado	gestionar_inventario	Aprobar Rechazar Movimiento Request	—
GET	/api/productos/movimientos/{id}/pdf	ver_inventario	—	PDF de comprobante de movimiento
PUT	/api/productos/{id}/stock-config	gestionar_inventario	UpdateStock ConfigRequest	— (mín/máx, global — ver Capítulo 20)
POST	/api/productos/{id}/imagen	gestionar_inventario	IFormFile (máx. 5MB, multipart)	—
DELETE	/api/productos/{id}/imagen	gestionar_inventario	—	—
GET	/api/productos/categorias	ver_inventario	—	listado de Categoría Producto — sin controller propio
POST	/api/productos/categorias	gestionar_inventario	Create Categoría Producto Request	—
PUT	/api/productos/categorias/{id}	gestionar_inventario	Update Categoría Producto Request	—
DELETE	/api/productos/categorias/{id}	gestionar_inventario	—	— (desactiva)

Método	Ruta	Policy	Request	Response
DELETE	/api/productos/ categorias/ {id}/permanente	gestionar_ inventario	—	— (borrado físico)

A.6.2. MarcasController — /api/marcas ([Authorize])

Cuadro A.15: MarcasController

Método	Ruta	Policy	Request	Response
GET	/api/marcas	ver_inventario	—	listado
POST	/api/marcas	gestionar_ inventario	CreateMarca Request	—
PUT	/api/marcas/{id}	gestionar_ inventario	UpdateMarca Request	—
DELETE	/api/marcas/{id}	gestionar_ inventario	—	—
GET	/api/marcas/ modelos?marcaId	ver_inventario	—	listado de Modelo — sin controller propio
POST	/api/marcas/modelos	gestionar_ inventario	CreateModelo Request	—
PUT	/api/marcas/modelos/{id}	gestionar_ inventario	UpdateModelo Request	—
DELETE	/api/marcas/modelos/{id}	gestionar_ inventario	—	—

A.6.3. TipoLentesController — /api/tipos-lente ([Authorize])

Cuadro A.16: TipoLentesController

Método	Ruta	Policy	Request	Response
GET	/api/tipos-lente	ver_inventario	—	listado
POST	/api/tipos-lente	gestionar_ inventario	CreateTipo LenteRequest	—
PUT	/api/tipos-lente/{id}	gestionar_ inventario	UpdateTipo LenteRequest	—
DELETE	/api/tipos-lente/{id}	gestionar_ inventario	—	— (desactiva)

MétodoRuta	Policy	Request	Response
DELETE/api/tipos-lente/{id}/permanente	gestionar_inventario	—	—

A.6.4. TratamientosController — /api/tratamientos ([Authorize])

Cuadro A.17: TratamientosController

MétodoRuta	Policy	Request	Response
GET /api/tratamientos	ver_inventario	—	listado
POST /api/tratamientos	gestionar_inventario	Create Tratamiento Request	—
PUT /api/tratamientos/{id}	gestionar_inventario	Update Tratamiento Request	—
DELETE/api/tratamientos/{id}	gestionar_inventario	—	—

A.6.5. ServiciosController — /api/servicios ([Authorize])

Cuadro A.18: ServiciosController

MétodoRuta	Policy	Request	Response
GET /api/servicios	ver_ventas	—	listado
POST /api/servicios	gestionar_ventas	CreateServicio Request	—
PUT /api/servicios/{id}	gestionar_ventas	UpdateServicio Request	—
DELETE/api/servicios/{id}	gestionar_ventas	—	—
POST /api/servicios/{id}/tarifas	gestionar_ventas	CreateServicio TarifaRequest	— (precio por profesional/especialidad)
DELETE/api/servicios/tarifas/{tarifaId}	gestionar_ventas	—	—
GET /api/servicios/{id}/precio?professionalId	ver_ventas	—	resuelve el precio aplicable (tarifa específica o default)

A.6.6. MotivosMovimientoController — /api/motivos-movimiento ([Authorize])**Cuadro A.19: MotivosMovimientoController**

Método	Ruta	Policy	Request	Response
GET	/api/ motivos-movimiento?tipo	ver_inventario	—	listado, filtrable por tipo de movimiento
POST	/api/motivos-movimiento	gestionar_inventario	CreateMotivoMovimientoRequest	—
PUT	/api/motivos-movimiento/{id}	gestionar_inventario	UpdateMotivoMovimientoRequest	—
DELETE	/api/motivos-movimiento/{id}	gestionar_inventario	—	—

A.6.7. StockLotesController — /api/stock/lotes (policy de clase ver_inventario)**Cuadro A.20: StockLotesController**

Método	Ruta	Policy	Request	Response
GET	/api/stock/lotes? productoId&vencidos	ver_inventario (heredado)	—	lotes con trazabilidad de vencimiento
POST	/api/stock/lotes/conteo	ver_inventario (heredado)	RegistrarConteoRequest	registra un conteo físico de inventario
GET	/api/stock/lotes/ conteos?estado	ver_inventario (heredado)	—	listado de conteos
GET	/api/stock/lotes/conteos/{id}	ver_inventario (heredado)	—	detalle de conteo (diferencias sistema-vs-físico)
POST	/api/stock/lotes/conteos/{id}/gestionar	gestionar_inventario	GestionarConteoRequest	aprueba/rechaza el conteo (ajusta stock si corresponde)

A.6.8. UbicacionesController — /api/ubicaciones ([Authorize])

Cuadro A.21: UbicacionesController

Método	Ruta	Policy	Request	Response
GET	/api/ubicaciones/departamentos?isActive	[Authorize] (here-dado)	—	listado de Departamento
POST	/api/ubicaciones/departamentos	gestionar_configuracion	Create Departamento Request	—
PUT	/api/ubicaciones/departamentos/{id}	gestionar_configuracion	Update Departamento Request	—
GET	/api/ubicaciones/ciudades?departamentoId&isActive	[Authorize] (here-dado)	—	listado de Ciudad
POST	/api/ubicaciones/ciudades	gestionar_configuracion	CreateCiudad Request	—
PUT	/api/ubicaciones/ciudades/{id}	gestionar_configuracion	UpdateCiudad Request	—

A.7. 7. Ventas

A.7.1. VentasController — /api/ventas ([Authorize])

Cuadro A.22: VentasController

Método	Ruta	Policy	Request	Response
GET	/api/ventas?estado&tipo&fechaDesde&fechaHasta&clienteId&page&pageSize	ver_ventas	—	listado paginado
GET	/api/ventas/{id}	ver_ventas	—	detalle
POST	/api/ventas	registrar_venta	CrearVenta Request — incluye el bloque opcional Trabajo Pedido de la venta “a pedido” (ver Capítulo 12)	—
PUT	/api/ventas/{id}	registrar_venta	Actualizar VentaRequest	—

Método	Ruta	Policy	Request	Response
PUT	/api/ventas/{id}/confirmar	registrar_venta	— (usa CurrentUserId)	— (pasa de borrador a confirmada; exige RecetaId si es a pedido)
DELETE	/api/ventas/{id}	registrar_venta	—	— (elimina presupuesto, no venta confirmada)
PUT	/api/ventas/{id}/cancelar	registrar_venta	CancelarVenta Request	—
GET	/api/ventas/cobros-pendientes	ver_ventas	—	ventas a crédito con saldo pendiente
POST	/api/ventas/cobros	registrar_venta	RegistrarCobro Request	—
POST	/api/ventas/{id}/comprobante	registrar_venta	— (usa CurrentUserId)	— (emite comprobante, agrega ingreso a caja si no hay cobros previos)
POST	/api/ventas/facturas	registrar_venta	EmitirFactura Request	—
POST	/api/ventas/{id}/devoluciones	registrar_venta	Solicitar Devolucion Request	—
GET	/api/ventas/{id}/devoluciones	ver_ventas	—	devoluciones de una venta
POST	/api/ventas/devoluciones/{devolucionId}/gestionar	gestionar_ventas	Gestionar Devolucion Request	— (aprobar/rechazar)

A.7.2. TimbradosController — /api/timbrados ([Authorize])

Cuadro A.23: TimbradosController

Método	Ruta	Policy	Request	Response
GET	/api/timbrados	ver_ventas	—	listado
GET	/api/timbrados/activos	ver_ventas	—	timbrados vigentes (por sucursal)
GET	/api/timbrados/{id}	ver_ventas	—	detalle
POST	/api/timbrados	gestionar_ventas	CreateTimbrado Request	—

Método	Ruta	Policy	Request	Response
PUT	/api/timbrados/{id}	gestionar_ventas	UpdateTimbrado Request	—
DELETE	/api/timbrados/{id}	gestionar_ventas	—	—

A.8. 8. Laboratorio

A.8.1. LaboratorioController — /api/laboratorio ([Authorize])

Cuadro A.24: LaboratorioController

Método	Ruta	Policy	Request	Response
GET	/api/laboratorio/pedidos?estado	ver_laboratorio	—	listado de TrabajoPedido (excluye Borrador)
POST	/api/laboratorio/pedidos/{id}/gestionar	gestionar_laboratorio	Gestionar TrabajoPedido Request	— (aprobar/-rechazar, pasa de Pendiente Aprobacion)
PUT	/api/laboratorio/pedidos/{id}/enviar	gestionar_laboratorio	RegistrarEnvio Request	—
PUT	/api/laboratorio/pedidos/{id}/recibir	gestionar_laboratorio	—	— (NO cambia el estado de la venta — cobro desacoplado, ver ADR 0006)
POST	/api/laboratorio/pedidos/{id}/factura	gestionar_laboratorio	EmitirFactura Laboratorio Request	—

A.9. 9. Compras

A.9.1. ComprasController — /api/compras/pedidos (clase: gestionar_pedidos)

Cuadro A.25: ComprasController

Método	Ruta	Policy	Request	Response
GET	/api/compras/pedidos?proveedorId&estado&page&pageSize	ver_inventario	—	listado paginado de OC

Método	Ruta	Policy	Request	Response
GET	/api/compras/pedidos/{id}	ver_inventario	—	detalle
POST	/api/compras/pedidos	gestionar_pedidos (heredado)	CrearPedido Request	—
PUT	/api/compras/pedidos/{id}	gestionar_pedidos (heredado)	CrearPedido Request	—
PUT	/api/compras/pedidos/{id}/confirmar	aprobar_pedidos	—	—
POST	/api/compras/pedidos/{id}/factura	gestionar_pedidos (heredado)	Registrar FacturaPedido Request	—
POST	/api/compras/pedidos/{id}/devolucion	gestionar_pedidos (heredado)	Registrar Devolucion Request	—
PUT	/api/compras/pedidos/{id}/cancelar	cancelar_pedido (OR: creador gestionar_pedidos / aprobador aprobar_pedidos)	—	—

A.9.2. ProveedoresController — /api/proveedores ([Authorize])

Cuadro A.26: ProveedoresController

Método	Ruta	Policy	Request	Response
GET	/api/proveedores?page&pageSize&search&isActive	ver_inventario	—	listado paginado
GET	/api/proveedores/laboratorios	ver_inventario	—	subconjunto marcado como laboratorio óptico
POST	/api/proveedores	gestionar_pedidos	Create Proveedor Request	—
PUT	/api/proveedores/{id}	gestionar_pedidos	Create Proveedor Request (se reutiliza para update)	—
DELETE	/api/proveedores/{id}	gestionar_pedidos	—	—

A.9.3. FacturasCompraController — /api/compras/facturas (policy de clase ver_inventario)

Cuadro A.27: FacturasCompraController

Método	Ruta	Policy	Request	Response
GET	/api/compras/facturas?proveedorId&condicionVenta&estado&origen&fechaDesde&fechaHasta&search&page&pageSize	ver_inventario (heredado)	—	listado paginado
GET	/api/compras/facturas/{id}	ver_inventario (heredado)	—	detalle
POST	/api/compras/facturas	gestionar_pedidos	Registrar FacturaDirecta Request	— (factura sin OC previa)
PUT	/api/compras/facturas/{id}/anular	gestionar_pedidos	AnularFactura Request	—

A.9.4. RecepcionesController — /api/compras/recepciones (policy de clase ver_inventario)

Cuadro A.28: RecepcionesController

Método	Ruta	Policy	Request	Response
GET	/api/compras/recepciones?proveedorId&estadoOC&fechaDesde&fechaHasta&page&pageSize	ver_inventario (heredado)	—	listado paginado
GET	/api/compras/recepciones/{id}	ver_inventario (heredado)	—	detalle
GET	/api/compras/recepciones/facturas-disponibles	ver_inventario (heredado)	—	facturas de compra sin recepción asociada
POST	/api/compras/recepciones	gestionar_pedidos	Registrar Recepcion Request	— (usa CurrentUserId; genera MovimientoStock de entrada más StockLote)

A.10. 10. Caja y Egresos

A.10.1. CajaController — /api/caja (policy de clase ver_ventas)

Cuadro A.29: CajaController

Método	Ruta	Policy	Request	Response
GET	/api/caja/resumen?fecha	ver_ventas (here- dado)	—	resumen del día
GET	/api/caja/movimientos? fechaDesde&fechaHasta& tipo&page&pageSize	ver_ventas (here- dado)	—	listado pa- ginado de MovimientoCaja
GET	/api/caja/sesion-actual	ver_ventas (here- dado)	—	SesionCaja abierta del usuario/sucursal
GET	/api/caja/ apertura-sugerida	ver_ventas (here- dado)	—	monto de apertu- ra sugerido
POST	/api/caja/sesiones	gestionar_caja	AbrirSesion Request	—
GET	/api/caja/sesiones/{id}	ver_ventas (here- dado)	—	detalle de sesión
POST	/api/caja/sesiones/ {id}/cerrar	gestionar_caja	CerrarSesion Request	— (pasa a Pendiente Aprobacion según tolerancia)
GET	/api/caja/sesiones? page&pageSize&estado	ver_ventas (here- dado)	—	historial de sesio- nes
POST	/api/caja/sesiones/ {id}/aprobar-cierre	aprobar_cierres_ caja	—	—
POST	/api/caja/sesiones/ {id}/rechazar-cierre	aprobar_cierres_ caja	RechazarCierre Request	—

A.10.2. EgresosController — /api/egresos (policy de clase gestionar_egresos)

Cuadro A.30: EgresosController

Método	Ruta	Policy	Request	Response
GET	/api/egresos?tipo& estado&fechaDesde& fechaHasta& soloVencidos&page& pageSize	ver_egresos	—	listado pagi- nado (Egreso, jerarquía TPH)
GET	/api/egresos/{id}	ver_egresos	—	detalle
POST	/api/egresos/facturas	gestionar_egresos (heredado)	CrearFactura CompraRequest	— (subtipo FacturaCompra)

Método	Ruta	Policy	Request	Response
POST	/api/egresos/honorarios	gestionar_egresos (heredado)	CrearHonorario Request	— (subtipo Honorario)
POST	/api/egresos/salarios	gestionar_egresos (heredado)	CrearSalario Request	— (subtipo SalarioEmpleado)
POST	/api/egresos/gastos	gestionar_egresos (heredado)	CrearGasto GeneralRequest	— (subtipo GastoGeneral)
PUT	/api/egresos/{id}/aprobar	aprobar_egresos	—	—
PUT	/api/egresos/{id}/rechazar	aprobar_egresos	RechazarEgreso Request	—
PUT	/api/egresos/{id}/pago	pagar_egresos (override explícito a nivel de método, no hereda gestionar_egresos de la clase — pago externo exige además el claim aprobar_egresos y MotivoExterno obligatorio)	RegistrarPago Request	—
PUT	/api/egresos/{id}/anular	gestionar_egresos (heredado)	AnularEgreso Request	—
GET	/api/egresos/categorias	ver_egresos	—	listado de CategoriaGasto — sin controller propio
POST	/api/egresos/categorias	gestionar_egresos (heredado)	CrearCategoria GastoRequest	—
PUT	/api/egresos/categorias/{id}	gestionar_egresos (heredado)	Actualizar CategoriaGasto Request	—

A.11. 11. Configuración

A.11.1. ConfiguracionController — /api/configuracion ([Authorize])

Cuadro A.31: ConfiguracionController

Método	Ruta	Policy	Request	Response
GET	/api/configuracion	[Authorize] (here- dado)	—	Configuracion Negocio (single- ton)
PUT	/api/configuracion	gestionar_ configuracion	Update Configuracion NegocioRequest	—

A.11.2. EstadosConfigController — /api/estados-config ([Authorize])

Cuadro A.32: EstadosConfigController

Método	Ruta	Policy	Request	Response
GET	/api/ estados-config?entidad	[Authorize] (here- dado)	—	estados cosméti- cos configurables por entidad (ej. EstadoConfig de Turnos)
POST	/api/estados-config	gestionar_ configuracion	CreateEstado ConfigRequest	—
PUT	/api/estados-config/{id}	gestionar_ configuracion	UpdateEstado ConfigRequest	—
DELETE	/api/estados-config/{id}	gestionar_ configuracion	—	—

A.12. 12. Sucursales

A.12.1. SucursalesController — /api/sucursales ([Authorize])

Cuadro A.33: SucursalesController

Método	Ruta	Policy	Request	Response
GET	/api/ sucursales?soloActivas	[Authorize] (he- redado — lectu- ra abierta a cual- quier autenticado, incl. pacientes, pa- ra el flujo de reserva de turno)	—	listado

Método	Ruta	Policy	Request	Response
GET	/api/sucursales/{id}	[Authorize] (heredado)	—	detalle
POST	/api/sucursales	gestionar_sucursales	CreateSucursal Request	—
PUT	/api/sucursales/{id}	gestionar_sucursales	UpdateSucursal Request	—
DELETE	/api/sucursales/{id}	gestionar_sucursales	—	—

A.12.2. TransferenciasController — /api/transferencias (policy de clase transferir_stock)

Cuadro A.34: TransferenciasController

Método	Ruta	Policy	Request	Response
GET	/api/transferencias?estado	transferir_stock (heredado)	—	listado
POST	/api/transferencias	transferir_stock (heredado)	Create Transferencia Request	— (Salida en origen, estado Pendiente)
POST	/api/transferencias/{id}/gestionar	transferir_stock (heredado)	Gestionar Transferencia Request	— (aceptar = Entrada en destino; rechazar = Entrada de vuelta al origen; solo el destino gestiona)

A.13. 13. Notificaciones

A.13.1. NotificacionesController — /api/notificaciones (policy de clase ver_notificaciones)

Cuadro A.35: NotificacionesController

Método	Ruta	Policy	Request	Response
GET	/api/notificaciones?soloNoLeidas&page&pageSize	ver_notificaciones (heredado)	—	notificaciones del usuario autenticado

Método	Ruta	Policy	Request	Response
GET	/api/notificaciones/ contador	ver_ notificaciones (heredado)	—	cantidad de no leídas (para el badge)
PUT	/api/notificaciones/ {id}/leer	ver_ notificaciones (heredado)	—	—
PUT	/api/notificaciones/ leer-todas	ver_ notificaciones (heredado)	—	—

A.13.2. NotificacionPreferenciasController — /api/notificaciones/
preferencias ([Authorize] a nivel de clase)

Cuadro A.36: NotificacionPreferenciasController

Método	Ruta	Policy	Request	Response
GET	/api/notificaciones/ preferencias/me	[Authorize] (cualquier autenticado)	—	preferencias del usuario autenticado (defaults si no hay fila)
PUT	/api/notificaciones/ preferencias/me	[Authorize] (cualquier autenticado)	Update Notificacion Preferencia Request	Notificacion Preferencia Response
GET	/api/notificaciones/ preferencias/ persona/{personId}	gestionar_ notificaciones	—	preferencias de una persona (staff)
PUT	/api/notificaciones/ preferencias/ persona/{personId}	gestionar_ notificaciones	Update Notificacion Preferencia Request	Notificacion Preferencia Response

A.14. 14. Reportes

A.14.1. ReportesController — /api/reportes ([Authorize])

Cuadro A.37: ReportesController

Método	Ruta	Policy	Request	Response
GET	/api/reportes/ventas?desde&hasta&agrupacion	ver_reportes	—	serie temporal agregada (día/semana/mes)
GET	/api/reportes/citas?desde&hasta&agrupacion	ver_reportes	—	turnos más consultas más recetas agregado
GET	/api/reportes/inventario?desde&hasta&agrupacion	ver_reportes	—	snapshot de stock (valorización, crítico, por categoría) más movimientos del rango
GET	/api/reportes/compras?desde&hasta&agrupacion	ver_inventario	—	OC, facturas, recepciones, compras por proveedor
GET	/api/reportes/operativo/{tipo}?desde&hasta&sucursalId&metodoPago&categoria&operadorId&tipoMov&page&pageSize	ver_reportes	— (tipo: ventas compras inventario caja)	listado operativo paginado y filtrable
GET	/api/reportes/operativo/{tipo}/export?formato&... (mismos filtros, sin paginar)	ver_reportes	—	archivo application/pdf o text/csv (QuestPDF / CSV UTF-8+BOM)

Relacionado: Capítulo 3 (capas y pipeline de seguridad), Capítulo 20 (entidades referenciadas por cada DTO), Apéndice B (decisiones detrás de reglas como “cobro desacoplado del laboratorio” o “TrabajoPedido en Borrador”).

Apéndice B

Decisiones de Diseño

Un ADR (*Architecture Decision Record*) documenta una decisión de diseño no trivial: el contexto que la motivó, qué se decidió, y qué se gana y qué se pierde con esa elección. Las 12 decisiones de este apéndice no son nuevas — formalizan patrones que ya estaban implícitos en el diseño de SIGA, dejándolos explícitos para quien mantenga o extienda el sistema.

B.1. ADR 0001 — Person como raíz única de identidad; Cliente cuelga de Person

ADR 0001 — Person como raíz única de identidad

Estado. Aceptada (implementada).

Contexto. El sistema necesita modelar distintos roles funcionales de una misma persona física (paciente, profesional, cliente, empleado, usuario administrativo) sin duplicar sus datos personales ni forzar relaciones directas entre esos roles entre sí.

Decisión. **Person** es la entidad raíz única (CI, nombre, contacto). Cada rol funcional (**User**, **Patient**, **Professional**, **Cliente**, **Empleado**) cuelga de **Person** (o de **User**) vía FK propio, nunca al revés. En particular, **Cliente** no es una entidad de facturación separada de **Patient**: ambos referencian el mismo **Person** cuando corresponden a la misma persona física, sin FK directa entre **Cliente** y **Patient** — se relacionan solo vía **PersonId** compartido. Los datos de facturación (**TipoFacturacion**, **RazonSocial**, **RucCiFiscal**, **Direccion**, etc.) viven directamente en **Cliente**. Esto reemplazó un diseño anterior con una entidad **DatosFacturacion** colgando de **Patient**, migrada y eliminada (migración 035_Clientes).

Consecuencias. Gana: un paciente puede convertirse en cliente (o viceversa) sin duplicar sus datos personales ni migrar identidad; consultas cross-rol (“¿este cliente también es paciente?”) se resuelven por **PersonId** compartido, sin joins forzados entre tablas de rol.

Pierde: cualquier query que necesite “todas las facetas de una persona” tiene que hacer join manual vía **PersonId** en vez de tener una sola tabla; no hay integridad referencial que fuerce que un **Cliente** y un **Patient** con el mismo **Person** sean intencionalmente la misma persona (un alta descuidada podría crear un **Person** duplicado con datos ligeramente distintos para la misma persona real).

Referencias: Capítulo de [Modelo de Datos](#) — Grupo A (**Person**, **Cliente**, **Patient**); [módulo de Clientes](#).

B.2. ADR 0002 — Stock derivado de movimientos aprobados, no un campo mutable

ADR 0002 — Stock derivado de movimientos

Estado. Aceptada (implementada).

Contexto. El modelo original tenía `StockActual/StockMinimo/StockMaximo` como campos directos de `Producto`, lo que hacía imposible auditar de dónde salió cada cambio de stock ni soportar múltiples sucursales sin ambigüedad sobre a qué sucursal pertenecía ese número.

Decisión. `Producto` ya no tiene campos de stock. El stock se deriva sumando `MovimientoStock` (`Entrada +`, `Salida -`) filtrados por `Estado = "Aprobado"`, expuesto vía la vista SQL `vw_stock_actual` (mapeada como entidad de solo lectura `StockActualView`, `HasNoKey()`), agrupada por producto más sucursal. Todo servicio que antes tocaba el stock directamente ahora crea un `MovimientoStock` en su lugar. Mín/más de stock quedaron en una tabla separada 1:1 (`ProductoStockConfig`), hoy global (no por sucursal) por decisión explícita para no romper la relación 1:1 durante la migración multi-sucursal.

Consecuencias. **Gana:** trazabilidad completa de cada cambio de stock (quién, cuándo, por qué motivo vía `MotivoMovimiento`); soporte natural para stock por sucursal sin duplicar `Producto`; movimientos pueden pasar por un estado `Pendiente/Rechazado` sin afectar el stock real hasta ser `Aprobado`.

Pierde: leer el stock actual siempre requiere una agregación (vista SQL) en vez de un simple `SELECT` de un campo; hay que mantener disciplina de que absolutamente ningún write-path modifique stock sin pasar por `MovimientoStock` — una excepción rompería la trazabilidad silenciosamente.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo B (`MovimientoStock`, `StockActualView`, `ProductoStockConfig`); [módulo de Inventario y Catálogo](#).

B.3. ADR 0003 — Autorización basada en permisos individuales, no en roles con nombre fijo

ADR 0003 — Autorización por permisos

Estado. Aceptada (implementada).

Contexto. Un sistema de roles fijos (ej. “Admin”, “Recepcionista” codificados en el backend) acopla la lógica de autorización al nombre del rol, dificultando que el negocio cree roles ad-hoc (ej. “Recepcionista Senior” con un permiso extra) sin modificar código ni hacer un despliegue.

Decisión. La autorización real de cada endpoint se basa en permisos individuales (`Permission`), no en el nombre ni el tipo del `Role`. Un `Role` es solo una colección con nombre de permisos (`RolePermission`, many-to-many); el JWT emite un claim “permission” por cada permiso resuelto, no el nombre del rol. `Program.cs` define una policy por permiso (`RequireClaim("permission", perm)`), más dos policies compuestas con `RequireAssertion` para OR de permisos. `Role.Type` (`admin/professional/patient`, nullable e inmutable) es la única noción de “rol de sistema”, usada solo para lógica interna

que necesita distinguir un rol inmutable (ej. bootstrap del admin), nunca para autorizar endpoints.

Consecuencias. Gana: el negocio puede crear/editar roles y su combinación de permisos desde `/roles` sin ningún cambio de código ni despliegue; los permisos son la única fuente de verdad para “qué puede hacer este usuario”, sin casos especiales por nombre de rol.

Pierde: el JWT puede volverse más pesado con muchos claims de permiso si un rol tiene muchos permisos asignados; el código nunca puede asumir “si es rol X entonces puede hacer Y” — tiene que verificar el permiso explícito incluso para casos que parecen obvios por el rol, lo que exige disciplina consistente en cada nuevo endpoint.

Referencias: Capítulo de [Arquitectura](#) — §3.5; Capítulo de [Modelo de Datos](#) — Grupo A (Role, Permission, RolePermission); [módulo de Identidad y Acceso](#).

B.4. ADR 0004 — Precio de venta siempre derivado del margen de categoría

ADR 0004 — Precio de venta derivado por margen

Estado. Aceptada (implementada).

Contexto. El formulario de producto no exponía el precio de venta y el campo quedaba en 0, forzando a tipear el precio a mano en cada línea de venta — inconsistente entre vendedores y propenso a error.

Decisión. `Producto.PrecioVenta` se calcula siempre como `round(PrecioCosto * (1 + CategoriaProducto.Margen/100))`, centralizado en el método de dominio `Producto.AplicarCosto(costo, margen)`. Se llama desde todo write-path que toca el costo: alta/edición de producto, recepción de mercadería, y actualización del margen de una categoría (que recalcula todos los productos de esa categoría). El campo `PrecioVenta` de los DTOs de request quedó sin uso — el backend es la única fuente de verdad. En la venta, el precio autocompleta desde `producto.PrecioVenta` pero queda editable por línea para un ajuste puntual.

Consecuencias. Gana: consistencia de márgenes garantizada por diseño; un solo lugar (el margen de la categoría) para ajustar la rentabilidad de toda una línea de productos a la vez.

Pierde: no hay forma de fijar un precio de venta “manual” de catálogo sin editar el margen de la categoría o el costo del producto — decisión explícita, no un descuido, pero limita casos de pricing por producto individual fuera del margen estándar salvo el ajuste manual por línea de venta.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo B (Producto, CategoriaProducto); [módulo de Inventario y Catálogo](#).

B.5. ADR 0005 — TrabajoPedido nace en Borrador junto al presupuesto

ADR 0005 — TrabajoPedido nace en Borrador

Estado. Aceptada (implementada).

Contexto. El flujo de venta “a pedido” (armazón + cristal + tratamientos + laboratorio) necesita capturarse desde el momento del presupuesto, antes de que el cliente confirme la compra — pero no debía aparecer en la cola de trabajo del laboratorio hasta que la venta esté efectivamente confirmada.

Decisión. `EstadoTrabajoPedido.Borrador` (agregado con valor 5, sin reenumerar los estados existentes) es el estado inicial de todo `TrabajoPedido` creado junto a un presupuesto, en el mismo paso que `CrearVentaAsync.GetTrabajosPedidoAsync` excluye explícitamente `Borrador` de la cola visible del laboratorio. Al confirmar la venta (`ConfirmarVentaAsync`), si el TP sigue en `Borrador` se exige asignar un laboratorio y pasa a `PendienteAprobacion`, recién ahí entrando al ciclo real de laboratorio.

Consecuencias. Gana: el vendedor puede armar la configuración óptica completa (diseño de lente, tratamientos, armazón, laboratorio tentativo) desde el presupuesto sin comprometer al laboratorio ni generar ruido en su cola de trabajo real; el presupuesto sigue siendo libremente editable mientras está en `Borrador`.

Pierde: cualquier query o reporte sobre `TrabajoPedido` tiene que recordar excluir `Borrador` explícitamente si no quiere contar presupuestos no confirmados como trabajo real de laboratorio — un filtro fácil de olvidar en código nuevo.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo C (`TrabajoPedido`, `EstadoTrabajoPedido`); [módulo de Ventas](#); [módulo de Laboratorio](#).

B.6. ADR 0006 — Cobro desacoplado del ciclo del laboratorio

ADR 0006 — Cobro desacoplado del laboratorio

Estado. Aceptada (implementada).

Contexto. Originalmente una venta “a pedido” quedaba en estado `EnProceso` hasta que el laboratorio recibía el trabajo, y solo entonces se habilitaba el cobro — un bloqueo de negocio real, porque impedía cobrarle al cliente (seña o total) mientras el pedido seguía en tránsito con un laboratorio externo, a veces por días o semanas.

Decisión. `ConfirmarVentaAsync` deja **toda** venta (directa o a pedido) en estado `ListaParaCobrar` inmediatamente al confirmar, sin esperar al laboratorio. `LaboratorioService.RegistrarRecepcionAsync` ya no cambia el estado de la venta. El ciclo del laboratorio (envío → recepción → factura) corre en paralelo al cobro, sin bloquearse mutuamente.

Consecuencias. Gana: el negocio puede cobrar (seña o total) apenas se confirma la venta, sin esperar la logística externa del laboratorio.

Pierde: `EstadoVenta.EnProceso` quedó sin uso para ventas nuevas — el enum se mantiene por compatibilidad con datos/reportes históricos, lo que es una fuente potencial de confusión para quien lea el código o el enum sin conocer este historial.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo C (`Venta.Estado`); [módulo de Ventas](#).

B.7. ADR 0007 — El cristal graduado ya no se modela como Producto con stock

ADR 0007 — El cristal ya no es Producto con stock

Estado. Aceptada (implementada) — supersede un diseño anterior.

Contexto. El diseño original del catálogo óptico modelaba el cristal graduado como un Producto más, distinguido por `CategoriaProducto.Tipo = Cristal`, con `TrabajoPedido.CristalProductoId` como FK. En la práctica, un cristal a pedido no tiene stock ni SKU fijo: se fabrica según receta, diseño de lente y tratamientos elegidos por el cliente, por lo que forzarlo a la misma tabla que armazones con stock real generaba fricción de modelado (¿qué significa “stock” de un cristal que no existe hasta fabricarse?).

Decisión. El cristal se especifica hoy directamente en `TrabajoPedido` mediante `TipoLente` (clasificación de diseño: monofocal/bifocal/progresivo/ocupacional) más una colección de `Tratamientos` más precio, **sin FK a Producto**. `TrabajoPedido.CristalProductoId` fue removido del modelo. `TipoCategoriaProducto.Cristal` se mantiene en el enum pero está marcado `[Obsoleto]` explícitamente en el propio código (se conserva por compatibilidad de datos existentes, no se ofrece al dar de alta categorías nuevas). En `VentaLinea`, `TipoLineaVenta.Lente` identifica la línea del cristal a pedido, que no descuenta stock.

Consecuencias. Gana: el cristal a pedido se modela con su semántica real (especificación de fabricación, no inventario), sin forzar un `Producto/SKU` ficticio por cada combinación de diseño más tratamiento.

Pierde: cualquier dato ya cargado bajo el diseño anterior (`CategoriaProducto.Tipo = Cristal`) queda como dato legado, conservado por trazabilidad histórica pero sin ofrecerse en altas nuevas; el modelo arrastra esa categoría y sus productos sin que ninguno de los dos tenga ya un rol en el flujo de venta.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo B, nota “Catálogo óptico”; [módulo de Ventas](#); [módulo de Laboratorio](#).

B.8. ADR 0008 — Receta puede ser clínica o externa/manual

ADR 0008 — Receta clínica o externa

Estado. Aceptada (implementada).

Contexto. No toda venta de lentes graduados se origina en una consulta clínica registrada en el sistema — un cliente puede traer una receta emitida por otra óptica u oftalmólogo externo, y el negocio necesita poder cargarla igual para generar el trabajo a pedido.

Decisión. `Receta.ConsultaClinicaId` es nullable (antes era obligatorio, ligado 1:1 a una `ConsultaClinica`). Se agregó `Receta.PersonId` (nullable, FK a `Person`) para representar una receta “externa” cargada manualmente sin consulta previa. Ambos campos son nullable y mutuamente informativos: si `ConsultaClinicaId` es null, la receta es externa. El endpoint `POST /api/recetas` permite el alta manual (`CreateRecetaManualRequest`), separado del flujo clínico normal que sigue generando la receta desde `ConsultaClinicaService`.

Consecuencias. Gana: el flujo de venta a pedido no depende de que el paciente haya tenido una consulta en el sistema; cubre el caso real de negocio de una óptica que también vende con receta de terceros.

Pierde: hay dos caminos distintos para crear una **Receta** (clínico vs. manual) que hay que mantener sincronizados en reglas de validación; una receta externa no tiene la trazabilidad de diagnóstico/observaciones clínicas que sí tiene una receta nacida de una **ConsultaClínica**.

Referencias: Capítulo de [Modelo de Datos](#) — Grupo B (**Receta**); [módulo de Clínica](#); [módulo de Ventas](#).

B.9. ADR 0009 — Una sucursal fija por usuario, sin selector

ADR 0009 — Sucursal fija por usuario

Estado. Aceptada (implementada).

Contexto. Al volver el sistema multi-sucursal había que decidir cómo un usuario del staff determina en qué sucursal está operando en cada momento. Un selector manual en el header es flexible pero propenso a error humano (olvidarse de cambiarlo y cargar una venta en la sucursal equivocada).

Decisión. Cada **User** de staff tiene una sucursal fija (**User.SucursalId**), asignada desde el form de Profesional o de Empleado — no desde la pantalla de Usuarios, que solo gestiona roles y activar/desactivar. No hay selector de sucursal en el header: la sucursal del usuario logueado determina automáticamente el scoping de todo lo que crea, resuelto centralmente vía **ICurrentUserContext.SucursalId = null** significa usuario global: el admin (ve/filtra todas las sucursales) y los pacientes (intrínsecamente globales, eligen sucursal solo al reservar un turno). Catálogos (productos, marcas, servicios, etc.) y personas (pacientes, clientes, profesionales) quedan explícitamente globales, no scopeados por sucursal.

Consecuencias. Gana: elimina una clase entera de errores humanos (cargar una operación en la sucursal equivocada por olvidar cambiar un selector); el scoping se resuelve una sola vez de forma centralizada en vez de repetirse en cada pantalla o servicio.

Pierde: un empleado que reparte su tiempo entre dos sucursales físicas no puede operar en la segunda sin que un admin le reasigne la sucursal — no hay soporte de “sucursal del día” ni multi-sucursal por usuario. Un selector cross-sucursal solo para el admin en algunos listados (stock/productos) quedó identificado como extensión posible.

Referencias: Capítulo de [Arquitectura](#) — §3.6; Capítulo de [Modelo de Datos](#) — § Scoping por sucursal; [módulo de Sucursales](#).

B.10. ADR 0010 — Centro de notificaciones interno antes que integración externa

ADR 0010 — Notificaciones internas antes que externas

Estado. Aceptada (implementada — Fase 1 de 5 del módulo).

Contexto. La especificación completa del módulo de notificaciones contemplaba notificaciones externas a pacientes (email/WhatsApp/SMS) y un sistema de plantillas — alcance demasiado grande para una sola entrega. Había que elegir con qué empezar para entregar valor real cuanto antes.

Decisión. La Fase 1 del módulo se limitó a un centro de notificaciones **interno** al

staff (`NotificacionInterna`), sin ningún canal externo todavía. Scoping por destinatario individual (`DestinatarioUsuarioId`), broadcast por sucursal (`DestinatarioSucursalId`) o broadcast global (ambos null). Tres triggers reales conectados desde el día uno: bajo stock (poller `BackgroundService` cada 30 min, no hook inline — porque los movimientos “Aprobado” se originan desde al menos 5 servicios distintos), transferencia de stock pendiente, y pedido de laboratorio recibido.

Consecuencias. Gana: valor inmediato para el staff (alertas de bajo stock, transferencias, laboratorio) sin la complejidad de integrar un proveedor de email/WhatsApp todavía; el diseño de destinatario (usuario/sucursal/global) es reutilizable tal cual para las fases externas futuras.

Pierde: Leído es un único flag compartido para notificaciones de broadcast (no por-usuario), así que un usuario puede marcar como leída una notificación que otro compañero de la misma sucursal todavía no vio — decisión explícita aceptada, no un descuido.

Referencias: Capítulo de [Modelo de Datos](#) — § Entidades transversales (`NotificacionInterna`); [módulo de Notificaciones](#).

B.11. ADR 0011 — Egreso como jerarquía TPH unificando las salidas de dinero

ADR 0011 — Egreso como jerarquía TPH

Estado. Aceptada (implementada).

Contexto. El negocio tiene varios tipos de salida de dinero con semántica distinta (factura de proveedor, honorario a profesional, gasto general, salario de empleado, factura de laboratorio) pero que comparten casi todos sus campos (monto, estado, fechas, método de pago) y necesitan aparecer juntos en reportes de caja/egresos sin duplicar esa lógica común cinco veces.

Decisión. Egreso es la clase base abstracta de la que heredan `FacturaCompra`, `Honorario`, `GastoGeneral`, `SalarioEmpleado` y `EgresoFacturaLaboratorio`, mapeada por EF Core como **Table-Per-Hierarchy (TPH)**: una única tabla física `egresos` con columna discriminadora `Tipo` (`TipoEgreso` enum). Cada subtipo solo agrega 1-3 columnas específicas propias (ej. `Honorario.ProfessionalId`, `SalarioEmpleado.EmpleadoId`, `GastoGeneral.CategoriaGastoId`).

Consecuencias. Gana: reportes y consultas de caja/egresos tratan cualquier tipo de salida de dinero de forma uniforme (mismo `Estado`, mismo flujo de aprobación/pago) sin necesidad de UNION entre tablas separadas; agregar un sexto tipo de egreso no requiere tocar la lógica de caja existente, solo un nuevo subtipo más valor de enum.

Pierde: la tabla física `egresos` acumula columnas nullable de todos los subtipos (patrón típico y esperado de TPH); cualquier índice o constraint a nivel de un subtipo específico es más difícil de expresar que en tablas separadas (TPT).

Referencias: Capítulo de [Modelo de Datos](#) — Grupo C (Egreso); [módulo de Caja y Egresos](#).

B.12. ADR 0012 — Cambio de contraseña obligatorio solo para cuentas nuevas

ADR 0012 — Cambio de contraseña solo cuentas nuevas

Estado. Aceptada (implementada).

Contexto. Al implementar el flujo de cambio de contraseña obligatorio para cuentas creadas por un admin (profesionales/empleados dados de alta con una contraseña que ellos no eligieron), había que decidir si aplicar el requisito retroactivamente a las cuentas ya existentes o solo hacia adelante.

Decisión. `User.MustChangePassword` (default `false`) solo se setea en `true` para cuentas nuevas creadas desde ese momento en adelante (`ProfessionalService.CreateAsync` / `EmpleadoService.CrearAsync`); no hay migración retroactiva sobre cuentas existentes. El admin puede forzar el cambio en cualquier cuenta puntual vía `UserService.ResetPasswordAsync`. El flag no bloquea el login (a diferencia de `IsEmailVerified`): el frontend redirige después de autenticar, vía guard de router que cubre también navegación directa y refresh, no solo el login inicial.

Consecuencias. Gana: cambio simple y sin fricción para las cuentas existentes, que no se ven interrumpidas por un requisito retroactivo; el admin conserva la herramienta puntual (reset) para forzar el cambio cuando haga falta caso por caso.

Pierde: cuentas viejas creadas con contraseña temporal antes de esta funcionalidad nunca son forzadas a cambiarla automáticamente — depende de que el admin lo haga manualmente si le preocupa un caso puntual. No hay invalidación de JWTs ya emitidos al cambiar/resetear una contraseña (limitación conocida, sin infraestructura de revocación).

Referencias: Capítulo de [Arquitectura](#); módulo de [Identidad y Acceso](#).

Apéndice C

Manual de Usuario

Sistema de gestión para óptica.

Escrito a partir de las pantallas reales del sistema. Si una opción descrita aquí no aparece en el menú de un usuario, es por permisos — ver la Sección [C.9](#).

C.1. Qué es SIGA y para quién

SIGA es el sistema de gestión de la óptica. Desde un mismo lugar se atiende a los pacientes (turnos, consultas y recetas), se administra el catálogo y el stock de productos, se hacen las compras a proveedores, se venden lentes y servicios, se maneja la caja del día y se sacan reportes. Se usa desde el navegador: cada persona entra con su usuario y contraseña y ve solamente las pantallas que le corresponden según su rol.

C.1.1. Los roles que existen

SIGA arranca con tres roles ya cargados de fábrica. Lo que cada rol puede hacer está definido por sus permisos:

- **Administrador:** tiene todos los permisos del sistema, con una sola excepción (no tiene el permiso de “Mis Turnos”, que es la vista pensada para pacientes). En la práctica el Administrador ve y puede usar todos los módulos: pacientes, agenda, clínica, catálogo, inventario, compras, egresos, ventas, laboratorio, caja, reportes, personal, usuarios, roles, sucursales y configuración. Es el único rol que el sistema mantiene sincronizado solo: sus permisos se recalculan cada vez que se reinicia y no se editan a mano.
- **Profesional:** de fábrica viene sin permisos asignados. Esto es a propósito: el Administrador le asigna a mano los permisos que necesite ese profesional en particular (por ejemplo ver la agenda, ver y registrar consultas, ver el historial clínico). Hasta que el Administrador le cargue permisos, un Profesional recién creado no ve ningún módulo de gestión.
- **Paciente:** tiene dos permisos, “ver el panel de control” y “ver mis turnos”. Con eso el paciente entra a su propio panel, reserva y consulta sus turnos, ve su historial clínico y descarga sus recetas. No ve nada de la operación interna de la óptica.

C.1.2. Roles a medida

Además de esos tres, el Administrador puede crear todos los roles que la óptica necesite (por ejemplo Recepcionista, Vendedor, Cajero, Encargado de Depósito, Optometrista) desde la pantalla “Roles y Permisos”, eligiendo uno por uno qué puede hacer cada rol. Así, un Recepcionista puede tener solo agenda y recepción; un Vendedor, ventas y clientes; un Encargado de Depósito, inventario y stock; y así con cada puesto. El manual describe todas las pantallas, pero cada persona verá solo aquellas para las que su rol tenga permiso.

En resumen: lo que ve y puede hacer cada usuario no depende de “quién es” sino de “qué permisos tiene su rol”. Por eso dos personas distintas pueden ver menús muy diferentes.

C.2. Primeros pasos

C.2.1. Cómo se ingresa

Se entra por la pantalla de inicio de sesión. Pide dos datos:

- Usuario o Correo electrónico
- Contraseña (hay un botón de ojo para “Mostrar contraseña” u “Ocultar contraseña”)

Se confirma con el botón Iniciar Sesión (mientras verifica muestra “Verificando...”). Si los datos están mal, avisa “Credenciales incorrectas. Intenta de nuevo.”; si no hay conexión, “No se pudo conectar con el servidor. Verifica tu conexión.”

No hay opción de “recordarme”: la sesión queda guardada en el navegador y sigue abierta aunque se cierre y se vuelva a abrir, hasta que se use “Cerrar sesión”.

Apenas se ingresa, el sistema lleva al Panel de Control si el usuario tiene permiso para verlo; si no, a la primera pantalla a la que tenga acceso.

C.2.2. La primera vez: cambio de contraseña obligatorio

Cuando el Administrador crea una cuenta, le asigna una contraseña provisoria. La primera vez que se entra con ella, el sistema obliga a cambiarla antes de dejar hacer cualquier otra cosa: lleva a la pantalla “Cambiá tu contraseña” y no deja salir de ahí hasta que se cambie. Pide:

- Contraseña provisoria (la que se recibió)
- Contraseña nueva (mínimo 8 caracteres, con al menos una letra y un número)
- Confirmar contraseña nueva

Se confirma con Cambiar contraseña y continuar. Recién ahí el sistema deja usar el resto de las pantallas. Lo mismo pasa cuando un Administrador “restablece la contraseña” de un usuario: la próxima vez que entre, tendrá que elegir una nueva.

C.2.3. Cómo se recupera la contraseña

Si se olvidó, en la pantalla de ingreso se toca la opción Olvidó su contraseña. Eso lleva a una pantalla donde se carga el Correo electrónico, se completa el captcha de seguridad y se toca Enviar enlace. El sistema, por privacidad, siempre responde “Si el correo ingresado corresponde a una cuenta, te enviamos un enlace”. Se revisa el email, se entra al enlace y ahí se elige la nueva contraseña (Nueva contraseña y Confirmar contraseña, con la misma regla de complejidad) y se confirma con Restablecer contraseña. Si el enlace ya venció, la pantalla ofrece Pedir un nuevo enlace.

C.2.4. Registro de pacientes

Los pacientes pueden crearse su propia cuenta desde el enlace “¿Sos paciente y no tenés cuenta? Registrate aquí”, en la pantalla de ingreso. Completan nombre, apellido, cédula, fecha de nacimiento, teléfono, email y contraseña, más un captcha. Después reciben un correo para verificar el email y activar la cuenta. El personal de la óptica no se registra solo: sus cuentas las crea el Administrador.

C.2.5. Cómo se cierra sesión

Arriba a la derecha está el nombre del usuario con sus iniciales. Al tocarlo se abre un menú que muestra nombre, email y roles, y ofrece:

- Cambiar contraseña: abre una ventana para cambiarla cuando se quiera (Contraseña actual, Contraseña nueva y confirmación).
- Preferencias de notificación: qué avisos recibir por email y una ventana de silencio horaria opcional.
- Cerrar sesión: cierra la cuenta y devuelve a la pantalla de ingreso.

En esa misma barra de arriba hay un botón para cambiar entre tema claro y oscuro, y una campana de notificaciones (solo si se tiene permiso para verlas).

C.2.6. Por qué cada uno ve un menú distinto

El menú de la izquierda está organizado en zonas (General, Atención, Óptica, Comercial, Gestión) y dentro de cada zona hay módulos. El sistema muestra solamente los módulos y las opciones para los que el rol del usuario tiene permiso. Si no hay permiso de un módulo, ese módulo directamente no aparece, y si una zona entera queda sin módulos, la zona no se muestra. Por eso es normal no ver algunos capítulos de este manual: significa que el rol no tiene ese permiso. Hay además un buscador rápido de módulos (se abre con la tecla “/”) para saltar directo a lo que se busca.

C.3. Zona General

C.3.1. Panel de Control

Para qué sirve: es la pantalla de inicio. Cambia según quién entra.

- **Paciente:** saludo según la hora y la fecha de hoy, una tarjeta grande con su Próximo turno (fecha, hora, profesional, motivo y estado) o el aviso “No tenés turnos próximos”, accesos a Reservar turno e Historial clínico, y una lista de Turnos recientes.
- **Profesional:** saludo con su especialidad y una tira de indicadores (Consultas hoy, Esta semana, Este mes, Pacientes atendidos, Recetas emitidas), más un panel de Últimas consultas con paciente, diagnóstico, fecha y una etiqueta “Con receta” cuando corresponde. El botón Ver todas abre la lista de consultas.
- **Personal de la óptica:** saludo e indicadores del negocio (Pacientes activos, Citas hoy, Ingresos semanales, Stock crítico), la agenda del día y un panel de novedades. Abajo a la derecha hay un botón redondo con un más que abre Acciones rápidas: Nueva Cita (lleva a Agenda) y Nuevo Paciente (lleva a Pacientes).

Cómo se llega: es la pantalla que aparece al ingresar, o el primer ítem del menú.

C.3.2. Notificaciones

Para qué sirve: ver los avisos del sistema (por ejemplo stock bajo, una transferencia pendiente o un pedido de laboratorio que llegó).

Qué se ve: el título con el total, un botón Solo no leídas para filtrar, y la lista de avisos con su ícono, el mensaje y cuándo ocurrió. Las no leídas quedan resaltadas con un punto. Abajo hay paginación cuando son muchas.

Tareas:

- Marcar todas como leídas: botón arriba a la derecha (aparece si hay pendientes).
- Ver solo pendientes: botón Solo no leídas.
- Marcar una como leída: se toca la notificación.

C.3.3. Mis Turnos (paciente)

Para qué sirve: que el paciente reserve, vea y pida cancelar sus turnos.

Qué se ve: si tiene un turno activo, una tarjeta Tu próximo turno con fecha, hora, profesional, motivo y estado. Un calendario mensual navegable (con Hoy y flechas de mes) donde cada día puede mostrar la hora del turno, una leyenda de colores, y abajo una tabla Historial con Fecha, Profesional, Motivo y Estado.

Tareas:

- Reservar turno: botón Reservar turno (o tocar un día futuro del calendario). Se abre la ventana Reservar turno con pasos: elegir Sucursal (si hay más de una), elegir Profesional, elegir Horario disponible y escribir un Motivo de consulta. Se confirma con Reservar Turno. No se puede reservar un turno nuevo si ya se tiene uno activo; primero hay que cancelarlo.
- Solicitar cancelación: en la tarjeta del turno, botón Solicitar Cancelación. El sistema aclara que un miembro del equipo confirmará la cancelación. Queda como “Cancelación solicitada” hasta que el equipo la aprueba.

C.3.4. Mi Historial (paciente)

Para qué sirve: que el paciente vea sus consultas y diagnósticos. Es solo de lectura.

Qué se ve: una lista de tarjetas, una por consulta, con la fecha, el profesional y los campos Motivo, Diagnóstico, Plan de tratamiento y Observaciones. Si la consulta tiene receta, aparece la etiqueta “Receta emitida”.

C.3.5. Mis Recetas (paciente)

Para qué sirve: que el paciente vea sus prescripciones ópticas y descargue el PDF de cada una.

Qué se ve: una tarjeta por receta, con fecha y profesional, y una tabla con Esférico, Cilindro, Eje y Adición para el ojo derecho (OD) y el izquierdo (OI), más DIP, agudeza visual sin corrección y con corrección, y observaciones.

Tareas: descargar el PDF con el botón PDF en cada receta.

C.4. Zona Atención

C.4.1. Pacientes

Para qué sirve: administrar la base de pacientes.

Cómo se llega: menú Atención, Pacientes.

Qué se ve: filtros por estado (Activo / Inactivo) y un buscador por nombre, cédula o contacto. Una tabla con Nombre del Paciente, C.I., Fecha de Registro, Estado y acciones. Al pie, el conteo y la paginación, más dos tarjetas resumen (Pacientes Activos y Total Registros).

Tareas:

- Crear: botón Añadir Paciente. Se completa Nombre, Apellido, Nro. de Cédula, Fecha de Nacimiento, Sexo, Teléfono y Email (se exige al menos teléfono o email). Se guarda con Guardar Paciente. También existe una página completa “Registrar Paciente” con los mismos campos.
- Ver / Editar / Desactivar: el menú de cada fila (los tres puntos) ofrece Ver detalle, Editar y Desactivar. Al desactivar, el sistema aclara que el paciente no podrá ingresar pero sus datos se conservan.
- Ver detalle: al tocar una fila se abre la ficha, con pestañas Información, Citas y Turnos, Historial Clínico y Ventas (esta última marcada como “Pronto”). Desde la ficha también se puede Editar y Desactivar.

C.4.2. Agenda

Para qué sirve: gestionar los turnos de toda la óptica.

Cómo se llega: menú Atención, Agenda.

Qué se ve: la fecha actual con modos día, semana y mes, un filtro por profesional, seis tarjetas de conteo (Total, Pendientes, Confirmados, En sala, Completados, Cancelados) y chips para filtrar por estado. La tabla muestra Hora, Profesional, Paciente, Motivo, Estado y Acciones.

Tareas:

- Nuevo turno: botón Nuevo Turno. Se elige Profesional, Fecha, Horario (según disponibilidad), Paciente (buscando por nombre o cédula), Motivo y Notas internas. Se confirma con Confirmar Turno. El sistema no deja reservar en fechas pasadas ni en horarios ya ocupados, y solo ofrece los días y horas en que el profesional trabaja (cada turno dura 30 minutos).
- Confirmar llegada: en el menú de la fila, Confirmar llegada marca al paciente como Presente (así pasa a la sala de espera / Recepción).
- Cancelar turno: en el menú de la fila, Cancelar turno, con confirmación.
- Gestionar solicitud de cancelación: cuando un paciente pidió cancelar, la fila muestra “Solicita cancelar” y el menú ofrece Gestionar solicitud, donde se puede Aprobar o Rechazar.

C.4.3. Clínica: Recepción

Para qué sirve: que recepción vea los pacientes que ya llegaron (estado Presente) y les arranque la consulta.

Cómo se llega: menú Atención, Clínica, Recepción.

Qué se ve: la fecha de hoy, una tarjeta Pacientes en sala con el número, y una tabla con Hora, Profesional, Paciente, Motivo y Acciones. Si no hay nadie, dice “Sin pacientes en sala”.

Tareas:

- Actualizar: botón Actualizar para recargar la lista.
- Iniciar consulta: en el menú de la fila, crea la consulta del paciente y lleva al formulario clínico.
- Cancelar turno: en el menú de la fila, con confirmación.

C.4.4. Clínica: Consultas

Para qué sirve: listar y gestionar las consultas clínicas.

Cómo se llega: menú Atención, Clínica, Consultas.

Qué se ve: buscador por paciente, cédula o diagnóstico, filtro por profesional, y una tabla con Fecha, Paciente, Profesional, Diagnóstico y Estado (con etiqueta “Con receta” cuando corresponde).

Tareas:

- Nueva consulta: botón Nueva Consulta, que abre el formulario de alta. Ahí se elige el Paciente y el Profesional, se cargan Fecha y Motivo de consulta (obligatorios) y el Diagnóstico Principal (obligatorio). Un interruptor Incluir detalles de consulta agrega Anamnesis, Examen Físico, Diagnóstico Secundario, Plan de Tratamiento y Observaciones. Otro interruptor, Incluir receta óptica, despliega la prescripción (Fecha de Emisión, tabla OD/OI con Esfera, Cilindro, Eje y Adición, Distancia Interpupilar, agudeza visual sin y con corrección, y observaciones). Se guarda con Guardar Consulta.
- Cambiar de estado: en el menú de la fila aparecen, según corresponda, Abrir consulta, Cerrar consulta o Cancelar consulta.
- Editar consulta: en el menú, Editar consulta (abre una página de edición; el paciente y el profesional quedan fijos y no se cambian). Al editar se exige que quede cargado el Diagnóstico Principal.
- Receta: en el menú, Agregar receta o Ver / Editar receta. La ventana de receta tiene un botón Descargar PDF cuando ya existe.
- Eliminar: en el menú, Eliminar, con confirmación.

Estados de una consulta: Pendiente, Abierta, Cerrada y Cancelada.

C.4.5. Clínica: Historial

Para qué sirve: consultar el historial clínico completo de un paciente. Es solo de lectura.

Cómo se llega: menú Atención, Clínica, Historial.

Qué se ve: un buscador Buscar paciente (por nombre o cédula). Al elegir uno, aparece su ficha y una línea de tiempo de consultas con todos los campos clínicos y, si hay, la prescripción óptica. La etiqueta “Con receta” marca las consultas con prescripción.

C.5. Zona Óptica

C.5.1. Catálogo: Productos

Para qué sirve: administrar el catálogo de productos (armazones, accesorios, lentes de contacto, soluciones, etc.).

Cómo se llega: menú Óptica, Catálogo, Productos.

Qué se ve: un selector de Categoría, un botón Bajo stock para resaltar los que están por debajo del mínimo, y un buscador por nombre o SKU. La tabla muestra Producto (con miniatura), Categoría, SKU y Estado.

Tareas:

- Crear: botón Nuevo Producto. Campos Nombre, Categoría, SKU / Código (opcional), Precio de costo (debajo muestra el Precio de venta calculado según el margen de la categoría),

Marca, Modelo (se habilita al elegir marca), Color, Talle y Descripción. Se guarda con Crear Producto.

- Editar: en el menú de la fila, Editar. Además de los datos permite gestionar la Imagen (Subir imagen, Cambiar imagen, Quitar imagen) y ver el Estado.
- Ver detalle: al tocar la fila se abre un panel con imagen, datos y descripción.
- Desactivar / Activar / Eliminar: en el menú. Desactivar deja el producto oculto pero conserva sus datos; Eliminar es permanente y el sistema recomienda desactivar en lugar de eliminar si el producto tiene movimientos o ventas.

C.5.2. Catálogo: Categorías

Para qué sirve: gestionar las categorías de productos, con su margen de ganancia y su descuento. Cómo se llega: menú Óptica, Catálogo, Categorías.

Qué se ve: filtro por estado y buscador por nombre. Tabla con Nombre, Tipo, Descripción, Descuento, cantidad de Productos y Estado.

Tareas:

- Crear: botón Nueva Categoría. Campos Nombre, Tipo (Genérico o Armazón; el tipo Armazón habilita el producto en la venta a pedido), Descripción, Margen en porcentaje (ganancia sobre el costo) y Descuento en porcentaje. Se guarda con Crear Categoría.
- Editar / Desactivar / Activar / Eliminar: en el menú de la fila.

C.5.3. Catálogo: Armazón (Marcas y Modelos)

Marcas. Para qué sirve: administrar las marcas de armazones y lentes de contacto. Se llega por menú Óptica, Catálogo, Armazón, Marcas. Tabla con Nombre, cantidad de Modelos y Estado. Se crea con Nueva Marca (solo Nombre), y se puede Editar, Desactivar o Activar desde el menú de la fila.

Modelos. Para qué sirve: administrar los modelos, cada uno atado a una marca. Se llega por Armazón, Modelos. Se filtra por Marca y por estado. Se crea con Nuevo Modelo (Marca y Nombre) y se Edita, Desactiva o Activa desde el menú.

C.5.4. Catálogo: Óptico (Tipos de Lente y Tratamientos)

Tipos de Lente. Para qué sirve: administrar los diseños de lente a pedido (por ejemplo Monofocal, Progresivo) y su precio base sugerido. Se llega por Óptica, Catálogo, Óptico, Tipos de Lente. Se crea con Nuevo Tipo (Nombre y Precio base, que después es editable en cada venta). Se Edita, Desactiva o Elimina desde el menú.

Tratamientos. Para qué sirve: administrar los tratamientos de lentes (por ejemplo Antirreflejo, Fotocromático) y su precio, que se agrega como una línea al cargarlo en una venta. Se llega por Óptico, Tratamientos. Se crea con Nuevo Tratamiento (Nombre y Precio) y se Edita o Desactiva desde el menú.

C.5.5. Inventario: Niveles de Stock

Para qué sirve: ver y ajustar los niveles mínimo y máximo de cada producto y detectar alertas de bajo o sobre stock.

Cómo se llega: menú Óptica, Inventario, Consultas, Niveles de Stock.

Qué se ve: buscador por nombre, SKU o categoría, y una tabla con Producto, Categoría, Stock actual, Stock mínimo, Stock máximo y Estado (badge Bajo stock, Sobre stock u OK).

Tareas: editar niveles desde el menú de la fila, con Editar niveles de stock. Se cargan Stock mínimo y Stock máximo (opcional). El sistema no deja poner un máximo menor al mínimo.

C.5.6. Inventario: Movimientos

Para qué sirve: consultar el historial de entradas y salidas de stock, ver el detalle y descargar comprobante.

Cómo se llega: menú Óptica, Inventario, Transacciones, Movimientos.

Qué se ve: filtros por Tipo (Entradas / Salidas) y Estado (Pendientes / Aprobados / Rechazados). Tabla con Fecha, Producto, Sucursal, Tipo, Cantidad (con signo), Motivo, Registrado por y Estado.

Tareas:

- Nuevo movimiento: botón Nuevo Movimiento, que abre una página de alta donde se elige el producto (buscando por nombre o SKU), la Fecha y hora, el Tipo (Entrada / Salida), la Cantidad y el Motivo. Se confirma con Confirmar Movimiento. Solo se muestran los motivos aplicables al tipo elegido.
- Ver detalle: al tocar la fila se abre la ficha del movimiento; si ya fue resuelto, muestra quién lo aprobó o rechazó y las observaciones.
- Descargar PDF: en el menú de la fila o desde el detalle.

C.5.7. Inventario: Transferencias

Para qué sirve: mover mercadería entre sucursales.

Cómo se llega: menú Óptica, Inventario, Transacciones, Transferencias.

Qué se ve: una lista de tarjetas, una por transferencia, con Origen a Destino, estado, productos con su cantidad, fecha, quién la creó y observaciones.

Tareas:

- Crear: botón Nueva Transferencia. Se elige Origen (si el usuario no tiene sucursal fija), Destino, los Productos (buscando y cargando cantidades) y Observaciones. Se confirma con Crear transferencia.
- Aceptar o Rechazar: en las transferencias pendientes cuyo destino corresponde al usuario, aparecen los botones Aceptar y Rechazar.

Estados: Pendiente, Aceptada, Rechazada.

C.5.8. Inventario: Registrar Conteo e Inventario Físico

Para qué sirve: hacer el inventario físico, es decir contar la mercadería real y compararla con lo que dice el sistema.

Cómo se llega: menú Óptica, Inventario, Transacciones, Registrar Conteo (para cargar uno nuevo); y Consultas, Inventario Físico (para ver el historial).

Registrar un conteo (Nuevo Conteo de Inventario): se eligen las categorías a contar, se busca el producto y se carga la cantidad realmente contada en la columna Cant. contada (los que se dejan en blanco no se cuentan). Hay un botón Descargar planilla que genera un PDF en blanco para contar a mano. Se puede escribir una Observación y se confirma con Confirmar Conteo.

Ver el historial (Historial de Inventario): tabla con Fecha, Registrado por, cantidad de Productos, Con diferencia y Estado. Cada conteo se puede abrir para ver el detalle y descargar en PDF (una planilla “Inventario Físico” con Producto, SKU, Categoría, Sistema, Físico, Diferencia y Ajuste).

Estados de un inventario físico: Pendiente, Aprobado, Rechazado.

C.5.9. Inventario: Aprobaciones (de movimientos y de conteos)

Aprobaciones de movimientos. Se llega por Óptica, Inventario, Consultas, Aprobaciones. Muestra solo los movimientos Pendientes. En el menú de la fila (o desde el detalle) se puede Aprobar (el sistema aclara que al confirmar se actualiza el stock; permite escribir Observaciones) o Rechazar (con Motivo del rechazo; no se aplican cambios al stock). También se puede Descargar PDF.

Aprobaciones y revisión de inventarios. Al abrir un inventario físico pendiente se ve la “Planilla de comparación” con Producto, Categoría, Sistema (stock al momento del conteo), Físico (lo contado), Diferencia y Ajuste. Con permiso, se puede Aprobar (genera los movimientos de ajuste de stock) o Rechazar (no aplica ningún ajuste), en ambos casos con observaciones.

C.5.10. Configuración: Motivos

Para qué sirve: configurar los motivos que se pueden elegir al registrar movimientos de stock.
Cómo se llega: menú Óptica, Inventario, Configuración, Motivos.

Tareas: crear con Nuevo Motivo (Nombre y Aplica a: Entrada, Salida o Ambos), y Editar, Desactivar o Activar desde el menú.

C.6. Zona Comercial

C.6.1. Compras: Proveedores

Para qué sirve: administrar los proveedores y laboratorios, con datos fiscales, ubicación, redes y contactos.

Cómo se llega: menú Comercial, Compras, Configuración, Proveedores.

Qué se ve: filtro por estado y buscador por nombre o RUC. Tabla con Proveedor, RUC, Ciudad, cantidad de Contactos y Estado.

Tareas:

- Crear: botón Nuevo Proveedor. Campos Nombre comercial, RUC (con formato validado), Razón social, Dirección y Ciudad, redes (Sitio web, WhatsApp, Facebook, Instagram), un interruptor Es laboratorio óptico (para que aparezca al asignar laboratorio en trabajos a pedido) y un bloque de Contactos (Nombre, Cargo, Teléfono, Email; se exige al menos un contacto con nombre). Se guarda con Crear Proveedor.
- Editar / Desactivar / Activar: en el menú de la fila.

C.6.2. Compras: Órdenes de Compra

Para qué sirve: pedir mercadería a los proveedores.

Cómo se llega: menú Comercial, Compras, Transacciones, Órdenes de Compra.

Qué se ve: chips de filtro por estado (Borradores, Confirmadas, Facturadas, Parciales, Recibidas, Canceladas) y una tabla con Número de OC, Proveedor, Fecha, Ítems / Total y Estado.

Tareas:

- Crear: botón Nueva Orden. Se selecciona el Proveedor (con un buscador por nombre, RUC o ciudad), la Fecha, las Observaciones y los Productos a pedir (con cantidad y precio; el sistema autocompleta el precio de costo). Se guarda con Crear Borrador. La orden nace como Borrador y queda pendiente de aprobación.
- Editar: solo mientras está en Borrador, desde el menú de la fila.
- Cancelar: desde el menú, solo en Borrador. No se puede cancelar una orden ya recibida por completo.
- Ver detalle: al tocar la fila. La ficha tiene pestañas Detalle, Factura, Recepciones y Devoluciones, un botón Descargar PDF y, cuando la orden ya fue recibida, un botón Registrar Devolución (para devolver al proveedor, con producto, cantidad y motivo).

Estados de una orden de compra: Borrador, Confirmada, Recibida parcial, Recibida total, Facturada y Cancelada.

C.6.3. Compras: Aprobación de OC

Para qué sirve: revisar las órdenes en Borrador y aprobarlas o rechazarlas.

Cómo se llega: menú Comercial, Compras, Transacciones, Aprobación de OC.

Qué se ve: una tarjeta por orden pendiente, con sus ítems y el total estimado.

Tareas:

- Aprobar: botón Aprobar (la orden pasa a Confirmada y queda lista para facturar).
- Rechazar: botón Rechazar, con Motivo del rechazo opcional (la orden pasa a Cancelada).

C.6.4. Compras: Facturas de Compra

Para qué sirve: registrar y seguir las facturas de los proveedores.

Cómo se llega: menú Comercial, Compras, Transacciones, Facturas de Compra.

Qué se ve: filtros por Estado (Vigente / Anulada), Condición (Contado / Crédito) y Origen (Con OC / Directa), más selector de proveedor, rango de fechas y buscador por número. Tabla con Nro. Factura, Proveedor, Origen, Fecha Emisión, Monto Total, Condición y Estado.

Tareas:

- Crear: botón Nueva Factura. Primero se elige el origen: Compra directa (sin orden previa) o Con OC (vinculada a una orden confirmada). Se completa Número de Factura (con formato automático), Fecha de Emisión, Condición (Contado o Crédito), Método de Pago (obligatorio si es contado) o Vencimiento (obligatorio si es crédito). En Con OC los ítems se copian de la orden; en Directa se cargan a mano (los que tienen producto asociado suman stock). El sistema calcula solo el desglose de IVA (Exento, 5 %, 10 %) y el Total. Se guarda con Guardar Factura.
- Ver detalle y Anular: al abrir la factura se ven todos sus datos; con permiso se puede Anular Factura (con Motivo de anulación). Si la factura tenía una orden asociada, esa orden vuelve al estado Confirmada.

C.6.5. Compras: Recepciones

Para qué sirve: registrar la llegada de la mercadería de una factura con orden.

Cómo se llega: menú Comercial, Compras, Transacciones, Recepciones.

Qué se ve: filtros por estado de la orden, proveedor y fechas. Tabla con número de recepción, Fecha, Proveedor, Factura, OC / Estado, Ítems y Usuario.

Tareas:

- Crear: botón Nueva Recepción. Se busca y elige la Factura con OC pendiente, se pone la Fecha de recepción, y en el detalle de ítems se carga cuánto se Recibe de cada producto (no puede superar lo pendiente), con Lote y Vencimiento opcionales (si se pone uno, el otro es obligatorio). Botones Recibir todo y Limpiar ayudan a completar. Se confirma con Confirmar Recepción. Al recibir, cada producto suma stock (entrada aprobada) y se recalcula su precio de venta según el costo y el margen de la categoría.
- Ver detalle: al tocar la fila (solo lectura), con enlace a la factura asociada.

C.6.6. Compras: Reportes de Compras

Para qué sirve: ver los indicadores de compras del período.

Cómo se llega: menú Comercial, Compras, Reportes, Reportes de Compras.

Qué se ve: selector de período (día, semana, mes), tarjetas de Órdenes de compra, Monto facturado, IVA, Pendiente de pago y Recepciones, un gráfico de Compras facturadas, un gráfico de Estado de OC y una tabla de Compras por proveedor.

Tareas: exportar con los botones CSV y PDF.

C.6.7. Egresos

Para qué sirve: registrar y controlar los pagos y gastos de la óptica (gastos generales, honorarios a profesionales, salarios y facturas de compra o de laboratorio).

Cómo se llega: menú Comercial, Egresos.

Lista de Egresos (Transacciones, Lista de Egresos): chips de filtro por Tipo (Honorario, Gasto General, Factura de compra, Factura de laboratorio) y por Estado (Pendiente, Aprobado, Pagado, Rechazado, Anulado), más un botón Vencidos. Tabla con Tipo, Concepto, Monto, Emisión y Estado. En el menú de la fila: Ver detalle y, si está Pendiente, Anular egreso (con motivo).

Nuevo Egreso (Transacciones, Nuevo Egreso): se elige el Tipo (Gasto General o Pago de Personal). Para Pago de Personal se elige entre Profesional o Empleado, la persona y el Período. Para Gasto General se elige la Categoría (se puede crear una nueva con el botón más). Campos comunes: Concepto, Monto a pagar, Fecha, Vencimiento (opcional), Método de pago y Observaciones. Se envía con Enviar Solicitud; el egreso queda pendiente de aprobación.

Aprobación (Transacciones, Aprobación): bandeja de egresos pendientes. En el menú de la fila: Ver detalle, Aprobar (deja el egreso listo para que caja registre el pago) y Rechazar (con Motivo obligatorio).

Pagos Pendientes (Transacciones, Pagos Pendientes): lista de egresos ya aprobados esperando pago. En el menú, Registrar pago abre la pantalla de pago, donde se cargan Fecha de pago, Método de pago, N. Comprobante y Observaciones, y se confirma con Confirmar Pago. Quien tiene permiso de aprobar egresos puede marcar Pago externo (no afecta la caja), con un motivo obligatorio.

Categorías de Gasto (Configuración): administra las categorías con que se clasifican los gastos generales. Se crea con Nueva Categoría (Nombre y Descripción) y se Edita o Activa/Desactiva desde el menú.

Estados de un egreso: Pendiente, Aprobado, Pagado, Rechazado, Anulado (y la marca Vencido cuando corresponde).

C.6.8. Ventas: Configuración (Timbrados, Clientes, Servicios)

Clientes. Para qué sirve: administrar los clientes y sus datos de facturación. Se llega por Comercial, Ventas, Configuración, Clientes. Filtros por estado y por tipo (Persona Física / Persona Jurídica) y buscador. Se crea con Añadir Cliente. Al editar se cargan los Datos personales y los Datos de facturación (Tipo de Facturación, Razón Social, RUC o CI Fiscal, dirección, email y teléfono de facturación). Se puede Activar o Desactivar desde el menú.

Timbrados. Para qué sirve: administrar los timbrados fiscales (la numeración autorizada) para poder facturar. Se llega por Ventas, Configuración, Timbrados. Se crea con Nuevo Timbrado (Sucursal, Número de Timbrado, Establecimiento y Punto de Expedición de 3 dígitos, N. Desde, N. Hasta, y fechas de inicio y fin de vigencia). Se Editar o Desactiva desde el menú.

Servicios. Para qué sirve: administrar el catálogo de servicios (consulta, ajuste, reparación) y sus tarifas. Se llega por Ventas, Configuración, Servicios. Se crea con Nuevo Servicio (Nombre, Descripción, Precio). Desde el menú, Tarifas por profesional permite cargar precios distintos por profesional o por especialidad. Se puede Editar y Desactivar.

C.6.9. Ventas: Presupuestos

Para qué sirve: armar cotizaciones que todavía no son ventas.

Cómo se llega: menú Comercial, Ventas, Transacciones, Presupuestos.

Qué se ve: tabla con Número de Presupuesto, Cliente, Fecha, Tipo, Total y Vigencia (con semáforo de vencimiento).

Tareas:

- Crear: botón Nuevo Presupuesto (abre el mismo formulario que Nueva Venta, en modo presupuesto). Además de los datos de la venta, tiene un campo Validez en días (por defecto 15) y muestra la fecha de vencimiento. El laboratorio no es obligatorio aquí. Se guarda con Guardar presupuesto.
- En el menú de la fila: Descargar PDF, Generar venta (lleva a Nueva Venta con los datos precargados) y Eliminar.

C.6.10. Ventas: Nueva Venta

Para qué sirve: cargar una venta, ya sea un trabajo óptico a pedido (receta más armazón más lente) o una venta directa de mostrador (productos y servicios).

Cómo se llega: menú Comercial, Ventas, Transacciones, Nueva Venta.

Qué se ve: un selector de modo con dos botones, A pedido (óptica) y Directa (mostrador). La columna principal tiene el selector de cliente, y según el modo el selector de receta y la tarjeta del trabajo óptico. Un bloque de líneas permite agregar productos o servicios (con un conmutador Producto / Servicio, un buscador y un botón Línea manual). La columna lateral tiene Condiciones (Contado / Crédito, Fecha, Observaciones) y el Resumen con el desglose de IVA y el Total.

Tareas (venta a pedido):

1. Elegir el modo A pedido (óptica).
2. Seleccionar el cliente (si no se elige ninguno, la venta queda a Consumidor Final).
3. Seleccionar la receta (es obligatoria; si falta, avisa que la receta es obligatoria para una venta a pedido).
4. Completar la tarjeta óptica (armazón, lente, tratamientos) y elegir el laboratorio (obligatorio para confirmar).

5. Opcional: agregar productos o servicios extra, o una Línea manual (Descripción, Cantidad, Precio Unitario, Categoría Fiscal). Si un servicio tiene tarifas por profesional, se abre una ventana para elegir el profesional y ver el origen del precio.
6. Definir Contado o Crédito, fecha y observaciones.
7. Botón Crear venta. Al guardar, la venta se confirma y salta directo a la pantalla de cobro.

Tareas (venta directa): elegir Directa (mostrador), cliente (opcional), agregar productos o servicios, condiciones, y Crear venta.

C.6.11. Ventas: Lista de Ventas y Detalle

Lista de Ventas. Para qué sirve: consultar todas las ventas (solo consulta, no se crea ni edita desde aquí). Se llega por Comercial, Ventas, Transacciones, Lista de Ventas. Filtros por Estado y Tipo y buscador por cliente o comprobante. Tabla con Comprobante, Cliente, Tipo, Fecha, Total, Saldo y Estado. Al tocar una fila se abre el detalle.

Detalle de la venta. Muestra el encabezado con estado y tipo, la botonera de acciones (según el estado: Confirmar, Registrar envío, Registrar recepción, Devolver / Cambiar, Emitir comprobante, Cancelar), las Líneas de venta, los Cobros, el Pedido al laboratorio (si es a pedido) y las Devoluciones, más el Resumen y los datos del Comprobante y la Factura.

- Confirmar (solo en Borrador): si es Directa pasa a Lista para cobrar; si es a pedido pasa a En proceso y el trabajo entra a la cola del laboratorio.
- Devolver / Cambiar (solo con comprobante emitido): ventana con Tipo (Devolución o Cambio), Motivo y los productos a devolver. Solo se pueden devolver productos con stock, no servicios ni lentes a pedido.
- Cancelar: solo se puede en Borrador, Confirmada o En proceso; no una venta ya con comprobante emitido.

Estados de una venta: Borrador, Confirmada, En proceso, Lista para cobrar, Comprobante emitido y Cancelada.

C.6.12. Ventas: Cobros Pendientes y Cobrar

Cobros Pendientes. Para qué sirve: ver todas las ventas con saldo por cobrar. Se llega por Comercial, Ventas, Transacciones, Cobros Pendientes. Muestra el Saldo total pendiente, una tabla con Comprobante, Cliente, Fecha, Antigüedad, Total, Cobrado, Saldo y Estado, y un panel que agrupa el saldo Por cliente. En el menú de la fila: Cobrar, Emitir comprobante y Ver detalle.

Cobrar venta. Para qué sirve: registrar los cobros (seña, cuota o pago) de una venta. Se ve el estado de la venta y, si se puede cobrar, un formulario de Pago con: Fecha; si es a crédito, el Tipo (Seña o Cuota); y una sección Métodos de pago donde cada línea tiene el método (Efectivo, Tarjeta, Transferencia, Cheque), el Monto y una Referencia opcional. El botón Agregar método suma líneas. Se registra con Registrar cobro. Un cobro no puede superar el saldo pendiente. Si el método es efectivo, hace falta tener la caja abierta: si no, aparece el aviso “No hay una caja abierta” con un botón Abrir caja (tarjeta, transferencia y cheque sí se pueden registrar sin caja). Cuando el saldo llega a cero, el sistema recuerda emitir el comprobante para cerrar la venta.

C.6.13. Ventas: Emitir documento (comprobante o factura)

Para qué sirve: emitir el documento de la venta, ya sea un Recibo Simple (sin datos fiscales) o una Factura Timbrada (documento fiscal). Al emitir se descuenta el stock, se generan los

movimientos de caja y la venta pasa a Comprobante emitido. Esta acción no se puede deshacer.

Cómo se llega: desde Cobrar venta o desde el detalle de la venta, con Emitir comprobante.

Tareas:

- Recibo: seleccionar Recibo Simple y tocar Emitir recibo.
- Factura: seleccionar Factura Timbrada, elegir el Timbrado (si no hay uno activo, ofrece Ir a Timbrados), revisar el próximo número y los datos de la serie, cargar la Fecha de Emisión y Observaciones, y tocar Emitir factura. El sistema controla que el timbrado esté activo, sea de la sucursal de la venta, que la fecha caiga dentro de su vigencia y que no se haya llegado al número máximo.

C.6.14. Ventas: Facturas de Venta

Para qué sirve: consultar el historial de facturas timbradas emitidas a clientes.

Cómo se llega: menú Comercial, Ventas, Transacciones, Facturas de Venta.

Qué se ve: filtros por Tipo y Condición, rango de fechas y buscador. Tabla con Nro. Factura, Cliente, Tipo, Condición, Fecha Emisión, Total e IVA total. En el menú de la fila: Ver venta y Descargar PDF.

C.6.15. Laboratorio

El circuito del laboratorio es para las ventas a pedido: cuando se confirma una venta con lente a pedido, se genera un pedido de trabajo que hay que aprobar, enviar al laboratorio y recibir cuando vuelve terminado.

Pedidos (Comercial, Laboratorio, Pedidos): lista de todos los pedidos, con filtro por Estado y buscador. Tabla con Comprobante, Cliente, Tipo de lente, Laboratorio, Fecha, Estado y Factura. Al abrir un pedido se ve su ficha (receta, lente, armazón, tratamientos) y se puede Imprimir orden o Ver venta completa.

Aprobaciones (Laboratorio, Aprobaciones): pedidos pendientes de aprobación. Con permiso, se puede Aprobar o Rechazar (con Observaciones). Al aprobar, el pedido queda pendiente de envío.

Envíos y Recepciones (Laboratorio, Envíos y Recepciones): filtro por Estado y un semáforo de atraso de entrega. Registrar envío al lab abre una ventana con la Entrega estimada y el Medio de envío (WhatsApp, Email, Portal del laboratorio, Teléfono, En persona, Otro), más un botón para Imprimir orden de trabajo; se confirma con Registrar envío. Cuando vuelve, Registrar recepción marca el pedido como recibido y avisa a la sucursal que está listo para retirar. El cobro de la venta es independiente y no se ve afectado.

Facturas (Laboratorio, Facturas): las facturas que el laboratorio emite por los pedidos ya recibidos (lo que la óptica le debe al laboratorio). Con el botón Registrar se carga N. Factura, Timbrado, Fecha Emisión, Monto y Observaciones. Al registrar la factura del laboratorio se genera un egreso pendiente de pago.

Estados de un pedido de laboratorio: Pendiente de aprobación, Pendiente de envío, Enviado, Recibido y Rechazado.

C.6.16. Caja

Caja (Comercial, Caja, Caja): maneja la caja del día. Muestra si hay una sesión abierta (fecha y usuario) o no, la Posición de efectivo (Efectivo inicial, Ingresos efectivo, Egresos efectivo,

Movimientos) y la lista de Movimientos de la sesión. Los cobros y pagos que no son en efectivo aparecen con la etiqueta “No afecta caja”. Abrir y cerrar caja requieren permiso.

- Abrir caja: botón Abrir caja. El sistema propone como Monto inicial el efectivo del último cierre; se puede ajustar a mano. Se confirma con Abrir caja. No puede haber dos cajas abiertas a la vez en la misma sucursal.
- Cerrar caja: botón Cerrar caja. Se muestran monto inicial e ingresos y egresos en efectivo como referencia, y se carga el Efectivo contado (lo que se contó físicamente). El sistema indica en vivo si coincide o cuál es la diferencia, y permite una Observación. Se confirma con Confirmar cierre.

Historial de Caja (Caja, Historial de Caja): todas las sesiones pasadas, con Apertura, Cierre, Duración, quién la abrió y cerró, Ingresos, Egresos, Saldo y Estado. En el menú, Ver detalle abre el arqueo (Esperado, Contado, Diferencia) y los movimientos, con las marcas Cuadrada, Sobrante o Faltante.

Aprobaciones (Caja, Aprobaciones): los cierres que quedaron Pendientes de aprobación por diferencia. El supervisor puede Ver arqueo, Aprobar cierre (queda cerrado) o Rechazar cierre (con Motivo obligatorio; el cajero deberá hacer un nuevo conteo).

Estados de una sesión de caja: Abierta, Cerrada y Pendiente de aprobación.

C.7. Zona Gestión

C.7.1. Reportes

Se llega por menú Gestión, Reportes. Es un tablero de entrada con tarjetas que llevan a cada reporte. Se dividen en gerenciales (Reportes de Citas, de Ventas, de Inventario) y operativos (Ventas, Compras, Movimientos de inventario, Movimientos de caja).

- Reportes de Citas: turnos, asistencia, consultas y recetas del período (día, semana, mes), con indicadores, una tendencia, un gráfico por estado, una tabla por profesional y un panel de ausentismo. Exporta a CSV y PDF.
- Reportes de Ventas: facturación, cobros, productos y servicios más vendidos y desempeño por cajero, con indicadores y gráficos. Exporta a CSV y PDF.
- Reporte de Inventario: stock actual, valorización y movimientos del período, con indicadores, stock crítico y top de productos por valor. Exporta a CSV y PDF.
- Reportes operativos: un listado detallado fila por fila (Ventas, Compras, Movimientos de inventario o Movimientos de caja) con filtros por fecha, método de pago, categoría, tipo, operador y sucursal, y un botón Limpiar. Exporta a CSV y PDF.

C.7.2. Personal

Profesionales (Gestión, Personal, Profesionales): administra el equipo profesional. Filtro por estado y buscador por nombre, cédula o matrícula. Se crea con Añadir Profesional (nombre, apellido, cédula, nacimiento, email, teléfono, contraseña, matrícula, especialidades, sucursal). En el menú de la fila: Horario (define los días y horas de trabajo y las pausas), Editar y Desactivar. El horario de trabajo es lo que determina qué turnos se pueden reservar.

Empleados (Personal, Empleados): administra el personal no médico (vendedores, recepcionistas, cajeros). Se crea con Añadir Empleado (datos personales, email, contraseña, Cargo, Fecha de Ingreso, Salario Base, sucursal). Se Edita y Desactiva desde el menú.

Cargos (Personal, Cargos): la lista de puestos que se asignan a los empleados. Se crea con Nuevo Cargo (Nombre y Descripción) y se Editar o Activa/Desactiva.

Especialidades (Personal, Especialidades): las especialidades que se asignan a los profesionales. Se crea con Nueva Especialidad (Nombre y Descripción) y se puede Editar o Eliminar (aquí la eliminación es real, no baja lógica).

C.7.3. Administración

Usuarios (Gestión, Administración, Accesos, Usuarios): administra las cuentas del sistema y sus roles. Filtro por tipo y estado. Esta pantalla no crea usuarios nuevos: gestiona los existentes. En cada fila, Gestionar Roles abre una ventana para asignar o quitar roles, Restablecer contraseña (la nueva es provisoria y el usuario deberá cambiarla en su próximo ingreso) y Desactivar usuario.

Roles y Permisos (Accesos, Roles y Permisos): la lista de roles y cuántos permisos tiene cada uno, con el tipo Sistema o Personalizado. Se crea con Nuevo Rol y se editan los permisos con el botón de lápiz. Los roles de sistema no se pueden borrar. El formulario de rol tiene el Nombre del rol y una grilla de permisos por módulo (se puede marcar todo un módulo o permiso por permiso). El rol Administrador es gestionado por el sistema y no se edita desde aquí.

Configuración (Administración, Empresa, Configuración): los datos del negocio (Nombre Fantasia, Razón Social, RUC, Dirección, Teléfono, Email, Sitio Web), que se usan por ejemplo en los encabezados de reportes. Con permiso, se guarda con Guardar.

Sucursales (Empresa, Sucursales): administra los locales del negocio. Se crea con Nueva Sucursal (Nombre, Código, Dirección, Teléfono, Email, Ciudad, Establecimiento) y se Editar o Activa/Desactiva. Una sucursal inactiva no se puede asignar a nuevos usuarios ni operaciones.

Ubicaciones (Empresa, Ubicaciones): administra los departamentos y ciudades que se usan como ubicaciones (por ejemplo al crear sucursales). Tiene pestañas Departamentos y Ciudades, con alta, edición y activación/desactivación.

Auditoría (Empresa, Auditoría): en esta versión es una sección en preparación.

C.8. Flujos completos de punta a punta

C.8.1. Flujo A: Atender a un paciente

1. Agendar el turno. Desde Agenda, botón Nuevo Turno: se elige profesional, fecha, horario y paciente, y un motivo. El turno queda en estado Pendiente. (El paciente también puede reservarlo solo desde Mis Turnos, y puede confirmarlo desde el enlace que le llega por correo, con lo que pasa a Confirmado.)
2. Recepción. Cuando el paciente llega, se lo marca como presente: desde Agenda, en la fila del turno, Confirmar llegada. El turno pasa a Presente y el paciente aparece en la pantalla Recepción.
3. Iniciar la consulta. Desde Recepción, en la fila del paciente, Iniciar consulta. Eso abre el formulario clínico con el paciente ya cargado. El profesional completa Motivo y Diagnóstico Principal, opcionalmente los detalles (anamnesis, examen, plan) y guarda con Guardar Consulta.
4. Cargar la receta. En el mismo formulario de la consulta se activa el interruptor Incluir receta óptica y se cargan los valores (OD/OI con esfera, cilindro, eje, adición, DIP, etc.). También se puede agregar después desde Consultas, en el menú de la fila, con Agregar

receta. La receta queda disponible para el paciente (Mis Recetas, con PDF) y es la que después alimenta la venta a pedido.

5. Cerrar la consulta y marcar el turno como Completado.

Una aclaración: el paso del turno a Completado se hace por el cambio de estado del turno; el sistema no lo fuerza automáticamente al cerrar la consulta, es una acción que hace el usuario.

C.8.2. Flujo B: Vender unos lentes

1. Seleccionar cliente y cargar la venta. Desde Ventas, Nueva Venta, se elige el modo A pedido (óptica), se selecciona el cliente (o queda Consumidor Final), se selecciona la receta del paciente, se completa la tarjeta óptica (armazón, lente, tratamientos) y se elige el laboratorio. Se pueden agregar productos o servicios extra. Se define Contado o Crédito. Botón Crear venta: la venta se confirma y lleva directo a cobrar.
2. Cobrar. En Cobrar venta se registran los pagos (seña, cuota o total), eligiendo método (efectivo, tarjeta, transferencia, cheque). Si es efectivo, la caja tiene que estar abierta. Botón Registrar cobro.
3. Emitir el comprobante. Con Emitir comprobante se elige Recibo Simple o Factura Timbrada. Para factura se elige el timbrado, se pone la fecha y se emite. Al emitir, el stock se descuenta, se genera el movimiento de caja y la venta pasa a Comprobante emitido.
4. Laboratorio (para el lente a pedido). El pedido de trabajo generado al confirmar la venta se aprueba (Laboratorio, Aprobaciones), se envía (Envíos y Recepciones, Registrar envío) y, cuando vuelve, se recibe (Registrar recepción). El cobro es independiente de este circuito.

Dos detalles importantes: se puede cobrar apenas la venta está confirmada (Lista para cobrar), sin esperar al laboratorio; y el descuento de stock ocurre recién al emitir el comprobante, no al crear la venta.

C.8.3. Flujo C: Abrir y cerrar la caja del día

1. Abrir. Desde Caja, botón Abrir caja. El sistema propone como monto inicial el efectivo con que quedó el último cierre (o cero si es la primera vez); se puede ajustar. Se confirma con Abrir caja. Con la caja abierta ya se pueden registrar cobros y pagos en efectivo. No puede haber dos cajas abiertas a la vez en la misma sucursal.
2. Operar el día. Cada cobro en efectivo de una venta y cada pago en efectivo de un egreso genera un movimiento en la caja. Los pagos con tarjeta, transferencia o cheque se registran pero no afectan el efectivo de la caja.
3. Cerrar. Botón Cerrar caja. Se cuenta el efectivo real y se carga en Efectivo contado. Se confirma con Confirmar cierre.

Qué valida el sistema al cerrar: la caja tiene que estar Abierta. El sistema calcula el efectivo esperado como monto inicial más ingresos en efectivo menos egresos en efectivo, y lo compara con lo contado. Hay una tolerancia fija de 50.000 guaraníes: si la diferencia (de más o de menos) es de 50.000 o menos, la caja queda Cerrada directamente; si la diferencia supera los 50.000, la caja queda Pendiente de aprobación y un supervisor tiene que revisarla en Caja, Aprobaciones (donde puede Aprobar el cierre o Rechazarlo, y si lo rechaza la sesión vuelve a estar Abierta para un nuevo conteo). El cierre es un arqueo de efectivo; el sistema no bloquea el cierre por tener ventas sin cobrar.

C.8.4. Flujo D: Reponer stock

1. Pedido al proveedor. Desde Compras, Órdenes de Compra, Nueva Orden: se elige el proveedor y se cargan los productos con cantidad y precio. Se guarda como Borrador. Con permiso, se aprueba en Aprobación de OC (pasa a Confirmada).
2. Factura de compra. Desde Facturas de Compra, Nueva Factura, con origen Con OC: se elige la orden confirmada, se cargan número de factura, fecha, condición (contado o crédito) y, en contado, el método de pago. La orden pasa a Facturada. (Si es contado en efectivo, hace falta caja abierta y la factura queda pagada; si es crédito, queda pendiente de pago.)
3. Recepción de la mercadería. Desde Recepciones, Nueva Recepción: se elige la factura con orden pendiente y se carga cuánto se recibe de cada producto (con lote y vencimiento si aplica). Al confirmar, cada producto suma stock (entrada aprobada) y se recalcula su precio de venta según el costo y el margen de la categoría. La orden pasa a Recibida parcial o Recibida total según lo recibido.

Una aclaración sobre el orden: la recepción se registra contra la factura, así que la secuencia efectiva es Orden de Compra, luego Factura de Compra, y luego Recepción (la recepción necesita que la orden ya esté Facturada), encadenadas a través de la factura y no de forma independiente. El stock lo actualiza la recepción, no la factura.

C.9. Preguntas frecuentes y errores comunes

Qué significa cada estado de un turno. Pendiente: reservado, todavía sin confirmar. Confirmado: el paciente confirmó que viene. Presente: el paciente llegó y está en sala. Completado: ya fue atendido. Cancelado: se anuló. Si el paciente pidió cancelar, aparece además la marca Solicita cancelar hasta que el equipo la resuelve.

Qué significa cada estado de una consulta. Pendiente, Abierta (en curso), Cerrada (terminada) y Cancelada.

Qué significa cada estado de una venta. Borrador (es un presupuesto, todavía editable), Confirmada, En proceso (venta a pedido cuyo lente está en el circuito del laboratorio), Lista para cobrar, Comprobante emitido (ya se emitió recibo o factura y se descargó el stock) y Cancelada.

Qué significa cada estado de una orden de compra. Borrador, Confirmada (aprobada y enviada al proveedor), Facturada (se cargó la factura del proveedor), Recibida parcial, Recibida total y Cancelada.

Qué significa cada estado de un egreso. Pendiente (esperando aprobación), Aprobado (listo para pagar), Pagado, Rechazado y Anulado. La marca Vencido aparece cuando pasó su vencimiento y todavía no se pagó.

Qué significa cada estado de un pedido de laboratorio. Pendiente de aprobación, Pendiente de envío, Enviado, Recibido y Rechazado.

Qué significa cada estado de la caja. Abierta, Cerrada y Pendiente de aprobación (cuando el cierre tuvo una diferencia mayor a la tolerancia y espera revisión de un supervisor).

Por qué no veo tal opción o módulo. Porque el rol del usuario no tiene ese permiso. El menú muestra solo lo que el rol tiene habilitado; si un módulo entero falta, es que no tiene ninguno de sus permisos. Lo mismo pasa dentro de una pantalla: botones como Nuevo, Editar, Desactivar, Aprobar o Anular aparecen solo si se tiene el permiso correspondiente. Si se necesita una opción, hay que pedirle al Administrador que ajuste los permisos del rol.

Por qué no se puede cerrar la caja. Puede ser porque no hay una sesión Abierta (ya se cerró, o nunca se abrió), o porque no se tiene el permiso para operar la caja. Además, si el conteo

da una diferencia mayor a 50.000 guaraníes, la caja no queda cerrada del todo: queda Pendiente de aprobación hasta que un supervisor la apruebe.

Por qué no se puede cobrar en efectivo. Porque no hay una caja abierta en la sucursal. Los cobros en efectivo necesitan la caja abierta; los de tarjeta, transferencia o cheque se pueden registrar igual. Hay que abrir la caja y volver a intentar.

Por qué no se puede facturar. Puede faltar un timbrado activo, vigente y de la sucursal, o el timbrado puede haber llegado a su número máximo. Conviene revisar Timbrados.

Por qué no deja editar una venta o una orden. Porque solo se pueden editar mientras están en Borrador. Una vez confirmadas, ya no.

Qué es una sucursal y cómo afecta lo que se ve. Una sucursal es un local del negocio. La óptica arranca con una sucursal llamada Casa Central. Muchas operaciones están atadas a una sucursal: el stock, la caja, los timbrados y las transferencias son por sucursal. Según el usuario, es posible ver y operar solo sobre la sucursal propia. Con un permiso especial para ver todas las sucursales, se ve la información de todas. Por eso, por ejemplo, la caja que se abre es la de la propia sucursal, y una transferencia mueve stock de una sucursal a otra.

Qué pasa la primera vez que se entra. Si la cuenta es nueva o se restableció la contraseña, el sistema obliga a cambiar la contraseña antes de dejar hacer cualquier otra cosa.

C.10. Cobertura

Este manual cubre todas las zonas del menú (General, Atención, Óptica, Comercial, Gestión) y sus pantallas, más los cuatro flujos de trabajo completos.

Pantallas documentadas:

- General: Panel de Control, Notificaciones, Mis Turnos, Mi Historial, Mis Recetas.
- Atención: Pacientes (lista, alta rápida, página de alta y ficha de detalle), Clientes, Agenda, Recepción, Consultas (lista, formulario de alta, edición), Historial Clínico.
- Óptica: Productos, Categorías, Marcas, Modelos, Tipos de Lente, Tratamientos, Niveles de Stock, Movimientos y su formulario, Motivos, Aprobaciones de movimientos, Registrar Conteo, Historial de Inventario, revisión de inventario, Transferencias.
- Comercial: Proveedores, Órdenes de Compra (lista, formulario, detalle, aprobación), Facturas de Compra (lista, formulario, detalle), Recepciones (lista, formulario, detalle), Reportes de Compras, Egresos (lista, nuevo, aprobación, pagos, registrar pago, categorías de gasto), Clientes, Timbrados, Servicios, Presupuestos, Nueva Venta, Lista de Ventas, Detalle de Venta, Cobros Pendientes, Cobrar, Emitir documento, Facturas de Venta, Laboratorio (pedidos, aprobaciones, envíos y recepciones, facturas), Caja (apertura y cierre, historial, aprobaciones).
- Gestión: Reportes (hub, citas, ventas, inventario, operativos), Profesionales, Empleados, Cargos, Especialidades, Usuarios, Roles y Permisos, Configuración, Sucursales, Ubicaciones.
- Autenticación: ingreso, cambio de contraseña obligatorio, recuperar contraseña, verificar email, confirmar turno, registro de pacientes, cerrar sesión.

Pantallas que no se pudieron documentar como funcionales:

- Auditoría (Gestión, Administración, Empresa, Auditoría): en esta versión es una sección en preparación, no una pantalla operativa.
- La pestaña Ventas dentro de la ficha del paciente figura como “Pronto” / “En desarrollo”.

- En la pantalla de ingreso aparece un botón Entrar con Google que, según el código, no tiene una acción conectada; por eso no se documenta como funcional.

Aclaraciones sobre los límites de esta verificación:

- Los roles y permisos se tomaron tal cual están definidos en el sistema. De fábrica existen tres roles: Administrador, Profesional y Paciente. Cualquier otro rol (Recepcionista, Vendedor, Cajero, etc.) es un rol a medida que crea el Administrador; sus permisos exactos dependen de cómo lo configure cada óptica, por lo que no se pueden documentar de antemano.
- El paso de un turno a Completado y el cierre de una consulta se hacen por cambio de estado del usuario; el sistema no encadena automáticamente uno con otro.

Apéndice D

Manual Técnico

Este manual está dirigido a quien tenga que levantar SIGA en un entorno de desarrollo, mantenerlo o extenderlo. Es una guía operativa: cómo poner el sistema a correr en una máquina nueva, cómo agregar funcionalidad siguiendo las convenciones que ya usa el proyecto, y qué particularidades del código conviene conocer antes de tocarlo.

Deliberadamente no repite lo que ya está en el cuerpo del documento. Para entender *por qué* el sistema está construido como está, la referencia es:

- **Arquitectura** (capas, pipeline de seguridad, aislamiento multi-sucursal, servicios de fondo): Capítulo 3.
- **Arquitectura del frontend** (estado, enrutado, permisos en la UI): Capítulo 4.
- **Modelo de datos** (entidades, relaciones, enums, convenciones de tipos): Capítulo 20.
- **Despliegue en producción** (servidor, `docker-compose.prod.yml`, variables de entorno, TLS): Capítulo 21.
- **Endpoints** (los 234, con su policy y sus DTOs): Apéndice A.
- **Decisiones de diseño** y su justificación: Apéndice B.

D.1. Entorno de desarrollo

D.1.1. Requisitos

- SDK de .NET 10, para compilar y correr el backend y para crear migraciones.
- Node `~20.19.0 || >=22.12.0`, para el frontend.
- Docker, para la base de datos local.

A diferencia de producción — donde todo corre en contenedores y el host solo necesita Docker (ver Capítulo 21) — en desarrollo conviene tener el SDK y Node instalados en la máquina, para aprovechar la recarga en caliente de ambos lados.

D.1.2. Levantar el sistema

Los dos repositorios (**SIGA** y **SIGA-Web**) se clonan como directorios hermanos. Después, tres pasos, cada uno en su terminal:

```
# 1. Base de datos: Postgres 16 en un contenedor
cd SIGA && docker compose up -d

# 2. Backend: escucha en http://localhost:5038
dotnet run --project SIGA/src/SIGA.Api

# 3. Frontend: Vite en modo desarrollo
cd SIGA-Web && npm install && npm run dev
```

El contenedor de la base se llama `sig-postgres` y publica el puerto **5433** del host (mapeo 5433:5432), no el 5432 — así no choca con un Postgres ya instalado localmente.

Al arrancar, el backend aplica las migraciones pendientes y siembra los datos base (permisos, roles y la cuenta administradora inicial) sin intervención manual:

```
await db.Database.MigrateAsync();
await DbSeeder.SeedAsync(db);
```

El frontend no necesita configuración de CORS: el proxy de Vite (`vite.config.ts`) reenvía `/api/*` y `/uploads/*` al backend (`VITE_API_URL`, por defecto `http://localhost:5038`). Por eso todos los servicios del frontend usan rutas relativas, y ese mismo esquema se reproduce en producción con el proxy de borde.

Scripts disponibles en el frontend:

```
npm run dev      # levanta Vite en modo desarrollo
npm run build    # type-check (vue-tsc --build) EN PARALELO con vite
                 build
npm run preview  # sirve el dist ya construido
npm run format  # prettier --write sobre src/
```

D.2. Dónde está cada cosa

El mapa del código ya está en el cuerpo del documento y no se repite aquí, para que exista en un solo lugar: los cuatro proyectos del backend (`SIGA.Domain`, `SIGA.Application`, `SIGA.Infrastructure`, `SIGA.Api`), con qué contiene cada uno y en qué orden dependen entre sí, están en el Capítulo 3, §3.3; el árbol de `SIGA-Web/src/` (vistas, componentes, servicios, composables, stores, router, configuración del menú) está en el Capítulo 4, §4.2.

Vale la pena tener presente una sola cosa al buscar dónde hacer un cambio: la lógica de negocio vive íntegramente en `SIGA.Infrastructure/Services/`. Los controllers no la tienen, y las entidades de `SIGA.Domain` solo tienen las reglas que dependen de sí mismas.

D.3. Trabajar con migraciones

Crear una migración nueva:

```
dotnet ef migrations add <Nombre> --project SIGA/src/SIGA.Infrastructure
--startup-project SIGA/src/SIGA.Api
```

Se aplican solas en el siguiente arranque de la API. Tres cosas a tener en cuenta:

- **El prefijo NNN del nombre no indica el orden.** Hay 88 migraciones y el NNN más alto es 064: la numeración no es una secuencia completa, hay prefijos duplicados (003, 008, 021

a 027, 031 a 033, 041 a 043, 047, 048 y 059 aparecen dos veces cada uno) y 7 migraciones tempranas no llevan prefijo. El único orden real es el *timestamp* de 14 dígitos con el que empieza el nombre del archivo, que es también el que usa el historial de migraciones de EF.

- **Con más de una réplica, todas ejecutan `MigrateAsync` a la vez** y compiten por aplicar la misma migración. Conviene arrancar de a una la primera vez, o correr la migración como paso separado del despliegue.
- **Conviene leer la migración autogenerada antes de aplicarla** y probarla en limpio, subida y bajada: a veces arrastra cambios no relacionados que quedaron pendientes en el modelo.

D.4. Cómo agregar un endpoint nuevo

Receta paso a paso, con los archivos a tocar en orden:

1. **DTOs** en `SIGA.Application/DTOs/<Area>/`: el request (`Crear{Entity}Request`) y la response (`{Entity}Response` o `{Entity}Dto`). Nunca exponer la entidad de Domain.
2. **Método en la interfaz** del servicio, en `SIGA.Application/Interfaces/I{Entity}Service.cs`, devolviendo `Task<Result<T>` (o `Task<Result<PagedResult<T>>` para listados).
3. **Implementación** en `SIGA.Infrastructure/Services/{Entity}Service.cs`. Ahí va la lógica: validar toda la entrada y devolver `Result.Failure(msg, ErrorType.X)` en cada caso de error. Si la operación persiste stock, dinero u otra entidad con dimensión de sucursal, fijar `SucursalId` con `SucursalResolver.WriteBranchAsync`. **Para leer no hace falta filtrar por sucursal a mano**: el filtro global de EF ya lo hace (ver Capítulo 3, §3.6).
4. **Registrar el servicio**, si es nuevo, en `SIGA.Infrastructure/DependencyInjection.cs` dentro de `AddInfrastructure` (`services.AddScoped<I{Entity}Service, {Entity}Service>()`). Nunca en `Program.cs`.
5. **Método en el controller** de `SIGA.Api/Controllers/`: liviano, decorado con `[Authorize(Policy = "...")]`, que llama al servicio y termina en `return ToHttpResponse(result)`. Sin lógica de negocio ni acceso al `AppDbContext`.

Un permiso nuevo se declara en dos lugares. Si el endpoint usa un permiso que todavía no existe, hay que agregarlo tanto al arreglo `permissionPolicies` de `Program.cs` — que es el que registra la `AuthorizationPolicy` de ASP.NET — como a `DbSeeder.AllPermissions` y al rol correspondiente, que es lo que lo siembra en la base. Son dos listas separadas y nada las sincroniza: sembrar el permiso sin registrar la policy hace que el endpoint responda 500 (*AuthorizationPolicy not found*) en vez de autorizar, y registrarla sin sembrar el permiso hace que nadie lo tenga y responda 403. La asimetría que existe hoy es `ver_recepcion`: está en el seed pero no tiene policy declarada, así que no puede usarse en un `[Authorize(Policy=...)]` aunque exista en la base.

D.5. Cómo agregar una vista nueva

1. **Servicio** (`src/services/<entidad>Service.ts`). Conviene seguir el estilo del archivo vecino: si el área ya usa `useHttp`, usarlo; si no, `import { http } from "@api/http"` directo. Se declaran las interfaces de request/response y una función por operación. Muchos listados devuelven `PagedResult<T>` (`items`, `totalCount`, `totalActive`, `page`, `pageSize`,

totalPages) y reciben { page?, pageSize?, search?, status? } serializados con URLSearchParams.

2. **Vista** (src/views/<Nombre>View.vue), con <script setup lang="ts">. Layout autenticado estándar:

```
<div class="min-h-screen" style="background-color: var(--color-background)">
  <AppSidebar />
  <AppHeader />
  <main style="margin-left: 280px; padding-top: 64px">
    <div class="p-8"><!-- contenido edge-to-edge, sin max-w ni mx-auto --></div>
  </main>
</div>
```

Se reusan los componentes base: encabezado con <h1 class="text-4xl font-extrabold tracking-tight"> y BaseButton primary lg; *toolbar* con FilterChips (izquierda) más SearchInput class="w-72" (derecha); BaseTable para el listado; BaseModal para crear/editar (size="lg") y confirmar (size="sm"); inputs especializados (MontoInput, DateInput, SearchableSelect) en vez de nativos. Estilos de campo vía inputStyle() de useFieldStyles, nunca hex ni font-family inline. Los estados de carga, vacío y error se manejan con try/catch mapeando err.status.

3. **Ruta** en src/router/index.ts, con importación dinámica:

```
{
  path: "/mi-entidad",
  name: "mi-entidad",
  component: () => import("@/views/MiEntidadView.vue"),
  meta: { requiresAuth: true, permission: "ver_mi_entidad", label:
    "Mi Entidad" },
}
```

4. **Ítem de menú** en src/config/menuConfig.ts: en la zone que corresponda, con su route, su icon (Material Symbols) y su permission. Sin esto la vista existe pero no aparece en el *sidebar*.
5. **Permisos de acción**: envolver los botones sensibles con v-if="auth.hasPermission('...')", o usar el campo hidden de RowContextMenu.

Antes de integrar el cambio: npm run build (que incluye el *type-check*) y probar la pantalla logueado, en modo claro y en modo oscuro.

D.6. Convenciones a tener en cuenta

Particularidades del código que no son evidentes al leerlo por primera vez y que conviene conocer antes de extenderlo:

- **El error del backend no tiene una forma única.** BaseController.ToHttpResponse devuelve el error como string JSON pelado; EmpleadosController — el único que no hereda de BaseController — define su propio ToResponse que devuelve { message }; y hay *returns* ad-hoc con { message } en algunos puntos. El frontend lo absorbe en un solo

lugar, el interceptor de `src/api/http.ts`, con la cascada `typeof data === "string" ? data : (data?.message ?? data?.error ?? "Error {status}").` Es tolerante, pero un endpoint nuevo que devuelva el detalle bajo otra clave (`detail`, `title`) hará que el usuario vea solo el código de estado. Al agregar endpoints conviene apuntar a `{ message }`.

- **Conviven dos estrategias de paginación.** *Server-side:* 8 servicios devuelven `PagedResult<T>` y el pie de página usa `totalCount/totalPages` del backend. *Client-side:* cerca de 18 vistas hacen *slicing* local con una constante `PAGE_SIZE` (típicamente 10) sobre la lista completa ya traída. Antes de armar el pie de página de una pantalla conviene mirar cuál usa su servicio, sin asumir.
- **No hay contrato de paginación uniforme en la API.** `sortBy/sortOrder` no existen en ningún controller; el filtro de activo/inactivo no tiene estándar (`isActive` en unos, `status` string en otros, `soloActivos` en Empleados); y el `pageSize` por defecto varía por controller (10, 20 o 50).
- **La búsqueda por texto no usa índice.** Casi toda usa `.ToLower().Contains(q)` en EF, que se traduce a `lower(col) LIKE '%q%'`: con el comodín al principio el índice btree no sirve y termina en *sequential scan*. Solo dos servicios usan `EF.Functions.ILike`. Cuando el volumen lo justifique, la salida es la extensión `pg_trgm` con índices GIN.
- **BaseController vive al final de AuthController.cs**, sin archivo propio. Es fácil no encontrarlo.
- **La carga de archivos tiene un solo caso y su patrón.** `ProductosController.UploadImagen` recibe un `IFormFile`, valida que no esté vacío ni supere 5 MB y delega en `ProductoService.UploadImagenAsync`, que valida la extensión (jpg, jpeg, png, webp), guarda bajo `wwwroot/uploads/productos/` y persiste la ruta relativa en `Producto.ImagenUrl`. Se sirve como estático vía `app.UseStaticFiles()`. Es el único punto del sistema que guarda archivos en disco: conviene seguir ese patrón si aparece otro.
- **No hay pruebas automatizadas ni linter.** No existe proyecto de test en la solución, así que `dotnet test` no corre nada, y del lado del frontend no hay eslint. Los únicos controles automáticos son `dotnet build` y el *type-check* de `vue-tsc`. Toda verificación funcional es manual: levantar el sistema, loguearse y recorrer el flujo.
- **Persona no es una sola tabla.** Coexisten `Patient` (clínico, con login) y `Cliente` (comercial, sin login) como dominios separados, cada uno con su controller y su servicio, colgando ambos de `Person`. El documento de identidad es `Person.CI` (cédula), no DNI. El razonamiento está en el [ADR 0001](#).

D.7. Construcción de las imágenes

El despliegue completo — servidor, `docker-compose.prod.yml`, variables de entorno y TLS — está en el Capítulo 21. Acá quedan solo los detalles de construcción de cada imagen.

Backend. El `Dockerfile` es multi-etapa: compila con `mcr.microsoft.com/dotnet/sdk:10.0`, publica `SIGA.Api` y corre sobre `aspnet:10.0`, exponiendo el puerto 8080. Para construirlo suelto:

```
docker build -t siga:local SIGA/
```

Frontend. También multi-etapa: Node construye y nginx sirve el estático.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_HCAPTCHA_SITE_KEY=10000000-ffff-ffff-ffff-000000000001
ENV VITE_HCAPTCHA_SITE_KEY=$VITE_HCAPTCHA_SITE_KEY
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx-spa.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

La *site-key* de hCaptcha se resuelve en tiempo de *build*: Vite reemplaza `import.meta.env.VITE_HCAPTCHA_SITE_KEY` por el valor literal en las pantallas públicas. El valor por defecto es la *test key*, siempre válida; para el registro público real hay que pasar la de producción como *build-arg*. No es un secreto: es una clave pública de cliente.

La segunda etapa usa `nginx-spa.conf`, cuyo único trabajo es hacer *fallback* a `index.html` para que el enrutado del lado del cliente sobreviva a una recarga directa sobre una ruta interna:

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
}
```